

APPLIED IMPLICIT COMPUTATIONAL COMPLEXITY

Neea Rusch

Submitted to the Faculty of The Graduate School
of Augusta University in partial fulfillment
of the Requirements of the Degree of
Doctor of Philosophy
September 2025

Dissertation defended on Friday, 15 August 2025, in Augusta, Georgia
before the dissertation committee composed of:

Dr. Clément Aubert, *major advisor* Augusta University, United States
Dr. Martin Avanzini Centre Inria d'Université Côte d'Azur, France
Dr. Yuyan Bao Augusta University, United States
Dr. Bogdan Chlebus Augusta University, United States
Dr. Harley Eades III Augusta University, United States

© 2025 by Neea Rusch

CC Attribution 4.0 International

Acknowledgements

Multiple individuals deserve repeated recognitions for their impact in my doctoral journey. However, in interest of brevity, I have adopted a principle from *linear types*: mentioning each name exactly once.¹ Let that principle not diminish the significance of anyone's role.



Family. Kiitos, Eila ja Lefa. Oli kiva nähdä ja toivottavasti tavataan taas. I am thankful to my children, Ileana and Tommy. They both influenced my doctoral studies in their own ways. I am incredibly proud of both of you. Tank, my fondest companion and best buddy, was with me every year I spent at Augusta University, 2011–2025.

Academic thanks. My history at Augusta University is extensive. It involves three institutional names,² various roles, learning moments, and interactions with many exceptional individuals. My current achievements are thanks to the last one. In these acknowledgements, I aim to capture everyone who contributed overall positively to this outcome.³

From my undergraduate years, 2011–2014, I appreciate every professor who was teaching computer science at the time: Joanne Sexton, Mike Dowell, Onyeka Ezenwoye, and Paul York. From them, I learned the foundations of my discipline. My undergraduate experience was encouraging, nurturing, and provided the skills that helped me succeed later, at Georgia Institute of Technology and various companies. I am thankful to Joseph Hauger,

¹Except Clément, because his actions and influence require multiple mentions.

²Augusta State University, Georgia Regents University, and Augusta University.

³Though, I suspect I still end with a sound under-approximation.

Tom Colbert, and Sam Robinson (Professor Emeritus), from the College of Science and Mathematics, for supporting me financially during those years through the Savannah River Scholar's Program. I am also thankful for the instruction of Predrag Punoševac as I understood the value of his teachings only years later. Finally, I appreciate Jurgen Brauer (Professor Emeritus) and Walter Evans (Professor Emeritus), for creating me opportunities to develop my software engineering skills before I even graduated.

After an enriching hiatus, I returned to Augusta University in 2019. First as an instructor and soon as a doctoral student.⁴ This return was prompted in response to the many invitations and encouragement I received (again) from my undergraduate professors. I recognize and thank Steve Weldon for his role in supporting this re-entry. I appreciate my fellow pioneering students, James O'Meara and William Cocke, who were present with me in those early days of the computer science doctoral program, in the spring of 2021.

The years of my doctoral study were made pleasant and memorable thanks to the many friends and colleagues I met at the *Palazzo*⁵ in Summerville. I thank Peter Hanukaev for his friendship, and leadership of $\Delta\Delta\Delta$, and organizing with me the programming languages reading group. I thank Deivid Vale, Gabriele Cecilia, Jason Weeks, Mahady Hassan, Mark Holcomb, and Vignesh Sivakumar for the shared experiences of the reading group and many social events. A special recognition to my fellow Palazzo ladies and friends. Zain Halloush, I will always remember the "escape mission" before the movie *Millennium Actress*. Nour Alhussien, my special friend, thank you for everything – including the many delicious Arabic foods I got to sample. I am thankful for the many magical moments we all shared as students. I am excited for all our futures and learning what happens in the next chapters.

Being in the first cohort in a doctoral program was minimally a character-building experience. It required much resilience and tenacity, since various policies were not in place, or

⁴In 2020 or 2021, depending on where we set the official start.

⁵The name was inspired by Konstantin Britikov and Rodrigo Otoni; and validated by Simone Gazza.

were not yet fitted for computer science. I accumulated many “firsts” and uncovered several edge cases in the program logistics, which I hope will be resolved over time. I appreciate Gagan Agrawal, for his commitment to student success in the early days of the program, and his direct support to me at various stages of my doctoral study. As a testament to Gagan’s momentous influence—although he left Augusta University two years ago, just two years after the launch of the program—he had a direct impact on the successes of three students that were first to defend their dissertations.

During these turbulent times, I also greatly appreciate the stability, peace, and support I found at The Graduate School (TGS). I particularly recognize the strong leadership, commitment, and consistency in action of the TGS Dean, Jennifer Sullivan. I appreciate Patricia Cameron, for her responsiveness and support, especially during the Three Minute Thesis competition. Finally, I want to thank Emily Crider, for the many prompt responses and overall support of the graduate students. Also thanks to The Graduate School for hosting the many wonderful events that fueled and nourished the students of the Palazzo.

Across other departments, I am thankful for the supportive and positive staff at the Center of Writing Excellence, where I worked with Brilynn Janckila, Candis Bond, Hannah Soblo, Makayla Mathews, and Romana Hinton. I also appreciate the staff at the Reese Library, which is my special retreat in Summerville. I am thankful for all the times I had the privilege of attending lectures of John Hayes of Pamplin College of Arts. Finally, I greatly appreciated the numerous performance I experienced at the Maxwell Performing Arts Theatre, especially the many organized by Matthew Buzzell.

But, beyond friends and events, doctoral study is foremost a phase of individual growth. Easily the most influential person in my journey was my advisor and scientific mentor, Clément Aubert. I started my doctoral study without research experience, but through his guidance developed an appreciation for the elegance and beauty of theoretical computer science. Through years of mentoring and countless conversations, he helped me grow and instilled

the research skills that will carry me in the future. It saddens me greatly that our interactions are inevitably ending, and that I am completely incapable of adequately expressing my gratitude for the past. The Augusta University community is infinitely enriched by your presence and mentorship. I hope many more students get to experience it.

In research, I am thankful for my collaborators, Thomas Rubiano and Thomas Seiller, for the many projects we completed together. In my mind, our projects represent the ideal model of scientific collaboration, and I am happy to have experienced that model early. I am also thankful to the members of my doctoral committee. Bogdan Chlebus, for his impact in those early days of the doctoral program, and for the guidance that led me to finding the best advisor. Harley Eades III, for instituting programming languages research in Augusta and for introducing me to functional reactive programming. Martin Avanzini, for graciously hosting me in Sofia Antipolis, and for the many fascinating conversations about the flow calculus. Yuyan Bao, for the many thoughtful conversations, reading group experiences, and our many shared experiences.

I am also immensely thankful to Neil Jones and Lars Kristiansen, for creating the flow calculus of mwp-bounds. The calculus has been a source of much inspiration and wonder throughout my doctoral study, and expectedly will be also in the future.

Outside Augusta, my doctoral journey was greatly elevated by the many fascinating people I met on my travels. Besides the names already mentioned, I appreciate the following individuals – for the conversations, guidance, inspiration, memories, shared adventures, and friendship – that shaped my doctoral experience.

Ahmed K. Zaher, Alaia Solko-Breslin, Alan Jeffrey, Alfons Laarman, Amirali Ebrahimzadeh, Anssi Yli-Jyrä (for the reissumies), Arjun Ramesh, Ármin Zavada, Arthur Correnson, Brigitte Pientka, Bryan Richlinski, Caleb Stanford, Chengsong Tan, Chris Brzuska, Christine Rizkallah, Delphine Demange, Dhanushka Jayasuriya, Dionysios Spiliopoulos,

Elliot Bobrow, Eric Cornelissen, Ewen Denney, Fabian Muehlboeck (for the dissertation format choice), Haoyi Zeng, Ignacio Ballesteros, Jared Pincus (for Computer Junkyard), Jonathan Aldrich, João C. Pereira, Judith Perera, Julian Loss, Jérémy Thibault, Krystal Maughan, Laura Kovacs, Leona Odole, Linard Arquint, Lisa Oakley, Long Pham, Luca Maio, Mandana Farhang Ghahfarokhi, Marsha Chechik, Mihai Nicola, Mishel Carelli, Mohammed Foughali, Natarajan Shankar, Norine Coenen, Onur Şahin, Pamela Zave, Paul Patault, Rémi Desmartin, Romain Péchoux, Ross Horne, Rustan Leino, Santiago Bautista, Shengyu Huang, Shriram Krishnamurthi, Sumit Gulwani, Thomas Lamiaux, Tim Nelson, Tobias Nießen, Tomáš Dacík, Wilf Offord, Yannick Forster, and Zahra Moezkarimi.

I hope we meet again, sometime, somewhere!

Recognitions of support in creation of this dissertation. This dissertation benefits from many open source software projects, developed benevolently by contributors around the world. I thank the development teams of the many excellent scientific programming tools—like the Rocq Prover, Dafny, and Python—that I regularly use in my research.

I thank Clément Aubert for the initiative and action of preparing the general L^AT_EX template for doctoral dissertations, and making it open source. Now it benefits all students of The Graduate School at Augusta University.

Several aspects and styling of this dissertation were inspired by other researchers, through their work. I appreciate Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti, for their artistry in styling code blocks in (Giallorenzo, Montesi, & Peressotti, 2024), and for sharing those sources on arXiv. The same style is used throughout this dissertation. I was greatly impressed by the work of Swaraj Dash, Younesse Kaddar, Hugo Paquet, and Sam Staton in (Dash, Kaddar, Paquet, & Staton, 2023); and its companion artifact, the Haskell library LazyPPL (Dash, Kaddar, Paquet, & Staton, 2024). The presentation influenced the

preparation of the pymwp tool user guide that appears in §2.1.7. Finally, I am thankful to Arnaud Spiwack, Csongor Kiss, Jean-Philippe Bernardy, Nicolas Wu, and Richard A. Eisenberg for their work in “*Linearly qualified types*,” (Spiwack, Kiss, Bernardy, Wu, & Eisenberg, 2022). From the manuscript, I learned many neat techniques for formatting type systems and working with unicode.

Viimeisenä ISO kiitos Tonille ja Sipelle musiikista joka säästi monia väitöskirjan kirjoitushetkiä. Nyt ei pitäisi olla puutteita koodissa!

Acknowledgements of financial research support. The primary funding agency behind this work is me. During the doctoral study, my scientific development was also greatly enhanced by numerous research visits and professional events. Participating in these events was made possible thanks to the financial support I received from many generous agencies (in alphabetic order). The list includes all agencies that impacted or supported the dissertation research in any meaningful way, and is complete up to the date of writing these acknowledgements.

The ACM Special Interest Group on Programming Languages (SIGPLAN), AFCEA Educational Foundation, Association Internationale pour les Technologies Objets (AITO) e.V., Booz Allen Hamilton, Computing with Infinite Data (CID) programme of the European Commission, Department of Informatics at the Technische Universität München, ETAPS e.V., Graduate Student Government Association (GSGA) at Augusta University, Institute of Electrical and Electronics Engineers (IEEE), NATO Science for Peace and Security Program, National Science Foundation (NSF), Office of the Provost at Augusta University, the Programming Methodology Group at ETH Zürich, SRI International, The Graduate School (TGS) at Augusta University, Udo Keller Stiftung, fortiss GmbH, organizers and sponsors of the mentoring workshop at the International Conference on Computer Aided Verification (CAV) 2023, organizers and sponsors of the European Conference on Object-Oriented

Programming (ECOOP) 2025, organizers and sponsors of the European joint conferences on Theory and Practice of Software (ETAPS) 2024, organizers and sponsors of the International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI) 2022, organizers and sponsors of the International Programming Language Implementation Summer School (PLISS) 2025, the Japan Society for the Promotion of Science (JSPS) Core-to-Core Program, the Transatlantic Research Partnership, and the Translational Research Program of the Department of Medicine at the Medical College of Georgia.

Abstract

NEEA RUSCH

Applied Implicit Computational Complexity

(Under the direction of DR. CLÉMENT AUBERT)

Implicit computational complexity (ICC) complements classic complexity theory by developing machine-independent characterizations of complexity classes. The idea is to introduce a restriction, at the level of a programming language, that guarantees every program satisfying the restriction belongs to a particular complexity class. There are several strong motivations for this approach, including that ICC systems drive better understanding of complexity classes, yield natural definitions and proofs, and produce techniques for automatable program analysis for free. However, despite the numerous advantages, ICC remains largely a theoretical novelty. This means the practical power, limitations, and advantages of ICC-based program analyses are not well-understood. The dissertation research “puts ICC theories to test” by investigating their capabilities outside the theoretical domain. A guiding intuition is that, if applied, implicit computational complexity could provide us new techniques for program analysis and verification.

Over a series of projects, the dissertation work yields three main findings. The first shows that ICC offers complementary techniques for analyzing resource usage. In other words, it allows to produce quantitative reports about programs execution behaviors in cases that are overlooked by other techniques. Second, it is possible to adjust the analyses to track other program properties. This suggests that unrealized potential is available in ICC systems, and we should learn to leverage it. The third finding is reflective and visionary. An essential

feature of ICC is that it enables guaranteeing program properties by construction, before any program exists. This enables providing behavioral guarantees without needing to run any post-analysis. This is a powerful concept in principle, but still beyond reach. The practical attainability of this goal depends crucially on a viewpoint shift – we should consider ICC in the broader context it offers and continue exploring its applied potential.

KEYWORDS: programming languages · static program analysis · implicit computational complexity · verification · formal methods

Table of Contents

1	Introduction	1
1.1	Problem Statement and Specific Aims	2
1.1.1	Dissertation Core Topics	4
1.1.2	Addressed Problem	5
1.1.3	Dissertation Goals	7
1.1.4	Research Plan	8
1.1.5	Manuscript Overview	12
1.1.6	Important Tips to the Reader	13
1.2	Literature Review	18
1.2.1	Implicit Computational Complexity	19
1.2.2	Static Resource Analysis	29
1.2.3	The Flow Calculus of mwp-Bounds	42
1.2.4	Loop Transformations for Parallel Programming	70
1.2.5	Confidentiality and Non-Interference	92
1.2.6	Programs Proofs for Formal Reasoning	112
2	Published Manuscripts	148
2.1	pymwp: A Static Analyzer Determining Polynomial Growth Bounds	149
2.1.1	Introduction – Making Use of Implicit Complexity	150
2.1.2	Calculating Bounds with mwp-Analysis	152
2.1.3	Technical Overview of pymwp	154
2.1.4	Implementation Advancements	155
2.1.5	Experimental Evaluation	157

2.1.6	Conclusion	160
2.1.7	Tool User Guide	163
2.2	Distributing and Parallelizing Non-canonical Loops	179
2.2.1	Original Approaches to Automatic Parallelization	181
2.2.2	Background: Language and Dependency Analysis	183
2.2.3	Loop Fission Algorithm	190
2.2.4	Limitations of Existing Alternative Approaches	193
2.2.5	Evaluation	195
2.2.6	Conclusion	200
3	Unpublished Research	204
3.1	Polynomial Postconditions via mwp-Bounds	205
3.1.1	Introduction	206
3.1.2	A High-Level Picture	211
3.1.3	Technical Preliminaries of the Flow Calculus	216
3.1.4	Variable-Guided Matrix Exploration	224
3.1.5	mwp-bounds as Postconditions	233
3.1.6	Comparing Related Techniques	238
3.1.7	Experimental Evaluation	243
3.1.8	Conclusions and Future Directions	247
3.1.9	Appendix A: Application to Program Verification	249
3.1.10	Appendix B: Technical Details of Analyzer Comparison	250
3.1.11	Appendix C: Details of Experimental Evaluation	254
3.2	A Logic for Anytime Non-Interference	257
3.2.1	Introduction	258
3.2.2	High-level Overview	263
3.2.3	The Non-interference Logic	265
3.2.4	Capturing Anytime Non-Interference	274

3.2.5	Interpreting Function Calls in an Anytime Non-Interfering Context . .	277
3.2.6	Practical Applications and Comparison	283
3.2.7	Conclusion: Strengths, Limitations and Future Directions	287
3.2.8	Appendix A: Proof of Theorem 3	289
3.2.9	Appendix B: Examples	292
3.3	Certifying Complexity Analysis	297
3.3.1	Motivation	298
3.3.2	Preliminary Action Plan	299
3.3.3	Related Work	302
4	Discussion	304
4.1	Observations About Specific Techniques	305
4.1.1	Advancements of the Flow Calculus	305
4.1.2	Extending the QI Framework	308
4.2	Broader Research Findings	312
4.2.1	Utilities in Automatic Resource Analysis	312
4.2.2	Motivating Analyzer Developments	313
4.2.3	The Implicit Costs of Applied ICC	315
4.3	Open Problems for Future Work	316
4.3.1	Questions About the Flow Calculus	316
4.3.2	Other Emerging Research Questions	317
4.4	Completion of Dissertation Goals	318
4.5	Research Conclusions	320
5	Summary	322
5.1	Applied Implicit Computational Complexity: Extended Abstract	323
5.1.1	Seeking Strong Software Quality Guarantees	324
5.1.2	Addressed Problem and Goals	325

5.1.3	Research Questions and Approaches	327
5.1.4	Methodology and Research Deliverables	330
5.1.5	Research Findings and Conclusions	331
6	References	335
7	Additional Published Manuscripts	361
7.1	mwp-Analysis Improvement and Implementation: Realizing Implicit Com- putational Complexity	362
7.1.1	Introduction: Letting ICC Drive the Development of Static Analyzers .	364
7.1.2	Background: The Original Flow Analysis	365
7.1.3	A Deterministic, Always-Terminating, Declension of the mwp Analysis	368
7.1.4	Extending and Improving the Analysis: Functions and Efficiency . . .	372
7.1.5	Implementing, Testing and Comparing the Analysis	377
7.1.6	Conclusion: Limitations, Strengths and Future Work	378
7.1.7	Appendix A: Technical Appendix on Semi-rings (Abridged)	382
7.1.8	Appendix B: Omitted Proofs	383
7.1.9	Appendix C: Benchmarks	384
8	Index of Artifacts	386
9	Co-author Statements	388
10	Index of Acronyms	395
11	Symbol Index	397
12	Term Index	401

List of Tables

1	The programming languages of code listings	15
2	Manuscript background dependency associations	18
3	Theoretical implicit computational complexity systems and results	28
4	Technical developments of the flow calculus of mwp-bounds	51
8	Tools for formal methods	119
9	The Rocq prover formalization results	125
10	Comparison of obtained resource bounds	154
11	Benchmark results	159
12	Examples of obtained bounds	159
13	Definition of Out, In and Occ for commands	185
14	Parallelization tools feature support comparison	195
15	Speedup comparison between parallel benchmarks	199
16	Descriptions of evaluated parallel benchmarks	199
19	Summary of analyzer capabilities, behaviors, and assumptions	244
21	Experiment results for mwp_f and mwp_ℓ	245
23	Benchmark suite characteristics by count and (mean).	255
24	Definition of Out, In and Occ for commands	266
25	Statement examples, sets, and possible violation(s)	269
27	Anytime non-interference with pure functions	278
28	Statement examples, interpretation and sets involving effects	281
29	Benchmark results produced by pymwp on C programs	385

30	Published and archived artifacts	386
----	--	-----

List of Figures

1	Dissertation manuscripts and their associations	9
2	Implicit complexity in a nutshell	23
3	Program analysis as an approximation	31
4	Sound and incomplete bug detector vs. verifier	32
5	The pymwp static analyzer workflow	49
6	Variable labels of an mwp-matrix	53
7	The inference rules of the original mwp-calculus	55
8	Visualizing loop transformations	75
9	The fork-join programming model	83
10	Parallel worksharing in loop executions	86
11	Loop fission transformation	91
12	Document classification scheme	94
13	Grammar of the non-interference security type system	105
14	Non-interference security type system	105
15	Formal verification approaches	118
16	Development view of the simple Rocq proof	123
17	Overview of Dafny implementation and verification workflow	126
18	Automation degree chart	127
19	Dafny running in Visual Studio Code	128
20	Simple imperative while language	184
21	Statement examples, sets, and representations of their dependencies	187
22	Data-Flow Graph of composition	188
23	Distributing a more complex while loop	192

24	Code transformation example	196
25	Speedup of selected benchmarks	198
26	Simple imperative programs and their corresponding data flow graphs . . .	213
27	The mwp-analysis workflow	216
28	A selection of flow calculus rules for assigning vectors to expressions. . . .	218
29	A data-flow graph of LucidLoop	222
30	Flow calculus inference rules for while and loop	229
31	A demonstration of improved derivation failure handling	231
32	Loop cases for analyzer comparison	241
33	Postcondition analysis scopes	254
34	A simple imperative while language	265
35	Security-Flow Matrix of compositions	270
36	Primary research findings	321
37	Original non-deterministic flow analysis rules	366
38	Deterministic improved flow analysis rules	368

List of Listings

1	Equivalent programs	3
2	A cost equation system	35
3	Derivable program	58
4	Loop that always fails the derivation	59
5	Loop that fails for some derivation paths	59
6	Program with a single assignment	60
7	Program with exponential variable value growth	61
8	Program with a while loop	63
9	“Infinite” program	65
10	Challenge program whose derivability is unknown	66
11	Program with control flow	68
12	Transformed program with arithmetic	69
13	Loop of insertion sort	71
14	Canonical loop form	72
15	Loop fusion transformation	74
16	Loop fission transformation	74
17	Loop splitting transformation	75
18	Loop tiling transformation	75
19	Loop peeling transformation	75
20	Loop unrolling transformation	75
21	Before interchange	75
22	After interchange	75
23	True dependence	77

24	Antidependence	77
25	Output dependence	78
26	Control dependence	78
27	Loop-carried dependence	79
28	The OpenMP parallel construct	85
29	Serial execution	86
30	Parallel execution	86
31	Parallel worksharing	86
32	Sequential version	88
33	Parallel version	88
34	Applying full unroll	89
35	Fully transformed loop	89
36	Applying partial unroll	90
37	Partially transformed loop	90
38	Secure and insecure Boolean operations	97
39	Implicit information flow leak in an array	97
40	High conditional incremental leak	97
41	Uncovering loop invariants	116
42	A simple proof in Rocq	122
43	A Rocq proof of program equivalence	130
44	Proof of program equivalence in Dafny	132
45	Leftpad in action	133
46	Leftpad in Rocq (1)	135
47	Leftpad in Rocq (2)	136
48	Leftpad in SSReflect proof language	137
49	Formally verified Leftpad in Dafny	138
50	Moscow messaging functions	142

51	Receiver knows sender	142
52	Person A cannot recover the allocation	143
53	Running a certified message exchange	144
54	An imperative program that manipulates variables	165
55	Program assign_expression	169
56	Program exponent_2	171
57	Program notinfinite_3	172
58	Program infinite_3	174
59	Challenge program dense_loop	175
60	Precise context	208
61	Imprecise context	208
62	Benchmark: LucidLoop	241
63	Benchmark: Function condition	241
64	Benchmark: Finite iteration	241
65	LucidLoop verified in Dafny	249
66	Running pymwp in loop analysis mode	250
67	Program analysis with Duet	251
68	Creating a trace with Daikon	252
69	Infer invariants using Daikon	252
70	Displaying invariants inferred by Daikon	252
71	Benchmark: mwp/example 3.4	253
72	Benchmark: mwp/not infinite #4	253
73	Benchmark: Linear #02	253
74	Expository program	263
75	Assignment Case	290
76	Loop Case	290

77 Branching Case 290

List of Algorithms

1	Program analysis with flow calculus of mwp-bounds	57
2	Command analysis with flow calculus of mwp-bounds	57
3	A loop fission algorithm for parallelizing loops	191
4	mwp-matrix evaluation	226

1 Introduction

1.1 Problem Statement and Specific Aims

A programmable computer is a gadget without precedent that can be appreciated from many perspectives. For example, it can be viewed as an engineering marvel of ever-increasing capacity, as a medium for new flashy applications, or as the machine-independent scientific discipline that forms its foundations (Dijkstra, 1979a; Hoare, 2006). This dissertation focuses on the intellectual challenge of *programming* these gadgets.

This challenge seems unique in the combination of the possibility for unmastered complexity – programs are among the most complex things ever conceived – and the ultimate, but misleading, simplicity of a world of zeros and ones alone (Dijkstra, 1979a).

Programming languages provide us a convenient interface for crafting programs. A programming language is an abstraction layer, because it relieves us from working with 0s and 1s directly. We can write programs in a human-intelligence form, and then “translate” (compile) it to 0s and 1s for processing by computers. However, the abstraction provided by programming languages does nothing to reduce the intellectual challenge involved in programming (Dijkstra, 1979b). If we write a syntactically legal but faulty program, we are at the mercy of our faulty creation when the program executes.

Equivalence. A fascinating feature about programs is that we can write multiple distinct programs that compute the same result for the same input. Two programs $\llbracket p_1 \rrbracket$ and $\llbracket p_2 \rrbracket$ are functionally *equivalent*, i.e., $\llbracket p_1 \rrbracket \equiv \llbracket p_2 \rrbracket$, iff for every possible input i , $\llbracket p_1 \rrbracket(i) = \llbracket p_2 \rrbracket(i)$. In other words, the criteria is to maintain the input/output behavior, but we are allowed to change *how* the result is computed. A scientifically inclined reader will recognize functional equivalence as multiple algorithms computing the same function. A software engineer will recognize it as the art of refactoring.

To demonstrate the idea, consider the two programs in Listing 1. The programs consider two “input variables”, X and Y , and store the result in variable Z . The variable `bit` is either 0 or 1. By visual inspection, it is obvious that the programs look different. Yet, for all values of X and Y , they compute the same result.¹ The programs are thus functionally equivalent.

<pre> if (bit) { Z = X } else { Z = Y } </pre>	<pre> Z = X * bit + Y * (1 - bit) </pre>
--	--

Listing 1: Equivalent programs.

It is natural to ask, “which program is better?” The program on left contains no arithmetic operations. Its behavior is easy to read and understand from how the program is written. The clarity is a useful if the program is intended for frequent visual inspection. However, we may not care about readability. Many programs, like the ones generated by a compiler, are never read by humans. The program on right has the advantages that it does not involve conditional branching and always evaluates the full arithmetic expression. Such features are beneficial in certain application domains, like information security (discussed in §1.2.5). Therefore, choosing the preferable program form depends on what is meant by “better”. A question of interest is how can we quantify and measure such aspects of programs?

Non-functional properties. Extending beyond functional behavior, we evaluate programs based on their quality attributes, i.e., *non-functional properties*. Non-functional properties include aspects like program security and resource consumption. For example, between two functionally equivalent programs, we may want to identify the program that computes the result faster or using less memory. A main theme of this dissertation is to treat *programs as mathematical objects to analyze their non-functional properties*. Among the goals are developing techniques that enable quantifying such properties. Furthermore,

¹This is an informal claim, and it will be revisited in §1.2.6.5.

instead of trusting informal claims—like the earlier one about program equivalence—we seek rigorous guarantees that the stated properties truly hold in the inspected program(s).

Scientific challenges. The challenge of programming thus involves much more than “just” the act of writing functionally correct programs. How to achieve desirable behaviors and exclude bad ones, and how to detect desirable non-functional properties in programs are scientifically independent and compelling questions.

1.1.1 Dissertation Core Topics

The quest to analyze non-functional properties in programs brings us to the topic of static program analysis. In addition, obtaining strong guarantees requires formal methods. These are the two core topics of this dissertation. For a higher sense of adventure, and to make the dissertation exploration worthy of a multi-year doctoral study, we add *implicit computational complexity* to the list of topics.

Static program analysis and formal methods. The goal in program analysis is to inspect programs and check whether they satisfy some desirable behavioral criterion. While many families of techniques exist, this dissertation focuses on analyzing programs statically, i.e., by how programs are written. Static analysis techniques study program syntax. They aim to make judgements about how the program will behave when it is executed, but without ever executing the program. Formal methods complement the setting by providing the mathematical techniques and tools to obtain rigorous guarantees. Formal methods enable providing *proofs* about the conclusions of (informal) program analysis. This way, we can substitute informal claims with maximally strong assertions.

Implicit computational complexity. The core idea of implicit computational complexity (ICC) is to quantify computational power through programming languages. Although ICC can be viewed from many perspectives, in the scope of this dissertation, its role is to provide a conceptual “toolbox” for constructing techniques of static program analysis. Considered broadly, ICC finds ways to characterize program properties—mainly resource consumption—by introducing a restriction at the level of a programming language. The restriction is such that it guarantees the target property holds in every expressible program. Programming languages thus become a *mechanism* to guarantee ideal runtime behavior. There are multiple compelling motivations for this approach. For example, ICC drives better understanding of resource consumption and provides natural ways to express and control resource usage of programs (Kristiansen, 2017). Because behavioral guarantees are part of the language design, this creates a *temporal shift* that enables reasoning about properties *before* any program exists. If we can identify the construction principles of satisfactory programs, this would remove the need to analyze programs after they are written. However, materializing such ideas in practice requires deeper investigations of implicit computational complexity.

*** Primary topics summarized.** The dissertation investigates problems in static program analysis, ideally with formally verifiable guarantees, and from the starting point of implicit computational complexity.

1.1.2 Addressed Problem

Given the many compelling motivations, there exists a long series of theoretical results in implicit computational complexity (reviewed in § 1.2.1.4). These works focus primarily on *defining* theoretical systems. There are substantially fewer explorations of *applications*

of those theories (see §1.2.2.4 and §1.2.5.6 for related presentations).² The difference in allocated efforts could be explained by many factors, including the following.³

1. *Broad insight is required to identify suitable application domains.* The claim is evidenced by, e.g., Parikh’s Theorem from automata theory. Classically, the theorem is about letter-counting and language expressiveness. Its practical usefulness was enabled only *decades later* by the development of efficient algorithms. Since then, it has been exploited in a range of applications like SMT solvers, verification of cryptographic protocols and concurrent programs, and query evaluations in graph databases (Hague, Jež, & Lin, 2024, pg. 2).
2. *There is a conceptual gap between pure theory and practical applications.* For example, the flow calculus of mwp-bounds (reviewed in §1.2.3) was designed for theoretical analysis of programs. Although the initial presentation (Jones & Kristiansen, 2009) seems to provide an automatable technique, much subsequent work was needed to materialize this objective. In general, it is extremely difficult to foresee application challenges without attempting to bridge the gap.
3. *An application may require changing the research domain.* For example, a result in theoretical computer science may find relevant uses in formal verification, symbolic execution, or secure compilation. The cross-domain adaptation then requires communicating findings to new communities who may be motivated differently to assess the usefulness of the findings.

Some application challenges are specific to implicit computational complexity. Restricting a programming language sacrifices Turing-completeness. A non-Turing-complete programming language means certain programs become inexpressible. The task of writing (or

²By “application”, we refer to any use case beyond complexity theory, including implementations.

³The claims come from observations and conversations; they are unlikely to exist in literature.

finding) a satisfactory program is then delegated upstream to the software engineer (Moyen, 2017, p. 14). If the restriction is too strong, it is difficult, or perhaps impossible, to express natural algorithms. This creates a trade-off between theoretical soundness (i.e., providing a guarantee) and expressiveness (Férée, Hym, Mayero, Moyen, & Nowak, 2018). It is thus justifiable to ask, what are the uses of such restricted programming languages?⁴

With these considerations in mind, the *main hypothesis* of this dissertation pushes to investigate the application potential of implicit computational complexity.

*** Main hypothesis.** Implicit computational complexity offers applied utilities when lifted from the theoretical domain.

Every manuscript in this dissertation is an instantiation of this main hypothesis. Through applications, we can stress test capabilities of ICC systems; pinpoint limitations and discover use cases in other domains.

1.1.3 Dissertation Goals

In support of the main hypothesis, we define four actionable goals.

- **Goal 1.** *Extend applied capabilities in automatic program analysis and verification.* The motivation is to show that ICC theories can provide new complementary techniques for automatic program analysis and support verification of non-functional properties.
- **Goal 2.** *Take ICC techniques closer to integration with real-world software development workflows.* Such integration would drive theoretical improvement, support

⁴For an inspirational warm-up, see <https://stackoverflow.com/questions/315340>; a discussion about practicality of non-Turing-complete languages.

relevance of ICC techniques, and motivate thinking about ICC from fresh perspectives.

- **Goal 3.** *Initiate conversations about the relevance of ICC applications.* Improvements in ICC are typically achieved by showing improvement in expressiveness. A common strategy is to show that the newly presented system gives a guarantee for some subset of programs that were not expressible previously.⁵ However, this strategy is limited in quantifying practical usefulness of the ICC systems. Particularly, it does not account for the “methodological cost” at which the improvement is achieved. Applications would provide complementary insight and metrics to assess the capabilities of ICC systems.
- **Goal 4.** *Expose ideas from implicit computational complexity to broader research communities.* Wider dissemination of ideas could lead to discovery of new applications and ease communication about ICC results.

The completion of these goals is discussed in §4.4.

1.1.4 Research Plan

To make the main hypothesis and goals more concrete, we define two research directions. One focuses on automatic resource analysis. The other investigates the uses of implicit computational complexity in ensuring non-functional properties (besides complexity). These directions are visualized in Fig. 1. An alternative way to think about these directions is that the first aims to apply ICC systems in the originally designed ways, while the second seeks to discover unexpected applications.

⁵The strategy is used e.g., in (Hainry & Péchoux, 2023, p.16–17), (Jones & Kristiansen, 2009, p. 18), and (Férée, Hym, Mayero, Moyen, & Nowak, 2018, p. 147).

 Concept
  Prior foundational work
  Dissertation work
 * Preprint

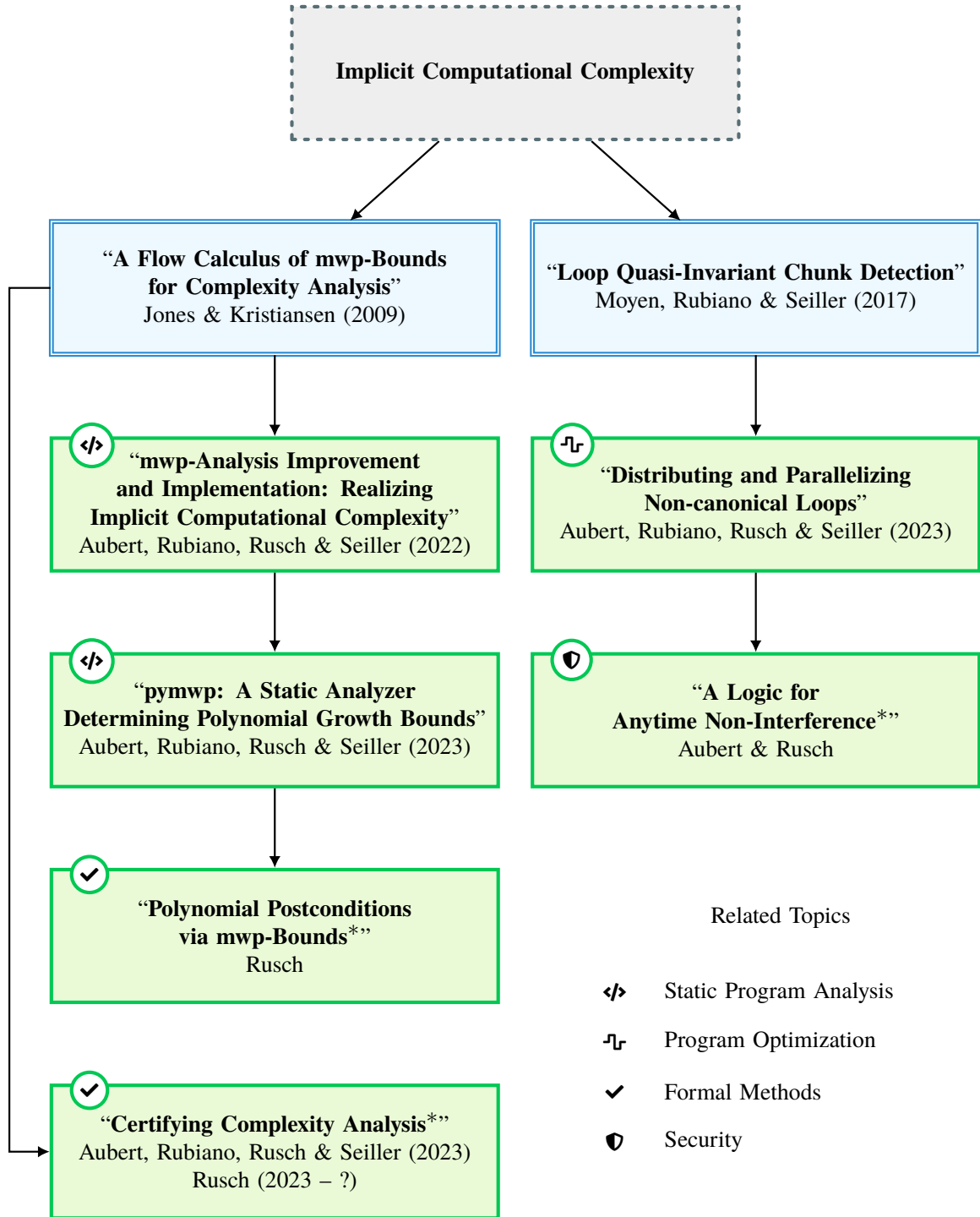


Figure 1: Dissertation manuscripts and their dependency associations.

1.1.4.1 Automatic Resource Analysis

In the first direction, we focus on the flow calculus of mwp-bounds, introduced by Jones and Kristiansen (2009). It is a canonical example of an ICC system for analyzing variable value growth (data size) in imperative programs. The flow calculus is introduced in detail in §1.2.3.

Building on the flow calculus, we define three research questions.

RQ1 Can we develop an automatic program analysis based on the flow calculus?

RQ2 Given its paper proofs, is the theory correct? I.e., can we prove formally the soundness of the flow calculus?

RQ3 Assuming the theory can be automated, what are its use cases?

The manuscripts building on the flow calculus demonstrate its uses in static program analysis and in formal verification for specification inference. The manuscripts support all dissertation goals. The manuscripts concerned with the implementation of the flow calculus (RQ1) support Goals 1–3. A formally verified soundness proof (RQ2), and an application to formal verification (RQ3) support Goal 4.

One manuscript, “*mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity*” is in the Appendix (§7) due to an Augusta University policy. The placement should not distract from the significance of the work, as several other manuscripts follow from it.

The results of the research questions are discussed in §4.1.1.

1.1.4.2 Analyzing Extended Non-Functional Properties

The second direction starts with complexity-theoretic techniques but repurposes them toward analysis of other non-functional properties. The technical foundation is inspired by “*Loop Quasi-Invariant Chunk Detection*” by Moyen, Rubiano, and Seiller (2017a). In the original formulation, the idea is to identify fragments of loops (called “blocks”), whose variables become invariant after a finite number of iterations. Such loop quasi-invariant code blocks can then be lifted from the loop. The program transformation improves the loop’s complexity profile if the lifted block is a nested loop. For convenience, we refer to this technique as the *QI framework*, after quasi-invariants.

This original QI framework is not presented in this dissertation. This is because each refinement changes the behavior of the system and requires fully redefining the system each time. The manuscripts in §2.2 and §3.2 thus provide the technical details.

We define two research questions relating to the QI framework.

RQ4 How to develop a program transformation to increase parallelization potential?

RQ5 How to use it to analyze security properties, specifically non-interference?

The research questions thus change the motivation from complexity theory to other application domains. Critically, the original complexity result may be lost, but some other guarantee is gained. On the surface, the research questions appear to be about properties, but more deeply they are about understanding better the underlying theory and its flexibility.

Both goals are practically motivated. The expected deliverables support Goals 1 and 2. Moreover, they are strongly aligned to support Goal 4.

The results of the research questions are discussed in §4.1.2.

1.1.5 Manuscript Overview

The dissertation manuscripts connect implicit computational complexity with related research topics. Each manuscript is decorated with an icon that denotes the secondary topic. The topics include: static program analysis , security , program optimization , and formal methods .

Author contributions. The authors are always listed in alphabetical order. The manuscripts in Chapters §2–§3 are “first author” works. The manuscripts in Chapter §7 are “non-first author” works. The co-authors contributions are detailed in Chapter §9.

Peer review. The entire Chapters §2, §3, §5, and §7 have been peer reviewed. In other words, the only new content is Chapters §1 and §4, i.e., Introduction and Discussion.

- The published works in Chapters §2 and §7, were inspected by 7–10 reviewers before acceptance.
- The unpublished works, in Chapter §3, have been reviewed by at least 4 reviewers thus far. Preliminary versions have been accepted and presented at respectable workshops. Be advised that the unpublished works must be viewed with caution, as they are works in progress. All works have some limitations that prevent their inclusion among the published manuscripts.
- Chapter §5 is an extended abstract. It describes about the full-length dissertation in a standalone manner. It has been peer-reviewed and presented at the Doctoral Symposium of the European Conference on Object-Oriented Programming 2025.

1.1.6 Important Tips to the Reader

This dissertation contains three varying-length versions of its content. This allows accessing the presentation at different levels of detail, by need.

* Content organization.

1. The *abstract* contains the highlights only.
2. *Chapters §1–§4 and §7* form the full-length presentation.
3. *Chapter §5* is a standalone extended summary of the full presentation.

1.1.6.1 Software Artifacts and Data Availability

All software developed as part of this dissertation is publicly available. Each published manuscript in the dissertation has an associated software artifact. The artifacts are archived, according to the policies of the Association for Computing Machinery (ACM, 2020), for long-term retention. The artifacts are archived even if the publication venue did not provide an official artifact evaluation round. Therefore, each manuscript consists of more than just the text pages of the dissertation. How to locate the artifacts is explained in Chapter §8.

Dissertation source code. The source code of the dissertation (Rusch, 2025d) is publicly available at the following repository.

<https://github.com/nkrusch/dissertation>

www

Dissertation artifact. The dissertation has a companion artifact. The artifact is preloaded with all required software and enables reproducing all executable examples that appear in the dissertation. For example, §2.1.7 can be followed interactively with the artifact. The URL of the artifact is:

<https://doi.org/10.5281/zenodo.17148077>

www

1.1.6.2 Notational Conventions

Programming languages, syntax, and code blocks. Syntactic constructs (variables, expressions, commands, etc.) that are embedded in text are typeset in `teletype`. Larger or more significant code blocks are displayed as dedicated code listings. The listings will display, in the bottom right corner, the associated programming language or context, according to Table 1. In the table, *version* is the language release version assumed in the listings, when applicable (pseudo-languages do not have a version). Plain text listings, that describe command outputs, are labelled “output.” Internet addresses are marked by  “www.” The Rocq Prover is undergoing a name change from its former name Coq. We will use the new name whenever possible.

Line numbers. References to a line of source code (or an algorithm) begin with an L, for line. The L is followed by a number (or a numeric range) that specifies the row(s) of interest. For example, L3 means *line number 3*, and L10–12 means *lines 10 to 12*.

Distinguishing variable states. We will often want to refer to the same variable in different states of computation. In the style of *Z specification language* (Spivey & Abrial, 1992), we use notation that identifies the *new* (post) states. A plain variable refers to a state where

Language	Description	Version
 C	the C programming language	C99
 C*	imperative C-like pseudo-language	–
 CES	cost equation system	–
 cmd	executable shell/terminal command	–
 Dafny	the Dafny programming language	4.10.0
 Imp	imperative language of mwp-calculus	–
 Java	the Java programming language	SE 24
 OMP	C code that includes OpenMP directives	6.0
 Rocq	the Rocq interactive theorem prover	8.20.1
 SSReflect	the SSReflect proof language	2.4.0
 While	simple imperative while language	–

Table 1: The programming languages used in code listings.

the variable holds its initial value. A variable with a postfix decoration ' refers to a state where the variable holds its final value. For example, X refers to the initial value and X' refers to the final value.

1.1.6.3 Lookup Indices

The dissertation includes three indices for terms, acronyms, and symbols. The Term Index (§12) lists technical terms and their uses. Acronyms are generally defined at first use and the Index of Acronyms (§10) lists the long form. The Symbol Index (§11) lists definitions of symbolic notations.

1.1.6.4 Dynamic Bibliographic Entries

Some bibliography entries refer to dynamic resources like software, web pages, or lecture notes (without DOIs). References to such documents may become unreliable over time. To improve the long-term availability and recoverability, they are pre-emptively preserved in suitable online archives, on Software Heritage and the Internet Archive.

Software repositories. Version-controlled third-party software repositories are archived at Software Heritage. For a repository hosted at <URL>, the recovery address pattern is:

```
https://archive.softwareheritage.org/browse/origin/directory/  
?origin_url=<URL>
```

www

Example 1 (Recovering a software repository). *The dissertation source code archival recovery address is:*

```
https://archive.softwareheritage.org/browse/origin/directory/?origin_u  
rl=https://github.com/nkrusch/dissertation
```

www



Written documents. Web pages and PDF files are archived in the Wayback Machine of the Internet Archive. Use the following pattern to recover a document from the Internet Archive – substitute <URL> by the document’s URL:

```
https://web.archive.org/web/<URL>
```

www

A few cited documents could not be archived by the Internet Archive; for example because the hosting server was blocking automatic clients. These documents are denoted in the bibliography with the symbol †.

Example 2 (Recovering archived documents). *The following URL is fragile and it may disappear in the future.*

```
https://types22.inria.fr/files/2022/06/TYPES_2022_paper_14.pdf
```

www

As a backup, the following Internet Archive address produces the same document.

```
https://web.archive.org/web/https:
```

```
//types22.inria.fr/files/2022/06/TYPES_2022_paper_14.pdf
```

www



1.2 Literature Review

For research papers, it is often optimal to arrive to new findings as soon as possible. This need is motivated by a page limit, but also the need to keep readers interested. Thus, it is natural to make certain assumptions about the readers' prior knowledge and sacrifice extensive explanations of the basics. This description also applies to the manuscripts bundled in this dissertation.

However, the dissertation does not have the same constraints as a research paper. There is neither a page limit nor a compelling reason to omit technical foundations. Since it took several years to internalize and complete the work presented in this dissertation, it seems unlikely every reader is familiar with the same foundations. Therefore, this section presents the technical background needed to enjoy to full extent the manuscripts that will follow.

Manuscript Title	Section	Background
mwp-Analysis Improvement and Implementation...	7.1	1.2.1, 1.2.2, 1.2.3
pymwp: A Static Analyzer Determining Polynomial...	2.1	1.2.1, 1.2.2, 1.2.3
Distributing and Parallelizing Non-canonical Loops	2.2	1.2.4
Polynomial Postconditions via mwp-Bounds	3.1	1.2.3, 1.2.6
A Logic for Anytime Non-Interference	3.2	1.2.5
Certifying Complexity Analysis	3.3	1.2.3, 1.2.6

Table 2: Manuscript background dependency associations.

The background is organized thematically. The subsections are (mostly) self-contained introductions to the specified topic. It is advisable to read first §1.2.1, but the other sections can follow in any order. It is also possible to initially skip the background and return to this section when a manuscript creates a need to understand its foundations. Refer to Table 2 for guidance on how to connect the dissertation manuscripts with the background topics.

1.2.1 Implicit Computational Complexity

Theoretical computer science studies the fundamental capabilities and limitations of computation. A function is *computable* if it is algorithmically solvable in principle (Sipser, 2012, p. 234). However, computability is independent of the required computational resources, i.e., *cost* or *resource usage* of computation. A computable function that requires an excessive amount of resources may not be solvable in practice.

Computational complexity theory enhances the study of computable functions by classifying them into *complexity classes*⁶ based on resource requirements. Algorithms computing those functions are categorized by the amount of required *resources* when executed on a specified *machine model*, such as a Turing Machine (TM). Which particular model of computation is used does not matter, as long as one adheres to Slot and van Emde Boas Invariance Thesis⁷ (Slot & van Emde Boas, 1984), and is concerned with complexity classes, i.e., order of magnitudes in the resources needed. The resource cost is estimated relative to the input size. This way algorithms are allowed to consume more resources when the input size grows while maintaining membership in the same complexity class. The common resources of interest are time and space (memory). Complexity measures are defined explicitly relative to the machine model. For example, time complexity refers to the maximum number of computation steps a machine uses on any input (Sipser, 2012, p. 276). Besides (deterministic) time and space, various other resources can be viewed as complexity measures; for example, parallel time and circuit complexity (Sipser, 2012, p. 428).

Asymptotic analysis is an approach for obtaining an estimation of an algorithm's resource requirements (Sipser, 2012, p. 267–277). The analysis result is expressed as *asymptotic*

⁶For a comprehensive exploration of complexity classes, see Aaronson, Kuperberg, and Habryka (2025).

⁷The Invariance Thesis asserts that a time (resp. space) cost model is reasonable for a computational model $\mathcal{C}$ if there are mutual simulations between TMs and $\mathcal{C}$ such that the overhead is polynomial in time (resp., linear in space) (Vanoni, 2022).

bounds. The “big-oh” $\mathcal{O}$ -notation denotes an asymptotic upper bound (Cormen, Leiserson, Rivest, & Stein, 2009, p. 47). However, the bound may or may not be asymptotically tight. The “small-oh” o -notation distinguishes an upper bound that is not asymptotically tight (Cormen, Leiserson, Rivest, & Stein, 2009, p. 50). Other relevant notations are Ω for an asymptotic lower bound, and Θ for an asymptotic tight bound (Cormen, Leiserson, Rivest, & Stein, 2009, Sect. 3.1). Among the benefits, asymptotic analysis develops insight of costs of computational patterns and identifies relationships between complexity classes. For algorithms, it produces measures of efficiency, and allows comparing relative performance of alternative algorithms (Cormen, Leiserson, Rivest, & Stein, 2009).

This overview of the classical complexity theory should be familiar from textbooks. However, a refresher on the basics gives a comparison baseline for the discussion that is about to follow. The remainder of this dissertation builds on *implicit* computational complexity. Implicit complexity complements the classical complexity theory by offering alternative perspectives and systems for reasoning about cost of computation.

1.2.1.1 Computational Complexity, *Implicitly*

Implicit Computational Complexity (ICC) is a subfield of computational complexity theory that explores *machine-free characterizations* of complexity classes. It inherently shifts focus from computational models to *programs*.

By implicit, we here mean that classes are not given by constraining the amount of resources a machine is allowed to use, but rather by imposing linguistic constraints on the way algorithms are formulated (Dal Lago, 2011, p. 90).

In other words, complexity classes are described independently of notions of time or space related to a machine’s definition. For example, polynomial time complexity has been thoroughly examined under these terms. The Cobham-Edmonds Thesis (Cobham, 1965; Ed-

monds, 1965) relates polynomial time class with the class of feasible functions.⁸ Multiple distinct implicit characterizations are known for many complexity classes – cf. Table 3.

Implicit computational complexity sees complexity bounds as consequences of restrictions on program structures. In this view, it studies the computational resources embedded in programming languages by *construction*. Another common feature is that the characterizations may omit references to explicit resource bounds (Moyen, 2017) – refer to e.g., suppress = MathFunctionText Bellantoni and Cook (1992a) or Kristiansen (2005) for examples. Thus, the focus is on studying the quality, rather than quantity, of the resources needed. Since an implicit characterization, or an *ICC system*, captures a specific complexity class, complexity analysis becomes a task of determining class membership, rather than classifying computations among many classes as in classical asymptotic analysis.

There are several good motivations for researching implicit characterizations of complexity classes (Jones, 1999; Kristiansen, 2017, p. 36), including the following.

- Implicit complexity drive better understanding of complexity classes and sheds light on the notorious hard open problems of complexity theory.
- From a programming language perspective, implicit complexity is motivating because it suggests ways to control complexity properties, and quantifies the computational power present in programming languages by construction.
- A programming language-based approach yields more natural definitions and proofs of central results than the classical approach.

⁸The Cobham-Edmonds Thesis asserts that the time complexities in any two “reasonable and general” models of computation are polynomially related; that is, a problem has time complexity t in some reasonable and general model of computation if and only if it has time complexity $\text{poly}(t)$ in the model of single-tape Turing Machines (Goldreich, 2008, p. 33).

- The characterizations demonstrate that complexity classes are intrinsic mathematical entities that do not depend on a particular machine model.
- Implicit complexity introduces orthogonal program analysis techniques with potential to convert complexity-theoretic insights to practical analyses.

1.2.1.2 Complexity via Programming Languages

Describing the goals of implicit complexity omits the details of *how* such implicit characterizations should be designed. Over time, numerous distinct systems have been developed based on, or inspired by, e.g., calculi, term rewriting, data flow analysis, primitive recursion, linear logic, modal logic, type systems, digital circuits, and more (Baillot, Dal Lago, & Moyen, 2012; Kristiansen, 2017; Marion, 2011; Moyen, 2017). With such rich versatility, it becomes challenging to identify ICC systems upon encounter and to find an overarching definition that matches them all.⁹

Recent reflective works (Hoffmann & Jost, 2022; P  choux, 2020) converge on descriptions centered on programming languages. Fig. 2 summarizes the idea succinctly. In this view, the unifying factor of implicit complexity is to find natural characterizations of complexity classes through *restrictions* embedded in *programming languages* (Hoffmann & Jost, 2022). It requires introducing a language-based *syntactic* restriction that guarantees a *semantic* property; typically, that a satisfactory program is a member of the characterized complexity class (Moyen, 2017). Then, if the restriction is maintained, the complexity bound will also be maintained.

⁹On the the challenges of describing ICC systems, Moyen (2017, p. 14) writes: “For a long time, I have been convinced that ICC systems are like the proverbial blind men approaching an elephant: each touching a different part and describing it in terms that let the others think this is a different beast.”

*** Implicit complexity in a nutshell.** The goal of implicit computational complexity can be summarized as follows. Let $\mathcal{L}$ be a programming language, $\mathcal{C}$ a complexity class, and $\llbracket p \rrbracket$ the function computed by program p . Then, the task is to find a restriction $\mathcal{R} \subseteq \mathcal{L}$, such that the following equality holds:

$$\{\llbracket p \rrbracket \mid p \in \mathcal{R}\} = \mathcal{C}.$$

The variables $\mathcal{L}$, $\mathcal{C}$, and $\mathcal{R}$ are the parameters that vary greatly between different ICC systems.

Figure 2: Description of ICC systems – adapted from Péchoux (2020, p. 7)

The programming language-based approach has minimally two significant effects compared to classical complexity theory. The first is a *temporal* shift that arises from making complexity an *a priori* consideration in the language design. In other words, implicit complexity systems permit reasoning about runtime properties *before a program exists*. The second is a *relational* shift between a program developer and the task of performing complexity analysis. By embedding into a language a restriction by which a complexity bound will be guaranteed, the effort of satisfying that restriction is raised upstream to the developer (Moyen, 2017). In return, to obtain runtime guarantees, the developer does not need to know about the intricacies of complexity analysis or classes. The developer can focus on writing programs and the program’s behavioral guarantees are obtained for free. Of course, this assumes that the restriction is not excessively prohibitive in expressing algorithms.

1.2.1.3 Implicit Complexity by Example: Safe Recursion

This example is adapted from a presentation by Dal Lago (2022). Consider Kleene's algebra of general recursive functions as a programming language. In this language, programs are built from basic building blocks (the initial functions) by composition, primitive recursion, and minimization. Primitive recursive functions are well beyond any reasonable complexity class due to the possibility of having nested recursive definitions. As an example, we can define addition from the initial functions

$$\text{add}(0, x) = x;$$

$$\text{add}(n + 1, x) = \text{add}(n, x) + 1.$$

Then for n , we can define exponentiation $n \mapsto 2^n$ from addition as follows:

$$\text{exp}(0) = 1;$$

$$\text{exp}(n + 1) = \text{add}(\text{exp}(n), \text{exp}(n)).$$

Composing exp with itself allows us to form towers of exponentials of arbitrary size. A recursion on exp leads us even beyond the class of elementary recursive functions, and complexity-wise, this is already too broad.

Rather than removing primitive recursion from the algebra up-front (which would have too strong an impact on the expressive power), we want to refine the algebra by distinguishing arguments that matter for complexity from arguments that do not. The arguments of any function f are dubbed either as *normal*, when their value can have a relatively big impact on f 's complexity, or *safe* when their value can only influence f 's behavior very mildly. If $\vec{x}$ are the normal parameters and $\vec{y}$ are the safe ones, we indicate the value of f on $\vec{x}$ and

$\vec{y}$ as $f(\vec{x}; \vec{y})$, where the semicolon separates the normal from the safe. This way, the usual scheme of primitive recursion can be restricted as follows:

$$f(0, \vec{x}; \vec{y}) = h(\vec{x}; \vec{y});$$

$$f(n + 1, \vec{x}; \vec{y}) = g(n, \vec{x}; \vec{y}, f(n, \vec{x}; \vec{y})).$$

The way we define f from g and h is morphologically identical to what happens in the usual primitive recursive scheme. That it is a restriction is a consequence of the distinction between normal and safe arguments: the recursive call $f(n, \vec{x}; \vec{y})$ is required to be forwarded to one of g 's safe argument, while the first parameter of f must be normal. This makes $\exp$ not expressible anymore, giving rise to new function algebra called *safe recursion*, discovered by Bellantoni and Cook (Bellantoni & Cook, 1992a) (BC in the following).

Theorem 1. *The class BC equals the class FP of polynomial time computable functions.*

□

The Bellantoni and Cook result extends an earlier result by Cobham (Cobham, 1965). The Cobham-Edmonds Thesis asserts that being feasible is the same as being computable in polynomial time (Cobham, 1965; Edmonds, 1965). Although the Cobham characterization is independent of any particular execution model, it is not fully syntactic. The limitations are that a size bound has to be proved on the semantics of recursion, and the proof uses a particular model of Turing Machine. This prevents constructing an *automatic procedure* to check whether a program satisfies Cobham's criteria (Heraud & Nowak, 2011). In contrast, the Bellantoni and Cook characterization is a fully syntactic and membership in BC can be checked automatically. Note that Safe recursion aims to guarantee that there exists a polynomial bounding the execution time; however, it does not provide the actual value of the polynomial (i.e., the complexity bound are implicit) Heraud and Nowak (2011). The

Bellantoni and Cook result was proved formally (Heraud & Nowak, 2018) by Heraud and Nowak (2011).

1.2.1.4 A Snapshot of Theoretical Results

The breakthrough result of safe recursion is regarded as the first implicit characterization of a complexity class (Dal Lago, 2011; Kristiansen, 2017; Rubiano, 2017). However, the same result was independently discovered by Leivant (1993) in a slightly different setting of stratified recurrence (Dal Lago, 2022, p. 762). Table 3 shows a very small collection of additional impactful theoretical results. The inclusion criteria is varied and based on representing the rich variety of underlying techniques or complexity classes; or for presenting foundational techniques that have been extended later; or for reflecting shifts in theoretical developments and interests over time. The table also marks with hyperlinks works that are discussed elsewhere in this dissertation. These theoretical results provide a conceptual “toolbox” of techniques that form the foundation for the rest of this dissertation.

Symbols. ♣ logics ★ type system ↻ data flow A syntactic x¹ algebras C recursion

Year		Venue	Description	Classes
1992*	A	Comput Complex.	safe recursion – §1.2.1.3 Bellantoni and Cook (1992a)	P
1993*	C	POPL	stratified recurrence Leivant (1993)	P
1995	x ¹	CSL	tiering a.k.a. ramification Leivant and Marion (1995)	PSPACE

Table 3: (Cont.)

Symbols.  logics  type system  data flow **A** syntactic  algebras **C** recursion

Year		Venue	Description	Classes
1999		LICS	non-size-increasing program Hofmann (1999)	P
2000	C	LPAR	quasi-interpretations Marion and Moyen (2000)	P
2001	A	J. Funct. Program.	CONS-free programs Jones (2001)	P, EXP
2001		POPL	size-change termination principle – §1.2.2.3 Lee, Jones, and Ben-Amram (2001)	PSPACE
2005		Comput. Complex.	LOOP ⁻ and CLIP programs Kristiansen (2005)	L, Linspace
2009		TOCL	flow calculus of mwp-bounds – §1.2.3 Jones and Kristiansen (2009)	P
2010		Theor. Comput. Sci.	quantum implicit computational complexity Dal Lago, Masini, and Zorzi (2010)	EQP, BQP, ZQP
2011		LICS	SAFE programs – §1.2.5.6 Marion (2011)	P
2013		ICALP	ramified programs – §1.2.5.6 Leivant and Marion (2013)	P, L
2013		FoSSaCS	SAFE processes – §1.2.5.6 Hainry, Marion, and Péchoux (2013)	P

Table 3: (Cont.)

Symbols.  logics  type system  data flow  syntactic  algebras  recursion

Year		Venue	Description	Classes
2015		LPAR	dℓT programs ^a – §1.2.5.6 Baillot, Barthe, and Dal Lago (2015)	P
2016		Inf. Comput.	Complexity with pointer machines Aubert and Seiller (2016)	L
2016		Inf. Comput.	Complexity of uniform Boolean circuits Bonfante, Kahle, Marion, and Oitavem (2016)	NC
2018		Inf. Comput.	object oriented SAFE programs – §1.2.5.6 Hainry and Péchoux (2018)	P
2021		MFCS	Probabilistic characterization ST _{pp} Dal Lago, Kahle, and Oitavem (2021)	PP
2023		POPL	stratified programs (STR) – §1.2.5.6 Hainry and Péchoux (2023)	P
2024		POPL	quantitative type theory with dependent types Atkey (2024)	P, NP, BPP
2024		LICS	aperiodic programs – §1.2.5.6 Hainry, Kapron, Marion, and Péchoux (2024)	P

^a The system can handle sub-computations not in P.

Table 3: A list of theoretical implicit computational complexity systems and results. The historically first publications are marked with the symbol *. The graphical icons describe the primary restriction technique used to enforce complexity bounds.

1.2.2 Static Resource Analysis

Asymptotic analysis, that classifies programs based on resource requirements, is the natural dual to the study of complexity classes. Since implicit computational complexity focuses on language-based characterizations, it produces analysis techniques for free. It is a foundational hypothesis of this dissertation that theoretical ICC systems—with sufficient expressive power and an efficient evaluation strategy—enable concrete program analysis. An analysis that evaluates a program’s resource usage is called resource analysis. Resource analysis is a subfield of static program analysis.

1.2.2.1 Foundations of Static Program Analysis

Static program analysis aims to answer questions about the runtime behaviors of programs. The term *static* means the analysis is conducted at compile-time on the program syntax and without execution. The main motivation is to improve the performance characteristics of the generated executable (Kennedy & Allen, 2001; Nielson, Nielson, & Hankin, 2010). Such improvements include detecting logical issues in the specification of the input program. Foundational techniques include, e.g., type and effect systems; control and data flow analysis; abstract interpretation (AI) (Cousot, 2021); constraint-based, pointer, and inter-procedural analysis; and fixed point algorithms (Møller & Schwartzbach, 2024; Nielson, Nielson, & Hankin, 2010). Static program analysis is strongly practice-oriented and frequently materialized in automatic analyzers. Real-world software development tools—like compilers, interactive development environments, and verifiers—rely on static analyses (Livshits et al., 2015). A central challenge of static analysis concerns the precision and scalability of the techniques. These topics are actively explored in research (Arzt et al., 2014; Dudina & Stark, 2025; Schiebel, Sattler, Schubert, Apel, & Bodden, 2024).

As a consequence of Rice’s Theorem (Rice, 1953),¹⁰ it is impossible to construct a perfect program analyzer for a Turing-complete language. All approaches must sacrifice something among soundness, completeness and termination (Møller, 2024). Although seemingly discouraging, Rice’s Theorem should be viewed with optimism. Since static program analysis is inherently undecidable, there is infinite opportunity in building increasingly precise approximations.¹¹ The challenge is then to design analysis techniques that produce *useful* answers *efficiently* and *often enough*. This challenge is colloquially called the “full employment theorem for static program analysis designers” (Møller & Schwartzbach, 2024, p. 4).

Static analyzers are designed to evaluate programs against semantic properties.¹² More specifically, the focus is on *safety properties* (Møller & Schwartzbach, 2024, p. 6), which assert that *something bad will not happen* (Lamport, 1977). Examples of safety properties are memory safety, absence of overflows, termination, and secure information flow (cf. §1.2.5.2). Safety properties have many applications, for example, in bug finding (Popeea & Chin, 2010), program optimization (Rinard & Diniz, 1996), and verification (Falcone, Fernandez, & Mounier, 2009).

1.2.2.2 The Challenge of Approximation

There are two broad categories of static analyzers (Jourdan, Laporte, Blazy, Leroy, & Pichardie, 2015).

- A *bug finder* must discover potential errors that are hard to find by testing. The analysis must be precise because excessive false alarms make the tool unusable. The

¹⁰Informally, Rice’s Theorem asserts that all interesting questions about the input/output behavior of programs written in a Turing-complete language are undecidable.

¹¹For inspiration see, e.g., Ding and Zhang (2023).

¹²Properties are discussed in §1.2.6.1, but previewed here briefly.

precision of an analysis is the degree to which it avoids erroneous results (Livshits et al., 2015). However, a bug finder offers no guarantee that all bugs will be found.

- A program *verifier* must establish that a given safety property holds with high confidence. Critically, the analyzer must account for all execution paths. If the analyzer reports no alarms, then it must be the case that the program is free of the errors tracked by the analyzer.

Obtaining the intended behavior depends on design choices, particularly on how the analysis approaches soundness and completeness.

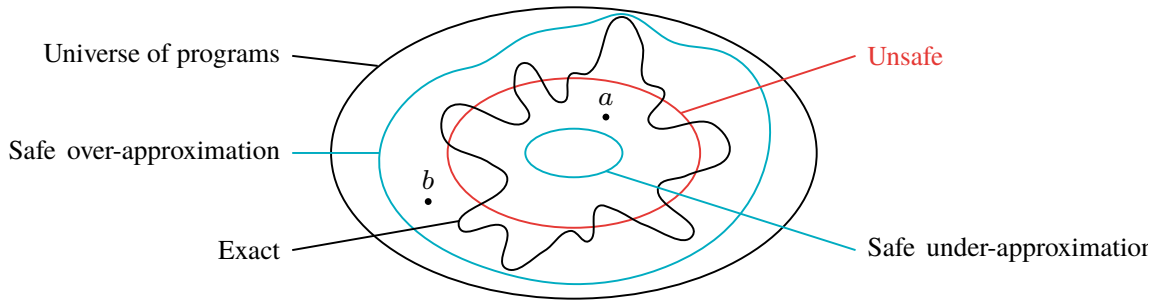


Figure 3: Program analysis as an approximation – an illustration inspired by Steffen (2020). The EXACT class of programs satisfying the analyzed property ϕ is undecidable. The program a satisfies ϕ , but the program b does not. For an analyzer making a SAFE UNDER-APPROXIMATION, every accepted program truly satisfies ϕ , but would give an erroneous result on a . An analyzer making a SAFE OVER-APPROXIMATION would accept a and b , even though b does not satisfy ϕ . However, an over-approximating analyzer never misses programs that satisfy ϕ . Although an UNSAFE analysis can be justifiable in some cases, interpreting its results is problematic because errors are not conservative.

Soundness and completeness. A terminating static analyzer can either be sound or complete, but not both. Soundness and completeness are terms from logic and not exclusive to static program analysis. In logic, they carry the following meaning.

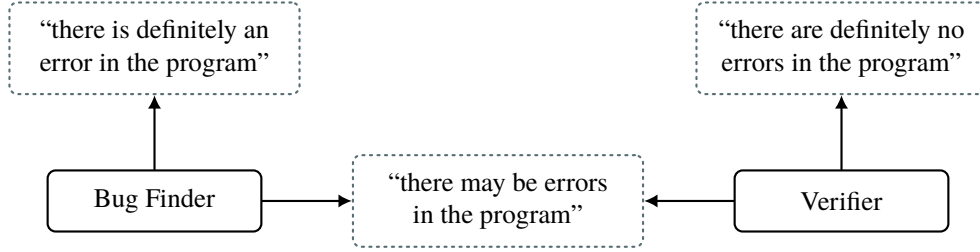


Figure 4: Behavior of sound and incomplete static analyzers: bug detector vs. verifier (Møller, 2024). An “error” is some property tracked by the analyzer.

- A *sound* system guarantees that everything that is provable is true.
- A *complete* system guarantees that everything that is true has a proof.

In the context of program analysis, if a provably sound system reports that some property holds, then it must be true for the analyzed program. A sound analyzer never misses a violation—i.e., when a property does not hold—but it may report an erroneous result for a satisfactory program (Torlak, 2015). A complete analyzer detects all satisfactory programs, but also accepts cases where the tracked property does not truly hold. The analysis approximation should be *conservative* whereby all errors occur on the “safe side” of the application domain (Møller & Schwartzbach, 2024, p. 5). Refer to Figures 3 and 4 for examples.

To offer guarantees, a sound analyzer must overestimate the program behaviors. It models all execution paths, including those that do not occur in any execution, like all branches of a conditional statement. Soundness is challenging because it can render the analysis unscalable or imprecise to the point of being useless. To maintain semantics, an optimizing analysis typically requires soundness (Møller & Schwartzbach, 2024, p. 5). A complete analyzer underestimates the program behaviors and thus errs on the other side.

Soundness. While academic literature widely emphasizes sound analyses, according to Livshits et al. (2015), strict soundness is a myth among realistic whole-program analyzers. Practically-orientated static analyzers typically insist on Turing-completeness

and termination, but trade between soundness and completeness (Møller & Schwartzbach, 2024; Steffen, 2020). As a compromise, modern analyzers for real programming languages are often soundy. A *soundy* analysis is as sound as possible, without excessively compromising precision or scalability (Livshits et al., 2015).

1.2.2.3 Techniques of Resource Analysis

Resource analysis¹³—or (static/automatic) complexity analysis (Leivant & Marion, 2013; Rosendahl, 1989)—aims to obtain static information about the runtime execution cost of an analyzed program. Such analysis naturally decomposes into two parts: one concerning the program’s execution time and termination; and the other change in the program’s data size (Jones & Kristiansen, 2009). Numerous techniques can be used to reason about such properties.

- Termination analyses tend to focus on values that are copied or decreased. Establishing termination is a critical aspect of correctness of recursive calls and commands. Termination analyses aim to discover conditions—like difference constraints (Sinn, Zuleger, & Veith, 2017), program-dependent well-founded orders on loops (Lee, Jones, & Ben-Amram, 2001), or establishing that recursive reduction sequence are finite¹⁴—that guarantee termination. A specific technique for termination analysis, that originates from implicit computational complexity, is the size-change termination principle of Lee, Jones, and Ben-Amram (2001).
- Data-size analyses focus on memory consumption, estimating how large various variable values may become (Lommen & Giesl, 2023). Determining data-size bounds requires studying patterns of data-size changes during computation. In imperative

¹³Also called *cost analysis* when using an explicit *cost measure* (Albert, Arenas, Genaim, & Puebla, 2008).

¹⁴Showing that every reduction sequence is finite is called the *strong normalization* property (Bertot & Castéran, 2004, p. 36)

programs, interesting data-size behaviors arise from counter increments and resets, particularly in loops (Ben-Amram & Hamilton, 2020; Sinn, Zuleger, & Veith, 2017).

This dissertation is mainly concerned with techniques of data-size analysis. Among the well-established techniques are automatic amortized resource analysis (AARA) and cost equation systems (CES). These techniques are briefly introduced next.

Automatic amortized resource analysis. *Amortized analysis* averages the time required to perform a sequence of operations over all the operations performed (Cormen, Leiserson, Rivest, & Stein, 2009, p. 451). Amortization enables to show that the average cost of an operation is small, even if a single operation might be expensive.

Automatic amortized resource analysis (AARA) (Hoffmann & Jost, 2022) builds on the same idea: bounding programs by amortization. In AARA, the analysis can account for amortization effects across sequences of operations. Internally, the analysis relies on inference rules and derivation trees provide proofs of the resource bounds. Thus, AARA provides an automatic static technique for deriving arithmetic resource bounds.

AARA originated from works in implicit computation complexity, and in particular from the contributions of Martin Hofmann (Hoffmann & Jost, 2022). The initial design was for tracking resource bounds in functional programs (Hoffmann, Aehlig, & Hofmann, 2012). In later works, techniques based on or influenced by AARA have been extended to imperative (Carbonneaux, Hoffmann, Reps, & Shao, 2017; Carbonneaux, Hoffmann, & Shao, 2015), parallel (Hoffmann & Shao, 2015), probabilistic (Avanzini, Moser, & Schaper, 2020; Ngo, Carbonneaux, & Hoffmann, 2018; Wang, Kahn, & Hoffmann, 2020), and concurrent session-typed programs (Das, Balzer, Hoffmann, Pfenning, & Santurkar, 2021; Das, Hoffmann, & Pfenning, 2018).

Cost equation systems. A *cost equation system* (CES)¹⁵ is an abstract representation format to express a program's resource cost (Albert, Bofill, Borralleras, Martin-Martin, & Rubio, 2019; Montoya, 2017a). A CES consists of a set of cost equations. A *cost equation* (CE) is a nondeterministic recurrence relation annotated with constraints, as shown in Listing 2 (Flores-Montoya & Hähnle, 2014).

The analysis of cost equations is based on the seminal work of Wegbreit (1975). Such analysis operates in two phases.

1. The first phase extracts a set of equations that capture the program cost w.r.t. data-size and a parametric cost measure. This set can be regarded as recurrence relations, or as time-bound programs à la Rosendahl (1989).
2. The second phase searches for a non-recursive representation of the equations known as *closed form*. In most cases, it is not possible to find an exact solution, and the closed form provides an upper bound (Albert, Arenas, Genaim, & Puebla, 2008).

Cost equations deal with loops and recursion in a uniform manner. However, they may have issues with multiphase loops and with programs that require complex termination proofs (Flores-Montoya & Hähnle, 2014).

```

1 % Entries
2 entry(a(N): [N>=0]).
3 % Equations
4 eq(a(N), 0, [], [N=0]).
5 eq(a(N), 1, [a(N-1), a(N-1)], [N>=1]).

```

CES

Listing 2: A cost equation system representing an exponential function. The term $a(n)$ is called the head and the rightward terms are the body. The body terms represent costs and constraints.

¹⁵Sometimes alternatively: cost *relation* system.

1.2.2.4 A Sampler of Public Static Resource Analyzers

The following list presents a small survey of automatic complexity analysis implementations (interpreted broadly). It prioritizes tools that are open source or, at minimum, provide a public interface. A comprehensive comparison of resource bounds computed by various tools is available online (Montoya, 2017b).

PUBS *Practical Upper Bounds Solver* (Albert, Arenas, Genaim, & Puebla, 2010) is a CES solver that computes closed form upper bounds for cost equation systems. Specifically, it computes bounds for the entry and all cost equations that depend on entry.

AProVE *Automated Program Verification Environment* (Giesl et al., 2016) produces termination proofs and complexity bounds for term rewrite systems (TRS), Java bytecode, and programs written in C, Prolog, or Haskell. AProVE is a regular contestant in international tool competitions, including the Termination Competition (Giesl, Rubio, Sternagel, Waldmann, & Yamada, 2019) and Competition on Software Verification (SV-Comp) (Beyer, 2022).

CoFloCo *Control Flow refinement of Cost Relations* (Flores-Montoya & Hähnle, 2014) is an open-source analyzer, and a cost equation system solver, that infers symbolic upper and lower complexity bounds for CESs (Flores-Montoya, 2016b). CoFloCo is not bound to any particular programming language; rather, it expects a input program translated to a CES (Flores-Montoya, 2016a).

C4B (Carbonneaux, Hoffmann, & Shao, 2015) is an AARA-based analyzer of certified compositional resource bounds. It infers linear resource bounds on C-like programs by amortization – Table 10.

TcT *Tyrolean Complexity Tool* (Avanzini, Moser, & Schaper, 2016) is a modular analyzer with front-ends to support TRSs and integer transition systems (ITS); high-order functional programs, and Java bytecode. The complexity problem and metric of interest are parametric. TcT has also competed in the Termination Competition.

KoAT *Komplexitäts-Analyse-Tool* (Brockschmidt, Emmes, Falke, Fuhs, & Giesl, 2016) is a complexity analyzer for computing time-, cost- and size bounds. As inputs, KoAT accepts Integer Programs (IPs) and C integer programs that follow the grammar specification of the Termination Competition – §3.1.7.

LQICM *Loop Quasi-Invariant Chunk Motion* (Moyen, Rubiano, & Seiller, 2017a) is a compile-time program optimization pass for C programs. It aims to peel (cf. §1.2.4.2) inner loops to reduce program’s time complexity.

RaML *Resource aware ML* (Hoffmann, Das, & Weng, 2017) is an analyzer of worst-case upper bounds and best-case lower bounds of cost, for functional programs written in OCaml. The cost resource can be any quantity that is of interest. The RaML analysis is based on AARA and can produce precise amortized bounds.

Pastis (Carbonneaux, 2018; Carbonneaux, Hoffmann, Reps, & Shao, 2017) is another AARA-based analyzer. It infers various resource bounds and size change information of programs expressed in LLVM bitcode (interprocedural CFGs). Additionally, it generates proof certificates that guarantee the validity of the inferred bounds.

HoSA *Higher-Order Size Analysis* (Avanzini & Dal Lago, 2017) is a time complexity analyzer for higher-order functional programs written in a (generic) strongly typed functional programming language. The analysis technique is based on a type system and inference of sized types, and constraint solving.

Cecoa (Férée, Hym, Mayero, Moyen, & Nowak, 2018) is a formal methods library based on implicit computational complexity. It enables creating proofs to show a program is poly-time computable – §1.2.6.7.

MaxCore *Max-SMT termination analyzer + C_{ost} Recurrence Equation solver* (Albert, Bofill, Borralleras, Martin-Martin, & Rubio, 2019) is an upper bounds complexity analyzer for C programs. The underlying technique is based on cost equations and termination analysis, and MaxCore uses CoFloCo and PUBS as back-end CES solvers.

eco-imp (Avanzini, Moser, & Schaper, 2020) computes expected cost bounds for programs written in a probabilistic imperative P_{WHILE} language.

ComplexityParser (Hainry, Jeandel, Péchoux, & Zeyen, 2021) is a complexity analyzer for Java programs. A typable program is guaranteed to run in polynomial time if the program terminates. The analyzer is based on (Hainry & Péchoux, 2015) and (Hainry & Péchoux, 2018), discussed more in §1.2.5.6.

pymwp (Aubert, Rubiano, Rusch, & Seiller, 2023b) is an implementation of the flow calculus of mwp-bounds. It is purely syntactical analysis of C programs, to determine if the program’s variable value growth is polynomially bounded in inputs – §2.1.

Quantitative probabilistic bounds (Chatterjee, Goharshady, Meggendorfer, & Žikelić, 2024) analyzer computes tail and expectation bounds for programs written in a probabilistic imperative programming language PROG.

1.2.2.5 Feature Considerations of ICC Techniques

Soundness and completeness. In characterizing a complexity class, a *soundness* statement says that all programs of a given language, or those satisfying a restriction $\mathcal{R}$, admit a certain complexity property, i.e., compute functions in the target class. A *completeness*

statement says that all functions of the corresponding functional complexity class can be programmed in this language (Baillot, Dal Lago, & Moyen, 2012). Alternatively, if $\mathcal{R}$ is (*extensionally*) *complete*, then for any problem in the complexity class, there exists a program in $\mathcal{R}$ that decides it. For instance, for polynomial time complexity, the soundness statement refers to a polynomial time evaluation of programs, whereas completeness refers to the class FP of functions computable in polynomial time (Baillot, Dal Lago, & Moyen, 2012). However, extensional completeness is of limited interest, as the goal is to validate as many interesting programs as possible.

ICC systems are typically proven sound w.r.t. the complexity class they characterize, whereby they are *sound and incomplete* under-approximations of the target class (Moyen, 2017, p. 125). A sound and incomplete analysis admits false negatives, but guarantees the absence of false positives. Complexity analysis can then give an erroneous result in the following way: a developer writes a program that belongs to the intended complexity class, but the program fails the analysis because it is not expressible in the ICC system. Fixing the issue requires rewriting the program to satisfy the class membership restriction $\mathcal{R}$. Due to undecidability, it is impossible to completely avoid false negatives. Stated differently, implicit complexity can decide non-trivial semantics properties of programs, but it will always exclude some programs that possess the property of interest.

Input languages and expressive power. Extensional correspondence with complexity classes is usually not enough. A programming language offering safety guarantees is plausible only if it captures enough interesting and natural algorithms (Baillot, Dal Lago, & Moyen, 2012). On the other hand, sacrificing Turing-completeness allows developing an analysis that is “completely correct” about membership in a particular complexity class. It requires defining a non-Turing-complete programming language and proving that it is sound and complete w.r.t. a particular complexity class. This sidesteps Rice’s Theorem, but can-

not be *intentionally* complete, since it will always exclude programs that yet belong to the targeted complexity class. Correctly analysing a real world programming language is very difficult because of the large number of constructions allowed in it.

We must understand all the difficulty because for a Turing-complete language for example the class of PTime programs is not recursively enumerable, and therefore an intentionally complete criterion would not be decidable (Mogbil, 2012, Sect. 4.1).

For this reason, most research¹⁶ in implicit complexity techniques focuses on “toy languages” (Moyen, 2017; Rubiano, 2017); like term rewrite systems or the Loop language (Kristiansen, 2005). While such languages capture the core of mainstream programming languages, they are far from the rich constructs allowed in real languages.

Analysis mechanics and obtained bounds. That ICC systems are designed to characterize a specific complexity classes has implications on the results they produce. If they generate bounds, they are asymptotic (symbolic) bounds, rather than concrete bounds (Baillot, Barthe, & Dal Lago, 2019). An existing research direction in ICC concerns improving precision of obtained resource bounds (Ben-Amram & Hamilton, 2020). However, if the system is designed as a sound under-approximation, it can only provide upper bounds (Moyen, 2017, p. 119). A satisfactory program can be provided run-time complexity guarantee, but nothing is known of the runtime behavior of unsatisfactory programs. ICC system is inherently restricted to analyzing programs whose sub-computations also satisfy the membership criterion of the target complexity class (Baillot, Barthe, & Dal Lago, 2019). On the other hand, unlike many other static program analysis techniques, ICC systems may not require establishing termination and avoid difficult control-flow analysis by abstraction and over-approximation (Jones & Kristiansen, 2009).

¹⁶Among exceptions, quasi-interpretations (Marion & Moyen, 2000) considers a Turing-complete language.

1.2.2.6 Prospects of ICC-Based Program Analysis

There are two general strategies for implementing static analyses based on implicit computational complexity. The first involves restricting the program syntax to guarantee all valid programs satisfy a desired property. This is challenging for a software engineer since it may require writing programs in an unnatural way (though not all programs are written by humans). However, if the program can be expressed in the given syntax, then the desired property is obtained *for free*.

The second strategy allows a software engineer to write any program and performs program analysis *a posteriori*. Instead of requiring every program to be written using the restriction $\mathcal{R}$, we would test the adhesion of the program to the restriction $\mathcal{R}$. While this is more preferable for the program creator, such analysis is necessarily incomplete. Some programs will be decided incorrectly and must be refactored to be acceptable to an analyzer. Nonetheless, this strategy already has produced applications in Moyen and Rubiano (2016) and Moyen, Rubiano, and Seiller (2017a).

Those research lines illustrates a shift from the design of ICC programming languages – from pre-bounding all programs – to the design of *criteria* that may or may not be met by programs written in a Turing-complete programming language. The first observation is crucial, considering that software engineers in general focus on much more tangible functional properties, than complexity classes. The second observation sends a mixed signal. It seems that ICC will lose the originality of its approach by accepting to act as a post-filter on Turing-complete programming languages. Let us not forget that by restricting the programming language itself, ICC offers the possibility of never having to run any analysis.

Compelling related examples of systems that provide properties by construction already exist. The safe subset of the programming language Rust guarantees e.g., memory safety and deadlock freedom by design. Strongly normalizing interactive theorem provers require

structural fixed point termination (Bertot & Castéran, 2004). In return, every valid programs is known to terminate. Although a software engineer cannot write every program, if they accept the restriction, they receive the termination guarantee for free. Conceivably, implicit computational complexity techniques can provide similar *X-by-Construction* (ter Beek, Cleophas, Schaefer, & Watson, 2018) guarantees.

1.2.3 The Flow Calculus of mwp-Bounds

The *flow calculus of mwp-bounds*, introduced by Jones and Kristiansen (2009), is a sound and compositional complexity analysis technique for reasoning about variable value growth in imperative programs. It tracks data-size growth in variables between commands. The information about variable value growth is captured and expressed as *mwp-bounds*.

More precisely, for program C the analysis aims to discover *polynomially bounded data flow relations*, between the *initial values* $x_1, \dots, x_n$ of natural-number variables $X_1, \dots, X_n$ and the *final values* x'_i of X_i (for $i = 1, \dots, n$), that hold whenever $\llbracket C \rrbracket(x_i \rightsquigarrow x'_i)$. If it is possible to determine that all variable values grow at most polynomially in inputs, the analysis assigns to program C a *bound*, i.e., a conjunction of mwp-bounds; that characterizes the value growth of all its variables. The conclusion is derived statically and syntactically by applying inference rules to the commands of the program. As a motivating preview, Examples 3 and 4 demonstrate the semantic property the analysis detects.

Example 3 (Polynomially bounded program). *The variable values of the program*

$$X1=X2+X3; \quad X1=X1+X1$$

are bounded by $x'_1 \leq 2x_2 + 2x_3$ and $x'_2 \leq x_2$ and $x'_3 \leq x_3$. Each variable can be bounded by at most a polynomial and the flow calculus accepts the program. The polynomially bounded data flow relation $\llbracket C \rrbracket(x_1, x_2, x_3 \rightsquigarrow x'_1, x'_2, x'_3)$ holds for all variables in all program states. $\square$

Example 4 (An always failing program). Consider a program that repeats, for X_2 times, an arithmetic operation on the variable X_1 .

$$X_1=1; \text{ loop } X_2 \{X_1=X_1+X_1\}$$

The value of X_2 never changes from its initial value; therefore, the final value of X_2 is trivially bounded by a polynomial $x'_2 \leq x_2$. However, variable X_1 grows at an exponential rate, with final value bounded by $x'_1 \leq 2^{x_2}$. Since variable X_1 cannot be bounded by a polynomial, the program is rejected by the flow calculus of mwp-bounds. $\square$

1.2.3.1 Tracking Value Growth With Coefficients

As a bookkeeping procedure, the flow calculus of mwp-bounds tracks *coefficients* (or “flows”) representing how data flows in variables between commands. The name of the calculus comes from these coefficients. In the mwp-calculus, harmless data flows are acceptable. However, computations that are harmful lead to values not bounded by a polynomial in the inputs and the coefficients capture these data flow facts.

The coefficients are, in order of lowest to the highest degree of dependency: 0 indicating no dependency, m for maximal of linear, w for weak polynomial, p for polynomial, and ∞ for failure, when no value growth bound can be established. The m -flow is harmless. The w - and p -flows are sometimes harmful and have restrictions on their use. The distinction between w and p is that w is iteration-independent and p is iteration-dependent (more

harmful). The ∞ -flow is always harmful. For example, if an exponential dependency exists between two variables, the analysis assigns an ∞ -coefficient to the target variable of the data flow.

Internally, the coefficients are collected and tracked in *mwp-matrices*. The mwp-matrices are assigned to program commands by the inference rules of the *mwp-calculus*. This dissertation will revisit the topic of mwp-matrices several times, presenting them at different levels of detail. For this introduction, it suffices to consider an mwp-matrix as an *array of coefficients* for simplicity.

A sound calculus of value growth. The analysis result is binary and indicates whether all final values can be bounded by polynomials in inputs. When affirmative, we call the analyzed program *derivable*. For a program C and an mwp-matrix M , the *syntactic relation* $\vdash C : M$ holds if a program is derivable. In other words, a derivable program can be assigned an mwp-matrix of at most p -coefficients at each variable. If a command C computes a function growing exponentially, the statement $C : M$ will be false for all matrices M . The flow calculus result is sound but incomplete. If a derivable program terminates, the soundness theorem (Jones & Kristiansen, 2009, p. 12) guarantees that the variable value growth is polynomially bounded.

Theorem 2 (Soundness of mwp-calculus). $\vdash C : M \text{ implies } \models C : M$. □

The soundness theorem connects the syntactic data flow relation to the *semantic property* $\models C : M$. Informally, derivability guarantees that the variable values of program C will grow at most polynomially at runtime. This in turn ensures program C runs in polynomial time (Kristiansen, 2017). The main result of the work introducing the flow calculus (Jones & Kristiansen, 2009) is precisely a proof of the soundness theorem. Since the analysis omits termination, the flow analysis provides a *partial correctness* guarantee.

The flow calculus offers no guarantee to programs that are not derivable. Then, variable value growth is interpreted as unknown. A program always fails if a variable value grows “too fast”, for example exponentially. A single variable can be the source of whole-program derivation failure. Alternatively, failure may occur from inability to express satisfiable behavior. The latter is a built-in limitation of the flow calculus. To increase expressiveness and capture a larger class of derivable programs, the mwp-calculus includes nondeterminism in its inference rules. Consequently, a single program may admit multiple derivations with distinct matrices, which in turn complicates determining the result of the analysis. However, a program is derivable if there exists a derivation without an ∞ -coefficient.

The mwp-bounds characterize value growth. The column vectors of an mwp-matrix encode variable value growth bounds. An mwp-bound (denoted $W, V, U, \dots$) characterizes the value growth of a single variable w.r.t. program inputs. An mwp-bound is an expression of form $\max(\vec{x}, \text{poly}_1(\vec{y})) + \text{poly}_2(\vec{z})$. Variables characterized by m -flow are listed in $\vec{x}$; w -flows in $\vec{y}$, and p -flows in $\vec{z}$. The notation $W(\vec{x}; \vec{y}; \vec{z})$ displays all the variables in an mwp-bound W where $\vec{x}$, $\vec{y}$ and $\vec{z}$ are respectively the m -, w - and p -variables of W . Variables characterized by 0-flow do not occur in the expression and no bound exists if some variable is characterized by ∞ . The poly_1 and poly_2 are *honest polynomials*, build up from constants and variables by applying $+$ and $\times$. Any of the three variable lists might be empty, and poly_1 and poly_2 may not be present. When characterizing value growth, an mwp-bound is approximative; it excludes precise constants and degrees of polynomials. We write an mwp-bound of variable X as $x' \leq W$ where x' is the final value of X and W is an mwp-bound. A *program bound* is a conjunction ($\wedge$) of its variables' mwp-bounds.

Example 5 (Interpreting mwp-bounds). *For an arbitrary variable, the mwp-bound indicates the final value of the variable is bounded by...*

0 ...a constant.
 X ...the initial value of variable X .
 $\max(X, Y)$...the initial values of X or Y , the greatest of the two.
 $\max(X, 0) + Y$...the initial values of X and Y .
 $\max(X \odot, X_1 + X_2) + X_3 \times X_4$...the maximum of $X \odot$ or $X_1 + X_2$, and $X_3 \times X_4$.

□

Multiple mwp-bounds of different form can evaluate to the same numerical value. For example, the mwp-bounds $W \equiv \max(0, X_1 + X_2) + 0$ and $V \equiv \max(X_1, 0) + X_2$ and $U \equiv \max(X_2, 0) + X_1$ are all numerically equal to $X_1 + X_2$. As each variable in the mwp-bound omits constants. Thus, an mwp-bound is best suited for recognizing the *shape* of the value growth rather than its explicit numerical value. Furthermore, mwp-bounds can be incomparable. For example, between $W \equiv \max(X_1, X_2) + X_3$ and $V \equiv X_1 + X_2 \times X_4$, it is impossible to determine which mwp-bound evaluates to a lower value numerically. This means mwp-bounds cannot be totally ordered.

1.2.3.2 Advancements of the Flow Calculus, Chronologically

The incremental technical advancement of the flow calculus of mwp-bounds is an important theme of this dissertation. The developments are a necessary prerequisite for enabling different *applications* of the flow calculus.

It may be challenging to extract the “big picture of developments” by reading the manuscripts alone. Therefore, this section aims discuss the topic in an accessible manner, with a summary in Table 4). The table corresponds to the following four publications.

The original flow calculus. The flow calculus of mwp-bounds (Jones & Kristiansen, 2009) was designed to characterize sequential imperative polytime programs, with an underlying ambition to study the boundary of decidability (Kristiansen, 2017). The complexity-theoretic result is obtained by bounding the growth of variable values by polynomials in inputs. In effect, the flow calculus provides a computational method $\mathcal{M}$ for deciding a run-time property,

$$\mathcal{M} : \text{programs} \rightarrow \{\text{yes}, \text{no}\}$$

such that $\{C_i \mid \mathcal{M}(C_i) = \text{yes}\} \subset \{C_i \mid C_i \text{ runs in polynomial time}\}.$

The formulation naturally provides the theoretical foundations and a concrete technique for program analysis. The analysis is defined over nondeterministic inference rules and coefficients $\{0, m, w, p\}$ where multiple matrices can be assigned to one program. A single program has 3^k derivations where k is the count of binary arithmetic operations (the binary operations are the sources of nondeterminism). The analysis proceeds in a “single-shot”, exploring one derivation path until the complete program syntax has been successfully analyzed, or the derivation fails. On derivation failure, the analysis restarts to explore a different deprivation path until all paths have been exhausted.

Among the advantage of the original flow calculus is that if a derivation succeeds, the mwp-bounds are obtained immediately. Although on success the method produces mwp-bounds straightforwardly, it does not support searching for mwp-bounds of specific form. The mathematical framework is sophisticated and recognizes a larger class of programs than earlier similar systems, e.g., (Niggl & Wunderlich, 2006). However, beyond pen and paper examples, the technique is infeasible for practical implementation, due to the exponential blowup and inefficient handling of derivation failure.

The enhanced mwp^∞ -calculus. In an effort to move the flow calculus toward practical static analysis, in (Aubert, Rubiano, Rusch, & Seiller, 2022a) (§7.1) the calculus was refined to resolve its theoretical inefficiencies. There are several well-founded motivations for this goal: a practical analysis permits stress-testing the technique beyond paper examples, exports ideas from implicit complexity to broader communities, and exposes hidden challenges within the analysis technique.

Most notably, the enhanced flow calculus “internalizes” the nondeterminism through deterministic inference rules. A key notion is the introduction of the ∞ -coefficient that tracks derivation failure. Instead of multiple matrices, every program is assigned exactly one (complex) mwp-matrix. The complex mwp-matrix captures all derivations at once, in a single data structure, resolving much of the combinatorial inefficiency of the original flow calculus. Program analysis now transforms, from performing multiple derivations, to a question of determining if there exists a derivation without an ∞ -coefficient. This splits the program analysis procedure into two phases of mwp-matrix construction and mwp-matrix evaluation. As a consequence, the mwp-bounds are no longer immediately obtained from the mwp-matrix.

The enhanced flow calculus is practically efficient as demonstrated by the prototype implementation, *pymwp*.¹⁷ The technique permits efficient construction of complex mwp-matrices, but only a brute force approach for mwp-matrix evaluation. At the time, the analyzer could answer a question of whether all variables could be bounded (yes/no). However, in absence of an efficient evaluation strategy, it could not provide concrete bounds (cf. Table 1 in §7.1).

The *pymwp* static analyzer. After continued development, the *pymwp* static analyzer (Aubert, Rubiano, Rusch, & Seiller, 2025) was enhanced in two ways (Aubert,

¹⁷Pronounced “par em double-you pē.”

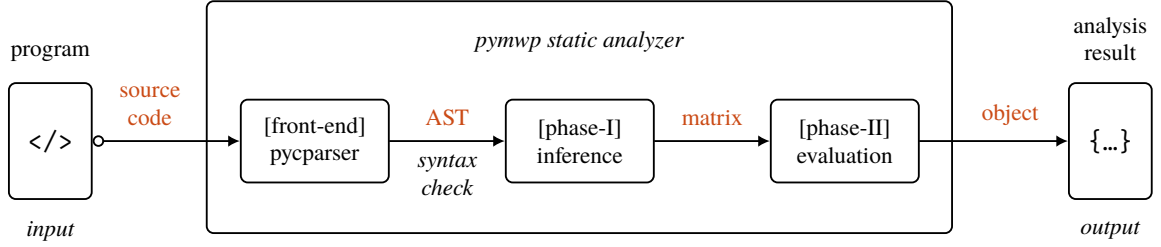


Figure 5: Workflow of the pymwp static analyzer. pymwp analyzes a C program and produces a result with information about the program’s variable value growth. If the program is derivable, the result contains mwp-bounds for all variables. Otherwise, the result identifies the variables with problematic data flows that cause derivation failure.

Rubiano, Rusch, & Seiller, 2023b). The first introduced an efficient mwp-matrix evaluation strategy. It enabled representing all permissible derivations in a compact form and producing an arbitrary instance of a program bound, if at least one bound exists. The second introduced feedback about source of derivation failure. This is possible because the ∞ -coefficient marks the locations of failure, yielding a more informative system than the original flow calculus. With these enhancements, the analysis produces results efficiently, while exceeding the informative capabilities of the original flow calculus.

The workflow of pywmp is visualized in Fig. 5. Functionally, pymwp is a command-line tool that enables running mwp-analysis through a terminal. The analyzer takes as input a program written in (a subset) of the C programming language. The analyzer front-end is pycparser (Bendersky, 2024). pycparser converts the program source code into an abstract syntax tree (AST). Before the inference, the AST is checked for consistency with the imperative language. In strict mode, if the syntax is not fully expressible, the analysis terminates.¹⁸ pymwp then runs the flow calculus of mwp-bounds on the (compatible) AST. By default, the result of the analysis displays at the screen and is saved to a file. Besides the CLI interface, pymwp can be incorporated into larger software development as an imported library.

¹⁸Importantly, the strict mode is sound because it only accepts programs that are fully-expressible in the imperative language of the flow calculus.

Postconditions via mwp-bounds. The most recent development applies the flow calculus to specification inference, namely postconditions §3.1. This is a new use-case, shifting the focus from static analysis toward formal verification. Technically it introduces three flow calculus enhancements: projecting the mwp analysis on individual variables, improving the derivation failure handling strategy, an evaluation strategy to obtain optimal mwp-bounds through a query procedure. For example, it permits inferring mwp-bounds of some variables in presence of whole-program derivation failure. Due to focus on verifying numerical loops, the programming language is restricted to a language of loops. The language adjustment is now a prerequisite to obtain the technical enhancements. The loop analysis variant is implemented in pymwp as a special mode called mwp_ℓ .

Open problems for future developments. Throughout the technical progression, the class of programs covered by the analysis has remained similar; with small adjustments. The semantic property of interest, variable value growth w.r.t. inputs, is never lost nor altered. Therefore, the flow calculus input/output behavior has remained the same. All notable changes concern the mechanics of the analysis and how to improve the process by which the analysis arrives to its conclusion. In the original mwp-calculus, the problem of determining program derivability is NP-complete. It is unknown if the efficiency has changed following the technical advancements.

Among the remaining open problems are: 1. extending the imperative language to cover a larger class of programs, 2. formally verifying the flow calculus for obtain a rigorous soundness guarantee (cf. §3.3), 3. exploring the extended analysis applications, and 4. enhancing the precision of the flow calculus. Matrix construction is currently the most costly analysis phase. It is possible to design data flows that make program analysis inefficient due to the resulting matrix size. More effort is needed to scale the analysis to such inputs.

Analysis phase	Description	The original flow calculus	The enhanced MWP^∞ calculus	The pymwp static analyzer	Postconditions via mwp-bounds
INPUT	Motivation	complexity theory	toward static analysis	static analysis	verification
	Language	$\mathcal{L}$ (baseline) ^a	$\mathcal{L} + f + \text{recursion}$	$\mathcal{L} \mapsto \text{C99}$	$\text{loops} \subset \mathcal{L}$
PROCEDURE	Calculus variant	MWP	MWP^∞	MWP^∞	MWP^∞
	Matrix type	simple flows	complex	complex	complex
	Matrix count	$0-3^k$	1	1	1
	Failure recovery	complete restart	∞	∞	∞
	Feedback on failure	none	none	failing variables	failing variables
OUTPUT	Generated bound	arbitrary	exists (y/n) ^b	arbitrary	optimal
	Bounds discovery	immediate	brute force	evaluation	evaluation
	Bounds granularity	whole-program	whole-program	whole-program	variables
	Implementation	none	pymwp 0.1.6	pymwp 0.4.2	mwp_ℓ

^a The language is similar to IMP in Winskel (1993, p. 11–26) and Pierce et al. (2025).

^b Analysis determines if a bound exists, but does not provide bound expressions reliably.

Table 4: Comparison of technical developments and features in works that extend the flow calculus of mwp-bounds.

1.2.3.3 Technical Details of Program Analysis

The flow calculus provides a static technique for analyzing programs written in an imperative language. Through its inference rules, the analysis assigns coefficients to program commands to determine if the program is derivable. This section gives an overview of the program analysis mechanics that suffices to understand the derivation examples that follow. We present the original flow calculus, because it generates mwp-matrices of simple coefficients which are easier to present and discuss on paper. In contrast, the mwp^∞ calculus generates complex mwp-matrices that requires two program analysis phases, for mwp-matrix construction and evaluation. For the best exposition of how to construct complex mwp-matrices see §7.1. The evaluation procedure that extracts mwp-bounds from complex mwp-matrices is best described in §3.1.

An imperative programming language. The inputs of the analysis are specified in a simple imperative language (referred to as `Imp` in code listings). Because the language and semantics are defined in §3.1 and §7.1, they are not repeated here. A *program* is a sequence of commands. A program manipulates sequentially a fixed number of natural number variables, with conditional branching and iteration permitted. Control expressions in `if` statements and `while` loops do not matter; all branches are analyzed and `while` loops are over-approximated to iterate infinitely. Conventionally, a program in the flow calculus corresponds to a procedure. Although simple, the imperative language captures a core fragment of mainstream programming languages, like C and Java; and makes all languages that share the same core analyzable.

The mwp-matrix algebra. We use the definitions from the original flow calculus (Jones & Kristiansen, 2009). The mwp-matrix algebra provides a mathematical framework for tracking variable value growth.

Elements. The mwp-matrix elements are coefficients (or *scalars*) in $\mathcal{V} = \{0, m, w, p\}$.

The elements in $\mathcal{V}$ are ordered $0 < m < w < p$ and letters $\alpha, \beta, \gamma, \dots$ denote elements in $\mathcal{V}$. The least upper bound of $\alpha, \beta \in \mathcal{V}$ is denoted by $\alpha + \beta$. The operation evaluates to $\alpha + \beta = \alpha$ if $\alpha \geq \beta$, otherwise $\alpha + \beta = \beta$. The product of $\alpha, \beta \in \mathcal{V}$ is denoted by $\alpha \times \beta$ and defined by $\alpha \times \beta = \alpha + \beta$ if $\alpha, \beta \in \{m, w, p\}$, otherwise $\alpha \times \beta = 0$.

$$\begin{matrix} & x_0 & \dots & x_n \\ x_0 & \ddots & & \ddots \\ \vdots & & \alpha & \\ x_n & \ddots & & \ddots \end{matrix}$$

Figure 6: Variable-labels of an mwp-matrix.

Matrices. For a program with n variables, an mwp-matrix

(denoted $M, A, B, C, \dots$) is a $n \times n$ matrix over $\mathcal{V}$. We use M_{ij} to denote the matrix element at row i and column j . A *zero matrix* is denoted by 0 and defined by $M = 0$ iff $M_{ij} = 0$ for all indices i, j . An *identity matrix* is denoted by 1 and defined by $M = 1$ iff $M_{ij} = m$ for $i = j$ and $M_{ij} = 0$ for $i \neq j$. The matrices are labelled by the program variables, as shown in Fig. 6, though the labels are typically omitted for compactness in derivations. If no labels are provided, we assume ascending alphabetical ordering of variables.

Matrix operations. The least upper bound $A \oplus B$ of the matrices A and B is defined component wise, i.e., $C = A \oplus B$ iff $C_{ij} = A_{ij} + B_{ij}$ for $i, j \in \{1, \dots, n\}$. The matrix product $A \otimes B$ of the matrices A and B is defined by $M = A \otimes B$ iff $M_{ij} = \sum_{k=1 \dots n} A_{ik} \times B_{kj}$, i.e., standard matrix multiplication. The closure (fixed point) of the matrix M is denoted by M^* and defined as an infinite sum

$$M^* = 1 \oplus M \otimes M^2 \oplus M^3 \oplus \dots$$

The inference rules of the mwp-calculus. The inference rules of the mwp-calculus cf. Fig. 7 specify how different program operations impact variable value growth during computation. Effectively, they provide the mechanics for connecting the mwp-matrix algebra with imperative programs.

Expressions. An *expression* e is assigned a vector V of coefficients $e : V$. The rules for analysing expressions, E1–E4, are defined in Fig. 7a. The notation $\{\alpha_i^\alpha\}$ means a vector with α in its i^{th} row and 0 everywhere else, and $\text{var}(e)$ means variables occurring in e . The product αV , where α is a coefficient and V is a vector, is defined by standard scalar multiplication of the vector elements. Observe that there are 3 ways to analyze a binary operation. This is the source of nondeterminism in the flow calculus. For k binary operation in the input program, there are 3^k different derivation paths to analyze.

Commands. A *command* C is assigned a $(n \times n)$ matrix M of coefficients $C : M$. The rules for analysing commands are defined in Fig. 7b. The notation $M \xleftarrow{j} V$ means replacing in matrix M the j^{th} column vector by V . The notation $\overset{p}{\leftarrow} j$ means adding coefficient p in column j at row $\mathfrak{l}$, where $\mathfrak{l}$ is the column index of the loop guard variable $X\mathfrak{l}$.

The assignment rule A has the effect that the *source* variables in the assignment expression impact the value growth of the assignment *target*. By the composition rule C, the impact of a command sequence is the product of the mwp-matrices of the commands. Since matrix multiplication is not commutative, the command order is relevant in mwp-calculus. The rule for *if* statements accounts for computations in both branches, irrespective of order. Only the iterative commands, *loop* and *while* have side conditions. This means if a derivation fails, the source of failure is an iteration command. Similarly, proving that variable value growth is polynomially bounded is trivial in absence of iteration. Between the two iteration commands, *while* is less permissive as it does not permit any p -flows in the mwp-matrix. The

$$\begin{array}{c}
\frac{}{\vdash x_i : \{^m\}} \text{ E1} \qquad \frac{}{\vdash e : \{^w \mid x_i \in \text{var}(e)\}} \text{ E2} \\
\qquad \qquad \qquad i \qquad \qquad \qquad i \\
\star \in \{+, -\} \frac{\vdash x_i : V_1 \quad \vdash x_j : V_2}{\vdash x_i \star x_j : pV_1 \oplus V_2} \text{ E3} \qquad \star \in \{+, -\} \frac{\vdash x_i : V_1 \quad \vdash x_j : V_2}{\vdash x_i \star x_j : V_1 \oplus pV_2} \text{ E4}
\end{array}$$

(a) Rules for assigning vectors to expressions

$$\begin{array}{c}
\frac{}{\vdash \text{skip} : \mathbf{1}} \text{ S} \qquad \frac{\vdash e : V_1}{\vdash x_j = e : \mathbf{1} \stackrel{j}{\leftarrow} V} \text{ A} \qquad \frac{\vdash C_1 : M_1 \quad \vdash C_2 : M_2}{\vdash C_1 ; C_2 : M_1 \otimes M_2} \text{ C} \\
\\
\frac{\vdash C_1 : M_1 \quad \vdash C_2 : M_2}{\vdash \text{if } b \text{ then } C_1 \text{ else } C_2 : M_1 \oplus M_2} \text{ I} \\
\\
\forall i, M_{ii}^* = m \frac{\vdash C : M}{\vdash \text{loop } x_l \{C\} : M^* \oplus \{^p_l \rightarrow j \mid \exists i, M_{ij}^* = p\}} \text{ L} \\
\\
\forall i, M_{ii}^* = m \text{ and } \forall i, j, M_{ij}^* \neq p \frac{\vdash C : M}{\vdash \text{while } b \text{ do } \{C\} : M^*} \text{ W}
\end{array}$$

(b) Rules for assigning matrices to commands

Figure 7: The nondeterministic inference rules of the original mwp-calculus of Jones and Kristiansen (2009).

skip command is simply an identity matrix and has no effect. However, it is necessary to formally handle e.g., an omitted else branch. The skip also permits handling commands that have no impact on variable value growth, but that occur in realistic programming languages, e.g., assertions.

The analysis procedure. Recall, a program is a sequence of commands. Program analysis using the flow calculus requires inspecting the sequence of commands, applying the inference rules, and composing the mwp-matrices of the commands. Each command is either derivable or raises a derivation failure.

In practice, the analysis is performed over the program’s abstract syntax tree. Since a tree is a hierarchical structure, the analysis proceeds in depth-first order, like evaluation of a sequential program. The derivation examples in §1.2.3.4 demonstrate the concept. For simplicity, in describing the analysis procedure we assume a flattened parse-tree of commands.

Let $P = (C_0, C_1, \dots, C_m)$ be a program in an imperative language where $m \geq 1$. A generalized program analysis proceeds by Algorithm 1, with the analysis of individual commands is described in Algorithm 2.¹⁹ Conceptually, the program analysis is straightforward: compose the mwp-matrices of commands in the order they appear in the AST. However, in practice care is needed to account for the exponential number of derivation paths and handling potential derivation failure. Failure handling varies between the different versions of the flow analysis (cf. Table 4); therefore, the specifics are omitted here. The mwp^∞ variant of the flow calculus internalizes the failure and always returns an mwp-matrix.

¹⁹Detailed implementations of these algorithms are available in the pymwp source code as `Analysis.cmds` and `Analysis.compute_relation`.

Input: sequence of commands P

Output: mwp-matrix M or failure

```
1  Take the first command  $C_0$  in  $P$ 
2  Analyze  $C_0$  by Algorithm 2 to obtain  $M$ 
3  (Maybe signal failure)
4  for Commands  $n = 1 \dots m$  do
5      Analyze  $C_n$  by Algorithm 2
6      (Maybe signal failure)
7      Compose  $M = M \circ M_n$  by rule C
8  return mwp-matrix  $M$   $\triangleright \vdash P : M$ 
```

Algorithm 1: Program analysis with flow calculus of mwp-bounds.

Input: command C

Output: mwp-matrix M or failure

```
1  Analyze expressions in  $C$  by rules E1-E4
2  Analyze  $C$  by rules A-W, possibly recursively
3  (Maybe signal failure)
4  return  $M$   $\triangleright \vdash C : M$ 
```

Algorithm 2: Command analysis with flow calculus of mwp-bounds.

1.2.3.4 Derivations I: Range of Analysis Outcomes

Example 6 (Polynomially bounded program is derivable). *Recall the program from Ex. 3,*

```
X1 = X2 + X3; X1 = X1 + X1
```

Imp

Listing 3: A derivable program.

The conclusion about derivability can be reached by $3^2 = 9$ distinct derivations, one of which is shown below.

$$\begin{array}{c}
 \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_3 : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_1 : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_1 : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} \text{EI} \\
 \hline
 \vdash X_2 + X_3 : \begin{pmatrix} 0 \\ m \\ p \end{pmatrix} \quad \vdash X_1 + X_1 : \begin{pmatrix} p \\ 0 \\ 0 \end{pmatrix} \\
 \hline
 \vdash X_1 = X_2 + X_3 : \begin{pmatrix} 0 & 0 & 0 \\ m & m & 0 \\ p & 0 & m \end{pmatrix} \quad \vdash X_1 = X_1 + X_1 : \begin{pmatrix} p & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{pmatrix} \\
 \hline
 \vdash X_1 = X_2 + X_3; X_1 = X_1 + X_1 : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ p & 0 & m \end{pmatrix}
 \end{array}$$

The mwp-bounds are obtained column-wise from the concluding mwp-matrix, cf. §1.2.3.1. For X_1 , the vector $\begin{pmatrix} 0 \\ p \\ p \end{pmatrix}$ corresponds to $x'_1 \leq \max(0) + X_2 \times X_3$ and simplifies to $x'_1 \leq X_2 \times X_3$. The mwp-bound is not as tight as possible (expression $X_2 + X_3$ would be an improvement), but discovering a better one requires exploring the derivation paths exhaustively. Variables X_2 and X_3 obtain the same mwp-bounds in every derivation. For X_2 , the vector $\begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}$ gives an mwp-bound $x'_2 \leq X_2$, and for X_3 , the vector $\begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}$ gives $x'_3 \leq X_3$. The program bound is a conjunction of the variables' mwp-bounds, i.e., $x'_1 \leq X_2 \times X_3 \wedge x'_2 \leq X_2 \wedge x'_3 \leq X_3$.

□

Example 7 (Failing program admits no derivation). *Consider the program fragment from Ex. 4,*

```
loop X2{X1 = X1 + X1}
```

Imp

Listing 4: A loop that always fails the derivation.

The command $X1 = X1 + X1$ admits three derivations.

$$\begin{array}{c}
 \frac{\frac{\overline{\vdash X1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} \text{ E1}}{\vdash X1 + X1 : \begin{pmatrix} p \\ 0 \end{pmatrix}} \text{ E3}}{\vdash X1 = X1 + X1 : \begin{pmatrix} p & 0 \\ 0 & m \end{pmatrix}} \text{ A}
 \end{array}
 \quad
 \begin{array}{c}
 \frac{\frac{\overline{\vdash X1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} \text{ E1}}{\vdash X1 + X1 : \begin{pmatrix} p \\ 0 \end{pmatrix}} \text{ E4}}{\vdash X1 = X1 + X1 : \begin{pmatrix} p & 0 \\ 0 & m \end{pmatrix}} \text{ A}
 \end{array}
 \quad
 \begin{array}{c}
 \frac{\overline{\vdash X1 + X1 : \begin{pmatrix} w \\ 0 \end{pmatrix}} \text{ E2}}{\vdash X1 = X1 + X1 : \begin{pmatrix} w & 0 \\ 0 & m \end{pmatrix}} \text{ A}
 \end{array}$$

The next step in the derivation requires accounting for the enclosing loop by applying the rule L . The application involves computing the closure (fixed point) M^* of the mwp-matrix assigned to the body command $X1 = X1 + X1$. In this instance, the fixed point computation does not change the mwp-matrix coefficients. The L side condition is $\forall i, M_{ii}^* = m$. That is, it requires m -coefficients on the diagonal of M^* . None of the derivations satisfy the side condition, therefore the program is not derivable. Although the analysis focused only on a program fragment, the complete program fails because it contains a command that is not derivable. $\square$

Example 8 (Derivability in presence of partial derivation failure). Consider the following program, that is a slightly modified version of Ex. 7,

```
loop X3{X2 = X1 + X2}
```

Imp

Listing 5: A loop that fails for some derivation paths.

The body of the `loop` command admits three different derivations, obtained by applying the rule A to one of the three derivations of the expression $X1 + X2$, that we name π_0 , π_1 and π_2 :

$$\begin{array}{c}
 \frac{\overline{\vdash X1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} \text{ E1} \quad \frac{\overline{\vdash X2 : \begin{pmatrix} 0 \\ m \end{pmatrix}} \text{ E1}}{\vdash X1 + X2 : \begin{pmatrix} p \\ m \end{pmatrix}} \text{ E3}
 \end{array}
 \quad
 \begin{array}{c}
 \frac{\overline{\vdash X1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} \text{ E1} \quad \frac{\overline{\vdash X2 : \begin{pmatrix} 0 \\ m \end{pmatrix}} \text{ E1}}{\vdash X1 + X2 : \begin{pmatrix} m \\ p \end{pmatrix}} \text{ E4}
 \end{array}
 \quad
 \begin{array}{c}
 \overline{\vdash X1 + X2 : \begin{pmatrix} w \\ 0 \end{pmatrix}} \text{ E2}
 \end{array}$$

From π_0 , the derivation of `loop X3 {X2=X1+X2}` can be completed using A and L , but since L requires having only m coefficients on the diagonal, π_1 cannot be used to complete the derivation, because of the p coefficient in a box below:

Similarly, using A after π_2 gives a w coefficient on the diagonal and makes it impossible to use L , hence only one derivation for this program exists. A single successful derivation is sufficient to make the input program derivable. $\square$

The `pymwp` tool user guide, in §2.1.7, discusses five cases of program analysis of programs written in C. The tool user guide gives the analyzer input and output and describes the rationale for the obtained result. This section complements the tool user guide by showing the derivations behind those results. To retail fidelity with the C language examples, we show inputs in C, but implicitly convert them to the core imperative language of the flow calculus. The examples omit function declarations since those are irrelevant for the analysis.

```
1 int foo(int y1, int y2) {
2     y2 = y1 + y1;
3 }
```

The program has three derivations and all of them succeed. One derivation instance is shown below.

$$\begin{array}{c}
\frac{}{\vdash Y1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash Y1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash Y1+Y1 : \begin{pmatrix} p \\ 0 \end{pmatrix} E3 \\
\hline
\vdash Y2=Y1+Y1 : \begin{pmatrix} 0 & p \\ m & 0 \end{pmatrix} A
\end{array}$$

The program bound is a $y'_1 \leq Y1 \wedge y'_2 \leq Y1$. □

Example 10 (Exponential Program). *The input program is in Listing 7.*

```

1 int foo(int base, int exp, int i, int result) {
2     while (i < exp) {
3         result = result * base;
4         i = i + 1;
5     }
6 }

```

Listing 7: A program with exponential variable value growth.

Step 1. *Analyze the command `result = result * base` (simplified to `r=r*b` in the following). It admits three derivations (π_0 , π_1 and π_2).*

$$\begin{array}{c}
\frac{}{\vdash r : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash b : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash r*b : \begin{pmatrix} p \\ 0 \\ m \end{pmatrix} E3 \\
\hline
\vdash r=r*b : \begin{pmatrix} m & 0 & 0 & p \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} A
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash r : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash b : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash r*b : \begin{pmatrix} m \\ 0 \\ p \end{pmatrix} E4 \\
\hline
\vdash r=r*b : \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & p \end{pmatrix} A
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash r*b : \begin{pmatrix} w \\ 0 \\ w \end{pmatrix}} E2 \\
\hline
\vdash r=r*b : \begin{pmatrix} m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & w \end{pmatrix} A
\end{array}$$

Step 2. *Analyze command `i=i+1`. Similarly, it admits three derivations (π_3 , π_4 and π_5).*

$$\begin{array}{c}
\frac{}{\vdash i : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash 1 : \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash i+1 : \begin{pmatrix} 0 \\ 0 \\ p \end{pmatrix} E3 \\
\hline
\vdash i=i+1 : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & m \end{pmatrix} A
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash i : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash 1 : \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash i+1 : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix} E4 \\
\hline
\vdash i=i+1 : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} A
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash i+1 : \begin{pmatrix} 0 \\ 0 \\ w \end{pmatrix}} E2 \\
\hline
\vdash i=i+1 : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & w & 0 \\ 0 & 0 & 0 & m \end{pmatrix} A
\end{array}$$

Step 3. *Compose the body commands, $r=r*b$; $i=i+1$. There are 9 different combinations, by the derivation choices made in the previous steps.*

$$\frac{\vdots \pi_0 \times \pi_3}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & p \\ 0 & m & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & m \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_0 \times \pi_4}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & p \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_0 \times \pi_5}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & p \\ 0 & m & 0 & 0 \\ 0 & 0 & w & 0 \\ 0 & 0 & 0 & m \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_1 \times \pi_3}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & p \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_1 \times \pi_4}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & p \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_1 \times \pi_5}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & w & 0 \\ 0 & 0 & 0 & p \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_2 \times \pi_3}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & p & 0 \\ 0 & 0 & 0 & w \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_2 \times \pi_4}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & w \end{pmatrix}} \mathcal{C}$$

$$\frac{\vdots \pi_2 \times \pi_5}{\vdash \mathbf{B} : \begin{pmatrix} m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & w & 0 \\ 0 & 0 & 0 & w \end{pmatrix}} \mathcal{C}$$

Step 4. Apply rule *W* to account for the enclosing `while` loop. It requires first computing *mwp-matrix closure* M^* for the body command. The side conditions is $\forall i, M_{ii}^* = m$ and $\forall i, j, M_{ij}^* \neq p$. That is, the rule can be applied when only *m*-coefficients appear on the diagonal and no *p*-coefficients occur anywhere in the *mwp-matrix*. In every derivation, the *mwp-matrix* violates the side condition by the boxed coefficients.

$$\begin{array}{ccc}
\begin{array}{c} \vdots \\ \pi_0 \times \pi_3 \end{array} & \begin{array}{c} \vdots \\ \pi_0 \times \pi_4 \end{array} & \begin{array}{c} \vdots \\ \pi_0 \times \pi_5 \end{array} \\
M^* = \begin{pmatrix} m & 0 & 0 & \boxed{p} \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{p} & 0 \\ 0 & 0 & 0 & m \end{pmatrix} & M^* = \begin{pmatrix} m & 0 & 0 & \boxed{p} \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} & M^* = \begin{pmatrix} m & 0 & 0 & \boxed{p} \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{w} & 0 \\ 0 & 0 & 0 & m \end{pmatrix} \\
\\
\begin{array}{c} \vdots \\ \pi_1 \times \pi_3 \end{array} & \begin{array}{c} \vdots \\ \pi_1 \times \pi_4 \end{array} & \begin{array}{c} \vdots \\ \pi_1 \times \pi_5 \end{array} \\
M^* = \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{p} & 0 \\ 0 & 0 & 0 & \boxed{p} \end{pmatrix} & M^* = \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & \boxed{p} \end{pmatrix} & M^* = \begin{pmatrix} m & 0 & 0 & m \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{w} & 0 \\ 0 & 0 & 0 & \boxed{p} \end{pmatrix}
\end{array}$$

$$M^* = \begin{pmatrix} \vdots & \pi_2 \times \pi_3 \\ m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{P} & 0 \\ 0 & 0 & 0 & \boxed{w} \end{pmatrix} \quad M^* = \begin{pmatrix} \vdots & \pi_2 \times \pi_4 \\ m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & 0 & \boxed{w} \end{pmatrix} \quad M^* = \begin{pmatrix} \vdots & \pi_2 \times \pi_5 \\ m & 0 & 0 & w \\ 0 & m & 0 & 0 \\ 0 & 0 & \boxed{w} & 0 \\ 0 & 0 & 0 & \boxed{w} \end{pmatrix}$$

Step 5. We conclude the input program is not derivable. □

Example 11 (While analysis). The input program, that we will refer to as P , is

```

1 int foo(int X0, int X1, int X2, int X3) {
2     if (X1 == 1) {
3         X1 = X2 + X1;
4         X2 = X3 + X2;
5     }
6     while (X0 < 10) {
7         X0 = X1 + X2;
8     }
9 }
```

Listing 8: A program with a while loop.

The tool user guide reveals that the program is derivable. Compared to derivation failure that requires exploring all derivation choices, a derivable program simply requires showing one successful derivation. For presentation reasons, we show the two branches of the derivation tree individually.

Step 1. The derivation of the `if` command (π_0) is straightforward. Observe that the control expression of the `if` statement has no impact on the constructed mwp-matrix.

$$\begin{array}{c}
\frac{}{\vdash X_1 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ 0 \\ m \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash X_3 : \begin{pmatrix} 0 \\ 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ 0 \\ m \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash X_2 + X_1 : \begin{pmatrix} 0 \\ m \\ p \\ 0 \end{pmatrix} \quad \vdash X_3 + X_2 : \begin{pmatrix} 0 \\ 0 \\ m \\ p \end{pmatrix} \\
\hline
\vdash X_1 = X_2 + X_1 : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & p & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} \quad \vdash X_2 = X_3 + X_2 : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & m & 0 \\ 0 & 0 & p & m \end{pmatrix} \\
\hline
\vdash \text{if}(X_1 == 1) \{ X_1 = X_2 + X_1; X_2 = X_3 + X_2 \} : \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & p & m & 0 \\ 0 & 0 & p & m \end{pmatrix} I
\end{array}$$

Step 2. The derivation of the while loop (π_1) is now permitted, because the rule *W side* condition is not violated by any coefficient.

$$\begin{array}{c}
\frac{}{\vdash X_1 : \begin{pmatrix} 0 \\ w \\ 0 \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ 0 \\ w \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash X_1 + X_2 : \begin{pmatrix} 0 \\ w \\ w \\ 0 \end{pmatrix} \\
\hline
\vdash X_0 = X_1 + X_2 : \begin{pmatrix} 0 & 0 & 0 & 0 \\ w & m & 0 & 0 \\ w & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} A \\
\hline
\vdash \text{while}(X_0 < 10) X_0 = X_1 + X_2 : \begin{pmatrix} m & 0 & 0 & 0 \\ w & m & 0 & 0 \\ w & 0 & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} W
\end{array}$$

Step 3. Composing the mwp-matrices of the two commands gives an mwp-matrix for the complete input program P . We conclude the syntactic data-flow relation $\vdash P : M$ holds.

$$\frac{\vdots \pi_0 \times \pi_1}{\vdash P : \begin{pmatrix} m & 0 & 0 & 0 \\ w & m & 0 & 0 \\ p & p & m & 0 \\ p & 0 & p & m \end{pmatrix}} C$$

If the program terminates for the input values of the variables (it does not terminate for all inputs), the variable value growth is polynomially bounded. The mwp-bounds of the variables are $x'_0 \leq \max(X_0, X_1) + X_2 \times X_3$ and $x'_1 \leq \max(X_1) + X_2$ and $x'_2 \leq \max(X_2) + X_3$ and $x'_3 \leq \max(X_3)$. This program bound matches the result of the tool user guide. $\square$

Example 12 (Infinite program). *The input program is*

```

1 int foo(int X1, int X2, int X3) {
2     if (X1 == 1) {
3         X1 = X2 + X1;
4         X2 = X3 + X2;
5     }
6     while (X1 < 10) {
7         X1 = X2 + X1;
8     }
9 }

```

Listing 9: An “infinite” program.

This program is similar to Ex. 11, with only a small change to the while loop. From the previous example, the if statement is derivable as before by π_0 . Letting C_2 represent the while command, the mwp-matrix closures M^ of C_2 are shown below.*

$$\begin{array}{ccc}
 \begin{array}{c} \vdots E3 \\ M^* = \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & \boxed{p} & 0 & 0 \\ 0 & m & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} \end{array} &
 \begin{array}{c} \vdots E4 \\ M^* = \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & \boxed{p} & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} \end{array} &
 \begin{array}{c} \vdots E2 \\ M^* = \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & \boxed{w} & 0 & 0 \\ 0 & w & m & 0 \\ 0 & 0 & 0 & m \end{pmatrix} \end{array}
 \end{array}$$

Every M^ violates the side condition of rule W. The problematic coefficient is boxed for emphasis. The program is not derivable. $\square$*

Example 13 (Challenge example). *The input program has four top-level commands, identified by the circled annotations.*


```

1  int foo(int X0, int X1, int X2) {
2      if (X0) {
3          X2 = X0 + X1;
4      } else {
5          X2 = X2 + X1;
6      }
7      X0 = X2 + X1;
8      X1 = X0 + X2;
9      while (X2) { X2 = X1 + X0; }
10 }

```

C

Listing 10: A challenge program whose derivability is unknown.

We will refer to these commands as C_0 – C_3 and their derivations as π_0 – π_3 , respectively. The program analysis proceeds in corresponding steps. To ease presentation, we will analyze command composition after each individual command.

Step 1. Analyze command C_0 i.e., the if-else statement.

$$\begin{array}{c}
 \frac{}{\vdash X_0 : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_1 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} \text{EI} \quad \frac{}{\vdash X_1 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} \text{EI} \\
 \hline
 \frac{}{\vdash X_0 + X_1 : \begin{pmatrix} m \\ p \\ 0 \end{pmatrix}} \text{E4} \quad \frac{}{\vdash X_2 + X_1 : \begin{pmatrix} 0 \\ p \\ m \end{pmatrix}} \text{E4} \\
 \hline
 \frac{}{\vdash X_2 = X_0 + X_1 : \begin{pmatrix} m & 0 & m \\ 0 & m & p \\ 0 & 0 & 0 \end{pmatrix}} \text{A} \quad \frac{}{\vdash X_2 = X_2 + X_1 : \begin{pmatrix} m & 0 & 0 \\ 0 & m & p \\ 0 & 0 & m \end{pmatrix}} \text{A} \\
 \hline
 \vdash \text{if}(X_0) \ X_2 = X_0 + X_1; \text{ else } X_2 = X_2 + X_1 : \begin{pmatrix} m & 0 & m \\ 0 & m & p \\ 0 & 0 & m \end{pmatrix} \text{I}
 \end{array}$$

Steps 2–3. Analyze commands C_1 and C_2 , i.e., $X_0 = X_2 + X_1$ and $X_1 = X_0 + X_2$.

$$\begin{array}{c}
\frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \quad \frac{}{\vdash X_1 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} E1 \\
\hline
\vdash X_2 + X_1 : \begin{pmatrix} 0 \\ p \\ m \end{pmatrix} E4 \\
\hline
\vdash X_\Theta = X_2 + X_1 : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{pmatrix} A
\end{array}
\qquad
\begin{array}{c}
\frac{}{\vdash X_\Theta + X_2 : \begin{pmatrix} w \\ 0 \\ w \end{pmatrix}} E2 \\
\hline
\vdash X_1 = X_\Theta + X_2 : \begin{pmatrix} m & w & 0 \\ 0 & 0 & 0 \\ 0 & w & m \end{pmatrix} A
\end{array}$$

Steps 4. Analyze command C_3 i.e., the while loop.

$$\begin{array}{c}
\frac{}{\vdash X_1 + X_\Theta : \begin{pmatrix} w \\ w \\ 0 \end{pmatrix}} E2 \\
\hline
\vdash X_2 = X_1 + X_\Theta : \begin{pmatrix} m & 0 & w \\ 0 & m & w \\ 0 & 0 & 0 \end{pmatrix} A \\
\hline
\vdash \text{while}(X_2) \ X_2 = X_1 + X_\Theta : \begin{pmatrix} m & 0 & w \\ 0 & m & w \\ 0 & 0 & m \end{pmatrix} W
\end{array}$$

Steps 5. Compose the mwp-matrices of derivations $\pi_0 \dots \pi_3$.

$$\begin{array}{c}
\vdots \pi_0 \qquad \qquad \qquad \vdots \pi_1 \\
\vdash C_0 : \begin{pmatrix} m & 0 & m \\ 0 & m & p \\ 0 & 0 & m \end{pmatrix} \quad \vdash C_1 : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{pmatrix} \\
\hline
\vdash C_0; C_1 : \begin{pmatrix} m & 0 & m \\ p & m & p \\ m & 0 & m \end{pmatrix} C \\
\hline
\vdash C_0; C_1; C_2 : \begin{pmatrix} m & w & m \\ p & p & p \\ m & w & m \end{pmatrix} C \\
\hline
\vdash C_0; C_1; C_2; C_3 : \begin{pmatrix} m & w & w \\ p & p & p \\ m & w & w \end{pmatrix} C
\end{array}$$

The derivation gives variable mwp-bounds $x'_0 \leq \max(X_\Theta, X_2) + X_1$ and $x'_1 \leq X_\Theta + X_2 + X_1$ and $x'_2 \leq X_\Theta + X_2 + X_1$. The program has 81 admissible derivations in total. $\square$

1.2.3.6 Analysis of the Equivalent Programs

Recall the equivalent programs of Listing 1. For program analysis, the programs must be first transformed to the imperative language of the flow calculus. Listings 11 and 12 show the programs after the transformation.

Program with control flow. Consider the input program in Listing 11.

```
if bit then Z = X else Z = Y
```

Imp

Listing 11: Program with control flow (P_C).

The program P_C is derivable by the following derivation. Moreover, it is the only derivation assigned to the program.

$$\begin{array}{c}
 \frac{}{\vdash X : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} \text{E1} \qquad \frac{}{\vdash Y : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} \text{E1} \\
 \frac{}{\vdash Z=X : \begin{pmatrix} m & 0 & m \\ 0 & m & 0 \\ 0 & 0 & 0 \end{pmatrix}} \text{A} \qquad \frac{}{\vdash Z=Y : \begin{pmatrix} m & 0 & 0 \\ 0 & m & m \\ 0 & 0 & 0 \end{pmatrix}} \text{A} \\
 \hline
 \vdash \text{if bit then } Z=X \text{ else } Z=Y : \begin{pmatrix} m & 0 & m \\ 0 & m & m \\ 0 & 0 & 0 \end{pmatrix} \text{C}
 \end{array}$$

The mwp-bound assigned to variable Z is $\max(X, Y)$. Because it is not possible to know which branch of the program will execute, this is the most precise upper bound to characterize the final value of variable Z .

Program with arithmetic. The initial program is $Z=X*\text{bit}+Y*(1-\text{bit})$. Transforming the program to the imperative language requires rewriting the arithmetic operations to binary form, and lifting constants to input variables. From these changes, we obtain the program in Listing 12. Post-transformation, the program contains 6 variables, 2 of which did not occur in the initial program. Variable c substitutes for the constant 1 and t is a necessary temporary variable to hold intermediate values during computation.

```

t = c - bit;    // 1 - bit
t = Y * t;      // Y * (1 - bit)
Z = X * bit;    // Z = X * bit
Z = Z + t;      // Z = X * bit + Y * (1 - bit)

```

Imp

Listing 12: Program with arithmetic (P_A), after transformation.

The program P_A is derivable,²⁰ though the derivation is omitted for brevity. The program admits 81 derivations in total. One of them produces the following mwp-matrix

$$\begin{array}{c}
 X \quad Y \quad Z \quad \text{bit} \quad c \quad t \\
 \begin{array}{l}
 X \\
 Y \\
 Z \\
 \text{bit} \\
 c \\
 t
 \end{array}
 \begin{pmatrix}
 m & 0 & w & 0 & 0 & 0 \\
 0 & m & p & 0 & 0 & w \\
 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & p & m & 0 & p \\
 0 & 0 & p & 0 & m & w \\
 0 & 0 & 0 & 0 & 0 & 0
 \end{pmatrix}
 \cdot
 \end{array}$$

In this mwp-matrix, variable Z is assigned an mwp-bound $\max(X) + Y \times \text{bit} \times c$.²¹ The mwp-bound simplifies to $X + Y \times \text{bit}$.

Assessment and comparison. The derivation of program P_C is more straightforward than of P_A . The former involves fewer variables, deterministic inference rules, and fewer derivation steps. There is precisely one derivation to characterize P_C . The mwp-bound $\max(X, Y)$ is more precise than $X + Y \times \text{bit}$. Therefore, for reasoning about variable value growth, P_C is the preferable program form.

²⁰A derivation failure only occurs in loops; every program without loops is trivially derivable.

²¹This mwp-bound may be sub-optimal.

1.2.4 Loop Transformations for Parallel Programming

A loop command is a focal construct in many areas of computer science. In complexity theory, interesting program behavior occurs in loops (Ben-Amram & Hamilton, 2020; Jones & Kristiansen, 2009). In verification, loop *invariants* (presented in §1.2.6) are essential for proving program correctness. However, finding loop invariants is one of the most challenging problems in verification (Dillig, Dillig, Li, & McMillan, 2013; Si, Dai, Raghothaman, Naik, & Song, 2018; Yu, Wang, & Wang, 2023). In compiler theory, loops provide a mechanism for applying *code optimizations*. These are syntactic transformations that aim to improve the efficiency of the generated executable program. The transformation must preserve semantic equivalence with the original program that contains the loop; but otherwise, the program structure can be altered to reduce running time, size, power consumption, etc. (Aho, Lam, Sethi, & Ullman, 2007, p. 10). In parallel programming, the repetitiveness of loop *iterations* creates potential for executing the iterations in parallel to reduce the program's runtime latency.

This section focuses on automatic compile-time loop *transformations* and relating those transformations to *parallel executions*. The starting point is a sequential imperative program. The program result is the one obtained by executing the program's commands in the order they appear; with appropriate exceptions for control flow constructs like branching and loops. Thus, the sequential program specifies a precise ordering on the operations that are expected to execute. It is impossible to exploit parallelism directly from this specification because parallelization changes the operation order. However, conditional on the *dependence* present in the program, it may be permissible to allow such an alternative command execution order if it computes the same result as the original sequential program.

1.2.4.1 Basic Loop Constructs and Canonicity

Throughout the presentation, we assume a *loop* to mean a *for* loop²² in an imperative programming language. We will refer to Listing 13 to highlight essential concepts and terms. The examples use the C language, but the concepts extend to other imperative languages. A loop has two parts: a header (L1) and a body (L2–3).

```
1 for(j=1; j<A.length; j++)  
2   for(i=j-1; i>=0; i--)  
3     CompareExchange(A, i, i+1)
```

C³

Listing 13: Adaptation of insertion sort from Cormen, Leiserson, Rivest, and Stein (2009).

The *header* contains three expressions that define in order the loop’s initialization, update or increment, and the termination condition. These expressions refer to the loop *iteration variable*, a scalar variable that is updated in each iteration. At L1, the iteration variable is *j*. The iteration variable is initialized, before any iterations occur, by the initialization expression. The iteration variable is updated after each iteration by the update expression. In the initialization expression, the constant 1 is the *lower bound* (lb) of iteration. In the termination condition, *A.length* is the *upper bound* (ub) of iteration (OpenMP Architecture Review Board, 2024, p. 198–199). In §3.1, when the upper bound is a variable, we call it the *loop guard*, or guard variable. The range of iterations is the *iteration space*.

The *body* consists of executable statements (OpenMP Architecture Review Board, 2024, p. 67). The termination condition is evaluated, at each iteration, before executing the commands of the body. The loop repeats execution of the body until the condition evaluates to false. The body may recursively contain another loop. Then, the inner loop is called

²²Extending the presentation to more general loop forms is generally challenging. Refer to §1.2.4.7 and §2.2 for an approach that circumvents the limitation.

a *nested loop*. In Listing 13, the header of the nested loop is at L2 and body at L3. Since there is no intervening code between the two loops, the loop of Listing 13 defines a *perfectly nested loop* (OpenMP Architecture Review Board, 2024, p. 84).

Canonical loop. The canonical loop form simplifies several program analyses and makes transformations more effective (LLVM project contributors, 2025). A *canonical loop* (OpenMP Architecture Review Board, 2024, p. 196–202)²³ has form,

```
1 for(init-expr; test-expr; incr-expr)
2     loop-body
```

C*

Listing 14: Canonical loop form.

The header must additionally satisfy the following restrictions.²⁴

1. The initialization `init-expr` has form `var=lb` where `var` is the iteration variable and `lb` is the lower bound of iteration.
2. The termination condition `test-expr` has form `var op ub` (commutatively) where `ub` is the upper bound of iteration and `op` is an operator in `{<, >, <=, >=, !=}`.
3. The update expression `incr-expr` applies to `var` an increment or a decrement (`incr`) of 1. The operation must be in unary (`++`, `--`), compound assignment (`+=`, `-=`), or binary (`var=var+incr`, `var=incr+var`, `var=var-incr`) form.

Non-canonical loop. If a loop that does not satisfy the criteria of a canonical loop, in §2.2, it is referred to a *non-canonical loop*.

²³The definition is specific to the programming model of §1.2.4.5 and §2.2.

²⁴Additional restriction are defined in (OpenMP Architecture Review Board, 2024, p. 201).

A note on loop representations. This representation is simplified, at least in two ways, for accessibility. Real compile-time code optimizations are performed on an *intermediate representation* (IR) of the source program. Intermediate representations, which are compiler-specific, differ from the high-level syntactic view described previously. Further, to facilitate various analyses, it is possible to detach programs from language-based representations altogether, via control-flow (and other) graphs. A *control-flow graph* (CFG) (Allen, 1970) is a directed graph for representing program’s execution paths. In a CFG, the nodes are basic blocks and the edges are control flow paths. A *basic block* is a linear sequence of program instructions with one entry and one exit point. Control-flow graphs are a common approach for compile-time program analyses and transformations (Moyen, 2017, p. 48) and are adopted, e.g., in the LLVM compiler (LLVM project contributors, 2025). Although making this note is important, the level of detail is unnecessary for this topic overview.

1.2.4.2 Sequential Loop Transformations

Compile-time loop optimization is motivated by the idea that a loop transformation yields—by some quantifiable metric like data access pattern—a more optimal loop form than the original source program (Aho, Lam, Sethi, & Ullman, 2007). A loop transformation replaces the *original* loop(s) with one or more semantically equivalent *generated* loops. We consider next a few common loop transformations to demonstrate the idea. Assuming a sequential source program, these transformations are straightforward for a compiler to apply automatically (Bertolacci, Strout, de Supinski, Scogland, Davis, & Olschanowsky, 2018).

Distributing and combining loops. Loop *fusion*, in Listing 15, merges multiple loops by replacing them with a single loop. In each iteration, the generated loop executes an iteration of each original loop. Loop *fission*, in Listing 16, operates in the reverse direction. It breaks the original loop into multiple loops, with each generated iteration taking only a part of

the original loop’s body (Zhao et al., 2018). In both transformations, the loop header and iteration space remain unchanged.

```

1 for(int i=0; i<N; i++) {
2   A[i] = B[i];
3   C[i] = rand();
4 }

```

C*

Listing 15: Loop fusion.

```

1 for(int i=0; i<N; i++)
2   A[i] = B[i];
3 for(int i=0; i<N; i++)
4   C[i] = rand();

```

C*

Listing 16: Loop fission.

Other loop transformations. Although the main focus of the dissertation work is loop fission (in §2.2), there are several other loop transformations. The transformations are summarized visually in Fig. 8. When considering the overall compilation strategy, the combination of applied transformations is important.

Iteration range-based transformations.

- The *split* transformation, in Listing 17, breaks the original loop into multiple generated loops. Each generated loop has the same body as the original, but iterates over a different contiguous subset of the original range.
- Loop *tiling* breaks a loop into contiguous chunks of fixed *tile size*. In Listing 18, the tile size is 8. Tiling improves cache behavior by reducing data reuse distance (Bertolacci, Strout, de Supinski, Scogland, Davis, & Olschanowsky, 2018).
- Loop *reversal* executes loop iterations in the reverse order. For example, if $0, 1, \dots, n-2, n-1$ are the logical iteration numbers of a loop, after transformation, the iterations occur in order $n-1, n-2, \dots, 1, 0$ (OpenMP Architecture Review Board, 2024).

```

1 for(int i=0; i<N/2; i++) {
2     A[i] = B[i];
3     C[i] = rand();
4 }
5 for(int i=N/2; i<N; i++) {
6     A[i] = B[i];
7     C[i] = rand();
8 }

```

C*

```

1 for(int j=0; j<N; j+=8) {
2     for(int i=j; i<j+8; i++) {
3         A[i] = B[i];
4         C[i] = rand();
5     }
6 }

```

C*

Listing 18: Loop tiling.

Listing 17: Loop splitting.

```

1 A[0] = B[0]; C[0] = rand();
2 A[1] = B[1]; C[1] = rand();
3 for(int i=2; i<N; i++) {
4     A[i] = B[i];
5     C[i] = rand();
6 }

```

C*

```

1 for(int i=0; i<N; i+=2) {
2     A[i] = B[i];
3     C[i] = rand();
4     A[i+1] = B[i+1];
5     C[i+1] = rand();
6 }

```

C*

Listing 19: Loop peeling.

```

1 for(int j=0; j<M; j++)
2     for(int i=0; i<N; i++) {
3         A[i][j] = B[i][j];
4         C[i][j] = rand();
5     }

```

C*

Listing 20: Loop unrolling.

```

1 for(int i=0; i<N; i++)
2     for(int j=0; j<M; j++) {
3         A[i][j] = B[i][j];
4         C[i][j] = rand();
5     }

```

C*

Listing 21: Pre-interchange.

Listing 22: Post-interchange.

Figure 8: Visualizing loop transformations.

- Finally, loop *peeling*, in Listing 19, is a special case of loop splitting. It removes the first few or last iterations from the body and performs them outside the loop. The loop iteration space is adjusted accordingly.

Modifications of the loop body. Loop *unrolling* expands and duplicates the body commands into similar independent commands. The update condition is adjusted by an *unroll factor*. In Listing 20 the loop body is unrolled twice, therefore the unroll factor is 2.²⁵ Unrolling aims to reduce or eliminate evaluations of the loop condition.

Transforming nested iterations. Loop *interchange* is a transformation on a nested loop. It changes the loop nesting order by exchanging two iteration variables referenced by a nested loop, as shown in Listings 21 and 22.

1.2.4.3 Dependence in Transformations

A transformation is permissible if the transformation preserves the semantics of the source program. Thus, the application of a transformation depend crucially on the execution relationships between program statements.²⁶ The source program specifies, via a series of statements and an ordering of those statements, a set of execution constraints. However, the constraints may be imprecise since not all constraints are needed to preserve the effect of the computation. For example, in the program fragment `X=4; Y=5;` it is safe to reorder the two statements. A fundamental challenge in compiler theory concerns determining whether an execution order, that differs from the one specified by the source program, always computes the same result as the source program. Dependence is the primary mechanism for determining if code transformation is safe.

²⁵The simplified example assumes the transformation does not exceed the array bounds.

²⁶Compiler literature refers to *statements*, rather than commands and expressions.

A *dependence* is a relation on statements that must be preserved by a transformation (Kennedy & Allen, 2001, p. 47). Informally, for any two statements S1 and S2, if S1 must be executed before S2 then S2 depends on S1. Dependence arises primarily from data access patterns – if S2 refers to a value computed by S1 – and from conditional branching, when the execution of S1 decides whether S2 will execute. Further, by parametrizing computation over iterations, loops introduce additional special cases of dependence. A *reordering transformation* is any transformation that changes the code execution order, but does not add or delete statements. By the fundamental theorem of dependence, a reordering transformation that preserves every dependence in a program preserves the meaning of that program (Kennedy & Allen, 2001, p. 66).

Data and control dependence. How data is accessed during computations creates dependencies in terms of *order* of shared memory operations. There is a *data dependence* from statement S1 to S2 if and only if both statements access the same memory location, at least one statement writes to the location, and there is a runtime execution path from S1 to S2 (Kennedy & Allen, 2001, p. 59–60). Dependence is further complicated by the program’s control flow. Conditional branching creates a *control dependence* between the control statement and the statements whose execution depends on the control statement (Kennedy & Allen, 2001, p. 374). Four common cases of data and control dependence are demonstrated in Listings 23–26. The interesting variables, that reference a memory location of interest, are emphasized.

```

X=42;      (S1)
Y=X;       (S2)

```

C*

Listing 23: True dependence.

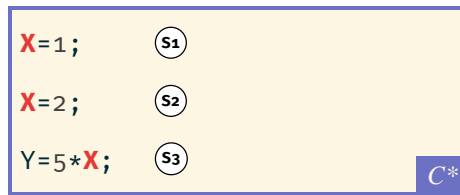
```

Y=X;      (S1)
X=42;     (S2)

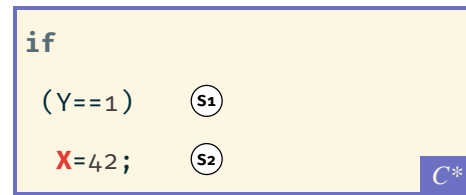
```

C*

Listing 24: Antidependence.



Listing 25: Output dependence.



Listing 26: Control dependence.

- A *true dependence* occurs when a write (S1) executes before a read (S2) at the same memory location (X). True dependence ensures the read observes the value of the write.
- An *antidependence* reverses the order, with a read (S1) executing before a write (S2) at the same memory location (X).
- An *output dependence* requires two writes (S1, S2) to the same memory location (X). It forbids reordering the writes to prevent an interchange that would cause a subsequent read (S3) to observe an incorrect value.
- In *control dependence*, the dependent statement (S2) is conditionally executed, based on evaluation of the control statement (S1).

Dependence in loops. Data and control dependence are applicable to loops; however, loops introduce additional dependence. Dependence in loops arises from statements being *parameterized by iterations* in which they execute.

A *loop-carried dependence* (Kennedy & Allen, 2001, p. 73), in Listing 27, occurs when a statement uses a value computed in a different iteration. More precisely, a statement S2 has a loop-carried dependence on statement S1 if and only if

1. S1 references a location X on iteration i ,

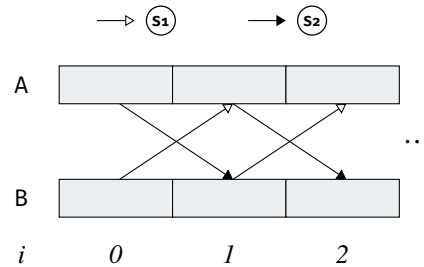
2. S2 references the location X on iteration j , and
3. the loop iterates at least once between the references of X, i.e., the distance d between iterations is $d(i, j) > 0$.

```

1  for(i=0; i<N-1; i++) {
2      A[i+1] = B[i];    S1
3      B[i+1] = A[i];    S2
4  }

```

C*



Listing 27: Loop-carried dependence. After the first iteration, in every iteration S2 uses a value of array A computed in the previous iteration by S1. Similarly, S1 uses a value of array B computed in the previous iteration by S2. There is a true dependence from S2 to S1 and from S1 to S2, and both dependencies are carried by the loop. If we select any individual iteration and execute it alone, no dependence exists.

If loop statements depend only on statements in the same iteration, then there exists a loop-independent dependence. A *loop-independent dependence* (Kennedy & Allen, 2001, p. 76) occurs when S1 references a location X on iteration i , S2 references X on iteration i , and there is a control flow path from S1 to S2.

Comparing the two loop dependencies, a loop-carried dependence determines the iteration order and whether that order can be transformed safely. A loop-independent dependence determines the statement execution order within a nest of loops.

Dependence analysis. Dependence analysis (or a *dependence test*) is a technique for identifying or characterizing program dependencies and dependence-preserving reorderings; which may then permit transformation or parallel execution. Among the well-established techniques of dependence analysis are Banerjee test (Banerjee, 1997), GCD test (Zima & Chapman, 1990), Omega test (Pugh, 1991), and commutativity analysis (Rinard & Diniz,

1996). Section §2.2 will present a complementary dependence analysis inspired by implicit computational complexity.

1.2.4.4 Incorporating Parallelism

If we only focus on sequential code, the notions around dependence are well-established. However, sequential reasoning is insufficient for effective use the parallelism that is available in modern processors. Developing and designing parallel software and algorithms is conceptually challenging. One of the major difficulties is the effort required to ensure that a parallel program is (functionally) correct. In addition to all the errors that arise in sequential programs, parallelism introduces a new class of bugs that includes race conditions. A race condition is devious because the runtime behavior of a program with such a bug is non-deterministic (Chapman, Jost, & Van Der Pas, 2007, p. 32). But in exchange for the challenge, parallelism provides the potential to improve the program’s runtime performance. In practice, exploiting parallelism is necessary for all performance-critical applications (Suomela et al., 2025).

There are several good motivations for introducing parallelism in programs and for improving the existing parallelization techniques. High-performance multicore processors are ubiquitous and possess the abilities to execute several operations in a single clock cycle. Due to physics, improvements in hardware design now come from parallel processing capabilities, rather than adjustments in clock cycles. On the software side, a vast majority of legacy applications, and the algorithms that power them, were developed without parallelization in mind. *Parallelizing compilers*, that automatically translate sequential programs to multi-core executables, have made big advances toward improving prevalence of parallelism. But despite the advances, these compilers make conservative choices and miss opportunities to parallelize (Holewinski et al., 2012). Correct and efficient parallel code is thus often hand-crafted in a process that is time-consuming and error-prone.

Ultimately, our abilities to parallelize programs relies on the potential parallelism available in the program and the processor; and the degree to which we can extract parallelism from a sequential program and find the best parallel schedule under the processor’s scheduling constraints (Aho, Lam, Sethi, & Ullman, 2007). No parallelization strategy fits all cases and it is important to facilitate the discovery of new strategies and parallel algorithms as complementary to existing techniques. For example, running automatic analysis on a large application can distinguish regions that can be parallelized automatically from those that need manual changes.

From dependence to independence. Since dependence determines how program executions may be reordered, it provides a strong foundation for developing techniques to increase the parallelism available in programs. Parallelism inherently requires *independence* between statements; in other words, we are interested in absence of dependence. Thus, if we can design algorithms, or transform programs in ways that increase prevalence of independent statements, we improve the program’s parallelization potential. One focus area in parallelization concerns *loop-level parallelism* that aims is to extract parallel tasks from loops. For example, by the Loop Parallelization Theorem, it is permissible to convert a sequential loop to a parallel one if the loop carries no dependence (Kennedy & Allen, 2001, p. 82). The remainder of this section focuses precisely on the terms and techniques of parallelizing loops.

Parallel programming models. Multiple sophisticated programming models exist to simplify the task of writing parallel programs. These models raise the abstraction level between the application code and the operating system-level parallelization mechanism of threading. The OpenMP application programming interface is presented in §1.2.4.5. Other approaches include the *Parallel Patterns Library* (PPL) (Campbell & Miller, 2011; Whitney, Sharkey, Jones, Blome, Hogenson, & Cai, 2021), that parallelizes tasks through built-in

algorithms and containerization; and the *oneAPI Threading Building Blocks* (oneTBB), a runtime-based model for C++ programs (oneTBB Contributors, 2025).

1.2.4.5 The OpenMP Programming Model

OpenMP, for *Open Multi-Processing*, is a parallel programming model for developing portable, shared memory parallel applications in C, C++ and Fortran (OpenMP Architecture Review Board, 2024). OpenMP is not a programming language. Rather, it provides *directives* for sequential programs that translate the programs to parallel execution instructions at compile-time. The OpenMP infrastructure includes compiler directives, library routines, environment variables, and tool support for creating, debugging, and analyzing parallel applications.

Program execution in OpenMP follows the *fork-join programming model* (Chapman, Jost, & Van Der Pas, 2007, p. 24) of Fig. 9. Program execution starts with an primary thread that runs in sequential mode. After encountering a specially-marked parallelizable program region, the primary thread spawns a team of threads in a *fork*-operation. The team collaborates to complete the tasks of the parallel program region. The execution concludes with a *join*-operation, where the forked threads terminate. The primary thread continues execution past the parallel program region, running again in sequential mode.

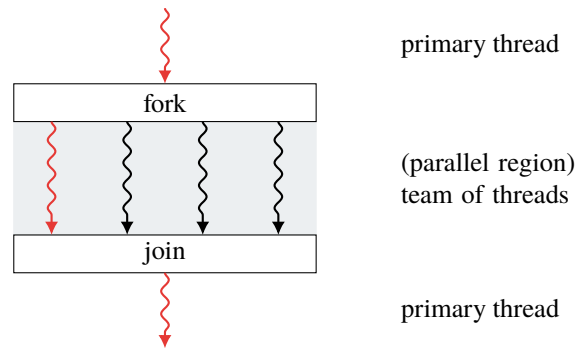


Figure 9: The fork-join programming model.

In OpenMP, a programmer annotates structured blocks with parallelization directives to control the parallel execution behavior. A *structured block* is a block of executable statements with a single point of entry and exit. A *parallelization directive* is a syntactical comment that specifies instructions for how the structured block should be executed. The syntax and semantics of parallelization directives are defined by OpenMP. It is the responsibility of the programmer to apply the directives safely. During compilation, an OpenMP-aware compiler translates the directives into parallel-executable instructions (Pophale, 2023). Other compilers are free to ignore the directives. OpenMP applications are suitable for running on multicore chips, GPUs, and various other devices attached to a CPU. OpenMP is developed and maintained by the OpenMP Architecture Review Board (ARB).

Foundational terms. The following are basic parallel programming terms needed to discuss OpenMP directives (OpenMP Architecture Review Board, 2024); refer to Listing 28 for a visual. A *thread* is a logical execution entity, equipped with a stack and associated thread-specific memory. The threads are identified by numeric ids. Once a non-empty set of threads is assigned to execute implicit tasks of a parallel region, the set of threads becomes a *team*. *Region* refers to all code encountered during execution, including code

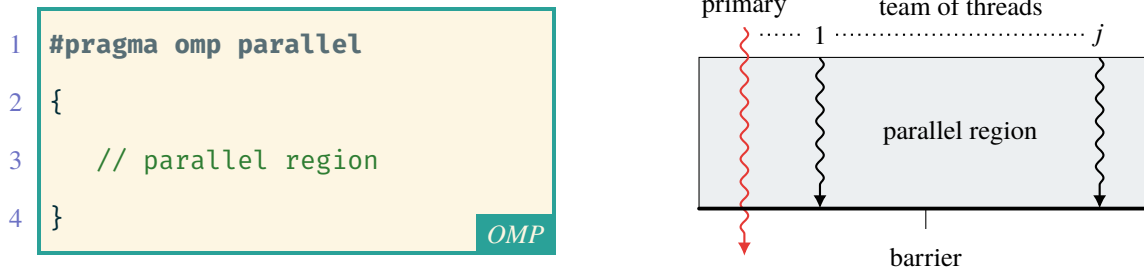
encountered in called procedures. A region is a *parallel region*, if it has an associated set of implicit tasks, and has been assigned a team to execute those tasks. A *task* is a specific instance of executable code and its data environment that can be scheduled for execution. If a task is generated by an implicit parallel region, or through an encounter with a parallel construct during execution, it is an *implicit task*. Finally, a *barrier* is a blocking synchronization construct at a program execution point. No thread in a team may execute beyond a barrier until all team members have reached the barrier.

Additional terminology relates to parallelization of loops. A *worksharing loop* is a loop-associated construct that satisfies the worksharing loop property. The *worksharing loop property* requires that a construct distributes the iterations of the associated loops among the threads of the team that encounters the loop (OpenMP Architecture Review Board, 2024, p. 113).

Essential parallel directives. Parallelization directives are at the core of OpenMP. This section introduces a few important ones, including the directives that appear elsewhere in this dissertation. Because OpenMP is a powerful and rich programming model, it is possible to present only a light introduction. A complete definition is available in the OpenMP API technical specification (OpenMP Architecture Review Board, 2024).

- `#pragma omp` is a directive specification in the style of C/C++. A parallelization directive must start with these keywords.
- The `parallel` construct marks a structured block parallelizable (OpenMP Architecture Review Board, 2024, p. 384–385). Listing 28 shows the syntactic use and illustrates its behavior. When a thread encounters the `parallel` construct, it forms a team of threads to execute the associated structured block. The structured block becomes a parallel region. The thread that encountered the `parallel` construct is

the *primary thread* of the team. All threads in the team execute the parallel region. An implicit *barrier* occurs at the end of a parallel region. After the parallel region, the primary thread resumes execution of the enclosing region.



Listing 28: The OpenMP `parallel` construct and parallel execution with j threads.

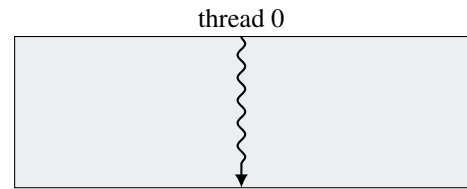
Synchronization constructs. Synchronization constructs (OpenMP Architecture Review Board, 2024, p. 473–482) enable controlling co-operation of threads over parallel executions.

- The `atomic` construct ensures a specific memory location is accessed atomically. It ensures that possible simultaneous reads and writes by multiple threads do not result in indeterminate values.
- The `barrier` construct specifies an explicit synchronization barrier. It prevents any thread from continuing past the barrier until all threads of the same team encounter the barrier.
- The `critical` construct means execution of a structured block is restricted to a single thread at a time.
- The `nowait` clause overrides any synchronization that would otherwise occur at the end of a structured block. Thus, `nowait` enables specifying that an implicit barrier should not occur.

One thread executes each iteration sequentially, total time = $N \times t_{\text{iter}}$.

```
1 for(int i=0; i<N; ++i)
2   C[i] = A[i] + B[i];
```

C

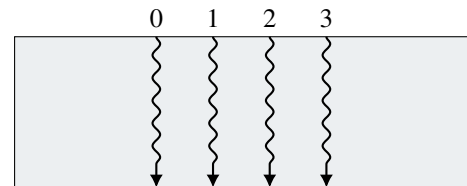


Listing 29: Serial execution.

All available threads will execute each iteration sequentially, overwriting values of array C, total time = $N \times t_{\text{iter}}$.

```
1 #pragma omp parallel
2 for(int i=0; i<N; ++i)
3   C[i] = A[i] + B[i];
```

OMP

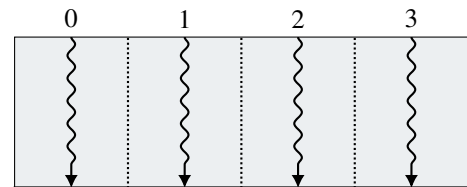


Listing 30: Parallel execution.

Assuming a thread count of 4, the threads distribute the loop iteration space uniformly, total time = $\frac{N}{4} \times t_{\text{iter}}$.

```
1 #pragma omp parallel for
2 for(int i=0; i<N; ++i)
3   C[i] = A[i] + B[i];
```

OMP



Listing 31: Parallel worksharing.

Figure 10: The impact of applying parallel worksharing directives to loop executions, adapted from Pophale (2023).

Work distribution and tasking constructs. Parallel executions of structured blocks are controllable by work distribution and tasking directives. Looping operations on arrays and matrices, like images and tabular data, are often prime candidates for parallelization. OpenMP has specialized directives to support parallelization of loops. When applying parallelization directives, the programmer must make sure that the loop is safe to parallelize and performs a sufficient amount of work to benefit from parallelization.

- The `for` construct is a worksharing loop. It specifies that the iterations of associated loop(s) will execute in parallel (OpenMP Architecture Review Board, 2024, p. 416). The team that encounters the loop will perform its execution. The sequential iteration order is consistent with the execution of the collapsed iterations of a parallel `for` region. The `for` construct supports e.g., explicit scheduling, and is compatible with the `nowait` clause. The `schedule` clause specifies how the loop iterations are divided into chunks and how the chunks are distributed between threads. The basic use and effect of the `for` construct are demonstrated in Fig. 10 and in Listing 31.
- The `loop` construct supports compiler optimization and parallelization of loops for which logical iterations may execute in any order, including concurrently (OpenMP Architecture Review Board, 2024, p. 423–424). More precisely, it specifies that the iterations of the associated loop(s) may execute concurrently, in the order specified by an `order` clause. A loop construct is a worksharing construct if and only if its binding region is the innermost enclosing `parallel` region. It is possible to use a shortcut expression `parallel loop`, combining the two constructs, if the construct refers to one or more loops and no other statements.
- The `single` construct restricts the execution of a structured to one thread (OpenMP Architecture Review Board, 2024, p. 405). The executing thread is not necessarily the primary thread; the choice is implementation-specific. An implicit barrier concludes a `single` region.

Shared vs. private data. In a multi-threaded shared memory environment, it is necessary to specify how variables are accessed by the executing threads. This behavior can be controlled with the data-sharing attribute clauses (OpenMP Architecture Review Board, 2024, p. 224–225) that are a kind of data environment modifiers. Both clauses presented here

take as a parameter a list of variables names or procedure pointers. The use of data-sharing attribute clauses is demonstrated in Listing 33.

- The `private` clause privatizes the variables in the parameter list. It creates a new variable for each variable in the parameter list, and for each thread executing the parallel region. Thus, each thread has its a thread-specific variable copy. A private variable is uninitialized.
- The `shared` clause declares that the variables appearing in the parameter list will be shared across the threads executing the parallel region. The programmer must ensure, with proper synchronization, that the shared memory remains available until execution of the parallel region completes.

Example 14 (Parallel transformation). *We demonstrate a parallelization transformation inspired by Singal, Gopalani, Kushwaha, and Badal (2019). The original sequential code is in Listing 32, where a loop iterates over fixed-size array length N , manipulating each array index in arrays A and B (we assume both arrays to be of length N).*

```
1 for (i=0; i<N; i++) {  
2   A[i]=i+1; B[i]=i*2;  
3 }
```

Listing 32: Sequential version.

```
1 #pragma omp parallel for \  
   shared(A, B) private(i)  
2 for (i=0; i<N; i++) {  
3   A[i]=i+1; B[i]=i*2;  
4 }
```

Listing 33: Parallel version.

In Listing 33, the loop iteration space is distributed equally between the threads of a team. Given a team of j threads, each thread will roughly complete the work of N/j iterations. OpenMP often makes the parallelization step straightforward, in this case requiring just one

line of directives. By the data-sharing attribute clauses, the arrays A and B are explicitly shared across the threads and the iteration variable `i` is privatized. □

1.2.4.6 Built-In Loop Transformations in OpenMP

OpenMP includes built-in directives that perform loop transformations (OpenMP Architecture Review Board, 2024). A loop-transformation replaces the associated loop with a structured block that is another loop or a loop sequence. Unless otherwise specified, all generated loops have canonical form. Loop-iteration variables of generated loops are always private in the innermost enclosing construct. The loop transformations available in OpenMP are: fusion, interchange, reverse, split, stripe (interleaving of loop iterations), tile, and unroll. As an example, we consider the unrolling transformation.

Loop unrolling. The `unroll` clause (OpenMP Architecture Review Board, 2024, p. 381–382) transforms the associated loop based on the specification provided in the parallelization directive. The `unroll` clause is followed by either `full` or `partial` clause. The effect of `full` clause is to unroll the associated loop fully. Full unroll requires that the iteration count of the associated loop is a constant. Listing 34 shows the application of the directive and the loop post-transformation form is shown in Listing 35.

```
1 #pragma omp for unroll full
2 for(int i=0; i<4; ++i)
3   A[i] += B[i] * c;
```

OMP

Listing 34: Applying full unroll.

```
1 A[1] += B[1] * c;
2 A[2] += B[2] * c;
3 A[3] += B[3] * c;
4 A[4] += B[4] * c;
```

C

Listing 35: Fully transformed loop.

The `partial` clause takes an integer-valued *unroll factor*. The effect of `partial` clause is to tile the loop, using the unroll factor as the tile size. Then, the generated tile loop is fully unrolled. If no unroll factor is specified, the unroll behavior depends on the implementation. Partial transformation is illustrated in Listing 36 and Listing 37.

```

1 #pragma omp for unroll \
   partial(4)
2 for(int i=0; i<n; i+=1)
3   A[i] += B[i] * c;

```

OMP

Listing 36: Applying partial unroll.

```

1 for(int i=0; i<n; i+=4) {
2   A[i] += B[i] * c;
3   if(i + 1 < n)
4     A[i+1] += B[i+1] * c;
5   if(i + 2 < n)
6     A[i+2] += B[i+2] * c;
7   if(i + 3 < n)
8     A[i+3] += B[i+3] * c;
9 }

```

C

Listing 37: Partially transformed loop.

Applying loop transformations requires suitable loop form. For example, the worksharing loop expects loops to be in canonical form. Afterward, a transformation that increases the program's parallelization potential enables making use of multicore hardware during execution. Thus, an important challenge of parallelizing compilers and program optimizations is to find generalized strategies to increase parallelization potential.

1.2.4.7 ICC for Parallel Loop Transformations

Since implicit computational complexity has developed many systems for reasoning about loops, it seems natural that in extended use, it has potential to support loop-based program

transformations. Founded on this concept, § 2.2 presents an ICC-based technique to increase parallelization potential in imperative programs. The technique uses a static data dependence analysis that detects independent blocks of commands inside loops. Through a loop fission transformation, then independent blocks are then transformed into multiple parallelizable loops, as illustrated in Fig. 11.

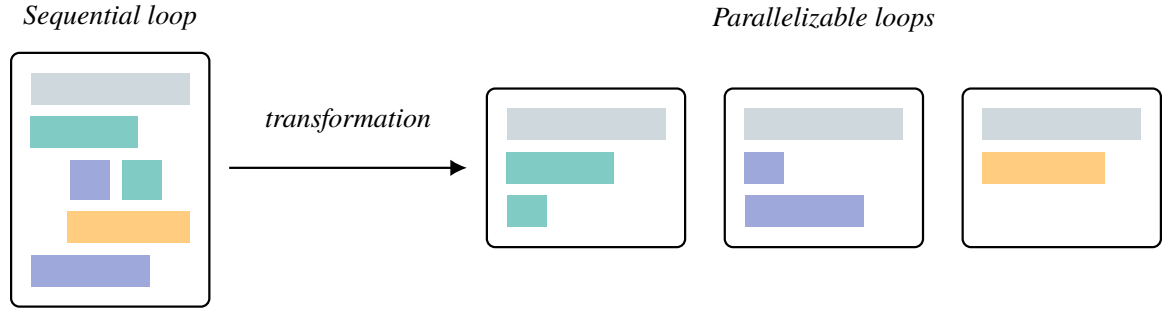


Figure 11: Loop fission of a sequential loop into parallelizable loops. The loop header (the leading ■ block) is replicated across the generated parallelizable loops. In the sequential loop body, the colors denote (disjoint) dependence between command blocks. Each generated parallelizable loop takes a part of the original loop’s body.

The technique is interesting because it is applicable even when the loop iteration space is unknown; enabling parallelization of e.g., while loops. Such non-canonical loops are traditionally outside the built-in transformation and parallelization capabilities of parallel programming models like OpenMP. Although the *iterations* of while loops must be executed under the single construct, the *multiple loops* generated by the transformation can be parallelized with OpenMP directives. The technique is useful as a preprocessor, before adding parallelization directives; and in conjunction with other loop transformations. The technique is inspired by (Moyen, Rubiano, & Seiller, 2017a), but was adjusted significantly to support the different application.

1.2.5 Confidentiality and Non-Interference

Computer security is a broad, interdisciplinary field with technical and non-technical challenges. An important subset of the technical aspects can be modeled and studied mathematically (Piessens, 2024). At its core, computer security is concerned with three properties: confidentiality, integrity, and availability (Du, 2017). Although the precise meaning of these terms is situational, they are generally described as follows (Bishop, 2019, p. 4–6).

Confidentiality is about concealment of protected assets, like secret information and resources. The goal of confidentiality is to ensure protected assets are viewed by authorized users only. By extension, it includes concealing knowledge about the existence of protected assets. Confidentiality is compromised if secrets are exposed, or “leak|seealsoviolation”, to unauthorized users.

Integrity is about trustworthiness of information and resources. It aims to ensure protected assets are modified by authorized users only and that those modifications preserve authenticity of the resource. For example, an update to a date field should yield a legitimate date. Integrity is difficult to guarantee because it concerns assets over mutations, transmissions, and time. Authentication is a critical (though an insufficient) mechanism to enforce integrity. Integrity is often considered the dual of confidentiality (Biba et al., 1977).

Availability refers to the ability to access and use authorized system and its assets. Availability is compromised if a computational system is unusable or incapable of supporting its intended use – to include unavailability due to excessive service latency.

The challenge of computer security is to find the right balance between the three properties – preventing unauthorized views and modifications, while preserving access and supporting authorized use. Among the strategies are designing techniques to prevent improper use,

detect issues, and recover from compromised scenarios. Although all three properties are important, the rest of this presentation focuses on confidentiality.

Confidentiality can be enforced with numerous mechanisms, e.g., firewalls, access control, and encryption. A firewall is a network-layer protection, and an operating system implements and maintains access control. Encryption protects secret information until decryption, but its excessive use is problematic for availability. In general, those mechanisms are insufficient to provide end-to-end guarantees through all layers of computation. One problem is that confidentiality is not guaranteed after secret information is released as input to a program (Zdancewic, 2004). Information flow security is concerned with achieving fine-grained confidentiality (and integrity) guarantees by tracking information propagation during program executions (Eggert, 2014; Hedin & Sabelfeld, 2012).

1.2.5.1 Information Flow Security

Protected assets may require different levels of confidentiality. Computer systems that protect assets at different security levels need controlled information flow. The capability to process assets at different security levels is called *multilevel security* (MLS) (Bossi, Macedonio, Piazza, & Rossi, 2005). Illustrated in Fig. 12, the United States federal government document classification scheme (Wikipedia contributors, 2025) is one example of a multilevel security system (Feiertag, Levitt, & Robinson, 1977). The permissions cascade: having access to a specific level grants access to all the levels below it. For example, accessing “secret” documents requires permission at secret or top secret level.

The main idea for achieving secure multilevel information flow is to impose constraints on how privileged information is permitted to flow within the system. Secure information flow is a confidentiality guarantee, because it aims to ensure secrets do not leak to unauthorized

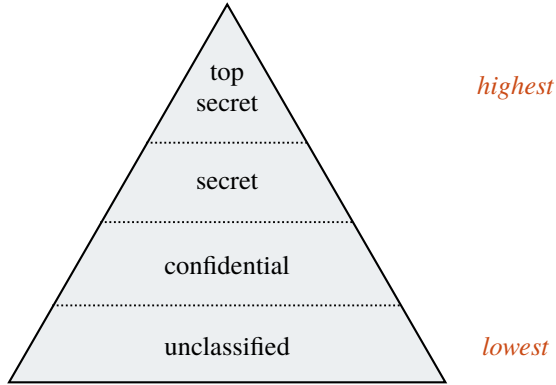


Figure 12: The United States federal government document classification scheme.

parties (Piessens, 2024). Implementing secure information flow requires three steps (Eggert, 2014).

1. Identifying *security classes* (alternatively, domains or user groups). In Fig. 12, the classes are top secret, unclassified, secret, and confidential.
2. Defining a confidentiality *policy*. The policy specifies how information is permitted to flow between the security classes. In Fig. 12, the the directed arrows define the policy.
3. Instrumenting an *control*. A control is a mechanism that enforces the behavior permitted by the policy. In Fig. 12, no control is specified.

For strong confidentiality guarantees, the information flow approach should also consider and protect against side channels. *Side channels* are all system characteristics that enable a malicious actor to deduce secrets, indirectly and without assistance from other users (Bishop, 2019, p. 280). In terms of program executions, a few examples of side channels are termination (Stefan, Russo, Buiras, Levy, Mitchell, & Mazières, 2012), divergence, deadlocking, and execution latency (Lawson, 2009).

1.2.5.2 The Elements of Information Flow

Information flow is an observable action between two agents²⁷ A and B (Eggert, 2014).

If an action performed by A is observable to B , then there exists an information flow from A to B . The information flow is *explicit* if it is directly observable from a single action. The insecure program in Listing 38 shows an example of direct information flow between `high` and `ret`. Information flow is *implicit* when it is not directly observable, but the initial action can be deduced from the sequence of actions that follow. The insecure program in Listing 40 shows an example of an implicit flow. The initial value of variable `h` is deducible from the returned value `l`, even though there is no direct flow from `h` to `l`.

Information flow policy (IFP) is a statement of what is, and what is not, permissible between security classes (Bishop, 2019, p. 9). Since the seminal works on information flow (Bell & La Padula, 1976; Biba et al., 1977), it has been a convention to model information flow policies with lattices (Denning, 1976). The most simple case considers two security classes, typically “high” and “low” (h, l) (or trusted and untrusted, secret and public, etc.). In the simple case, the policy condition is that no information should flow from high to low. (Bossi, Macedonio, Piazza, & Rossi, 2005) Then, showing that such a system is secure requires proving that executions that differ only on secrets are indistinguishable (Piessens, 2024). A policy that restricts information flow from a lower security class to a higher security class is called a non-interference policy, presented in §1.2.5.4.

Information flow control (IFC) is a mechanism that enforces a policy (Bishop, 2019).

Designing adequately expressive and effective controls is an active topic in security research (Bossi, Macedonio, Piazza, & Rossi, 2005; Sabelfeld & Myers, 2003; van

²⁷We use the term *agent* to emphasize an abstraction. An agent could be a human user, computer system, web request, program variable, etc.

der Meyden & Zhang, 2007). Like in static analysis (cf. §1.2.2.1), designing a sound and complete control for arbitrary programs is impossible. A sound IFC guarantees to find all policy violations and a precise IFC avoids raising excessive false alarms. However, an overly restrictive IFC is challenging because it compromises availability. The challenge of information flow control is more nuanced than classic data-flow analysis of functional properties, because an IFC must consider a program w.r.t. policies and multiple executions (Frumin, Krebbers, & Birkedal, 2021). Thus, the same program can have multiple judgments depending on the policy.

Declassification refers to controlled release of secret information (Sabelfeld & Sands, 2009). Declassification requires a *downgrading* operation; a mechanism that permits an elevated security judgement to be lowered. In other words, information is allowed to flow contrary to the policy (Cecchetti, Myers, & Arden, 2017). Declassification is necessary to support real world security applications. For example, after a failed login attempt, an unauthorized user is able to observe the effect of the failure, although this is an information leak. Declassification is generally not safe (Derakhshan, Balzer, & Yao, 2024), and thus poses a major design challenge to IFCs. In the context of integrity, declassification is also called endorsement (Marion, 2011).

Secure and insecure information flows. Figures 38–40 show basic examples of insecure and secure information flows. Variables `high`, `secret`, and `h` are assumed to be secret and their values should not leak from executions. The programs are fragments from the IF-SPEC benchmark suite (Hamann, Herda, Mantel, Mohr, Schneider, & Tasch, 2018a) available at (Hamann, Herda, Mantel, Mohr, Schneider, & Tasch, 2018b). The more advanced benchmarks involve, e.g., aliasing, reflection, context sensitivity, and exception handling.

```

1 boolean ret;
2 ret = (high && true);
3 return ret;

```

Java

Insecure.

```

1 boolean ret;
2 ret = (high || true);
3 return ret;

```

Java

Secure.

Listing 38: Boolean Operations. The programs are parametric on a Boolean variable `high`. In the insecure version, the value of `high` is deducible from the value of `ret`.

```

1 int[] arr = new int[5];
2 arr[0] = secret;
3 if (arr[0] == 42)
4     println("Found");

```

Java

Insecure.

```

1 int[] arr = new int[5];
2 arr[0] = secret;
3 if (arr[0] == 42)
4     println("checked");
5 else
6     println("checked");

```

Java

Secure.

Listing 39: Arrays Implicit Leak. The insecure program's output reveals if the value of `secret` is or is not 42. This leak is small, since it corresponds to 1 bit of information. However, the secure version reveals nothing about the value of `secret`.

```

1 int f(int h, int l) {
2     while (h > 0) { h--; l++; }
3     return l;
4 }

```

Java

Insecure.

```

1 int f(int h, int l) {
2     while (h > 0) { h--; }
3     return l;
4 }

```

Java

Secure.

Listing 40: High conditional incremental leak. The insecure version reveals implicitly the value of the secret variable `h` through variable `l`. The leak reveals every value of `h` precisely. In the secure version, value of `h` cannot be deduced. But even the secure version may be vulnerable over a side channel, if the loop execution time is observable to an attacker.

1.2.5.3 Techniques for Information Flow Modeling and Analysis

Information flow can be modeled in several distinct ways. One way to categorize these approaches is by how they model systems; e.g., state-based automata, trace-based model, and process algebras (van der Meyden & Zhang, 2007). The presentation in this dissertation is only concerned with approaches based on programming languages. The study of security through programming languages is called *language-based security* (Sabelfeld & Myers, 2003; Schneider, Morrisett, & Harper, 2001). The goal of language-based security is to use principles of programming languages—semantics, analysis, type systems, rewriting, etc.—to strengthen application security. However, to offer a point of comparison, we introduce briefly trace-based information flow. The language-based and trace-based approaches have very little in common (Eggert, 2014, p. 235).

Language-based information flow. Language based security formulates information flow constructs through programming languages. Program logics and security type systems are among the common techniques (Frumin, Krebbers, & Birkedal, 2021). A security type system aims to guarantee the security policy induced by its lattice (Marion, 2011). Each variable is assigned a security type, i.e., a label that indicates its security class. A satisfactorily secure program must pass a compile-time type check of the security labels. Sections 1.2.5.4 and 3.2 discuss the language-based information flow in more detail.

Trace-based information flow. The trace-based approach (Eggert, 2014, p. 24) models a system with a state machine. In response to actions, the machine transitions, deterministically or nondeterministically, from state to state. A *trace* is a sequence of actions of such a machine (Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020). Evaluating secure information flow over traces is about equivalence relations, or indistinguishability. For example, for a policy that requires that secrets should not leak to public, two traces must be

indistinguishable from the perspective of a public class (they only differ on secret actions). Further, executing those traces should produce publicly indistinguishable final states (Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020). Showing that a trace-based system satisfies this security requirements is done by simulation (Piessens, 2024) where the simulation proves the trace equivalence w.r.t. a policy.

Formal security analysis. Establishing that a “system is secure” demands security analysis. Rigorous formal security analysis considers the following three aspects.

1. System model: a precise mathematical definition of the system to be secured.
2. Security objective (or properties): a specification of system behaviors that are considered secure.
3. Attack model (or threat model): a definition of the class of attacks against which the system should be secure.

The parenthesized terms are included because the literature (Bau & Mitchell, 2011; Bognar, Van Bulck, & Piessens, 2022) varies in exact terminology. However, the goal of formal security analysis is uniform. For a satisfactory system, a formal analysis must conclude with a proof that shows the system satisfies its security specification under the identified threat. Designing an adequate system model, security objective, and attack model to formally prove system security is nontrivial (Piessens, 2024).

1.2.5.4 A Primer on Non-Interference

Non-interference is a foundational family of security policies and security properties that constrains information flow during computations. The important idea is that a low(er) security class is not permitted to observe a high(er) security class. In other words, information

flow from high to low security class is not allowed. Although non-interference is often described in binary terms, referring to just two security classes, it is applicable to arbitrary security hierarchies.²⁸ The primary application of non-interference is in supporting multi-level security systems (Roscoe & Goldsmith, 1999).

Classic non-interference. The concept of *non-interference* was introduced in 1982 by Joseph Goguen and José Meseguer. Their seminal paper (Goguen & Meseguer, 1982) presented the concept (only) informally, as follows.

One group of users, using a certain set of commands, is non-interfering with another group of users if what the first group does with those commands has no effect on what the second group of users can see.

Broadly, the paper provided an early framework for formal modeling and verification of information flow over deterministic state machines. This early view of deterministic non-interference is regarded as *classical non-interference* (Focardi & Gorrieri, 1997).

The terminology around non-interference and confidentiality is somewhat inconsistent (Sabelfeld & Myers, 2003; van der Meyden & Zhang, 2007). For example, surveying mathematical formalizations, Nelson et al. (Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020) identified four distinct characterizations of non-interference. It therefore more accurate to consider non-interference as a family of information flow properties that share the above common core. Non-interference has been studied in in a variety of models, e.g., trace-based models, process calculi, programming languages, probabilistic models, timed models, and cryptographic protocols (Bossi, Macedonio, Piazza, & Rossi, 2005). §1.2.5.5 will present how to formulate non-interference in terms of programming languages.

²⁸The policy in Fig. 12 is a 4-class non-interference policy.

Practical challenges with non-interference. Non-interference is a strong notion because it includes the absence of both explicit and implicit information flows (Bossi, Macedonio, Piazza, & Rossi, 2005). As noted by many authors (Bossi, Macedonio, Piazza, & Rossi, 2005; Cecchetti, Myers, & Arden, 2017), the requirement is too strong for practical applications. Strict non-interference is not achievable in real world systems (Bossi, Macedonio, Piazza, & Rossi, 2005). The non-interference requirement may also be excessively restrictive when additional knowledge about the system is available (Bossi, Macedonio, Piazza, & Rossi, 2005). For example, consider two “high” users communicating over an encrypted channel, and a “low” user who can observe the communication events. In this case, there is an implicit flow about the event, but the exchanged information remains secret. Such controlled leak may be practically acceptable, but classic non-interference would not permit it. Declassification (cf. §1.2.5.1) allows controlled and restricted information flows against the non-interference policy. It is an important mechanism to support security requirements of real world systems.

Proving that a system satisfies non-interference is harder than proving functional correctness (Frumin, Krebbers, & Birkedal, 2021). The reason is that functional correctness is a property of each single program execution. Non-interference, when expressed with trace-based information flow, is stated over multiple runs of the same program. A non-interference proof requires showing that for different behaviors of high security class the low security class cannot observe any different behavior (Frumin, Krebbers, & Birkedal, 2021). When a program property is expressed over a set of trace properties it is called a *hyperproperty* (Clarkson & Schneider, 2010). Since a non-interference proof requires sets of execution traces, non-interference is a hyperproperty (Mastroeni & Pasqua, 2019).

Extensions. The classic definition of non-interference has been extended in several directions. The following list contains several variants, though the list is necessarily incomplete.

See (Eggert, 2014; Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020; van der Meyden & Zhang, 2010) for comprehensive expositions of non-interference comparisons of the different varieties. Progress-sensitive non-interference is considered the gold standard of non-interference based information flow security (Derakhshan, Balzer, & Yao, 2024).

- *Bipartite non-interference* (Aceto, Gorla, & Lybech, 2024) is defined in terms of two levels, low and high.
- *Nondeterministic non-interference* (NNI) (Focardi & Gorrieri, 1997) is a generalization of classic non-interference to nondeterministic systems.
- *Termination-Insensitive noninterference* (TINI) (Hedin & Sabelfeld, 2012) requires that terminating executions satisfy non-interference.
- *Termination-sensitive noninterference* (TSNI) (Hedin & Sabelfeld, 2012) extends classic non-interference by considering divergence as distinguishable from termination, requiring that the behavior must be consistent for all executions.
- *Progress insensitive non-interference* (PINI) (Bay & Askarov, 2020) generalizes termination-insensitive non-interference to accommodate I/O interactions, where executions agree stepwise on same observable behaviors, permitting silent divergence.
- *Progress sensitive non-interference* (PSNI) (Hedin & Sabelfeld, 2012) demands that executions agree on making stepwise progress and match on the public observables.
- *Deadlock-sensitive non-interference* (DSNI) (van den Heuvel, Derakhshan, & Balzer, 2024) considers deadlocking as an observable behavior, and requires executions agree on deadlocking behavior.

- *Intransitive non-interference* (Roscoe & Goldsmith, 1999) is a relaxed notion of non-interference, where implicit flows may be permitted.

Finally, a note on *non-leakage* and *non-influence* (von Oheimb, 2004). The terms recur in information flow literature, e.g., (Ileri, Zeldovich, Chlipala, & Kaashoek, 2024; Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020); and are related to non-interference, but different. While non-interference restricts information flow *between* security classes, non-leakage is a view-partitioning constraint – it restricts the *parts of system state* a security class is allowed to infer. Non-leakage considers the same trace over two states (instead of two traces over same states). The expectation is that executing the same trace from the two states should result in indistinguishable states (Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020). Finally, non-influence is the combination of non-interference (as defined in (Rushby, 1992)) and non-leakage.

1.2.5.5 Non-Interference With Security Types

In the context of programming languages, a program is non-interfering if secret (high) inputs do not affect the calculation of public (low) outputs (Sabelfeld & Myers, 2003). The idea to use programming languages to model non-interference was introduced by Volpano, Irvine, and Smith (1996). The paper shows that non-interference can be soundly approximated using a security type system. The influential result has since then been followed by numerous works in language-based security. Among a few examples, non-interference has been studied in terms of static program analysis (Barthe, Pichardie, & Rezk, 2007; Huang, Dong, & Milanova, 2014), secure compilation (Cecchetti, Myers, & Arden, 2017; Myers, 1999; Patrignani & Garg, 2017), formal methods (Kammüller, 2008; Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020), program logics (Beringer & Hofmann, 2007; Frumin, Krebbers, & Birkedal, 2021; Garg & Pfenning, 2006; Karbyshev, Svendsen,

Askarov, & Birkedal, 2018), verification (Eilers, Dardinier, & Müller, 2023), and software testing (Hritcu et al., 2013). In general, evaluating non-interference over at different levels of abstractions (compilation, instruction set architecture, processor, hardware, digital circuits) is necessary, because even if a program is non-interfering w.r.t. its syntax, it is not protected against vulnerabilities occurring at lower abstraction levels (Piessens, 2024).

To develop intuition of programming languages based non-interference, the rest of this subsection shows examples using a security type system adapted from Sabelfeld and Myers (2003).²⁹

Programs, non-interference, and typing rules. For a programming language, we consider the grammar shown in Fig. 13. A program (command) in this language is denoted by C . Computation starts in an input state with high and low variables $s = (s_h, s_l)$. Either C terminates in an output state with final values $s' = (s'_h, s'_l)$, or diverges, which we denote by $\perp$. Letting S be the set of input states, $s \in S$, the semantics $\llbracket C \rrbracket$ of C is defined by a function $\llbracket C \rrbracket : S \rightarrow S_\perp$ where $S_\perp = S \cup \{\perp\}$ and $\perp \notin S$. The variation of high input is an equivalence relation $=_L$. Two inputs are equivalent if they agree on low values, i.e., $s =_L s'$ iff $s_l = s'_l$. The attacker's observational power is characterized by a relation on behaviors defined by the semantics, denoted $\approx_L$. Two behaviors are related by $\approx_L$ iff they are indistinguishable to the attacker, i.e., from the low security class viewpoint of the system. Formally, program C is secure w.r.t. non-interference iff

$$\forall s_1, s_2, \in S \cdot s_1 =_L s_2 \implies \llbracket C \rrbracket_{s_1} \approx_L \llbracket C \rrbracket_{s_2}$$

²⁹As per (Sabelfeld & Myers, 2003, p. 9, §B) The system is equivalent to Volpano, Irvine, and Smith (1996), but more conventional in style.

$$C ::= \text{skip} \mid \mathbf{v} := \mathbf{e} \mid C_1; C_2 \mid \text{if } \mathbf{e} \text{ then } C_1 \text{ else } C_2 \mid \text{while } \mathbf{e} \text{ do } C$$

Figure 13: Grammar of the non-interference security type system.

$\vdash \mathbf{e} : \tau$	<i>(expression typing)</i>			
$\frac{\text{E-HIGH}}{\vdash \mathbf{e} : \mathbf{h}}$ $\frac{\text{E-LOW} \quad \mathbf{h} \notin \text{vars}(\mathbf{e})}{\vdash \mathbf{e} : \mathbf{l}}$				
$\Gamma \vdash C$	<i>(command typing)</i>			
$\frac{\text{C-SKIP}}{\Gamma \vdash \text{skip}}$ $\frac{\text{C-ASSIGN1} \quad \vdash \mathbf{v} : \mathbf{h}}{\Gamma \vdash \mathbf{v} := \mathbf{e}}$ $\frac{\text{C-ASSIGN2} \quad \vdash \mathbf{e} : \mathbf{l}}{\mathbf{l} \vdash \mathbf{v} := \mathbf{e}}$ $\frac{\text{C-SEQ} \quad \Gamma \vdash C_1 \quad \Gamma \vdash C_2}{\Gamma \vdash C_1; C_2}$				
$\frac{\text{C-WHILE} \quad \vdash \mathbf{e} : \Gamma \quad \Gamma \vdash C}{\Gamma \vdash \text{while } \mathbf{e} \text{ do } C}$ $\frac{\text{C-IF} \quad \vdash \mathbf{e} : \Gamma \quad \Gamma \vdash C_1 \quad \Gamma \vdash C_2}{\Gamma \vdash \text{if } \mathbf{e} \text{ then } C_1 \text{ else } C_2}$ $\frac{\text{C-SUB} \quad \mathbf{h} \vdash C}{\mathbf{l} \vdash C}$				

Figure 14: A non-interference security type system.

which reads:

If two input states share the same low values, then the behaviors of the program executed on these states are indistinguishable by the attacker.

The security typing rules are shown in Fig. 14. For the typing rules, an expression $\vdash \mathbf{e} : \tau$ means $\mathbf{e}$ has type τ . The judgement $\Gamma \vdash C$ means the program C is typable in security context Γ . Expression types and security contexts can be either low (l) or high (h). The symbol Γ refers to the current context (low or high).

Derivations. Consider the programs in Figures 38–40. Since the programs are written in Java, the constructs are richer than available in the type system grammar. The programs are not fully analyzable. For example, the security type system does not specify how to handle the array in Listing 39. However, we can analyze sub-programs, like the `while` loops in Listing 40 through a mapping to the grammar. The following examples show basic non-interference derivations.

Example 15 (Secure high conditional incremental leak). *We will analyze the following program fragment.*

```
while (h > 0) { h--; }
```

Java

We assume `h` is an arbitrarily typed variable—neither low or high—and will analyze both cases. We map the program to the grammar by substituting `h > 0` with e_0 and `h--` with $v := e_1$ to obtain `while( $e_0$ ) {  $v := e_1$  }`.³⁰

Case 1: e_0 , v , and e_1 are of type high. Since the program is derivable as follows, it is non-interfering.

$$\frac{\frac{}{\vdash e_0 : \mathbf{h}} \quad E\text{-HIGH} \quad \frac{}{\mathbf{h} \vdash v := e_1} \quad C\text{-ASSIGN1}}{\mathbf{h} \vdash \text{while}(e_0) \{ v := e_1 \}} \quad C\text{-WHILE}$$

Case 2: e_0 , v , and e_1 are of type low. The program is also derivable in low context, and again, is non-interfering.

³⁰We will lose detail of the compound assignment, but the loss is acceptable for this example.

$$\begin{array}{c}
\frac{\mathbf{h} \notin \text{vars}(e_0)}{\vdash e_0 : \mathbf{l}} \text{E-Low} \quad \frac{\mathbf{h} \notin \text{vars}(v := e_1)}{\vdash v := e_1 : \mathbf{l}} \text{E-Low} \\
\frac{\vdash e_0 : \mathbf{l} \quad \vdash v := e_1 : \mathbf{l}}{\vdash v := e_1} \text{C-ASSIGN2} \\
\frac{\vdash e_0 : \mathbf{l} \quad \vdash v := e_1}{\mathbf{l} \vdash \text{while}(e_0) \{ v := e_1 \}} \text{C-WHILE}
\end{array}$$

Intuitively, the program is always non-interfering because it processes only high (secret) values, or low (public) values; never mixing the two security classes. $\square$

Example 16 (Insecure high conditional incremental leak). We analyze the following program. It is labeled insecure in the benchmark suite.

```
while (h > 0) { h--; l++; }
```

Java

The goal of this example is to reveal the assumptions that make it insecure. Using a similar procedure as in Ex. 15, we express the program in the type system grammar as

$$\text{while}(e_0) \{ v_1 := e_1; v_2 := e_2 \}$$

To maintain the original information flow behavior, expressions e_0 , v_1 , e_1 (resp. v_2 and e_2) must be typable in the same security class. We also know from Ex. 15 that without multiple security classes, the program is non-interfering. There are two interesting cases.

Case 1: e_0, v_1, e_1 are high, and v_2, e_2 are low. The derivation beings as follows.

$$\begin{array}{c}
\frac{\vdash e_0 : \mathbf{h}}{\vdash e_0 : \mathbf{h}} \text{E-HIGH} \quad \frac{\vdash v_1 := e_1}{\vdash v_1 := e_1} \text{C-ASSIGN1} \quad \frac{\textcircled{x} \vdash v_2 := e_2}{\textcircled{x} \vdash v_2 := e_2} \\
\frac{\vdash e_0 : \mathbf{h} \quad \vdash v_1 := e_1 \quad \textcircled{x} \vdash v_2 := e_2}{\mathbf{h} \vdash v_1 := e_1; v_2 := e_2} \text{C-SEQ} \\
\frac{\mathbf{h} \vdash v_1 := e_1; v_2 := e_2}{\mathbf{h} \vdash \text{while}(e_0) \{ v_1 := e_1; v_2 := e_2 \}} \text{C-WHILE}
\end{array}$$

Since v_2 is of type low, there is no way to handle $v_2 := e_2$. The derivation gets stuck at point marked $\textcircled{x}$ because the program has a non-interference violation. Intuitively, it is because the program operates on a low variable inside a loop with a guard variable that is of type high. This construction is not safe because it leaks information.

Case: e_0, v_1, e_1 are low and v_2, e_2 are of type high.

$$\begin{array}{c}
\frac{\mathbf{h} \notin \text{vars}(e_0)}{\vdash e_0 : \mathbf{l}} \text{E-Low} \quad \frac{\mathbf{h} \notin \text{vars}(e_1)}{\vdash e_1 : \mathbf{l}} \text{E-Low} \quad \frac{}{\mathbf{h} \vdash v_2 := e_2} \text{C-ASSIGN1} \\
\frac{}{\mathbf{l} \vdash v_1 := e_1} \text{C-ASSIGN2} \quad \frac{}{\mathbf{l} \vdash v_2 := e_2} \text{C-SUB} \\
\frac{}{\mathbf{l} \vdash v_1 := e_1; v_2 := e_2} \text{C-SEQ} \\
\frac{}{\mathbf{l} \vdash \text{while}(e_0) \{ v_1 := e_1; v_2 := e_2 \}} \text{C-WHILE}
\end{array}$$

Reversing the security label configuration makes the program typable. A critical rule is *C-SUB* (subsumption) that permits lowering the security class label of a high command. Unlike in the previous case, the loop guard is now low type. It permits manipulating a variable of high type inside the loop body. $\square$

1.2.5.6 Security Meets Implicit Computational Complexity

With a common focus on restrictions, it seems intuitive that techniques from implicit computational complexity would pair elegantly with information flow and applications in language-based security. The combination can potentially provide two types of correctness guarantees, relating resource analysis and security. Supporting this hypothesis, there are already multiple series of work exploring connections between the two domains. This section gives a brief summary of those results.

Complexity via non-interference: SAFE programs. The class of SAFE programs is foundational in demonstrating that non-interference security type systems can support implicit characterizations of complexity classes. The idea was introduced by Jean-Yves Marion in (Marion, 2011). SAFE programs continue to be actively investigated by Marion, Emmanuel Hainry, Romain Pécoux, and others.

Starting with the principle of *tiering*,³¹ Marion extended the principle to characterize the class of polynomial time computable functions. A critical step in obtaining this result was combining tiering with a type system for secure information flow. Each variable is assigned a type – a tier of either 0 or 1 – representing elements of a complexity lattice. The paper then shows that terminating and well-typed programs are computable in polynomial time. SAFE programs is precisely the class of programs captured by this characterization.

Judging by the number of works that followed, the technique was inspirational. The subsequent results have provided extensions mainly along two directions. The first has focused on programming language paradigms, including: concurrent fork-join processes (SAFE processes) (Hainry, Marion, & Péchoux, 2013), dynamic data structure (ramified programs) (Leivant & Marion, 2013), and object-oriented (OO) programs (Hainry & Péchoux, 2015). The OOP result is the first characterization of polynomial time computable functions in the object oriented paradigm. The second directions has focused on redefining the system restrictions, with the goal of improving intensional completeness. For example, the class of stratified programs (Hainry & Péchoux, 2023) characterizes a strictly larger class than SAFE programs, by using a combination of security type system, heap memory restriction, and shape analysis. With the same end-goal, aperiodic programs (AP) (Hainry, Kapron, Marion, & Péchoux, 2024) extends SAFE programs by using declassification; another mechanism adopted from the security domain. In the latter, the main result is a proof that SAFE and terminating aperiodic programs is the class of Basic Feasible Functionals, i.e., $\llbracket \text{SAFE} \cap \text{AP} \cap \text{terminating} = \text{BFF} \rrbracket$ (Hainry, Kapron, Marion, & Péchoux, 2024; Hainry, Kapron, Marion, & Péchoux, 2020).

³¹The intuition behind tiering, a.k.a. ramification, is that program execution time depends on the nature of information flow during executions. The flow can be constrained by imposing a precedence relation that regulates the information flow, e.g., from higher tier to lower tier (Leivant & Marion, 1995, 2013).

From complexity to cryptography. A separate series of works applies implicit computation complexity toward applications in cryptography. The application is natural, because the security of cryptographic schemes is formulated as a mathematical problem, where the goal is to show the scheme is not to be solvable by a feasible adversary. The term feasible refers to computable in polynomial time (Férée, Hym, Mayero, Moyen, & Nowak, 2018).

This application is naturally, because in showing that a cryptographic scheme is secure, adversarial capabilities must restrict available computational power (Heraud & Nowak, 2011).

Formalized safe recursion. The formalization of safe recursion was motivated by its envisioned usefulness in cryptographic proofs (Heraud & Nowak, 2011). At the time of its appearance, the technique was integrated into the Certicrypt proof assistant, for cryptographic proofs (Barthe, Grégoire, & Zanella Béguelin, 2009). However, Certicrypt has since then been superseded by EasyCrypt (Barthe, Dupressoir, Grégoire, Kunz, Schmidt, & Strub, 2014). The support for complexity proofs in EasyCrypt is limited (Barbosa, Barthe, Grégoire, Koutsos, & Strub, 2023). Thus, the integration of safe recursion did not survive the “test of time”.

Type system $d\ell T$. The type system $d\ell T$ (Baillot, Barthe, & Dal Lago, 2015, 2019) is simultaneously complexity-aware and has capabilities to support cryptographic proofs. Among its applications, $d\ell T$ allows proving that the constructed adversary for the Goldreich-Levin Theorem is polynomial time. Such applications are interesting because they involve computations that are inherently non-polynomial in time complexity. Because ICC systems are traditionally designed to characterize at most polytime computations, handling sub-computations that exceed this class requires unconventional flexibility from an ICC system. A limitation of $d\ell T$ is that it has not been implemented.

Both $d\ell T$ and the formalization of safe recursion have inspired other mechanized proofs (Barbosa, Barthe, Grégoire, Koutsos, & Strub, 2021; Férée, Hym, Mayero, Moyen, & Nowak, 2018).

1.2.5.7 From Quasi-Invariant Chunks to Non-Interference

Reinforcing the existing connections, §3.2 presents a program logic for non-interference. The logic is inspired by implicit computational complexity. The motivating idea was to explore the power and usefulness of ICC techniques in language-based security.

The logic analyzes information flows in imperative programs. It abstracts information flow patterns between variables into matrices. Once the matrix is paired with a security policy, it is possible to evaluate whether the program satisfies an *anytime non-interference* property, which is introduced in the paper.

The technique is a refinement of prior works (refer to §2.2 and (Moyen, Rubiano, & Seiller, 2017a)). A previous version of the logic was used to obtain a compile-time program optimization (in §2.2). Although the logic no longer guarantees complexity bounds, it demonstrates the benefit of connecting techniques between the ICC and security.

Adjusting the logic to track non-interference revealed various mathematical insights. For example, the treatment of loops is more straightforward for non-interference than complexity analysis, because the latter requires fixed point computations. As a data flow analysis, that tracks different program properties based on *dependence*, the QI framework is reminiscent of the Dependency Core Calculus (DCC) (Abadi, Banerjee, Heintze, & Riecke, 1999). The multiplicity of the program logic suggests it may possess representational capabilities similar to DCC.

1.2.6 Programs Proofs for Formal Reasoning

The size and complexity of modern software makes it almost impossible to avoid implementation errors. Asynchronous computation, the rich array of computing environments and their communication, and continuous modifications of source code are among the challenges that can lead to software “going wrong”.

Prominent and costly examples of failures include the Ariane 5 rocket explosion on its maiden flight (Ariane 501 Inquiry Board, 1996), the Therac-25 radiation therapy machine with six instances of lethal overdoses (Leveson & Turner, 1993); and the loss of NASA’s Mars Climate Orbiter (Mars Climate Orbiter Mishap Investigation Board, 1999). In more recent memory, the CrowdStrike incident caused multiple days of system outages across critical infrastructure like airports, banks, and hospitals (Wikipedia contributors, 2024). In a case involving the University System of Georgia (USG) (2024), a software-related failure exposed sensitive personal information of USG employees with potential harm to the Augusta University community.

Unreliability of software is not isolated to these few publicized instances. A pithy comment in Dijkstra (1970, p. 3) captures a similar sentiment.

Present-day computers are amazing pieces of equipment, but most amazing of all are the uncertain grounds on account of which we attach any validity to their output.

The context of Dijkstra’s statement is modest compared to exploding rockets. It refers to the operation of multiplying two 27-bit integers. Checking that a calculator actually performs the correct operation for all inputs—i.e., meets its *specification*—at a rate of few microseconds per input-pair, would require more than 10,000 years. Thus, skepticism around soft-

ware correctness is a long-held position among some computer scientists. It is also the core issue tackled by formal methods.

Formal methods motivations and challenges. Formal methods aims to remove uncertainty by providing techniques that allow establishing strong behavioral guarantees. Formal methods is deeply rooted in mathematics and logic (Shankar, 2023). In this view, programs are defined as precise mathematical *models* with specifications. The specification defines what behavior is expected from the model under study. Formally *verifying* a program involves constructing a *proof* that the model satisfies its specification. In contrast to manual inspection and tests—like a test of inspecting the multiplication result of *some* input-pairs—a proof conclusively ensures correctness for *all* inputs, at the model’s level of abstraction. As noted later by Dijkstra (1972), “[t]he only effective way to raise the confidence level of a program significantly is to give a convincing proof of its correctness.”

The motivation for using formal methods is based on the strength of guarantees it provides. Being formal helps when the goal is to be precise (Leino, 2023). However, thinking formally requires a different engineering mindset. Instead of thinking of all possible scenarios and how they might go wrong, formal reasoning requires defining how a system is *expected* to work, and identifying conditions that must be met to ensure the correct behavior (Cook, 2024). Those conditions are then formally verified with proofs.

Despite the clear benefits, wider adoption of formal methods continues to face many challenges. These include e.g., resource investment (crafting and maintaining formal proofs is time-consuming), tooling and tool maintenance, and technical education (ter Beek et al., 2024). Another challenge concerns the *methods* themselves. The Four Color theorem is the first famous result with a proof that requires large computer calculations. When the proof appeared in 2008, such proofs were still controversial. The source of the controversy was that computer programs could not be reviewed with mathematical rigor (Gonthier, 2008).

Thus, advancing formal methods also concerns improving communication about the available capabilities and offered guarantees.

We can envision [...] a world in which computer programs are always the most reliable components of any system or device (Hoare, Misra, Leavens, & Shankar, 2021).

1.2.6.1 Foundational Concepts

Formal models. A formal model is an abstract description of a computer system of interest. In the context of this dissertation, the systems of interest are programs. Then, a formal model is used to reason about the program and the consequences of design choices relating to the program (Ölveczky, 2017; Zave & Nelson, 2023a). The formal model is expressed in (some) modeling language (refer to Table 8 for examples of modeling languages). The choice modeling formalism should enable expressing the model as naturally and intuitively as possible, while still generating mathematically precise structures (Ölveczky, 2017; ter Beek et al., 2024). The formalism should (i) omit unnecessary technicalities, (ii) enable focusing on the problem of interest, at an appropriate level of abstraction, and (iii) facilitate analysis and reasoning about the program (Ölveczky, 2017).

Specifications. A *specification* is a part of the formal model that describes the *behavior* of the program (Zave & Nelson, 2023b). The specification should be simpler and easier to comprehend than the implementation (Zave & Nelson, 2023b). Temporally, the development of the specification should precede the implementation, or minimally be developed concurrently (Dijkstra, 1972). Critically, we want the specification to be *independent* from the implementation (Furia, 2014). Independence isolates the mathematical properties of interest and enables showing that the implementation is consistent with those properties. In

absence of independence, a formal correctness claim is compromised by the *Assertion Inference Paradox* (Furia, 2014). The paradox is essentially, “if the specification is inferred from the implementation, what do we prove?”

Hoare triples (Hoare, 1969) are one standard way to express specifications. In this view, we consider S to be some executable statement (a program). We then define a *precondition* P that describes conditions that hold before execution. A *postcondition* Q describes conditions that hold after execution. Then, a *Hoare triple* is a predicate that connects the statement with the pre- and postconditions by $\{P\}S\{Q\}$. The expression means, “starting from P , statement S will not crash and terminates in a state satisfying Q ”. For example, $\{x = 0\} x++ \{x = 1\}$ is a valid and provable Hoare triple.³²

Invariants. Proving that an implementation meets its specification requires more information than what is readily available in the program syntax (Chang, Rustan, & Leino, 2005). For example, proving a Hoare triple of a loop command often requires establishing additional verification conditions. An *invariant* is an assertion that is preserved by the program execution (Furia, Meyer, & Velder, 2014). An invariant is *inductive*, if it holds the first time a program location is reached and is preserved in every cycle returning to that location (Sankaranarayanan, Sipma, & Manna, 2004). There are many kinds of invariants, e.g., object, class, and loop invariants (refer to Ex. 17). Although invariants are useful for formal verification of programs, the process of discovering them is among the most difficult problems in verification (Dillig, Dillig, Li, & McMillan, 2013; Yu, Wang, & Wang, 2023).

Example 17 (Loop invariant). A loop invariant is an expression that holds on loop entry and after every loop iteration. The program in Listing 41 has two variables, i and n .

³²The mechanics of *how to prove it* is a topic for §1.2.6.5.

```
1 int i = 0; while (i < n) i++;
```

C*

Listing 41: Uncovering loop invariants.

At loop entry, the value of i is 0. If the value of n is negative, the loop never iterates. Otherwise, i will hold a value between 0 and n . Based on this analysis, the loop invariant is $0 \leq i \leq n \vee n < 0$. A postcondition of i is $i = 0 \vee i = n$. A loop invariant is a weakened form of its postcondition (Furia & Meyer, 2010). $\square$

Finally, a brief note about invariants and resource analysis. It is possible to infer resource bounds, of numerical loops, based on loop invariants (Nguyen, Antonopoulos, Ruef, & Hicks, 2017). However, the solution relies on dynamic techniques, that are beyond the capabilities of pure static program analysis.

Properties. Formal verification aims to guarantee that certain behavioral attributes, i.e., *properties*, hold for a program. A few examples of such properties are: memory safety,³³ absence of overflows, termination, data race freedom (Georges et al., 2025), deadlock freedom (Roscoe & Dathi, 1987), secure information flow (cf §1.2.5.1), staying within available resource bounds, and worst-case execution time (WCET). For example, showing that a program is free of overflows requires expressing overflow-freedom as a formal specification, then proving that the implementation satisfies the specification.

There are various different ways to categorize families of properties. *Functional* properties are technical characteristics that contribute to computation of the correct output. *Non-functional* properties are concerned with quality attributes like security, dependability, reliability, and resource requirements (ter Beek, Cleophas, Schaefer, & Watson, 2018). Two

³³I.e., absence of errors related to memory accesses: null-pointer dereferencing, access to unallocated memory, dangling pointers, out-of-bounds accesses, double-free, etc. (Müller, 2024)

other common terms are *safety* and *liveness*. They come from a paper by Lamport (1977) where they are formulated as follows. A *safety property* is a guarantee that “something (bad) will not happen”. A *liveness property* ensures “(good) things must happen”, eventually. The proof techniques across these safety and liveness properties are different. The works in this dissertation are primarily concerned with non-functional safety properties.

Partial and total correctness. Broadly, a program is correct if the implementation is consistent with the specification (Furia, 2014). However, when making assertions about correctness, we need to be more precise. There are two common notions that provide added specificity: partial and total correctness. *Partial correctness* means that every terminating execution is correct, however it does not ensure every execution terminates (Leino, 2023, p. 64). It is possible to obtain a stronger guarantee by proving total correctness. A program is *totally correct* if it is partially correct and always terminates (Leino, 2023, p. 64). Total correctness can be viewed as a combination of safety and liveness properties (Lamport, 1977). There are several strategies for proving termination in some cases, (cf. §1.2.2.3). However, a general method that proves termination of all programs cannot exist since termination is undecidable (Turing, 1936). This means it is impossible to show total correctness of all programs.³⁴ However, similar to §1.2.2.1, there is endless opportunity in building increasingly more capable program verifiers.

1.2.6.2 Tools for Formal Verification

Formal verification is inherently concerned with *computer-aided* reasoning. Software tools are essential in this pursuit. The tools include solvers, theorem provers, model checkers,

³⁴Total correctness is not always ideal. There are many applications where termination is undesirable—like a web server and cardiac pacemaker—but where we still want formal guarantees.

verifiers, etc. Each tool family has different use cases, degree of automation, and guarantees – as illustrated in Fig. 15, adapted from Leroy (2018).

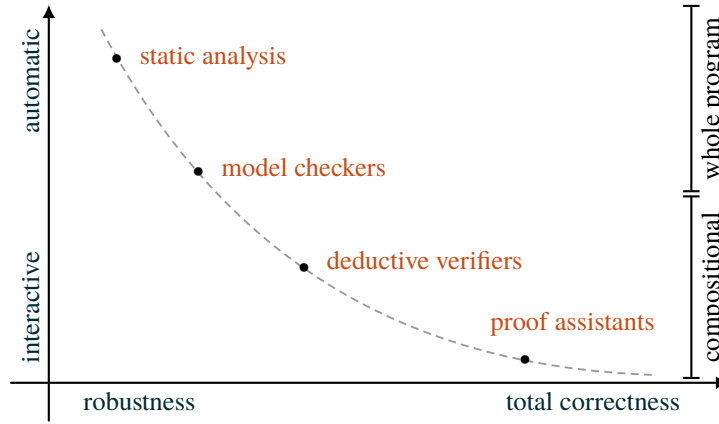


Figure 15: Formal verification approaches.

Table 8 shows a collection of formal methods tools. Interactive theorem provers and verification-aware programming languages are of special interest to this dissertation.

Mechanized proofs. One of the main goals of formal methods is to produce mechanized, i.e., machine-checked, proofs. A mechanized proof “detaches” proof-checking from the activity of crafting the proof. Checking a mechanized proof requires reviewing only the interface of the program, i.e., the statement of the theorem. The rest is self-checking and deferred to a theorem prover. This means everyone with sufficient technical abilities can check the proof once a proof exists (provided they trust the theorem prover). Mechanized proofs are sometimes compared to *paper proofs*, i.e., informal proofs written in natural language (whether actually on paper or written in a text editor). Since paper proof rely solely on the readers abilities to comprehend and confirm the detailed proof steps, mechanized proof are functionally distinct and provide more rigorous correctness guarantees (Gonthier, 2008).

Theorem provers. Mechanized (formal) proofs are created with theorem provers. There are broadly two categories of provers.

Automated theorem provers & solvers

Alt-Ergo, Bitwuzla, cvc5, Cyclist, dReal, E, iProver, MathSAT, PicoSAT, Princess, Prover9, Satallax, SPASS, Vampire, veriT, Yices, Z3

Interactive theorem proving

Agda, EasyCrypt, F*, HOL Light, Iris, Isabelle, Lean, Metamath, Mizar, PVS, Rocq, SASyLF, Tinker

Invariant synthesis

Code2Inv, Daikon, DIG, EvoSpex, FreqHorn, Prinsys

Language-based verification & modeling languages

Boogie, Caesar, Dafny, Eiffel, GRASShopper, K, Maude, Rebeca, SPARK, TLA+, VerCors, Viper, Whiley, Why3

Language extensions adding verification features

ACL2, Atelier B, Cameleer, Flux, Frama-C, Gobra, KeY, LiquidHaskell, Nagini, OpenJML, Prusti, Stainless, VeriFast

(Mixed Integer) Linear programming (MILP/LP) & optimization

Clp, CPLEX, CP-SAT, cuPDLP-C, Glop, GLPK, Gurobi, HiGHS, Sat4j

Model checking

Alloy, Attestor, CIVL, CPAchecker, CProver, Eldarica, JKind, Nidhugg, PRISM, RMC, Sally, TLC, UPPAAL

Symbolic execution

CIVL, CREST, CrossHair, Gillian, KLEE, Kontrol, owi, Symbolic JPF

Table 8: A collection of formal methods tools.

- An *automatic theorem prover* (ATP) aims to find proofs without assistance from a user. The proof *search* problem is challenging. For example, for first-order logic, the problem is NP-complete (Cook, 1971; Levin, 1973)). However, the solvers must be practically efficient to support various applications of automated theorem proving. Prominent ATPs compete bi-annually on solving the “Thousands of Problems for Theorem Provers” (TPTP problems) (Sutcliffe, 2024) at the CADE ATP System Competition (CASC) – the world championship for automated theorem proving (TPTP.org, 2025).
- An *interactive theorem prover* (ITP), a.k.a. *interactive proof assistant*, is a programming environment that helps users write mechanized proofs. Proof assistants vary in underlying logics and proof style. To provide some metric of comparability, there is a ranking for tracking the formalization extent of 100 well-known mathematical theorems across proof assistants. As of April 2025, the top 3 proof assistants with highest coverage of the problems are Isabelle (92%), HOL Light (89%), and Rocq (79%) (Wiedijk, 2025).

Automatic theorem provers search for proofs of isolated problems and frequently serve as building block of other verification tools. Interactive theorem provers enable large and complex modular proof developments.

Verification-aware programming languages. Verification-aware programming languages provide a conceptually different approach to mechanized proofs. Such languages are particularly well-suited for reasoning about functional correctness of programs. Verification-aware programming languages have built-in capabilities to express specifications and proofs. Additionally, they have an associated verifier, that checks the proofs during program development. The verifier is typically a background ATP, that offers an interaction interface via a programming language (Leino, 2023). One prominent style among the verification-aware languages is *deductive*

verification, where the verification process is based on some form of logical inference, i.e., deduction. The properties to be proven are expressed in a formal specification language, as structured comments, next to the program constructs they relate to (Hähnle & Huisman, 2019). A deductive verifier then aims at to prove that all possible behaviors of a program satisfy the specification using the available program annotations (Cassez, Fuller, & Quiles, 2022).

The remainder of this section briefly introduces two tools for formal methods: the (interactive) Rocq theorem prover (§1.2.6.3) and the verification-aware programming language Dafny (§1.2.6.4). These tools are relevant for some of the dissertation manuscripts and assumed to be familiar to the reader.

1.2.6.3 The Rocq Theorem Prover

The Rocq theorem prover is an interactive proof assistant for machine-checked formal reasoning. It provides a rich and flexible environment for various proof developments. Rocq is a widely-adopted tool in the programming languages research community. Among its applications are formal assurance of mathematics, semantics, and verification. In 2013, The ACM has recognized Rocq with the Software System Award – the highest distinction awarded to research software (The Rocq Development Team, 2025a). The success of Rocq has also created abundant interest in dependent type theory (Coquand & P. Huet, 1988), i.e., the core logic of Rocq.

Technical overview. As a guiding example, consider simple proof in Listing 42. The lemma keywords starts a statement that we wish to prove. The specification language for stating theorems is called Gallina. The code between `Proof` and `Qed` is a *proof script*. Proof scripts consists of *tactics*. Tactics perform operations on the *proof state* and make explicit

the steps needed to prove the associate lemma. The Rocq prover comes with dozens of built-in tactics. In the example, `intros`, `simpl` and `reflexivity` are tactics. The tactic language is called Ltac.

```
1 Lemma add_0_l :  $\forall (n : \text{nat}), 0 + n = n.$ 
2 Proof.
3   intros. simpl. reflexivity.
4 Qed.
```

Rocq

Listing 42: A simple proof in Rocq.

Proof engineers write proof scripts incrementally. At each step (marked with a period) it is possible to evaluate the proof and see what goal should be proved next. Refer to Fig. 16 for a visual example. In the editor, the goal (on top right) shows the current proof state.

A full-fledged proof development consists roughly of definitions, lemmas, and proofs. Definitions describe the abstract structures we want to reason about in the mechanization. Lemmas prove facts about the structures.³⁵ Definitions and lemmas provide the specification of a proof development. Formal verification then reduces to discovering the proofs that show that the specification is satisfied. A finished proof is machine-checkable to everyone familiar with these technical basics. A program proved in Rocq can be extracted to other target languages. The functional languages currently available as output targets are OCaml, Haskell and Scheme (The Rocq Development Team, 2025b).

Mathematical Components. The Mathematical Components library—colloquially `mathcomp`—extends the Rocq ecosystem with an extensive collection of formalized mathematical theories. The library covers a wide spectrum of topics, including formal theory

³⁵In Rocq the distinction between a lemma, theorem, corollary, etc., is just syntactic sugar.

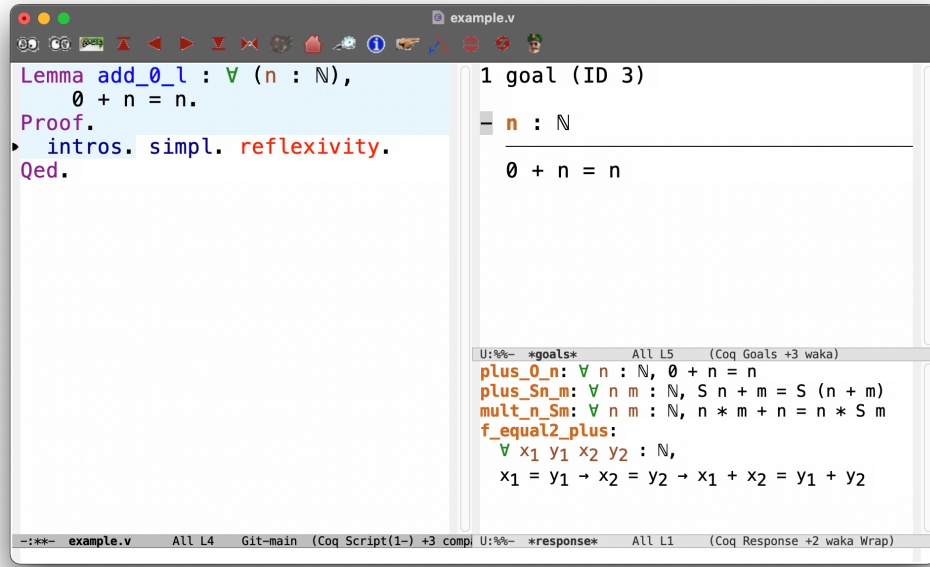


Figure 16: A development view of the simple Rocq proof. The Emacs editor is enhanced with the Proof General interface and the Company-coq plug-in. The left view contains the proof code under development. Focusing between Proof and Qed activates the proof mode. While in proof mode, the proof state is visible in the top-right view. The bottom-right view shows the search results, of a database lookup, for existing lemmas that are similar in form to the current goal.

of general purpose data structures—like lists, prime numbers, and finite graphs—and advanced topics in algebra. The theories are organized into hierarchical levels where the design facilitates reuse. The proof style is based on Rocq, but Mathematical Components uses a specialized language extension called SSReflect (small-scale reflection). SSReflect significantly influences proof writing style and poses its own learning curve. To demonstrate the difference, § 1.2.6.5 shows examples in “standard” Rocq and SSReflect. The Mathematical Components library has an essential role in the Rocq community because several landmark results of finite group theory are based on it. For example, the mechanical proofs of the Four Color theorem (Gonthier, 2008) and the Odd Order theorem (Gonthier et al., 2013) utilize the library extensively.

Demonstrated applications. Conventional uses of Rocq include proving properties of programming languages, formalizing mathematics, and teaching. Table 9 contains representative examples in the first two categories.

Additional resources. There are multiple literature sources for learning more about Rocq. Coq’Art (Bertot & Castéran, 2004) is the first book dedicated to the proof assistant and its theory. The Software Foundations (Pierce et al., 2025) book series is a hands-on, active learning experience. The books are implemented in Rocq and updated regularly. The Mathematical Components book (Mahboubi & Tassi, 2022) provides a guide to using the library and the SSReflect proof language. Karate Coq of Affeldt (2025) is an additional resource about Mathematical Components. There are multiple books about engineering Rocq proofs with dependent types and type systems (Chlipala, 2013, 2022; Sergey, 2014; Smolka, 2025). For a curated list of awesome Coq libraries, plugins, tools, and resources see the *Awesome Coq* list (The Rocq Community, 2025). Finally, the Rocq prover has an active community, with discussion forums and a mailing list (The Rocq Development Team, 2025a).

1.2.6.4 Dafny: The Verification Aware Programming Language

Dafny is an open source verification-aware programming language, and a verifier for functional correctness (Dafny Language Developers, 2025; Leino, 2010a). Dafny was designed for reasoning, and it can automatically check programs against specifications during program development. Fig. 17 shows a visual overview of the verification workflow.

Technical overview. Dafny is a hybrid language, influenced by Java and C#. It has both functional and object-oriented features. They include curly-bracket block-style, (mathematical) functions, and inductive and co-inductive data types. The programming constructs

Formalization target	Authors
π -calculus in (Co)inductive-type theory	Honsell, Miculan, & Scagnetto (2001)
Gödel-Rosser Incompleteness Theorem	O’Connor (2005)
Modal model of impredicative semantics	Appel, Melliès, Richards, & Vouillon (2007)
Four Color Theorem ^a	Gonthier (2008)
Verified Optimizing C Compiler, CompCert ^{bc}	Leroy (2009)
Flocq: floating-point numbers	Boldo & Melquiond (2011)
Bedrock: low-level programming library	Chlipala (2011)
Safe Recursion – §1.2.1.3	Heraud & Nowak (2011)
Desargues’s Theorem in projective geometry	Magaud, Narboux, & Schreck (2012)
Feit-Thompson’s Odd Order Theorem	Gonthier et al. (2013)
Coquelicot: Real Analysis	Boldo, Lelay, & Melquiond (2014)
Homotopy Type Theory (HoTT)	Bauer, Gross, Lumsdaine, Shulman, Sozeau, & Spitters (2017)
Regular Language representations	Doczkal & Smolka (2018)
Caspar blockchain finality	Palmskog, Gligoric, Pena, Moore, & Roşu (2018)
Interaction Trees: Recursive and impure programs	Xia et al. (2019)
Undecidable Problems	Forster et al. (2020)
SSProve: modular cryptographic proofs	Haselwarter et al. (2023)

^a based on (Robertson, Sanders, Seymour, & Thomas, 1997) and (Appel & Haken, 1989)

^b A verifying compiler is one of the grand challenges for computing research posed in Hoare (2003)

^c CompCert received the ACM Software System Award in 2021.

Table 9: A small sample of results formalized with the Rocq prover.

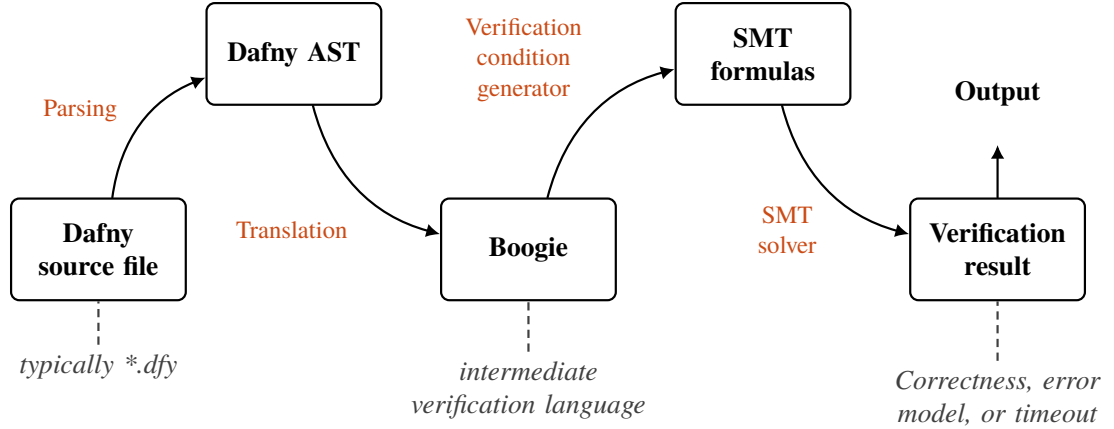


Figure 17: A (simplified) overview of Dafny implementation and verification workflow. “A well-designed language and verifier, plus a great SMT solver, go a long way.” – Leino (2010b).

include loops, `if`-statements, arrays, classes and methods; variables, types, generics, lambdas, inheritance, etc. (Dafny Contributors, 2025) The real power of Dafny comes from the ability to annotate methods to specify their behavior. For verification tasks, the idea is to define specifications and write proofs aside the implementation, in deductive verification-style. The built-in verification constructs include pre- and postconditions; loop invariants, termination metrics, lemmas, and ghost constructs.³⁶ This language design enables writing functionally-correct verified programs fully in Dafny (Leino, 2023). Dafny programs can be compiled to various target languages, like C#, Java, and Go (Dafny Contributors, 2025).

Dafny has an associated static program verifier. The role of the verifier is to check a program’s specification constructs.³⁷ As visualized by the chart in Fig. 18 (inspired by (Leino, 2010b)), the built-in verifier in Dafny adds a high degree of automation. The verifier resolves many low-level proof steps. For example, if we wanted to prove the lemma of Listing 42 in Dafny, the verifier succeeds at discharging the proof obligations automatically – see `AddLeft` in Fig. 19.

³⁶A *ghost* is any construct that is used for verification only; they are erased during compilation (Leino, 2023, p. 19).

³⁷These are Eiffel-like contracts from (Meyer, 1988).

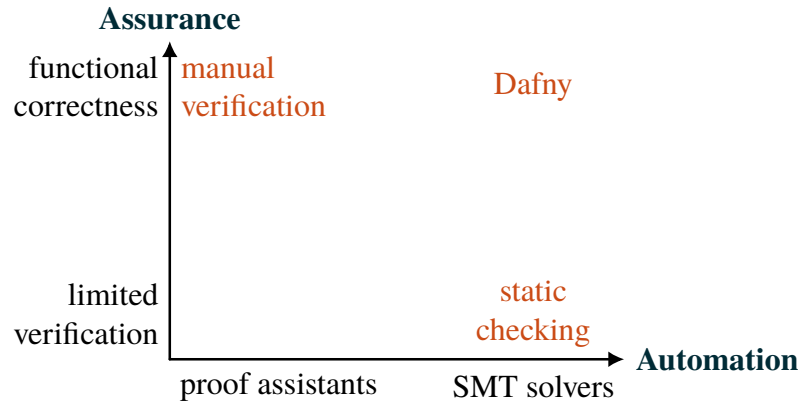
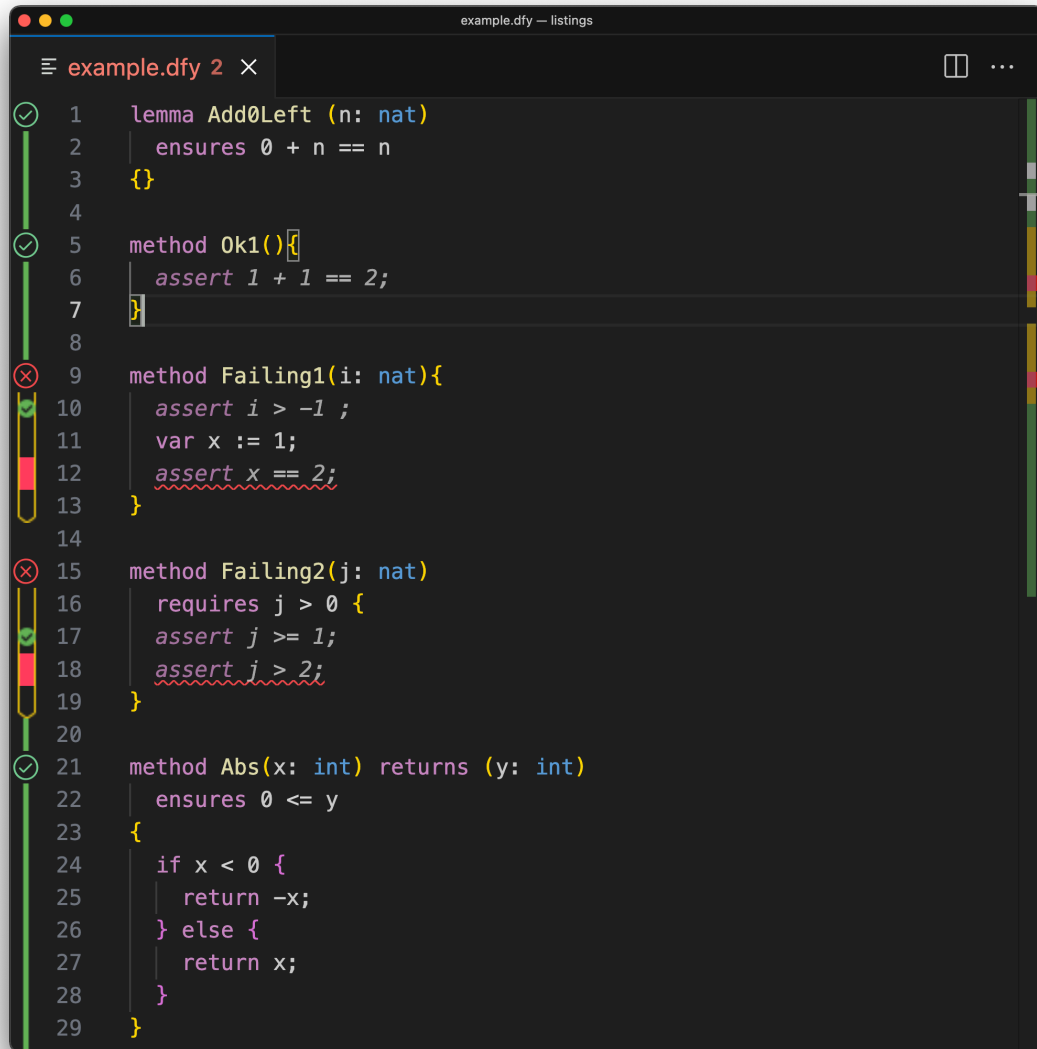


Figure 18: Automation degree.

A program accepted by the Dafny verifier is guaranteed to be totally correct, i.e., it terminates and satisfies its specification. During program development and when given a candidate specification, the verifier runs automatically on code edits, as shown in Fig. 19. The engineer receives immediate feedback on the verification efforts. When Dafny verifier fails, the engineer must annotate the program with additional guidance—assertions, preconditions, lemmas, etc.—to assist the verifier, until the verification succeeds. Logical contradictions, like the ones raised in Fig. 19, will obviously never succeed. The verifier helps to catch such errors early, during program development.

Proof style. In Dafny, proofs are developed in “top-down” style. This allows the developer to focus on the proof architecture before the detailed proof steps. To this end, Dafny provides assume statements, i.e., assertions without a proof; and ghost methods, i.e., lemmas that are not yet proven. Obviously, the proof is not complete until these temporary constructs have been replaced with concrete proofs. However, they are useful for facilitating the proof development.

Boogie. Boogie (Leino, 2008) is an intermediate verification language, used in the verification workflow of Dafny programs (cf. Fig. 17). More generally, Boogie is an open



The image shows a Visual Studio Code window titled 'example.dfy — listings'. The editor displays the following Dafny code:

```
1 lemma Add0Left (n: nat)
2   ensures 0 + n == n
3 {}
4
5 method Ok1(){
6   assert 1 + 1 == 2;
7 }
8
9 method Failing1(i: nat){
10  assert i > -1 ;
11  var x := 1;
12  assert x == 2;
13 }
14
15 method Failing2(j: nat)
16   requires j > 0 {
17   assert j >= 1;
18   assert j > 2;
19 }
20
21 method Abs(x: int) returns (y: int)
22   ensures 0 <= y
23 {
24   if x < 0 {
25     return -x;
26   } else {
27     return x;
28   }
29 }
```

On the left margin, there is a vertical status bar with icons for each line of code. Line 1 has a large green checkmark. Lines 2-3 have a green checkmark. Line 4 has a green checkmark. Line 5 has a green checkmark. Line 6 has a green checkmark. Line 7 has a green checkmark. Line 8 has a green checkmark. Line 9 has a red cross. Line 10 has a green checkmark. Line 11 has a green checkmark. Line 12 has a red block and a squiggly underline. Line 13 has a red block. Line 14 has a green checkmark. Line 15 has a red cross. Line 16 has a green checkmark. Line 17 has a green checkmark. Line 18 has a green checkmark. Line 19 has a red block and a squiggly underline. Line 20 has a green checkmark. Line 21 has a green checkmark. Line 22 has a green checkmark. Line 23 has a green checkmark. Line 24 has a green checkmark. Line 25 has a green checkmark. Line 26 has a green checkmark. Line 27 has a green checkmark. Line 28 has a green checkmark. Line 29 has a green checkmark.

Figure 19: The Dafny verifier checking Dafny code running in Visual Studio Code. A big green checkmark (✓) means the verifier succeeds at proving the corresponding program construct. A big red cross (✗) identifies program constructs that fail to verify completely, even if some inner declarations succeed. A green filled checkmark (✓) means the verifier succeeds at the proof obligation at the corresponding line. The verification fails at declarations marked with a red block (■) and squiggly underline.

source modeling language and a verification tool for sequential and concurrent programs, and distributed systems (Microsoft, 2025). It provides a layer on which to build program verifiers.³⁸ When viewed as a verification tool, Boogie takes as input a program written in the Boogie language. Then, the tool infers invariants, generates verification conditions, and passes them to an SMT solver. The default SMT solver is Z3 (de Moura & Bjørner, 2008).

Applications of Dafny. Dafny is a mature, industry-grade tool for program verification. It has been used in research projects, industrial applications, verification projects, and in teaching formal methods. Among the hallmark results, the AWS authorization engine—that handles 1 billion API calls per second (Wagner & McLaughlin, 2024)—is verified in Dafny (Chakarov et al., 2025).

1.2.6.5 Tool Comparison by Light Examples

This section covers three verification tasks that are suitable for paper presentation while illustrating differences between the formal proof techniques. To match the dissertation theme, the examples are about program proofs. However, proving programs is only a subset of the general utility of these tools. Refer to Table 9 for more cases.

Program Equivalence. Listing 1, in the Introduction Chapter, showed two programs with a claim that the programs are equivalent. Equivalence means that, for all inputs, the programs compute the same result. With manual inspection, one can become reasonably convinced that the claim is true. However, such inspection provides only weak informal guar-

³⁸Several program verifiers are built on Boogie, for example Dafny, Chalice, Spec#, and Move (Microsoft, 2025).

antees. We can do better – we now have the tools to prove formally the claim about equivalence.

```

1 Definition equiv (c1 c2 : com) : Prop :=
2   ∀ (st st' : state),
3   (st =[ c1 ]⇒st') ↔ (st =[ c2 ]⇒st').
4
5 Theorem program_equivalence (b: nat) (X Y : aexp) (Z : string) :
6   b = 0 ∨ b = 1 →
7   equiv <{ if b = 1 then Z := X else Z := Y end }>
8     <{ Z := X * b + Y * (1 - b) }>.
9 Proof.
10  intros; split.
11  (* if ... → Z := ... *)
12  - intros C1; inversion C1; subst; clear C1;
13    inversion H; subst; try discriminate;
14    inversion H6; subst;
15    apply E_Asgn; simpl;
16    rewrite mul_0_r, mul_1_r; auto.
17  (* Z := ... → if ... *)
18  - intros C2; destruct H as [H|H]; subst;
19    [apply E_IfFalse|apply E_IfTrue]; auto;
20    inversion C2; subst;
21    apply E_Asgn; simpl;
22    rewrite mul_0_r, mul_1_r; auto.
23 Qed.

```

Rocq

Listing 43: A Rocq proof of program equivalence (theorem only). The full proof with the programming language syntax, semantics, and notations is about 200 LoC.³⁹

The Rocq proof explained. The Rocq theorem (L5–8) makes explicit that the equivalence holds (at least) when X and Y are arithmetic expressions that evaluate to a $\mathbb{N}$; Z is a variable, and b is a bit with value 0 or 1. The precise programming language semantics are defined as part of the full Rocq proof.

The property of *equivalence* is defined as a proposition (L1–3). Informally, two programs (commands, c_1 and c_2) are equivalent iff they start execution in the same state (st) and reach the same final state (st'), for all states. Although program termination is not guaranteed by this specification, the state-based definition requires that equivalent programs must agree on divergence. That is, if one program diverges, the other must diverge also.

The actual proof steps (L9–23) show how to establish equivalence of the programs in Listing 1. The proof proceeds as follows. First, split the bijection $\leftrightarrow$ in two directions, using the `split` tactic (L10). Next, substitute b with concrete values, which creates additional subcases. Some of the cases are contradictory and resolved easily by the `discriminate` tactic (L13). Using `rewrite`, the multiplication operation is rewritten to simpler form, by algebra. After this, it becomes possible to finish the proof automatically with `auto` (L16, L22).

In absence of technical familiarity, the proof steps may not be easy to comprehend from the tactics. Yet, as a mechanized proof, by compilation, it is possible for everyone to confirm the correctness of the proof. Moreover, it is possible to inspect the different states of the proof script in an IDE.⁴⁰ Thus, a mechanized proof lifts the observer’s need to understand the details of the proof script. The observer must instead “just” understand the stated theorem and trust the proof assistant in checking its correctness.

³⁹The proof has some duplication at L15-16 and L21-22. It is possible to remove it for a shorter proof; however, the longer for is preferable for this presentation.

⁴⁰The proof needs light editing, to reduce automation, to make the intermediate states inspectable.

```

0  type bit = i: nat | i ≤ 1 witness 1
1
2  function Program2 (b : bit, X : nat, Y : nat) : nat {
3      X * b + Y * (1 - b)
4  }
5
6  method Program1 (b : bit, X : nat, Y : nat) returns (Z: nat)
7      ensures Z = Program2(b, X, Y) {
8      if b = 1 { Z := X; }
9      else { Z := Y; }
10 }

```

Dafny

Listing 44: Program equivalence verified in Dafny (complete). The listing includes a subtype definition of `bit` that does not occur in the Rocq version.

The Dafny verification explained. The Dafny version in Listing 44 contains the full details of the proof. To make life *a bit more* interesting, it defines a subtype of `bit`. Alternatively, the restriction on the values of `b` could be stated as a precondition.

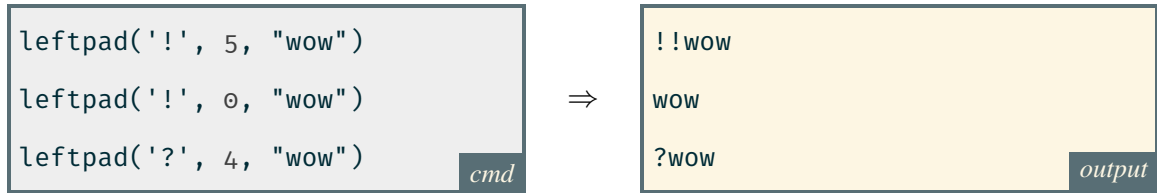
Next, the listing shows a function and a method. A *method* in Dafny is a piece of imperative executable code, which can have all sorts of statements in its body. The term *function* is reserved for constructs that can be used directly in specifications. Unlike a method, a function body must consist of exactly one expression, and it returns a value. For the ways the two programs were originally written and for demonstration, it makes sense to define the programs as a function and a method.⁴¹

Since `Program2` is a function, it can be used in the specification condition of `Program1`. The `ensures` clause says the value computed by `Program1` is exactly equal to that com-

⁴¹Specifying two methods is also possible, the proof will just look different.

puted by Program2, for all arguments. The equivalence holds in both directions. Because the equivalence is generally easy to show, the verifier succeed without additional lemmas. This proves the equivalence claim for the two programs.

Formally verified leftpad. Leftpad is a string manipulation function. It pads a string, using a specified character, up to a specified length. The padding character is applied to the left side of the string, as a prefix, hence the name. Listing 45 shows examples of leftpad behavior.



Listing 45: Leftpad in action.

The “Let’s Prove Leftpad” (Wayne, 2018) is public challenge project initiated by Hillel Wayne. The idea is to put different verification tools and proof techniques to test by using them to prove leftpad. At the time of writing this (April 2025), the repository contained 29 variants of leftpad proofs. The challenge is interesting, because a leftpad proof is small enough to be accessible without formal methods background, yet complex enough to differentiate the proof techniques. The leftpad implementation is simple, but the formal specification is surprisingly tricky. A formal proof requires showing that the following three specification conditions hold.

1. Given pad length n and a string str , the output length is $\max(n, \text{len}(str))$.
2. The output prefix is padding characters and nothing but padding characters.
3. The suffix of the output is the original string str .

Listings Listing 46–49 show the complete implementations and proofs of `leftpad` in Rocq; the SSReflect style of the Mathematical Components library, and Dafny. The proofs come from the “Let’s Prove Leftpad” project repository (Wayne, 2018). The original versions were crafted by Ezra Cooper (Rocq), Anton Trunov (SSReflect), and Hillel Wayne (Dafny). The proofs are modified here for paper presentation and to resolve language updates. The Rocq proof is especially heavily modified from its original form.

The padding character is represented as `c`, the pad length is `n`, and the input “string”⁴² is `s`. Each version includes an implementation and a proof to show correctness.

⁴²None of the proofs actually refer to *string* type, though one uses a sequence of characters.

```

1 From Coq Require Import Arith List. Import Nat.
2
3 Parameter char : Set.
4 Implicit Type (s xs ys: list char).
5
6 Definition allEqual xs y :=  $\forall x, \text{In } x \text{ xs} \rightarrow x = y$ .
7 Definition cutn n xs := (firstn n xs, skipn n xs).
8
9 Lemma cutn_app n:  $\forall$  xs ys,
10   n = length xs  $\rightarrow$  cutn n (xs ++ ys) = (xs, ys).
11 Proof.
12   induction n; destruct xs; simpl; try easy.
13   intros; unfold cutn in *; simpl.
14   lapply (IHn xs ys); auto; congruence.
15 Qed.
16
17 Lemma minus_plus_max m n:  $m - n + n = \max m n$ .
18 Proof.
19   intros; destruct (le_lt_dec m n).
20   - rewrite max_r; auto.
21     replace ( $_ - _$ ) with 0; auto.
22     rewrite  $\leftarrow$ sub_0_le in *; auto.
23   - rewrite max_l, sub_add; auto;
24     erewrite lt_le_incl; auto.
25 Qed.

```

Rocq

Listing 46: Leftpad in Rocq (part 1 of 2).

```

28 Definition leftpad c n s :=
29   repeat c (n - length s) ++ s.
30
31 Theorem leftpad_correctness c n s :
32   length (leftpad c n s) = max n (length s) ∧
33     ∃ m, let (prefix, suffix) := cutn m (leftpad c n s)
34     in allEqual prefix c ∧ suffix = s.
35 Proof.
36   unfold leftpad, allEqual. split.
37   - rewrite length_app, repeat_length; apply minus_plus_max.
38   - ∃ (n-length s); rewrite cutn_app.
39     + apply conj; auto; apply repeat_spec.
40     + erewrite repeat_length; auto.
41 Qed.

```

Rocq

Listing 47: Leftpad in Rocq (part 2 of 2).

Leftpad in Rocq. The Rocq proof of leftpad is shown in Listing 46. The leftpad implementation (at L27–28) works by concatenating $n - \text{length}(s)$ count of symbols c with s . Since the proof compiles successfully, we also know the implementation is guaranteed to terminate. In the proof, the key idea is that it is possible to cut the concatenated string to a prefix and suffix, at index m . The specification is then proved w.r.t. the prefix and suffix, and the concatenated length. Proving the theorem requires introducing the supporting definitions and the lemmas of L6–25.

```

1 From mathcomp Require Import ssreflect ssrbool ssrnat seq.
2
3 Section LeftPad.
4   Variables (T : Type) (def : T).
5   Implicit Types (c : T) (n : nat) (s : seq T).
6   Local Notation nth := (nth def). (* nth element *)
7
8   Definition leftpad c n s : seq T :=
9     ncons (n - size s) c s.
10
11   Lemma length_max_spec c n s :
12     size (leftpad c n s) = maxn n (size s).
13   Proof. by rewrite size_ncons addnC maxnC maxnE. Qed.
14
15   Lemma prefix_spec c n s :  $\forall i$ ,
16      $i < n - \text{size } s \rightarrow \text{nth } (\text{leftpad } c \ n \ s) \ i = c$ .
17   Proof. by move $\Rightarrow$ i; rewrite nth_ncons  $\Rightarrow\Rightarrow$ . Qed.
18
19   Lemma suffix_spec c n s :  $\forall i$ ,
20      $i < \text{size } s \rightarrow$ 
21      $\text{nth } (\text{leftpad } c \ n \ s) \ (n - \text{size } s + i) = \text{nth } s \ i$ .
22   Proof.
23     by move $\Rightarrow$ i _; rewrite nth_ncons addKn ifN//-ltnNge leq_addr.
24   Qed.
25 End LeftPad.

```

SSReflect

Listing 48: Leftpad in the SSReflect proof language of the Mathematical Components library.


```

0 function Max(a: int, b: int): int {
1   if a > b then a else b
2 }
3
4 method LeftPad(c: char, n: int, s: seq<char>)
5   returns (v: seq<char>)
6   ensures |v| = Max(n, |s|)
7   ensures  $\forall i \bullet 0 \leq i < n - |s| \Rightarrow v[i] = c$ 
8   ensures  $\forall i \bullet 0 \leq i < |s| \Rightarrow v[\text{Max}(n - |s|, 0) + i] = s[i]$ 
9 {
10  var pad, i := Max(n - |s|, 0), 0;
11  v := s;
12  while i < pad
13    invariant  $0 \leq i \leq \text{pad}$ 
14    invariant |v| = |s| + i
15    invariant  $\forall j \bullet 0 \leq j < i \Rightarrow v[j] = c$ 
16    invariant  $\forall j \bullet 0 \leq j < |s| \Rightarrow v[i + j] = s[j]$ 
17    decreases pad - i
18  {
19    v := [c] + v;
20    i := i + 1;
21  }
22 }

```

Dafny

Listing 49: Formally verified Leftpad in Dafny.

Leftpad in SSReflect. The Mathematical Components library does not contain definitions of string or character types. Therefore, Listing 48 is generalized to a parametric type T . The leftpad implementation, at L8–9, is functionally the same as in Listing 46, only

expressed in the style of SSReflect. The three lemmas that follow prove the specification. The first lemma `length_max_spec` matches the Rocq version in Listing 46, but the prefix and suffix proofs are achieved differently. The latter rely on a random access function `nth` that returns the i -th element in a sequence (numbered from 0), or the `def` element, if the sequence contains fewer than $i + 1$ elements. The imported libraries provide much assistance in completing the proof. In SSReflect, it is common for simple proofs to rely on `rewrite` tactic, and the “interesting proofs” to appear visually longer. The `leftpad` proof is simple since it requires only rewrites, and those rewrites are defined in the library. The tactics naming convention is a special design feature of the Mathematical Components library. The naming eases locating existing theorems, to avoid searching the proofs database.

Leftpad in Dafny. In Dafny, the specifications are “embedded” next to the implementation. To make the Listing 49 free of imports, the `Max` function is defined inline. The three specification conditions of `Leftpad` correspond (in order) to the `ensures` clauses. The same conditions re-appear as loop invariants (at L14–16). The `invariant` clauses are necessary to convince the verifier that the conditions are preserved over by the loop during iteration. Adding the invariants to the `while` loop is sufficient for the verifier to succeed at proving the implementation’s functional correctness. The final proof obligation is to show loop termination. The `decreases` clause serves this purpose. Thus, the `LeftPad` method is totally correct.

Observable differences. Although the `Leftpad` behavior is consistently the same, the examples show different styles of implementation and varying proof strategies. The Rocq prover is very flexible in supporting various kinds of proofs and provides fine-grained control over the specification. The deductive verification style of Dafny is natural and powerful for proving programs. Much of the power comes from automatic decision procedures. As a disadvantage, the implementation can be challenging to debug if the verification fails. In

general, it is important to understand the mechanics of the selected tool, since a tool choice greatly influences the proof style and available capabilities.

1.2.6.6 The Moscow Problem

The next problem is a more involved and interesting for formal techniques and logic. The problem was presented to me by Natarajan Shankar under the name “the Moscow problem”. Although it is too simple for research, it is also too fascinating to be left in obscurity. Therefore, it appears here, in my dissertation.

Problem Description. Assume there are three people, A, B, and C. Seven natural numbers in 1–7 are distributed randomly among the people. A gets 1 number, and B and C get 3 numbers each. The numbers are distinct, i.e., no two people hold the same number; and the distribution is secret, i.e., it is unknown who holds what numbers. The challenge is to design a message exchange that meets the following requirements.

1. B sends one message to C and from that message, C knows what numbers everyone holds. (And similarly in reverse: C sends one message to B...).
2. A observes the messages but A “does not learning anything”. In other words, A cannot determine who holds what numbers based on the observed messages.

Messages are exchanged in plain text and A is aware of the mechanics of the exchange protocol. The Moscow problem is to design a technique that meets the requirements. The following solution is what I have developed in response to the problem.

Informal Solution. Starting with a claim: it is sufficient to achieve the desired system with a message exchange where the message value is a single digit in 1–7.

The exchange is symmetric. Therefore, instead of referring to B and C, we say *sender* and *receiver*. The sender calculates the exchanged value (the single digit) using only the numbers they hold. The receiver calculates—using the single digit and number they holds—the number *held by A*. Finally, the receiver deduces the numbers held by the sender. There are three important functions.

1. *Send* function that calculates the exchanged digit.
2. *Receive* function that calculates number held by A.
3. *Recover* function that deduces values held by the sender.

The functions are defined as follows. The universe is $U = \{x \mid 1 \leq x \leq 7\}$. Letting X be a set of numbers in U , the numbers “outside” X are defined by the set complement $\overline{X} = U - X$. Letting y be a number in U , we *encode* a value by

$$E(X, y) = \begin{cases} 7, & \text{if } \sum \overline{X} + y \pmod{7} = 0 \\ \sum \overline{X} + y \pmod{7}, & \text{otherwise} \end{cases}$$

Letting the set of numbers held by the *sender* (resp. *receiver*) be X_s (resp. X_r), we define the function *send* as $S(X_s) = E(X_s, 7)$. The send function produces the exchanged message value m that is observable to everyone. The function *receive* is $R(X_r, m) = E(X_r, m)$, and it produces the number held by A, denoted as a . The receiver calculates numbers held by the sender using the *recover* function $\text{Rec}(X_r, a) = U - X_r - a$.

Formal proof. The interesting bit is showing that the solution meets the specification. The solution is provable in the Rocq Theorem Prover. We define first the messaging functions

as shown in Listing 50.

```

1  Definition outside s := [seq x ← U | x ∉ s].
2
3  Definition encode n1 n2 n3 x :=
4    let r := (sumn (outside [:: n1; n2; n3]) + x) %% 7 in
5    if r == 0 then 7 else r.
6
7  Definition send t := encode t.1.1 t.1.2 t.2 7.
8  Definition recv t m := encode t.1.1 t.1.2 t.2 m.
9  Definition recover t n :=
10   T3 [seq x ← outside (L t) | x != recv t n].

```

SSReflect

Listing 50: Implementation of the messaging functions. The t is a triple of natural numbers and T_3 converts an arbitrary-length sequence (back) to a triple.

The message exchange requirements are defined by two lemmas. First, “C (resp. B) learns everything from one message” is stated formally as shown in Listing 51. The lemmas essentially says the receiver (R) recovers (some permutation) of numbers that match exactly those held by the sender (S). The $\text{send } S$ is the value of the exchanged message.

```

1  (* symbol × is a permutation *)
2  Lemma receiver_knows_sender S R :
3    (S = B ∧ R = C) ∨ (S = C ∧ R = B)
4    → [recover R (send S)] × [S].

```

SSReflect

Listing 51: First requirement formally – receiver learns the numbers held by sender.

An *allocation* (type `alloc`) represents a distribution of numbers in U between A, B, and C. The second criterion is that “A does not learn anything”. We formalize this in Listing 52. The lemma says, for every allocation of numbers and after observing both message exchanges between B and C, A has *insufficient information* to determine with certainty what numbers everyone is holding. In other words, any observed exchanges can be explained by at least two different allocations, because there exists an alternate (different) `alloc2` that explains the observations of A.

```

1 Lemma send_ambiguous (alloc1 : alloc):
2   ∃ (alloc2: alloc),
3     (* observed exchange is the same *)
4     alloc1.(A) = alloc2.(A)
5     ∧ send alloc1.(B) = send alloc2.(B)
6     ∧ send alloc1.(C) = send alloc2.(C)
7     (* but allocated numbers are different *)
8     ∧ ~[alloc1.(B)] × [alloc2.(B)].
9     ∧ ~[alloc1.(C)] × [alloc2.(C)].

```

SSReflect

Listing 52: Second requirement formally – A does not learn anything.

A complete machine-checkable implementation and a proof is available in the dissertation artifact (cf. §8 in `code/moscow.v`). To machine-check the proof, we compile the file in a terminal following the instructions of the artifact readme. The expectation is that the proof compiles without errors (the compilation does not produce any terminal output).

Compilation of the proof also runs an extraction routine. The extraction generates a certified OCaml program (`message.ml`) from the messaging functions implemented in SSReflect. Another program—`exchange.ml`, also in the dissertation artifact (cf. Table 30)—is

a driver for running the certified program. The driver uses the certified program to perform a message exchange when run as shown in Listing 53.

```
ocamlbuild exchange.byte && ./exchange.byte
```

cmd

```
1 B sends 4 ==> C got A=2 and B=(5, 1, 4)
2 C sends 5 ==> B got A=2 and C=(7, 3, 6)
```

output

Listing 53: Running a certified message exchange. In this scenario, A holds the number 2, and everyone observes the messages “4” and “5”. This information is not enough for A to learn what numbers B (resp. C) is holding. However, by the end of the exchange, B and C are fully informed.

The presented moscow problem solution could be adjusted to a universe of $[0, 6]$ with a small change. However, it is unknown whether other modifications—e.g., larger set of numbers, different allocations, or more people—could be added.

1.2.6.7 Connecting Formal Methods and ICC

There are several ways to relate complexity theory and formal methods. Since they are founded on programming languages, ICC systems are potentially highly suitable for formal reasoning (cf. Pierce et al. (2024) for a starting point). A particularly strong motivator is that ICC systems yield more natural definitions and proofs of central results than classic complexity theory (Kristiansen, 2017).

Formalization of a theory requires significant time and effort. However, the effort of formalizing ICC results is justified, based on the continued demonstrated relevance of ICC results – cf. Table 3, absence of *formal* proofs among those results, and maturity of formalization techniques and tools.

Existing results in formally verified complexity. There exists a (small) collection of result formalizing classic complexity theory (cf. §3.3.3). For example, Universal Turing Machines (Forster, Kunze, & Wuttke, 2020), the $\mathcal{O}$ -notation (Guéneau, Charguéraud, & Pottier, 2018), and the Cook-Levin Theorem (Gäher & Kunze, 2021)⁴³ have been formalized previously.

Among programming language-based approaches, the type system of (Crary & Weirich, 2000) is notable. The type system assigns a “virtual clock” to a program. Then, a compile-time check aims to verify that the clock does not expire. Although this provides an automatic method for certifying resource usage, the theory is not formalized mechanically. The cost-aware logical framework Calf (Niu, Sterling, Grodin, & Harper, 2022)—and its extension Decalf (Grodin, Niu, Sterling, & Harper, 2024)—is similarly a type system-based approach for functional programs. Calf embeds, thought types, the amortized costs of computation. It also uses clocks, where the clocks record the recursion depth, to ensure termination of programs. Then, if a program can be typed, the Calf system ensures that the program satisfies the specified resource bound. The type system is formalized in Agda (Grodin, Niu, Sterling, & Harper, 2023).

Among works of implicit computational complexity, the hallmark result of Bellantoni and Cook (safe recursion cf. `safe-rec`) was formalized in Heraud and Nowak (2011). The Rocq library CecoaCecoa enables proving that a user-defined function is polytime computable (Férée, Hym, Mayero, Moyen, & Nowak, 2018). It is based on quasi-interpretations of Marion and Moyen (2000) and extends the Bellantoni and Cook formalization. The Cecoa library is between complexity theory and program analysis. However, it is not an automatic program analyzer, since user effort is needed to establish the complexity result.

⁴³The Cook-Levin Theorem states that Boolean SAT is NP-Complete (Cook, 1971; Levin, 1973).

To the best of our knowledge, no formalized automatic resource analyzer exists. The closest to the goal are the analyzers C4B and Pastis (cf. § 1.2.2.4), that produce *proof certificates* for the bounds they compute. The analysis technique—that is based on automatic amortized resource analysis (cf § 1.2.2.3)—is founded on inference rules. After program analysis, the derivation trees are translated to machine-checkable proofs. The proof certificates are produced via a support library that complements the core analysis. However, the analysis technique is not proven correct.

From implicit complexity towards formal methods. There is much room to improve the state-of-the-art in formal reasoning about complexity, both in terms of theory and analysis. Implicit computational complexity presents a potential avenue toward advancement.

Given the demonstrated applications and familiarity, the flow calculus of mwp-bounds is an appealing target for formalization. An important step is proving formally the mwp-calculus soundness theorem (Thm. 2). The aim has multiple sources of support. First, the paper proofs are available in (Jones & Kristiansen, 2009). Second, our confidence in the correctness of the theory is high. If a problem existed, it likely would have already surfaced in the prior projects.

However, converting the mwp-calculus to a formal proof poses various technical challenges. For example, how to model the mathematical framework of the flow calculus, and converting the proofs between logical formalisms, from set theory to type theory. Moreover, a proof of the original flow calculus would not extend to the mwp^∞ -calculus variant, that is used in program analysis. Yet, starting with the core mwp-calculus and paper proofs is a solid strategy toward formalizing complexity. The result would extend broadly to all programs that share the same imperative core. This project idea is discussed in §3.3.

It is important to note the utility of implicit computational complexity is not limited to for-

malizing complexity results. Another perspective is to use ICC systems in assisting formal methods tools in program verification efforts. This is the motivating idea in the manuscript of §3.1. The central idea is that the inferred mwp-bounds can aid the discovery of verification conditions in Dafny-style deductive verification.

2 Published Manuscripts

2.1 pymwp: A Static Analyzer Determining Polynomial Growth Bounds

 *Implicit computational complexity & static analysis*

Clément Aubert, Thomas Rubiano, Neea Rusch, Thomas Seiller

The 21st International Symposium on Automated Technology for Verification and Analysis (ATVA), 2023

Software artifact: 10.5281/zenodo.7908484

Citation: (Aubert, Rubiano, Rusch, & Seiller, 2023b)

Reproduced with permission from Springer Nature.

pymwp: A Static Analyzer Determining Polynomial Growth Bounds[★]

Clément Aubert¹[0000-0001-6346-3043], Thomas Rubiano², Neea Rusch¹[0000-0002-7354-5330], and Thomas Seiller^{2,3}[0000-0001-6313-0898]

¹ School of Computer and Cyber Sciences, Augusta University

² LIPN – UMR 7030 Université Sorbonne Paris Nord

³ CNRS

Abstract. We present pymwp, a static analyzer that automatically computes, if they exist, polynomial bounds relating input and output sizes. In case of exponential growth, our tool detects precisely which dependencies between variables induced it. Based on the sound mwp-flow calculus, the analysis captures bounds on large classes of programs by being non-deterministic and not requiring termination. For this reason, implementing this calculus required solving several non-trivial implementation problems, to handle its complexity and non-determinism, but also to provide meaningful feedback to the programmer. The duality of the analysis result and compositionality of the calculus make our approach original in the landscape of complexity analyzers. We conclude by demonstrating experimentally how pymwp is a practical and performant static analyzer to automatically evaluate variable growth bounds of C programs.

Keywords: Static Program Analysis · Automatic Complexity Analysis · Program Verification · Bound Inference · Flow Analysis.

1 Introduction – Making Use of Implicit Complexity

Certification of any program is incomplete if it ignores resource considerations, as runtime failure will occur if usage exceeds available capacity. To address this deficiency, automatic complexity analysis produced many different implementations [9,13,14,15] with varying features. This paper presents the development and specificities of our automatic static complexity analyzer, pymwp.

The first original dimension of our tool is its inspiration, coming from Implicit Computational Complexity (ICC) [10]. This field designs systems guaranteeing program’s runtime resource usage that tend to possess practically useful properties. For this reason, it is conjectured that ICC systems could be used to achieve realistic complexity analysis [18, p. 16]. Our series of work [5,6] is testing this

[★] This research is supported by the Transatlantic Research Partnership of the Embassy of France in the United States and the FACE Foundation, and has benefited from the research meeting 21453 “Static Analyses of Program Flows: Types and Certificate for Complexity” in Schloss Dagstuhl. Th. Rubiano and Th. Seiller are supported by the Ile-de-France region through the DIM RFSI project “CoHop”.

hypothesis, and resulted in the tool we present in this paper: pymwp is one of the first ICC-inspired applications, and the first mechanization of the specific technique it implements. Let us first exemplify what pymwp calculates.

Example 1. Consider an imperative program with a fixed number of parameters:

```
void increasing(int X1, int X2, int X3) {
    while (X2 < X1) { X2 = X1 + X1; }
    while (X3 < X2) { X3 = X2 + X2; }
}
```

Independently of the arguments passed (henceforth called initial values), once computation concludes, $X1$ will hold the same value, but the values held by $X2$ and $X3$ may have changed. By manual analysis, we can deduce that the variable values “growth bound” between the *initial values* $X1$, $X2$ and $X3$ (overloading initial values and parameter names) and their *final values* (denoted $X1'$, $X2'$ and $X3'$), omitting constants, is $X1' = X1$, $X2' \leq \max(X1, X2)$ and $X3' \leq \max(X3, X2 + X1)$.⁴ Therefore, for all initial values, the value growth of the variable’s value is bounded by a polynomial w.r.t. its initial values. Our analysis is designed to either produce such bounds, or to pinpoint variables that grow exponentially.

Introducing more variables, or potentially non-terminating iteration, or complicating the logic would make manual analysis difficult. However, our static analyzer handles all those cases automatically. It determines if a program accepts at least one polynomial bounding the final value of its variables in terms of their initial values—what we call its *growth bound*. If a bound cannot be established, it provides feedback on sources of failure, identifying variable pairs that have “too strong” dependencies. The technique is sound [16, p. 11], meaning a positive result guarantees program has satisfactory value growth behavior at runtime.

The mwp-flow analysis [16], that powers this tool, is of interest for its flexibility, originality, and uncommon features [9] such as being compositional and not requiring termination. However, using it to implement an automatic analyzer required important theoretical adjustments, and to sidestep or solve computationally expensive steps in the derivation of the bounds. For example, approaches to determine bounds were motivated by the need to compute them rapidly and present them in a concise human-interpretable manner, which is problematic for potentially exponential number of outputs. The theoretical improvements were presented previously [5] and serve as basis for pymwp. In this paper, we focus on the tool and its recent advancements, with following contributions.

1. We present the static analyzer pymwp in Sect. 3. It evaluates automatically if an input program has a polynomial growth bound and provides actionable feedback on failure. Our tool is easy to use and install; open-source, well-documented, and persistently available for future reuse.

⁴ Observe that the bound for $X3'$ involves $X1$ and $X2$: the presence of $X1$ in the bound of $X2'$ transitively impacts the bound for $X3'$, because the analysis is compositional.

2. Implementing the theoretical mwp-calculus required to solve several non-trivial implementation problems. Specifically, how to obtain fast and concise results was solved by recent tool developments, and discussed in Sect. 4.
3. Sect. 5 demonstrates that pymwp is a practical and performant static analyzer by experimentally analyzing a set of canonical C programs. The evaluation also includes every example presented in this paper.

2 Calculating Bounds with mwp-Analysis

Given a deterministic imperative program over integers constructed using `while`, `if` and assignments, the mwp-analysis aims at discovering the polynomials bounding the variables final values $X_1', \dots, X_n'$ in terms of their initial values $X_1, \dots, X_n$ [16, p. 5]. This section gives insight on how to interpret the results of pymwp, exemplifies those bounds in more detail, and identifies its distinctive features in the landscape of automatic complexity analysis.

2.1 Interpreting Analysis Results: mwp-Bounds and ∞

The mwp-flow analysis internally captures dependencies between program's variables to determine existence of growth bounds and locates problematic data flow relations. A flow can be 0, meaning no dependency; *maximal* of linear, *weak* polynomial, *polynomial* or ∞ , in increasing order of dependency. When the value of *every variable* in a program is bounded by at most a polynomial in initial values, the flow calculus assigns each variable an *mwp-bound*. It is a number-theoretic expression of form $\max(\mathbf{x}, \text{poly}_1(\mathbf{y})) + \text{poly}_2(\mathbf{z})$, where variables characterized by *m*-flow are listed in $\mathbf{x}$; *w*-flows in $\mathbf{y}$, and *p*-flows in $\mathbf{z}$. Honest polynomials poly_1 and poly_2 are build up from constants and variables by applying $+$ and $\times$. Any of the three variable lists might be empty and poly_1 and poly_2 may not be present. A bound of a program is conjunction ($\wedge$) of mwp-bounds. Variables that depend “too strongly” are assigned ∞ -flow, to indicate exponential growth.

Expression	reads as “The growth of X' is bounded by ...”
$X' \leq 0$	“... a constant.”
$X' \leq X$	“... a polynomial in X .” (its initial value)
$X' \leq \max(X, X_1) + X_2 \times X_3$	“... a polynomial in X or X_1 , X_2 and X_3 .”

Determining program bounds is complicated because the flow calculus is non-deterministic. This enables to analyze a larger class of programs, but also means that one program may be assigned multiple bounds. If a program is assigned a bound, it is derivable in the calculus. An impossibility result occurs when all derivation “paths” yields an ∞ -result.

2.2 Additional Foundational Examples

One important and original aspect is that the mwp-flow analysis ignores Boolean conditions, assuming that both `if`-branches evaluate, and that loops executes an

arbitrary number of cycles. This lets pymwp analyze non-terminating programs without complications, and justifies why all conditions will be abstracted as `b`.

Letting $C1 \equiv X2 = X1 + X1$ and $C2 \equiv X3 = X2 + X2$, Example 1 established that the iterative composition `while b C1; while b C2` has a polynomial growth bound, i.e., the property of interest.⁵ We now elaborate on mwp-flow analysis behavior by inspecting two more expository programs, letting $C3 \equiv X3 = X3 * X3$.

Example 2. Consider program `while b C3`. Even if $C3$ in itself admits the bound $X3' \leq X3$, the value stored in variable $X3$ will grow exponentially on each iteration. Therefore, the program cannot get a growth bound, due to the ∞ flow between $X3$ and itself introduced by the `while` statement.

Example 3. Combining elements from the previous two examples, we construct `while b C1; while b C2; C3`. Variables $X1$ and $X2$ are unaffected by $C3$, but $X3$ changes. We over-approximate the final value of $X3$ to obtain the program's growth bound $X1' \leq X1 \wedge X2' \leq \max(X2, X1) \wedge X3' \leq X1 + X2 + X3$. This example shows how partial results ($X3' \leq \max(X3, X1 + X2)$ and $X3' \leq X3$) can be combined to obtain new bounds ($X3' \leq X1 + X2 + X3$) by compositionality.

In the tool user guide, we present even more examples with in-depth discussion, to elaborate on the behavior and results of mwp-analysis.

2.3 Originalities of mwp-flow Analysis

The mwp technique offers many properties that make it unique and practically useful. It is a syntactic analysis, not based on general purpose reasoners e.g., abstract interpreters or model checkers. It requires little structure, and no manual annotation from the analyzed program. This enables its implementation on any imperative programming language, and potentially at different stages of compilation. Compositionality is another significant feature. Non-compositional techniques require inlining programs and are common among automated complexity analyses [9]. With compositionality, analysis can be performed on parts of whole-programs, and after refactoring, repeated only on those parts that changed.

Several tools that evaluate resource bounds already exist [1,2,8,12,13,14,15,17]; including LOOPUS [25] and C4B [9], that specialize in C language inputs. Comprehensive evaluations of these tools have also been performed recently [9,11,25]. The main distinguishing factor between these tools and pymwp is the program's complexity property of interest: pymwp evaluates the existence of polynomial growth bounds w.r.t. initial values. We illustrate the difference in obtained bounds in Table 1. It is not an extensive comparison but suffices to show that pymwp differs in its aims from the other related techniques.

⁵ pymwp actually outputs $X1' \leq X1 \wedge X2' \leq \max(X2, X1) \wedge X3' \leq \max(X3, X1 + X2)$.

Table 1. Comparison of obtained resource bounds for various C language analyzers, on examples from Carbonneaux et al. [9, p. 26]. LOOPUS and C4B find asymptotically tight bounds based on amortization. LOOPUS calculates bounds on loop iterations, C4B derives global whole-program bounds, and pymwp analyzes variables growths. The inputs are part of Sect. 5 benchmark suite, and available in the pymwp repository [24].

Input	LOOPUS	C4B	pymwp
t19.c	$\max(0, i - 10^2) + \max(0, k + i + 51)$	$50 + [-1, i] + [0, k] $	$i' \leq i + k \wedge k' \leq k$
t20.c	$2 \cdot \max(0, y - x) + \max(0, x - y)$	$ [x, y] + [y, x] $	$x' \leq x \wedge y' \leq y$
t47.c	$1 + \max(n, 0)$	$1 + [0, n] $	$n' \leq n \wedge \text{flag}' \leq 0$

3 Technical Overview of pymwp

In this section we present the main contribution of the paper: the pymwp static analyzer. It is a command-line tool that analyzes programs written in subset of C programming language presented in Sect. 3.3. The name alludes to its implementation language, Python, which we selected for its flexibility and use in previous related works [4,19,20]. Our tool takes as input a path to a C program, and returns for each function it contains a growth bound—if at least one can be established—or a list of variable dependencies that may cause the exponential growth.⁶ The pymwp development is open source [24] with releases published at Python Package Index (PyPI) [22], GitHub [24] and Zenodo [7]. A tool user guide is available at <https://statycc.github.io/.github/pymwp/>.

3.1 Program Analysis in Action

The default procedure for performing mwp-analysis is as follows:

1. Parse input file to obtain an abstract syntax tree (AST).
2. Initialize a **Result** object T .
3. For each function (or “program”, interchangeably) in the AST:
 - (a) Create an initial **Relation** R —briefly, this complex structure represents variables and their dependencies, at a program point (Sect. 4.1).
 - (b) Sequentially for each statement in function body:
 - i. Recursively apply inference rules to obtain R_i .
 - ii. Compose R_i with previous relation: $R = R \circ R_i$.
 - iii. If no bound exists, terminate analysis of function body.
 - (c) If bounds exist, evaluate R to determine the bounds (Sect. 4.3)
 - (d) Append function analysis result to T .
4. Return T .

⁶ Obtaining this feedback requires to specify the `--fin` argument.

3.2 Usage

There are multiple ways to use pymwp. It has a text-based application interface, and can be run from terminal, or it can be imported as a Python module into larger software engineering developments. The analysis is automatic and read-only, therefore it is possible to pair pymwp with other tools and integrate it into compilation or verification toolchains. The online demo provides one example use case. It is a web server application with pymwp as a package dependency. Other derived uses can be developed similarly. The easiest way to install pymwp is from PyPI, using command `pip install pymwp`. The default interaction command is

```
pymwp path/to/file.c [args]
```

where the first positional argument is required. By default, pymwp displays the analysis result with logging information, and writes the result to a file. This behavior is customizable by specifying arguments. For a list of currently supported arguments run `pymwp --help`.

3.3 Scope of Analyzable Programs

The programs analyzable with pymwp are determined by its supported syntax. pymwp delegates the task of parsing C files to its dependency, `pyccparser` [21], which aims to support the full C99 specification. Programs that cannot be parsed will expectedly throw an error. Otherwise, analysis proceeds on the generated AST, and pymwp handles nodes that are syntactically supported by its calculus.⁷ It skips unsupported nodes with a warning. We decided on this permissive approach, because it allows to obtain partial results and manually inspect unsupported operations. However, to establish a guaranteed bound, the input program must fully conform to the supported syntax of the calculus. Currently the syntax has limitations, e.g., arrays and pointer operations are unsupported. Extending the analysis to richer syntax is a direction for future work.

4 Implementation Advancements

Notable technical progress has occurred since the initial mention of pymwp in the literature [5].⁸ We will discuss those solutions in this section.

4.1 Motivations for Refining Analysis Results

Understanding pymwp's advances requires to briefly reflect on its past. The mwp-flow, as originally designed [16], is an inference system that has an unbearable computational cost, as it manipulates non-deterministically an exponential number of sizable matrices [5, Sect. 2.3] to try to establish a bound. Our enhanced

⁷ List of supported features: <https://statycc.github.io/pymwp/features>.

⁸ Full comparison: <https://github.com/statycc/pymwp/compare/FSCD22...0.4.2>.

mwp-technique [5] resolved this challenge by internalizing the non-determinism in a single matrix, containing coefficients and functions from choices into coefficients. This way, all derivations—including the ones that will fail—are constructed at the same time, moving the problem from “Is there a derivation?” to “Among all the derivations you constructed, is there one without ∞ coefficient?”—an equivalent question that however complicates the production of the actual bound.

While answering the first question is too computationally expensive, pymwp’s ♦ FSCD 2022 version can answer the second, and it can further, if all derivations contain ∞ coefficients, terminate early for faster result [5, Sect. 4.4]. This was achieved thanks to a complex `Relation` data structure,⁹ but extracting finer information from that data structure remained an outstanding problem. In particular, we wanted to provide the following feedback to the programmer: (i) If no bound exists, the location of the exponential growth. (ii) If bounds exist, the value of at least one of them. The current version of pymwp can now provide this feedback, thanks to a long maturation that we now detail.

4.2 Exposing Sources of Failure

Since pymwp identifies polynomial bounds, it reports failure on programs containing at least one variable whose value grows exponentially w.r.t. at least one of its initial value. Earlier tool versions would indicate that failure was detected without reporting the involved variables. Determining this information is complicated because of our treatment of non-determinism, but it is valuable, as addressing one of those points of failure would suffice to obtain a polynomial growth bound. Even if the program cannot be refactored satisfactorily, then analyzing the exponential growth allows to assess potential impact on the parent software application.

Our solution is to record additional information about ∞ -coefficient in the `Relation` data structure, and to list all variable pairs on which failure may occur. Since detailed failure information may not be relevant in some use-cases, and is costly to compute, it was added as an optional `--fin` argument.

Example 4. From our tool user guide (output abridged for clarity):

```
int foo(int X1, int X2, int X3){
    if (X1 == 1){
        X1 = X2+X1;
        X2 = X3+X2;
    }
    while(X1 < 10){
        X1 = X2+X1;
    }
}
```

```
$ pymwp infinite/infinite_3.c --fin
foo is infinite
Possibly problematic flows:
X1 → X1 || X2 → X1 || X3 → X1
```

Reads as “X1 depends too strongly on all variables.”

⁹ A complex data structure sounds daunting, but it is in fact one of the highlights of the system, and enables to solve a difficult derivation problem efficiently. For details, see the documentation at <https://statycc.github.io/pymwp/relation>.

4.3 Efficiently Determining Bounds

In the alternative case, where bounds are determined to exist, the next step is to evaluate the bounds—step 3(c) in the pymwp workflow (Sect. 3.1). This is problematic because the calculus can yield an exponential number of bounds w.r.t. the program size, as illustrated in Table 3 with e.g., benchmark 32. long. As a result, the evaluation phase—e.g., extracting the bounds from this conglomerate of derivations—is increasingly costly. Handling this task efficiently required us to discover a computational solution, and finding a compact format to represent the results in interpretable and memory-efficient manner. For simplicity we describe this process only at high-level, but refer to the implementation for complete details.

Determining mwp-bounds requires two separate steps, starting with the **Relation** data structure generated during analysis phase. The first challenge is to determine which paths in our conglomerate of derivations produce bounds (i.e., does not contain ∞). A naïve brute force solution would iterate over all paths, but this is too slow for practical use. Instead, we developed a set-theoretic approach, that determines first all derivation paths that lead to ∞ and then negates those paths. We capture this process in a structure called **Choice**, and the result of this computation is called a choice vector. A choice vector contains all derivation paths yielding a bound in a compact, regular expression-like representation. Once those paths are known, it is possible to extract from a **Relation** an mwp-bound (represented as a **Bound** object), by applying a selected path. Currently, we take the first choice from the choice vector, and display it as a result. Leveraging this set of bounds and its utilities is discussed in conclusion and left for future work.

5 Experimental Evaluation

To establish that pymwp is a practical and performant static analyzer, we evaluated it on a benchmark suite of canonical C programs. We ran the analyzer on the benchmarks and measured the results, thus conducting an evaluation of performance and behavioral correctness. We did not perform tool comparison or use a standard suite for two reasons: absence of a representative comparison target (cf. Sect. 2.3) and syntactic restrictions that limit the scope of analyzable programs (cf. Sect. 3.3). However, the choice methodology judiciously evaluates pymwp, and facilitates transparency and reproduction of experiments. We actively put heavy emphasis to ensure—with software engineering best practices e.g., tests, documentation [23], and long-term archival deposits [7]—that pymwp, and the evaluation presented here, are available and reusable for future comparisons.

5.1 Methodology

Benchmarks Description The suite contains 50 C programs, written in the subset of C99 syntax supported by pymwp. The benchmarks are designed purposely to exercise various data flows that pose challenges to the analyzer, e.g., increasing

parameters, binary operations, loops and decisions, and various combinations of those operations. The benchmarks are organized into seven categories based on their expected result (∞ vs. non- ∞); origin in related publications [5,16], and in this tool paper; and interest (basic examples, others). We omit these categories here, but they are apparent in the benchmarks distribution. The suite is available from pymwp repository [24], and as a release asset on GitHub, and Zenodo [7].

Metrics For each benchmark, we record 1. benchmark name, corresponding to C file name, followed by “: *program name*” if a file contains multiple programs. 2. The lines of code (loc) in the benchmark. Observe this number ranges between 4 and 45: this is reasonable and representative, because the analysis is compositional. Analysis of even a large C file reduces to analysis of its functions, that would be expectedly similar in size to these benchmarks. 3. The time (ms) required to complete program analysis. We use milliseconds for precision since all analyses conclude within seconds. For ∞ -programs, the time is for performing full evaluation with feedback, although a result of existential failure could be obtained faster. 4. Number of program initial values (vars), which internally impacts complexity of the analysis. 5. Number of polynomial bounds discovered by the analyzer. The number of bounds is 0 if the result is ∞ . 6. If a program is derivable, we capture one of its bounds.

Experimental Setup The measurements were performed on a Linux x86_64, kernel v.5.4.0-1096-gcp, Ubuntu 18.04, with 8 cores and 32 GB virtual memory. The machine impacts only observed execution time; other metrics are deterministic. The software environment was Python runtime 3.8.0, gcc 7.5.0, GNU Make 4.1, and the dev dependencies of pymwp. Because the measurement utilities of pymwp are not distributed with its release, the experiments must be run from source. We used source code version $\blacklozenge$ 0.4.2. The command to repeat experiments is `make bench`. It runs analysis on benchmarks and generates two tables of results.

5.2 Results

The evaluation results are presented in Table 2. We emphasize in these results the obtained bounds and their correctness, while the obtained execution times provide referential information of performance. The analyzer correctly finds a polynomial bound for noninfinite benchmarks, and rejects exponential and infinite benchmarks. The analyzer is also able to derive bounds for potentially non-terminating `while` benchmarks. Observe that the analysis concludes rapidly even for a long example with 45 loc, and for explosion, that has initial values count 18. The number of bounds for long is high, because it is a complicated derivation with high degree of internalized non-determinism.

For programs that have polynomial growth bounds, we give a simplified example bound in Table 3. We omit in this representation variables whose only dependency is on self, e.g., $\mathbf{x}' \leq \mathbf{x}$.¹⁰ The table serves to demonstrate that pymwp

¹⁰ Bound of example5_1 does not appear in Table 3 because of this simplification.

Table 2. Benchmark results for a canonical test suite of **C** programs. Benchmark that have 0 bounds represents case where analyzer reports an ∞ -result.

#	Benchmark	loc	ms	vars	bounds	#	Benchmark	loc	ms	vars	bounds
1.	assign_expression	8	0	2	3	26.	infinite_4	9	2189	5	0
2.	assign_variable	9	0	2	3	27.	infinite_5	11	518	5	0
3.	dense	16	15	3	81	28.	infinite_6	14	1031	4	0
4.	dense_loop	17	66	3	81	29.	infinite_7	15	298	5	0
5.	example14: <i>f</i>	4	2	2	1	30.	infinite_8	23	722	6	0
6.	example14: <i>foo</i>	11	0	2	3	31.	inline_variable	9	0	2	3
7.	example16	15	7	4	27	32.	long	45	2875	5	177147
8.	example3_1_a	10	1	3	9	33.	notinfinite_2	4	1	2	9
9.	example3_1_b	10	2	3	9	34.	notinfinite_3	9	7	4	9
10.	example3_1_c	11	3	3	1	35.	notinfinite_4	11	30	5	3
11.	example3_1_d	12	1	2	0	36.	notinfinite_5	11	29	4	9
12.	example3_2	12	2	3	0	37.	notinfinite_6	16	34	4	81
13.	example3_4	22	14	5	0	38.	notinfinite_7	15	283	5	9
14.	example5_1	10	0	2	1	39.	notinfinite_8	22	856	6	27
15.	example7_10	10	1	3	9	40.	simplified_dense	9	1	2	9
16.	example7_11	11	9	4	27	41.	t19_c4b	9	2	2	81
17.	example8	8	1	3	9	42.	t20_c4b	7	1	2	9
18.	explosion	23	405	18	729	43.	t47_c4b	12	1	2	3
19.	exponent_1	16	7	4	0	44.	tool_ex_1	7	5	3	1
20.	exponent_2	13	4	4	0	45.	tool_ex_2	7	0	2	0
21.	gcd	12	10	2	0	46.	tool_ex_3	9	8	3	3
22.	if	7	0	2	3	47.	while_1	7	1	2	3
23.	if_else	7	0	2	9	48.	while_2	7	1	2	1
24.	infinite_2	6	16	2	0	49.	while_if	9	3	3	9
25.	infinite_3	9	7	3	0	50.	xmu	26	17	5	6561

Table 3. Examples of obtained bounds for corresponding benchmarks. For compactness, the bounds are simplified to exclude variables that have dependency only on self.

#	Benchmark bound	#	Benchmark bound
1.	$y2' \leq y1$	34.	$x0' \leq \max(x0, x1) + x2 \times x3$
2.	$x' \leq y$		$\wedge x1' \leq x1 + x2 \wedge x2' \leq x2 + x3$
3.	$x0' \leq \max(x0, x2) + x1 \wedge x1' \leq x0 \times x1 \times x2$	35.	$x1' \leq \max(x1, x2 + x3) \wedge x2' \leq \max(x2, x3)$
	$\wedge x2' \leq \max(x0, x2) + x1$		$\wedge x4' \leq \max(x4, x5)$
4.	$x0' \leq \max(x0, x2) + x1 \wedge x1' \leq x0 \times x1 \times x2$	36.	$x1' \leq \max(x1, x4) + x2 \times x3$
	$\wedge x2' \leq \max(x0, x2) + x1$		$\wedge x2' \leq \max(x2, x4) + x3$
5.	$x2' \leq \max(x2, x1)$		$\wedge x3' \leq \max(x3, x4)$
6.	$x2' \leq x1$	37.	$x1' \leq \max(x1, x4) + x2 \times x3$
7.	$x1' \leq R + x1 \wedge x2' \leq x1 \wedge x_{-1}' \leq x1 \wedge R' \leq R + x1$		$\wedge x2' \leq \max(x2, x4) + x3$
8.	$x1' \leq x2 \times x3$	38.	$x1' \leq \max(x1, x2 + x3 + x4 + x5)$
9.	$x1' \leq x2 \times x3$		$\wedge x2' \leq \max(x2, x3 + x4 + x5)$
10.	$x1' \leq \max(x1, x2 + x3)$		$\wedge x3' \leq \max(x3, x4 + x5)$
15.	$x3' \leq x3 + x1 \times x2$		$\wedge x4' \leq \max(x4, x5)$
16.	$x1' \leq x1 + x2 \times x3 \times x4 \wedge x2' \leq x2 + x3 \times x4$	39.	$x1' \leq x1 + x2 \times x3 \times x4 \times x5$
	$\wedge x3' \leq x3 + x4$		$\wedge x2' \leq \max(x2, x1 + x3 + x4 + x5)$
17.	$x1' \leq x1 + x2 \times x3$		$\wedge x3' \leq \max(x3, x1 + x4 + x5) + x2$
18.	$x0' \leq x1 + x2 \wedge x3' \leq x4 + x5 \wedge x6' \leq x7 + x8$		$\wedge x4' \leq \max(x4, x1 + x5) + x2$
	$\wedge x9' \leq x10 + x11 \wedge x12' \leq x13 + x14$		$\wedge x6' \leq \max(x6, x1 + x3 + x4 + x5) + x2$
	$\wedge x15' \leq x16 + x17$	40.	$x0' \leq x0 + x1 \wedge x1' \leq x1 + x0$
22.	$y' \leq \max(x, y)$	41.	$i' \leq i + k$
23.	$x' \leq \max(x, y) \wedge y' \leq \max(x, y)$	43.	$flag' \leq 0$
31.	$y2' \leq y1$	44.	$x2' \leq \max(x2, x1) \wedge x3' \leq \max(x3, x1 + x2)$
32.	$x0' \leq x2 + x1 \times x4 \wedge x1' \leq \max(x2, x3, x4) + x1$	46.	$x2' \leq \max(x2, x1) \wedge x3' \leq x1 + x2 + x3$
	$\wedge x2' \leq \max(x2, x3) + x1 \times x4$	47.	$y' \leq \max(x, y)$
	$\wedge x3' \leq \max(x2, x3) + x1$	48.	$x' \leq \max(x, y)$
	$\wedge x4' \leq x1 \times x2 \times x3 \times x4$	49.	$y2' \leq \max(y2, y1) \wedge r' \leq \max(y2, y1)$
33.	$x0' \leq x0 + x1 \wedge x1' \leq x0 \times x1$	50.	$beg' \leq 0 \wedge end' \leq 0 \wedge i' \leq 0$

can derive complex multivariate bounds automatically, and to present the result of a non-deterministic computation in a digestible form. It also clarifies what the analyzer computes and that those results are original in form.

6 Conclusion

This paper presented pymwp, its recent technical advancements, and evaluated its performance. Our tool reasons efficiently about existence of the variables' growth bounds w.r.t. its initial value, and can be paired with other tools for extended verification and compound analyses. Possible enhancements of the tool involve extending it to support richer syntax, and exploring the space of discovered bounds. For example, we could investigate whether constraints such as “Is there a bound where this particular variable growth linearly?” can be satisfied. Another open question is to identify *distinct* bounds.

Beyond enhancements of pymwp, several future directions and extended applications can follow. Perhaps the most interesting of those is to formally verify the analysis technique, and work is already underway in that direction [3]. Since the analysis does not require much structure from an input program, it could be useful for analyzing intermediate representations during compilation. It could also find use cases in restricted domain-specific languages, and resource-restricted hardware, to establish guarantees of their runtime behavior. Long term, the fast compositional analysis could also be useful to construct IDE plug-ins to provide low-latency feedback to programmers.

Acknowledgments The authors wish to express their gratitude to the reviewers for their thoughtful comments, and to Antonio Flores Montoya, for the preparation and public sharing of his PhD thesis experimental evaluation resources [11].

References

1. Albert, E., Arenas, P., Genaim, S., Puebla, G., Zanardini, D.: Cost analysis of object-oriented bytecode programs. *Theoretical Computer Science* **413**(1), 142–159 (2012). <https://doi.org/10.1016/j.tcs.2011.07.009>
2. Alias, C., Darte, A., Feautrier, P., Gonnord, L.: Multi-dimensional rankings, program termination, and complexity bounds of flowchart programs. In: *Static Analysis: 17th International Symposium, SAS 2010, Perpignan, France, September 14-16, 2010. Proceedings* 17. pp. 117–133. Springer International Publishing (2010). https://doi.org/10.1007/978-3-642-15769-1_8
3. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: Certifying complexity analysis (2023), <https://hal.science/hal-04083105v1/file/main.pdf>, presented at the Ninth International Workshop on Coq for Programming Languages (CoqPL)
4. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: LQICM On C Toy Parser (3 2021), https://github.com/statycc/LQICM_On_C_Toy_Parser
5. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: mwp-analysis improvement and implementation: Realizing implicit computational complexity. In: Felty, A.P. (ed.)

- 7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022). Leibniz International Proceedings in Informatics, vol. 228, pp. 26:1–26:23. Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2022). <https://doi.org/10.4230/LIPIcs.FSCD.2022.26>
6. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: Realizing Implicit Computational Complexity (Mar 2022), <https://hal.archives-ouvertes.fr/hal-03603510>, presented at the 28th International Conference on Types for Proofs and Programs (TYPES 2022) (Recording)
 7. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: pymwp: MWP analysis on C code in Python (May 2023). <https://doi.org/10.5281/zenodo.7908484>
 8. Brockschmidt, M., Emmes, F., Falke, S., Fuhs, C., Giesl, J.: Analyzing runtime and size complexity of integer programs. *ACM Transactions on Programming Languages and Systems (TOPLAS)* **38**(4), 1–50 (2016). <https://doi.org/10.1145/2866575>
 9. Carbonneaux, Q., Hoffmann, J., Shao, Z.: Compositional certified resource bounds. In: Grove, D., Blackburn, S.M. (eds.) *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, Portland, OR, USA, June 15–17, 2015. pp. 467–478. Association for Computing Machinery (2015). <https://doi.org/10.1145/2737924.2737955>
 10. Dal Lago, U.: A short introduction to implicit computational complexity. In: Bezhanishvili, N., Goranko, V. (eds.) *ESSLLI. Lecture Notes in Computer Science*, vol. 7388, pp. 89–109. Springer (2011). https://doi.org/10.1007/978-3-642-31485-8_3
 11. Flores Montoya, A.: Cost Analysis of Programs Based on the Refinement of Cost Relations. Ph.D. thesis, Technische Universität, Darmstadt (August 2017), <http://tuprints.ulb.tu-darmstadt.de/6746/>
 12. Flores-Montoya, A., Hähnle, R.: Resource analysis of complex programs with cost equations. In: *Programming Languages and Systems (APLAS 2014)*. pp. 275–295. Springer International Publishing (2014). https://doi.org/10.1007/978-3-319-12736-1_15
 13. Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, Carstenand Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., Thiemann, R.: Analyzing program termination and complexity automatically with aprove. *Journal of Automated Reasoning* **58**(1), 3–31 (2017). <https://doi.org/10.1007/s10817-016-9388-y>
 14. Hainry, E., Jeandel, E., Péchoux, R., Zeyen, O.: Complexityparser: An automatic tool for certifying poly-time complexity of Java programs. In: Cerone, A., Ölveczky, P.C. (eds.) *Theoretical Aspects of Computing - ICTAC 2021 - 18th International Colloquium, Virtual Event, Nur-Sultan, Kazakhstan, September 8–10, 2021, Proceedings*. *Lecture Notes in Computer Science*, vol. 12819, pp. 357–365. Springer (2021). https://doi.org/10.1007/978-3-030-85315-0_20
 15. Hoffmann, J., Aehlig, K., Hofmann, M.: Resource aware ML. In: Madhusudan, P., Seshia, S.A. (eds.) *Computer Aided Verification - 24th International Conference, CAV 2012. LNCS*, vol. 7358, pp. 781–786. Springer (2012). https://doi.org/10.1007/978-3-642-31424-7_64
 16. Jones, N.D., Kristiansen, L.: A flow calculus of *mwp*-bounds for complexity analysis. *ACM Transactions on Computational Logic* **10**(4), 28:1–28:41 (2009). <https://doi.org/10.1145/1555746.1555752>
 17. Moser, G., Schneckenreither, M.: Automated amortised resource analysis for term rewrite systems. In: *Functional and Logic Programming (FLOPS 2018)*. pp. 214–229. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-90686-7_14

18. Moyen, J.: Implicit Complexity in Theory and Practice. Habilitation thesis, University of Copenhagen (2017), https://lipn.univ-paris13.fr/~moyen/papiers/Habilitation_JY_Moyen.pdf
19. Moyen, J., Rubiano, T., Seiller, T.: Loop quasi-invariant chunk detection. In: D'Souza, D., Kumar, K.N. (eds.) Automated Technology for Verification and Analysis - 15th International Symposium, ATVA 2017, Pune, India, October 3-6, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10482. Springer (2017). https://doi.org/10.1007/978-3-319-68167-2_7
20. Moyen, J., Rubiano, T., Seiller, T.: Loop quasi-invariant chunk motion by peeling with statement composition. In: Bonfante, G., Moser, G. (eds.) Proceedings 8th Workshop on Developments in Implicit Computational Complexity and 5th Workshop on Foundational and Practical Aspects of Resource Analysis, DICE-FOPARA@ETAPS 2017, Uppsala, Sweden, April 22-23, 2017. Electronic Proceedings in Theoretical Computer Science, vol. 248, pp. 47–59 (2017). <https://doi.org/10.4204/EPTCS.248.9>, <http://arxiv.org/abs/1704.05169>
21. pycparser - Complete C99 parser in pure Python, <https://github.com/eliben/pycparser>
22. pymwp at Python Package Index (2023), <https://pypi.org/project/pymwp/>
23. pymwp documentation (2023), <https://statycc.github.io/pymwp/>
24. pymwp source code repository (2023), <https://github.com/statycc/pymwp>
25. Sinn, M., Zuleger, F., Veith, H.: Complexity and resource bound analysis of imperative programs using difference constraints. *Journal of automated reasoning* **59**, 3–45 (2017). <https://doi.org/10.1007/s10817-016-9402-4>

2.1.7 Tool User Guide

Publication companion for the manuscript in §2.1.

The origin version of this guide is available at <https://statycc.github.io/.github/pymwp>.

2.1.7.1 Introduction

pymwp (“pai m-w-p”) is a tool for automatically performing static analysis on programs written in a subset of the C language. It analyzes resource usage and determines if program variables’ growth rates are no more than polynomially related to their inputs sizes.

The theoretical foundations are described in paper the “mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity” (cf. “Learn More” for additional references and links). The technique is generic and applicable to any imperative language. pymwp is an implementation demonstrating this technique concretely on C programs. The technique is originally inspired by “A Flow Calculus of mwp-Bounds for Complexity Analysis”.

This guide explains pymwp usage and behavior through several examples.

Property of Interest. For an imperative program, the goal is to discover a polynomially bounded data-flow relation, between the *initial values* of the variables (denoted $X_1, \dots, X_n$) and its *final values* (denoted $X_1', \dots, X_n'$).

For a program written in C language, this property can be presented as follows.

```

1 void main(int X1, int X2, int X3) {
2     // initial values ↑
3
4     /* various commands involving
5        variables X1, X2, X3 */
6
7     // X1', X2', X3' (final values)
8 }

```

C

Listing 54: An imperative program that manipulates numeric variables.

Question: $\forall n$, is $X_n \rightsquigarrow X'_n$ polynomially bounded in inputs?

We answer this question using the mwp-flow analysis, implemented in the pymwp static analyzer.

What mwp-Flow Analysis Computes. The mwp-flow analysis works to establish a polynomial growth bound for input variables by applying inference rules to program's commands.

Internally, the analysis tracks *coefficients* representing *dependencies* between program's variables. These coefficients (or “flows”) are 0, m , w , p and ∞ . They characterize how data flows between variables.

- 0 — no dependency
- m — maximal (of linear)
- w — weak polynomial

- p — polynomial
- ∞ — infinite

with ordering: $0 < m < w < p < \infty$. The analysis name also comes from these coefficients.

After analysis, two outcomes are possible. (A) The program's variables values can be bounded by a polynomial in the input's values, or (B) the analysis determines it is impossible to establish such a bound. Due to non-determinism, many derivation paths need to be explored to determine this result.

The analysis succeeds if – for some derivation – no pair of variables is characterized by ∞ -flow. That is, obtaining an ∞ -free derivation implies existence of a polynomial growth bound; i.e., the program has the property of interest, or we can say that the program is *derivable*. The soundness theorem of the mwp-calculus guarantees that if such derivation exists, the program variables' value growth is polynomially bounded in inputs.

Program fails the analysis if every derivation contains an ∞ coefficient. Then it is not possible to establish polynomial growth bound. For these programs, pymwp reports an ∞ -result.

Interpreting mwp-Bounds. If the analysis is successful, i.e., polynomial growth bound exists, it is represented using *mwp-bounds*.

An mwp-bound is a number theoretic expression of form: $\max(\vec{x}, poly_1(\vec{y})) + poly_2(\vec{z})$.

Disjoint variable lists $\vec{x}$, $\vec{y}$ and $\vec{z}$ capture dependencies of an input variable. Dependencies characterized by m -flow are in $\vec{x}$, w -flow in $\vec{y}$, and p -flow in $\vec{z}$. The polynomials $poly_1$ and

$poly_2$ are built up from constants, variables, and operators $+$ and $\times$. Each variable list may be empty and $poly_1$ and $poly_2$ may not be present.

For multiple input variables, the result is a conjunction of mwp-bounds, one for each input variable.

Example 18. Assume program has one input variable named X , and we have obtained a bound: $X' \leq X$. The bound expression means the final value X' depends only on its own initial value X . □

Example 19. Assume program has two inputs, X and Y , and we obtained a bound: $X' \leq X \wedge Y' \leq \max(X, 0) + Y$. □

- Final value X' depends on its own initial value X .
- Final value Y' depends on initial values of inputs X and Y .
- The bound expression can be simplified to $X' \leq X \wedge Y' \leq X + Y$.

2.1.7.2 Installation

This section explains how to get started using pymwp. We recommend installing pymwp locally for an interactive tutorial experience.

1. Check environment requirements. pymwp requires Python runtime 3.7 – 3.11 (current latest). To check currently installed version, run:

```
python3 --version
```

cmd

On systems that default to Python 3 runtime, use `python` instead of `python3`. Instructions for installing Python for different operating systems can be found at python.org ↗.

2. Install pymwp. Install `pymwp` from Python Package Index:

```
pip3 install pymwp==0.4.2
```

cmd

3. Download examples. Download a set of examples.

```
wget https://github.com/statycc/pymwp/releases/download/0.4.2/examples.  
zip
```

cmd

Alternatively, download the set of examples using a web browser:

<https://github.com/statycc/pymwp/releases/download/0.4.2/examples.zip>

4. Unzip the examples. Unzip `examples.zip` using your preferred approach.

This completes the setup.

2.1.7.3 Examples

We now demonstrate use of `pymwp` on several examples. To ease the presentation, we will use multiple command line arguments.

- `--fin` — always run analysis to completion (even on failure)

- `--info` — reduces amount of terminal output to info level
- `--no_time` — omits timestamps from output log

For a complete list of available command arguments, run:

```
pymwp --help
```

cmd

2.1.7.3.1 Binary Assignment. This example shows that assigning a compounded expression to a variable results in correct analysis.

Analyzed Program

```
1 int foo(int y1, int y2) {  
2     y2 = y1 + y1;  
3 }
```

C

Listing 55: `assign_expression.c`.

It is straightforward to observe that this program has a polynomial growth bound. The precise value of that bound is $y1' = y1 \wedge y2' \leq 2 \times y1$. Although the program is simple, it is interesting because binary operations introduce complexity in program analysis.

The current working directory should be the location of unzipped examples from Installation Step 4.

```
pymwp basics/assign_expression.c --fin --info --no_time
```

cmd


```
1 INFO (result): Bound:  $y_1' \leq y_1 \wedge y_2' \leq y_1$ 
2 INFO (result): Bounds: 3
3 INFO (result): Total time: 0.0 s (0 ms)
4 INFO (file_io): saved result in output/assign_expression.json
```

output

Discussion The analysis correctly assigns a polynomial bound to the program. The bound obtained by the analyzer is $y_1' \leq y_1 \wedge y_2' \leq y_1$. Comparing to the precise value determined earlier, this bound is correct, because we omit constants in the analysis results.

Due to non-determinism, the analyzer finds three different derivations that yield a bound. From the .json file, that captures the analysis result in more technical detail, it is possible to determine these three bounds are:

- $y_1' \leq y_1 \wedge y_2' \leq \max(0, 0) + y_1$
- $y_1' \leq y_1 \wedge y_2' \leq \max(0, 0) + y_1$
- $y_1' \leq y_1 \wedge y_2' \leq \max(0, y_1) + 0$

They all simplify to $y_1' \leq y_1 \wedge y_2' \leq y_1$. This concludes the obtained result matches the expected result.

2.1.7.3.2 Exponential Program. A program computing the exponentiation returns an infinite coefficient, no matter the derivation strategy chosen.

Analyzed Program

```
1 int foo(int base, int exp, int i, int result) {  
2     while (i < exp) {  
3         result = result * base;  
4         i = i + 1;  
5     }  
6 }
```

C

Listing 56: exponent_2.c.

This program's variable `result` grows exponentially. It is impossible to find a polynomial growth bound, and the analysis is expected to report ∞ -result. This example demonstrates how `pymwp` arrives to that conclusion.

```
pymwp infinite/exponent_2.c --fin --info --no_time
```

cmd

```
1 INFO (result): foo is infinite  
2 INFO (result): Possibly problematic flows:  
3 INFO (result): base → result || exp → result || i → result || result → ↵  
    result  
4 INFO (result): Total time: 0.0 s (2 ms)  
5 INFO (file_io): saved result in output/exponent_2.json
```

output

Discussion The output shows that the analyzer correctly detects that no bound can be established, and we obtain ∞ -result. The output also gives a list of problematic flows. This list indicates all variable pairs, that along some derivation paths, cause ∞ coefficients to

occur. The arrow direction means data flows from `source` $\rightarrow$ `target`. We can see the problem with this program is the data flowing to `result` variable. This clearly indicates the origin of the problem, and allows programmer to determine if the issue can be repaired, to improve program's complexity properties.

2.1.7.3.3 While Analysis. A program that shows infinite coefficients for some derivations.

Analyzed Program

```
1 int foo(int X0, int X1, int X2, int X3) {  
2     if (X1 == 1) {  
3         X1 = X2 + X1;  
4         X2 = X3 + X2;  
5     }  
6     while (X0 < 10) {  
7         X0 = X1 + X2;  
8     }  
9 }
```

C

Listing 57: `notinfinite_3.c`.

This program contains decision logic, iteration, and multiple variables. Determining if a polynomial growth bound exists is not immediate by inspection. It is therefore an interesting candidate for analysis with `pymwp`!

```
pymwp not_infinite/notinfinite_3.c --fin --info --no_time
```

cmd

```

1 INFO (result): Bound:  $X_0' \leq \max(X_0, X_1) + X_2 * X_3 \wedge X_1' \leq X_1 + X_2 \wedge X_2' \leq X_2 + X_3 \wedge X_3' \leq X_3$ 
2 INFO (result): Bounds: 9
3 INFO (result): Total time: 0.0 s (3 ms)
4 INFO (file_io): saved result in output/notinfinite_3.json

```

output

Discussion Compared to previous examples, the analysis is now getting more complicated. We can observe this in the number of discovered bounds and the form of the bound expression. The number of times the loop iterates, or which branch of the `if` statement is taken, is not a barrier to determining the result.

From the bound expression, we can determine the following.

- X_0 has the most complicated dependency relation. Its mwp-bound combines the impact of the `if` statement, the `while` loop, and the chance that the loop may not execute.
- X_1 and X_2 have fairly simple growth dependencies, originating from the `if` statement.
- X_3 is the simplest case – it never changes. Therefore, it only depends on itself.

Overall, the analysis concludes the program has a polynomial growth bound.

2.1.7.3.4 Infinite Program. A program that shows infinite coefficients for all choices.

Analyzed Program

```
1 int foo(int X1, int X2, int X3) {  
2     if (X1 == 1) {  
3         X1 = X2 + X1;  
4         X2 = X3 + X2;  
5     }  
6     while (X1 < 10) {  
7         X1 = X2 + X1;  
8     }  
9 }
```

C

Listing 58: infinite_3.c.

If you studied the previous example carefully, you might notice that this example is *very similar*. There is a subtle difference: variable X_0 has been removed and its usages changed to X_1 . This example demonstrates how this seemingly small change impacts the analysis result.

```
pymwp infinite/infinite_3.c --fin --info --no_time
```

cmd

```
1 INFO (result): foo is infinite  
2 INFO (result): Possibly problematic flows:  
3 INFO (result):  $X_1 \rightarrow X_1 \parallel X_2 \rightarrow X_1 \parallel X_3 \rightarrow X_1$   
4 INFO (result): Total time: 0.0 s (2 ms)  
5 INFO (file_io): saved result in output/infinite_3.json
```

output

Discussion We can observe the result is ∞ . Thus, even a small change can change the analysis result entirely.

The output reveals the problem arises from how data flows from source variables X_1 , X_2 , and X_3 , to target variable X_1 . Observe that even though there is no direct assignment from X_3 to X_1 , the analysis correctly identifies this dependency relation, that occurs through X_2 .

From the output, we have identified the point and source of failure. Conversely, we know other variable pairs are not problematic. By focusing on how to avoid “too strong” dependencies targeting variable X_1 , programmer may be able to refactor and improve the program’s complexity properties.

2.1.7.3.5 Challenge Example. Try to guess the analysis outcome before determining the result with pymwp.

Analyzed Program

```
1 int foo(int X0, int X1, int X2) {  
2     if (X0) { C0  
3         X2 = X0 + X1;  
4     } else {  
5         X2 = X2 + X1;  
6     }  
7     X0 = X2 + X1; C1  
8     X1 = X0 + X2; C2  
9     while (X2) { X2 = X1 + X0; } C3  
10 }
```

C

Listing 59: A challenge program dense_loop.c.

After seeing the various preceding examples – with and without polynomial bounds – we present the following challenge. By inspection, try to determine if this program is polynomially bounded w.r.t. its input values.

It is unknown which `if` branch will be taken, and whether the `while` loop will terminate, but this is not a problem for determining the result.

```
pymwp other/dense_loop.c --fin --info --no_time
```

cmd

```
1 INFO (result): Bound:  $X_0' \leq \max(X_0, X_2) + X_1 \wedge X_1' \leq X_0 * X_1 * X_2 \wedge X_2' \leq \downarrow$   
     $\max(X_0, X_2) + X_1$   
2 INFO (result): Bounds: 81  
3 INFO (result): Total time: 0.0 s (29 ms)  
4 INFO (file_io): saved result in output/dense_loop.json
```

output

Discussion Even with just 3 variables we can see—in the obtained bound expression and the number of bounds—that this is a complicated derivation problem. The analyzer determines the program has a polynomial growth bound. Let us reason informally and intuitively why this obtained result is correct.

We can observe in the bound expression, that all three variables have complicated dependencies on one another; this corresponds to what is also observable in the input program.

Regarding variables X_0 and X_1 , observe there is no command, with either as a target variable, that would give rise to exponential value growth (need iteration). Therefore, they must have polynomial growth bounds.

Variable X_2 is more complicated. The program has a `while` loop performing assignments to X_2 (potentially problematic), and the `while` loop may or may not execute.

- Case 1: loop condition is initially false. Then, final value of X_2 depends on the `if` statement, and in either branch, it will have polynomially bounded growth.
- Case 2: loop condition true – at least one iteration will occur. The program iteratively assigns values to X_2 inside the `while` loop. However, notice the command is loop invariant. No matter how many times the loop iterates, the final value of X_2 is $X_1 + X_0$. We already know those two variables have polynomial growth bounds. Therefore, X_2 also grows polynomially w.r.t. its input values.

This reasoning concurs with the result determined by `pymwp`.

2.1.7.4 Learn More

This guide has offered only a *very short* introduction to mwp-analysis and `pymwp`. If you wish to explore more, have a look at:

- “mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity”
Describes theoretical foundations behind `pymwp`.
- “A Flow Calculus of mwp-Bounds for Complexity Analysis”
Original mwp-flow analysis technique.
- Documentation – Includes information about supported C language features, etc.
- Online demo – Run `pymwp` online on more than 40 examples.
- Source code – `pymwp` is open source.

- License – pymwp is licensed under GPLv3.

2.2 Distributing and Parallelizing Non-canonical Loops

 *Implicit computational complexity & program optimization*

Clément Aubert, Thomas Rubiano, Neea Rusch, Thomas Seiller

The 24th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI), 2023

Software artifact: 10.5281/zenodo.7080145

Citation: (Aubert, Rubiano, Rusch, & Seiller, 2023e)

Reproduced with permission from Springer Nature.

Distributing and Parallelizing Non-canonical Loops[★]

Clément Aubert¹[0000–0001–6346–3043], Thomas Rubiano², Neea Rusch¹[0000–0002–7354–5330], and Thomas Seiller^{2,3}[0000–0001–6313–0898]

¹ School of Computer and Cyber Sciences, Augusta University

² LIPN – UMR 7030 Université Sorbonne Paris Nord

³ CNRS

Abstract. This work leverages an original dependency analysis to parallelize loops regardless of their form in imperative programs. Our algorithm distributes a loop into multiple parallelizable loops, resulting in gains in execution time comparable to state-of-the-art automatic source-to-source code transformers when both are applicable. Our graph-based algorithm is intuitive, language-agnostic, proven correct, and applicable to all types of loops. Importantly, it can be applied even if the loop iteration space is unknown statically or at compile time, or more generally if the loop is not in canonical form or contains loop-carried dependency. As contributions we deliver the computational technique, proof of its preservation of semantic correctness, and experimental results to quantify the expected performance gains. We also show that many comparable tools cannot distribute the loops we optimize, and that our technique can be seamlessly integrated into compiler passes or other automatic parallelization suites.



Keywords: Program Transformation · Automatic Parallelization · Loop Optimization · Abstract Interpretation · Program Analysis · Dependency Analysis

[★] This research is supported by the Transatlantic Research Partnership of the Embassy of France in the United States and the FACE Foundation. Th. Rubiano and Th. Seiller are also supported by the Île-de-France region through the DIM RFSI project “CoHop”. N. Rusch is supported in part by the Augusta University Provost’s office, and the Translational Research Program of the Department of Medicine, Medical College of Georgia at Augusta University.

1 Original Approaches to Automatic Parallelization

1.1 The Challenge of Unknown Iteration Space

Loop fission (a.k.a. loop distribution) is an optimization technique that breaks loops into multiple loops, with the same condition or index range, each taking only a part of the original loop’s body. Such transformation creates opportunity for parallelization and reduces program’s running time. For instance, the loop

```

while (t[i] != j){
    s1[i] = j*j;
    s2[i] = 1/j;
    i++;
}

```

would become

```

while (t[i1] != j) {s1[i1] = j*j; i1++;}
while (t[i2] != j) {s2[i2] = 1/j; i2++;}

```

under

this transformation. In the transformed program, variable *i* is substituted with two copies, *i1* and *i2*, and we obtain two **while** loops that can be executed in parallel.⁴ The gain, in terms of time, results from the fact that the original loop could only be executed sequentially, while the transformed loops can each be assigned to one core. If we consider similarly structured loops that perform resource-intensive computation or that can be distributed in e.g., 8 loops running on 8 cores, it becomes intuitive how this technique can yield measurable performance gain.

This example straightforwardly captures the idea behind loop fission. Of course, as a loop with a short body, it misses the richness and complexities of realistic software. It is therefore very surprising that all the existing loop fission approaches fail at transforming such an elementary program! The challenge comes from the kind of loop presented. Applying loop fission to “canonical” (Def. 15) loops or loops whose number of iterations can be pre-determined is an established convention. But our example of a non-canonical loop with a (potentially) unknown iteration space cannot be handled by those approaches (Sect. 4).

In this paper we present a loop fission technique that can resolve this limitation, because it can be applied to all kinds of a loops.⁵ The technique is applicable to any programming language in the imperative paradigm, lightweight and proven correct. The loop fission technique derives these capabilities from a graph-based dependency analysis, first introduced in our previous work [33]. Now we refine this dependency analysis and explain how it can be leveraged to obtain *loop-level parallelism*: a form of parallelism concerned with extracting parallel tasks from loops. We substantiate our claim of running time improvement by benchmarking our technique in Sect. 5. The results show, in cases where iteration space is unknown, that we obtain gain up to the number of parallelizable loops, and that in other cases the speedup is comparable to alternative techniques.

⁴ In practice, private copies of *i* are automatically created by e.g., the standard parallel programming API for C, OpenMP. Its **pragma** directives are illustrated in Fig. 5

⁵ We focus on **while** loops, but other kinds of loops (**for**, **do...while**, **foreach**) can always be translated into **while** and general applicability follows.

1.2 Motivations for Correct, Universal and Automatic Parallelization

The increasing need to discover and introduce parallelization potential in programs fuels the demand for loop fission. To leverage the potential speedup available on modern multicore hardware, all programs—including legacy software—should instruct the hardware to take advantage of its available processors.

Existing parallel programming APIs, such as OpenMP [25], PPL [32], and oneTBB [22], facilitate this progression, but several issues remain. For example, classic algorithms are written sequentially without parallelization in mind and require reformatting to fit the parallel paradigm. Suitable sequential programs with opportunity for parallelization must be modified, often manually, by carefully inserting parallelization directives. The state explosion resulting from parallelization makes it impossible to exhaustively test the code running on parallel architectures [12]. These challenges create demand for *correct* automatic parallelization approaches, to transform large bodies of software to semantically equivalent parallel programs.

Compilers offer an ideal integration point for many program analyses and optimizations. Automatic parallelization is already a standard feature in developing industry compilers, optimizing compilers, and specialty source-to-source compilers. Tools that perform local transformations, generally on loops, are frequently conceived as compiler passes. How those passes are intertwined with sequential code optimizations can however be problematic [14]. As an example, OpenMP directives are by default applied early in the compilation and hence the parallelized source code cannot benefit from sequential optimizations such as unrolling. Furthermore, compilers tend to make conservative choices and miss opportunities to parallelize [14,21].

The loop fission technique presented in this paper offers an incremental improvement in this direction. It enables discovery of parallelization potential in previously uncovered cases. In addition, the flexibility of the system makes it suitable to integration and pipelining with existing parallelization tools at various stages of compilation, as discussed in Sect. 6.

1.3 Our Technique: Properties, Benefits and Limitations

Our technique possesses four notable properties, compared to existing techniques:

Suitable to loops with unknown iteration spaces—our method does not require knowing loop iteration space statically nor at compile time, making it applicable to loops which are often ignored.

Loop-agnostic—our method requires practically no structure from the loops: they can be `while`, `do ... while` or `for` loops, have arbitrarily complex update and termination conditions, loop-carried dependencies, and arbitrarily deep loop nests.

Language-agnostic—our method can be used on any imperative language, and without manual annotations, making it flexible and suitable for application and integration with tools and languages ranging from high-level to intermediate representations.

Correct—our method is easy to prove correct and intuitive, largely because it does not apply to loop bodies with pointers or complex function calls.

All the approaches we know of fail in at least one respect. For instance, polyhedral optimizations cannot transform loops with unknown iteration spaces, since they work on static control parts of programs, where all control flow and memory accesses are known at compile time [20, p. 36]. More importantly, all the “popular” [35] automatic tools fail to optimize `do...while` loops, and require `for` and `while` loops to have canonical forms, that generally require the trip count to be known at compilation time. We discuss these alternative approaches in detail in Sect. 4.

The main limitation of our approach is with function calls and memory accesses. Although we can treat loops with pure function calls, we exclude treatment of loops that contain explicit pointer manipulation, pointer arithmetic or certain function calls. We reserve the introduction of these enhancements as future extensions of our technique. In the meantime, and with these limitations in mind, we believe our approach to be a good complement to existing approaches. Polyhedral models [24]—that are also pushing to remove some restrictions [13]—, advanced dependency analyses, or tools developed for very precise cases (such as loop tiling [14]), should be used in conjunction with our technique, as their use cases diverge (Sect. 6).

1.4 Contributions: From Theory to Benchmarks

We deliver a complete perspective on the design and expected real-time efficiency of our loop fission technique, from its theoretical foundations to concrete measurements. We present three main contributions:

1. The loop fission transformation algorithm—Sect. 3.1—that analyzes dependencies of loop condition and body variables, establishes cliques between statements, and splits independent cliques into multiple loops.
2. The correctness proof—Sect. 3.2—that guarantees the semantic preservation of loop transformation.
3. Experimental results [8]—Sect. 5—that evaluate the potential gain of the proposed technique, including loops with unknown iteration spaces, and demonstrates its integrability with existing parallelization frameworks.

But first, we present and illustrate the dependency analysis that enables our loop fission technique.

2 Background: Language and Dependency Analysis

2.1 A Simple While Imperative Language With Parallel Capacities

We use a simple imperative `while` language, with semantics similar to `C`, extended with a `parallel` command, similar to e.g., OpenMP’s directives [25], allowing to

execute its arguments in parallel.⁶ Our language supports arrays but not pointers, and we let **for** and **do...while** loops be represented using **while** loops. It is easy to map to fragments of C, Java, or any other imperative programming language with parallel support.

$var ::= i \mid j \mid \dots \mid s \mid t \mid \dots \mid x_1 \mid x_2 \mid \dots \mid z_n \mid var[exp]$	(Variables)
$exp ::= var \mid val \mid op(exp, \dots, exp)$	(Expression)
$com ::= var = exp \mid \text{if } exp \text{ then } com \text{ else } com \mid$ $\quad \text{while } exp \text{ do } com \mid \text{use}(var, \dots, var) \mid \text{skip} \mid$ $com; com \mid \text{parallel}\{com\}\{com\} \dots \{com\}$	(Command)

Fig. 1. A simple imperative **while** language

The grammar is given Fig. 1. A variable represents either an undetermined “primitive” datatype, e.g., not a reference variable, or an array, whose indices are given by an expression. We generally use **s** and **t** for arrays. An expression is either a variable, a value (e.g., integer literal) or the application to expressions of some operator *op*, which can be e.g., relational (**=**, **<**, etc.) or arithmetic (**+**, **-**, etc.). We let *V* (resp. *e*, *C*) ranges over variables (resp. expression, command) and *W* range over **while** loops. We also use combined assignment operators and write e.g., **x++** for **x += 1**. We assume commands to be correct, e.g., with operators correctly applied to expressions, no out-of-bounds errors, etc.

A program is thus a sequence of statements, each statement being either an *assignment*, a *conditional*, a *while* loop, a *function call*⁷ or a *skip*. *Statements* are abstracted into *commands*, which can be a statement, a sequence of commands, or multiple commands to be run in parallel. The semantics of **parallel** is the following: variables appearing in the arguments are considered local, and the value of a given variable **x** after execution of the **parallel** command is the value of the last modified local variable **x**. This implies possible race conditions, but our transformation (detailed in Sect. 3) is robust to those: it assumes given **parallel**-free programs, and introduces **parallel** commands that either uniformly update the (copy of the) variables across commands, or update them in only one command. The rest of this section assumes **parallel**-free programs, that will be given as input to our transformation explained in Sect. 3.1.

For convenience we define the following sets of variables.

⁶ OpenMP’s **pragma omp parallel** directive is illustrated in Sect. 5.

⁷ The **use** command represents any command which does not modify its variables but use them and should not be moved around carelessly (e.g., a **printf**). In practice, we currently treat all function calls as **use**, even if the function is pure.

Definition 1. Given an expression e , we define the variables occurring in e by:

$$\begin{aligned} \text{Occ}(x) &= x & \text{Occ}(t[e]) &= t \cup \text{Occ}(e) \\ \text{Occ}(\text{val}) &= \emptyset & \text{Occ}(\text{op}(e_1, \dots, e_n)) &= \text{Occ}(e_1) \cup \dots \cup \text{Occ}(e_n) \end{aligned}$$

Definition 2. Let C be a command, we let $\text{Out}(C)$ (resp. $\text{In}(C)$, $\text{Occ}(C)$) be the set of variables modified by (resp. used by, occurring in) C as defined in Table 1. In the **use**($x_1, \dots, x_n$) case, f is a fresh variable introduced for this command.

C	$\text{Out}(C)$	$\text{In}(C)$	$\text{Occ}(C) = \text{Out}(C) \cup \text{In}(C)$
$x = e$	x	$\text{Occ}(e)$	$x \cup \text{Occ}(e)$
$t[e_1] = e_2$	t	$\text{Occ}(e_1) \cup \text{Occ}(e_2)$	$t \cup \text{Occ}(e_1) \cup \text{Occ}(e_2)$
if e then C_1 else C_2	$\text{Out}(C_1) \cup \text{Out}(C_2)$	$\text{Occ}(e) \cup \text{In}(C_1) \cup \text{In}(C_2)$	$\text{Occ}(e) \cup \text{Occ}(C_1) \cup \text{Occ}(C_2)$
while e do C	$\text{Out}(C)$	$\text{Occ}(e) \cup \text{In}(C)$	$\text{Occ}(e) \cup \text{Occ}(C)$
use ($x_1, \dots, x_n$)	f	$\{x_1, \dots, x_n\}$	$\{x_1, \dots, x_n, f\}$
skip	$\emptyset$	$\emptyset$	$\emptyset$
$C_1; C_2$	$\text{Out}(C_1) \cup \text{Out}(C_2)$	$\text{In}(C_1) \cup \text{In}(C_2)$	$\text{Occ}(C_1) \cup \text{Occ}(C_2)$

Table 1. Definition of Out, In and Occ for commands

Our treatment of arrays is an over-approximation: we consider the array as a single entity, and that changing one value in it changes it completely. This is however satisfactory: since we do not split loop “vertically” (e.g., distributing the iteration space between threads) but “horizontally” (e.g., distributing the tasks between threads), we want each thread in the **parallel** command to have control of the array it modifies, and not to have to synchronize its writes with other commands.

2.2 Data-flow Graphs for Loop Dependency Analysis

The loop transformation algorithm relies fundamentally on its ability to analyze data-flow dependencies between loop condition and variables in the loop body, to identify opportunities for loop fission. In this section we define the principles of this dependency analysis, founded on the theory of *data-flow graphs*, and how it maps to the presented **while** language. This dependency analysis was influenced by a large body of works related to static analysis [1,26,29], semantics [27,38] and optimization [33]; but is presented here in self-contained and compact manner.

We assume the reader is familiar with semi-rings, standard operations on matrices (multiplication and addition), and on graphs (union and inclusion).

Definition of Data-Flow Graphs A data-flow graph for a given command C is a weighted relation on the set $\text{Occ}(C)$. Formally, this is represented as a matrix over a semi-ring, with the implicit choice of a denumeration of $\text{Occ}(C)$.⁸

⁸ We will use the order in which the variables occur in the program as their implicit order most of the time.

Definition 3 (DFG). A data-flow graph (DFG) for a command C is a $|\text{Occ}(C)| \times |\text{Occ}(C)|$ matrix over a fixed semi-ring $(S, +, \times)$, with $|\text{Occ}(C)|$ the cardinal of $\text{Occ}(C)$. We write $\mathbb{M}(C)$ the DFG of C and $\mathbb{M}(C)(y, x)$ for the coefficient in $\mathbb{M}(C)$ at the row corresponding to x and column corresponding to y .

How a data-flow graph is constructed, by induction over the command, is explained in Sect. 2.3. To avoid resizing matrices whenever additional variables are considered, we identify $\mathbb{M}(C)$ with its embedding in a larger matrix, i.e., we abusively call the DFG of C any matrix containing $\mathbb{M}(C)$ and the multiplication identity element on the other diagonal coefficients, implicitly viewing the additional rows/columns as variables not in $\text{Occ}(C)$.

2.3 Constructing Data-Flow Graphs

The data-flow graph (DFG) of a command is constructed by induction on the structure of the command. In the remainder of this paper, we use the semi-ring $(\{0, 1, \infty\}, \max, \times)$ to represent dependencies: ∞ represents *dependence*, 1 represents *propagation*, and 0 represents *reinitialization*.

Base cases (assignment, skip, use) The DFG for an assignment C is computed using $\text{In}(C)$ and $\text{Out}(C)$:

Definition 4 (Assignment). Given an assignment C , its DFG is given by:

$$\mathbb{M}(C)(y, x) = \begin{cases} \infty & \text{if } x \in \text{Out}(C) \text{ and } y \in \text{In}(C) & (\text{Dependence}) \\ 1 & \text{if } x = y \text{ and } x \notin \text{Out}(C) & (\text{Propagation}) \\ 0 & \text{otherwise} & (\text{Reinitialization}) \end{cases}$$

We illustrate in Fig. 2 some basic cases and introduce the graphical conventions of using weighted relations, or weighted bi-partite graphs, to illustrate the matrices. Note that in the case of dependencies, $\text{In}(C)$ is exactly the set of variables that are source of a dependence arrow, while $\text{Out}(C)$ is the set of variables that either are targets of dependence arrows or were reinitialized.

Note that we over-approximate arrays in two ways: the dependencies of the value at one index are the dependencies of the whole array, and the index at which the value is assigned is a dependence of the whole array (cf. the solid arrow from i to t in the last example of Fig. 2). This is however enough for our purpose, and simplify our treatment of arrays.

The DFG for **skip** is simply the empty matrix, but the DFG of **use** function calls requires a fresh “effect” variable to anchor the dependencies.

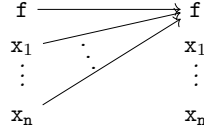
Definition 5 (skip). We let $\mathbb{M}(\text{skip})$ be the matrix with 0 rows and columns.⁹

Definition 6 (use). We let $\mathbb{M}(\text{use}(x_1, \dots, x_n))$ be the matrix with coefficients from each x_i to f , and from f to f equal to ∞ , and 0 coefficients otherwise, for f a freshly introduced variable. Graphically, we get:

⁹ Identifying the DFG with its embeddings, it is hence the identity matrix of any size.

C	$\text{Out}(C), \text{In}(C)$	$\mathbb{M}(C)$ (as a graph)	$\mathbb{M}(C)$
$w = 3$	$\text{Out}(C) = \{w\}$ $\text{In}(C) = \emptyset$	$w \xrightarrow{\text{reinitialization}} w$	$w \begin{pmatrix} w \\ 0 \end{pmatrix}$
$x = y$	$\text{Out}(C) = \{x\}$ $\text{In}(C) = \{y\}$	$x \xrightarrow{\text{dependence}} x$ $y \xrightarrow{\text{propagation}} y$	$\begin{matrix} x & y \\ x & \begin{pmatrix} 0 & 0 \\ \infty & 1 \end{pmatrix} \\ y & \end{matrix}$
$w = t[x + 1]$	$\text{Out}(C) = \{w\}$ $\text{In}(C) = \{t, x\}$	$w \xrightarrow{\text{dependence}} w$ $t \xrightarrow{\text{propagation}} t$ $x \xrightarrow{\text{propagation}} x$	$\begin{matrix} w & t & x \\ w & \begin{pmatrix} 0 & 0 & 0 \\ \infty & 1 & 0 \\ \infty & 0 & 1 \end{pmatrix} \\ t & \\ x & \end{matrix}$
$t[i] = u + j$	$\text{Out}(C) = \{t\}$ $\text{In}(C) = \{i, u, j\}$	$t \xrightarrow{\text{dependence}} t$ $i \xrightarrow{\text{propagation}} i$ $u \xrightarrow{\text{propagation}} u$ $j \xrightarrow{\text{propagation}} j$	$\begin{matrix} t & i & u & j \\ t & \begin{pmatrix} 0 & 0 & 0 & 0 \\ \infty & 1 & 0 & 0 \\ \infty & 0 & 1 & 0 \\ \infty & 0 & 0 & 1 \end{pmatrix} \\ i & \\ u & \\ j & \end{matrix}$

Fig. 2. Statement examples, sets, and representations of their dependences



Composition and multipaths The definition of DFG for a (sequential) *composition* of commands is an abstraction that allows treating a block of statements as one command with its own DFG.

Definition 7 (Composition). We let $\mathbb{M}(C_1; \dots; C_n)$ be $\mathbb{M}(C_1) \times \dots \times \mathbb{M}(C_n)$.

For two graphs, the product of their matrices of weights is represented in a standard way, as a graph of length 2 paths; as illustrated in Fig. 3—where C_1 and C_2 are themselves already the result of compositions of assignments involving disjoint variables, and hence straightforward to compute.

Correction Conditionals and loops both requires a *correction* to compute their DFGs. Indeed, the DFGs of **if** e **then** C_1 **else** C_2 and **while** e **do** C require more than the DFG of its body. The reason for this is that all the modified variables in C_1 and C_2 or C (e.g., $\text{Out}(C_1) \cup \text{Out}(C_2)$ or $\text{Out}(C)$) depend on the variables occurring in e (e.g., in $\text{Occ}(e)$). To reflect this, a *correction* is needed:

Definition 8 (Correction). For e an expression and C a command, we define e 's correction for C , $\text{Corr}(e)_C$, to be $E^t \times O$, for

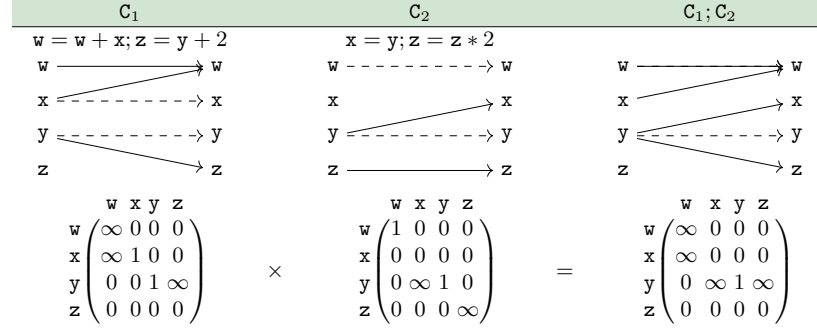


Fig. 3. Data-Flow Graph of Composition.

- E^t the (column) vector with coefficient equal to ∞ for the variables in $\text{Occ}(e)$ and 0 for all the other variables,
- O the (row) vector with coefficient equal to ∞ for the variables in $\text{Out}(C)$ and 0 for all the other variables.

As an example, let us re-use the programs C_1 and C_2 from Fig. 3, to construct $w > x$'s correction for $C_1; C_2$, that we write $\text{Corr}(w > x)_{C_1; C_2}$:

E^t	O	$E^t \times O$
$\begin{matrix} w \\ x \\ y \\ z \end{matrix} \begin{pmatrix} \infty \\ \infty \\ 0 \\ 0 \end{pmatrix}$	$\begin{matrix} & w & x & y & z \\ \begin{pmatrix} \infty & 0 & 0 & \infty \\ 0 & \infty & 0 & \infty \\ \infty & \infty & 0 & \infty \end{pmatrix} & \begin{matrix} (\text{Out}(C_1)) \\ (\text{Out}(C_2)) \\ (\text{Out}(C_1; C_2)) \end{matrix} \end{matrix}$	$\begin{matrix} & w & x & y & z \\ \begin{pmatrix} \infty & \infty & 0 & \infty \\ \infty & \infty & 0 & \infty \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} & \begin{matrix} w \\ x \\ y \\ z \end{matrix} \end{matrix}$

This last matrix represents the fact that w and x , through the expression $w > x$, control the values of w , x and z if C_1 and C_2 's execution depend of it.

Conditionals. To construct the DFG of **if** e **then** C_1 **else** C_2 , there are two aspects to consider:

1. First, our analysis does not seek to evaluate whether C_1 or C_2 will get executed. Instead, it will overapproximate and assume that both will get executed, hence using $M(C_1) + M(C_2)$.
2. Second, all the variables assigned in C_1 and C_2 (e.g., $\text{Out}(C_1) \cup \text{Out}(C_2)$) depends on the variables occurring in e . For this reason, $\text{Corr}(e)_{C_1; C_2}$ needs to be added to the previous matrix.

Putting it together, we obtain:

Definition 9 (if). We let $M(\text{if } e \text{ then } C_1 \text{ else } C_2)$ be $M(C_1) + M(C_2) + \text{Corr}(e)_{C_1; C_2}$.

Re-using the programs C_1 and C_2 from Fig. 3 and $\text{Corr}(w > x)_{C_1; C_2}$, we obtain:

$$\mathbb{M} \left(\begin{array}{l} \text{if } (w > x) \\ \quad \text{then } w = w + x; \\ \quad \quad z = y + 2 \\ \quad \text{else } x = y; \\ \quad \quad z = z * 2 \end{array} \right) = \begin{array}{c} \begin{array}{c} w \\ x \\ y \\ z \end{array} \begin{pmatrix} \infty & 0 & 0 & 0 \\ \infty & \boxed{1} & 0 & 0 \\ 0 & 0 & 1 & \infty \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{array}{c} w \\ x \\ y \\ z \end{array} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \boxed{0} & 0 & 0 \\ 0 & \infty & 1 & 0 \\ 0 & 0 & 0 & \infty \end{pmatrix} + \begin{array}{c} w \\ x \\ y \\ z \end{array} \begin{pmatrix} \infty & \infty & \boxed{0} & \infty \\ \infty & \boxed{\infty} & 0 & \infty \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

The boxed value represents the impact of x on itself: C_1 has the value 1, since x is not assigned in it. On the other hand, C_2 has 0 for coefficient, since the value of x is reinitialized in it. The correction, however, has a ∞ , to represent the fact that the value of x controls the values assigned in the body of C_1 and C_2 —and x itself is one of them. As a result, we have again the value ∞ in the matrix summing them three, since x controls the value it gets assigned to itself—as it controls which branch ends up being executed. On the other hand, the circled value at (w, y) is a 0 since y 's value is not controlled by w , since neither C_1 nor C_2 assign y : regardless of e 's truth value, y 's value will remain the same.

While Loops. To define the DFG of a command `while  $e$  do  $C$` from $\mathbb{M}(C)$, we need, as for conditionals, the correction $\text{Corr}(e)_C$, to account for the fact that all the modified variables in C depend on the variables used in e :

Definition 10 (while). We let $\mathbb{M}(\text{while } e \text{ do } C)$ be $\mathbb{M}(C) + \text{Corr}(e)_C$.¹⁰

As an example, we let the reader convince themselves that the DFG of

$$\begin{array}{l} \text{while } (t[i] \neq j) \{ \\ \quad s1[i] = j * j; \\ \quad s2[i] = 1/j; \\ \quad i++ \\ \} \end{array} \quad \begin{array}{c} t \quad i \quad j \quad s1 \quad s2 \\ \begin{pmatrix} 1 & \infty & 0 & \infty & \infty \\ 0 & \infty & 0 & \infty & \infty \\ 0 & \infty & 1 & \infty & \infty \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{array} \text{ . Intuitively, one can note that}$$

the rows for $s1$ and $s2$ are filled with 0s, since those variables do not control any other variable and are assigned in the body of the loop. On the other hand, t , i and j all three control the values of i , $s1$ and $s2$, since they determine if the body of the loop will execute. The variables t and j are the only one whose value is propagated (e.g., with a 1 on their diagonal), since they are not assigned in this short example. The command `i++` is the only command that has the potential to impact the loop's condition. We call it an update command:

Definition 11 (Update command). Given a loop $W := \text{while } e \text{ do } C$, the update commands C_u are the commands in C such that $\mathbb{M}(W)(y, x) = \infty$ for $x \in \text{Out}(C_u)$ and $y \in \text{Occ}(e)$.

¹⁰ This is different from our previous treatment of `while` loop [33, Definition 5], that required to compute the transitive closure of $\mathbb{M}(C)$: for the transformation we present in Sect. 3, this is not needed, as all the relevant dependencies are obtained immediately—this also guarantees that our analysis can distribute loop-carried dependencies.

3 Loop Fission Algorithm

We now present our loop transformation technique and prove its correctness.

3.1 Algorithm, Presentation and Intuition

Our algorithm, presented in Algo. 1, requires essentially to

1. Pick a loop at top level,
2. Compute its condensation graph (Def. 13)—this requires first the dependence graph (Def. 12), which itself uses the DFG,
3. Compute a covering (Def. 14) of the condensation graph,
4. Create a loop per element of the covering.

Even if our technique could distribute nested loops, it would require adjustments that we prefer to omit to simplify our presentation. None of our examples in this paper require to distribute nested loops. Note, however, that our algorithm handles loops containing themselves loops.

Definition 12 (Dependence graph). *The dependence graph of the loop $W := \text{while } e \text{ do } \{C_1; \dots; C_n\}$ is the graph whose vertices is the set of commands $\{C_1; \dots; C_n\}$, and there exists a directed edge from C_i to C_j if and only if there exists variables $x \in \text{Out}(C_j)$ and $y \in \text{In}(C_i)$ such that $\mathbb{M}(W)(y, x) = \infty$.*

The last example of Sect. 2.3 gives $s1[i] = j*j \rightarrow i++ \leftarrow s2[i] = 1/j$. Note that all the commands in the body of the loop are the sources of dependence edges whose target is the update commands: for our example, this means that every command will be the source of an arrow whose target is $i++$. This comes from the correction, even if the condition does not explicitly appear in the dependence graph.

The remainder of the loop transforming principle is simple: once the graph representing the dependencies between commands is obtained, it remains to determine the cliques in the graph and forms *strongly connected components* (SCCs); and then to separate the SCCs into subgraphs to produce the final parallelizable loops that contain a copy of the loop header and update commands.

Definition 13 (Graph helpers). *Given the dependence graph of a loop W ,*

- *its strongly connected components (SCCs) are its strongly connected sub-graphs,*
- *its condensation graph G_W is the graph whose vertices are SCCs and edges are the edges whose source and target belong to distinct SCCs.*

In our example, the SCCs are the nodes themselves, and the condensation graph is $s1[i] = j*j \rightarrow i++ \leftarrow s2[i] = 1/j$. Excluding the update command $i++$, there are now two nodes in the condensation graph, and we can construct

the parallel loops by 1. inserting a **parallel** command, 2. duplicating the loop header and update command, 3. inserting the command in the remaining nodes of the condensation graph in each loop. For our example, we obtain, as expected,

$$\text{parallel} \left\{ \begin{array}{l} \text{while}(\tau[i] \neq j) \{ \\ \quad s1[i] = j*j; \\ \quad i++ \} \end{array} \right\} \left\{ \begin{array}{l} \text{while}(\tau[i] \neq j) \{ \\ \quad s2[i] = 1/j; \\ \quad i++ \} \end{array} \right\}.$$

Formally, what we just did was to split the *saturated covering*.

Definition 14 (Coverings [16]). A covering of a graph G is a collection of subgraphs $G_1, G_2, \dots, G_j$ such that $G = \cup_{i=1}^j G_i$.

A saturated covering of G is a covering $G_1, G_2, \dots, G_k$ such that for all edge in G with source in G_i , its target belongs to G_i as well. It is proper if none of the subgraph is a subgraph of another.

The algorithm then simply consists in finding a proper saturated covering of the loop's condensation graph, and to split the loop accordingly. In our example, the only proper saturated covering is

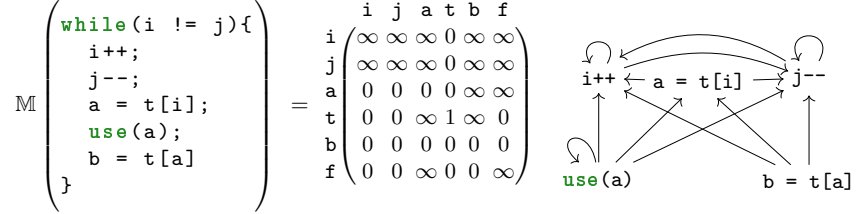
$$\{ s1[i] = j*j \rightarrow i++, i++ \leftarrow s2[i] = 1/j \}.$$

If the covering was not proper, then the $i++$ node on its own would be in it, leading to create a useless loop that performs nothing but updating its own condition.

Algorithm 1 Loop fission

Input: A loop $W := \text{while } e \text{ do } \{C_1; \dots; C_n\}$ ▷ Pick a loop W at top level
Compute the condensation graph G_W of W , ▷ cf. Def. 13
Compute the saturated covering $G_1, \dots, G_j$ of G_W : ▷ cf. Def. 14
while a node n in G_W is not part of a subgraph G_l **do**
 Create a new subgraph G_i containing n ,
 Recursively add to G_i the nodes targeted by edges whose source is in G_i ,
 Compute the proper saturated covering $G_1, \dots, G_k$ of G_W :
 for all G_i in the saturated covering **do**
 If $\exists G_l$ in the saturated covering s.t. G_i is a subgraph of G_l , then remove G_i
 end for
 Create one **while** loop per subgraph in the proper saturated covering:
 for all G_i in the proper saturated covering **do**
 Let $W_i := \text{while } e \text{ do } \{C_{i_1}; \dots; C_{i_m}\}$ where $\{C_{i_1}, \dots, C_{i_m}\}$ are the vertices of G_i ,
 inserted in the same order as they are in W .
 end for
Output: if $k > 1$, $\tilde{W} := \text{parallel}\{W_1\}\{\dots\}\{W_k\}$, else $\tilde{W} := W$.

Sometimes, duplicating commands that are not update commands is needed to split the loop. We illustrate this principle with a more complex example that involve function call and multiple update commands in Fig. 4.



The proper saturated covering has two subgraphs: one contains everything but `use(a)` and the other contains everything but `b = t[a]`. Since both `use(a)` and `b = t[a]` depend on `a = t[i]`, this latter command needs to be duplicated, even if it is not an update command:

$$\text{parallel} \left\{ \begin{array}{l} \text{while}(i \neq j) \{ \\ \quad i++; \\ \quad j--; \\ \quad a = t[i]; \\ \quad \text{use}(a) \} \end{array} \right\} \left\{ \begin{array}{l} \text{while}(i \neq j) \{ \\ \quad i++; \\ \quad j--; \\ \quad a = t[i]; \\ \quad b = t[a] \} \end{array} \right\}$$

Fig. 4. Distributing a more complex `while` loop

3.2 Correctness of the Algorithm

We now need to prove that the semantics of the initial loop W is equal to the semantics of $\tilde{W}$ given by Algo. 1. This is done by showing that for any variable x appearing in $\tilde{W}$, its final value after running W is equal to its final value after running $\tilde{W}$. We first prove that the loops in $\tilde{W}$ has the same iteration space as W :

Lemma 1. *The loops in $\tilde{W}$ have the same number of iterations as W .*

Proof. Let W_i be a loop in $\tilde{W}$. By property of the saturated covering, the update commands are in the body of W_i : there is always an edge from any command to the update commands due to the loop correction, and hence the update commands are part of all the subgraphs in the saturated covering. Furthermore, if there exists a command C that is the target of an edge whose source is an update command C_u , then C and C_u are always both present in any subgraph of the saturated covering. Indeed, since there are edges from C_u to C and from C to C_u , they are part of the same node in the condensation graph.

Since the condition of W_i is the same as the condition of W , and since all the instructions that impact (directly or indirectly) the variables occurring in that condition are present in W_i , we conclude that the number of iterations of W_i and W are equal. $\square$

Theorem 1. *The transformation $W \rightsquigarrow \tilde{W}$ given in Algo. 1 preserves the semantic.*

Proof (sketch). We show that for every variable x , the value of x after the execution of W is equal to the value of x after the execution of $\tilde{W}$. Variables are

considered local to each loop W_i in $\tilde{W}$, so we need to avoid race condition. To do so, we prove the following more precise result: for each variable x and each loop W_i in $\tilde{W}$ in which the value of x is modified, the value of x after executing W is equal to the value of x after executing W_i .

The previous claim is then straightforward to prove, based on the property of the covering. One shows by induction on the number of iterations k that for all the variables $x_1, \dots, x_h$ appearing in W_i , the values of $x_1, \dots, x_h$ after k loop iterations of W_i are equal to the values of $x_1, \dots, x_h$ after k loop iterations of W . Note some other variables may be affected by the latter but the variables $x_1, \dots, x_h$ do not depend on them (otherwise, they would also appear in W_i by definition of the dependence graph and the covering). Since the number of iteration match (Lemma 1), the claim is proven. $\square$

4 Limitations of Existing Alternative Approaches

In the beginning of this paper, we made the bold claim that other loop fission approaches do not handle unknown iteration spaces, which makes our loop-agnostic technique interesting. In this section we discuss these alternative approaches, their capabilities, and provide evidence to support this claim. We also give justification for the need to introduce our loop analysis into this landscape.

4.1 Comparing Dependency Analyses

Since its first inception, loop fission [2] has been implemented using different techniques and dependency mechanisms. Program dependence graph (PDG) [18] can be used to identify when a loop can be distributed [3, p. 844], but other—sometimes simpler—mechanisms are often used in practice. For instance, a patch integrating loop fission into LLVM [28] tuned the simpler data dependence graph (DDG) to obtain a Loop Fission Interference Graph (FIG) [30]. GCC, on the other hand, build a partition dependence graph (PG) based on the data dependency given by a reduced dependence graph (RG) to perform the same task [19]. In this paper, we introduce another loop dependency analysis, not to further obfuscate the landscape, but because it allows us to express our algorithm simply and—more importantly—to verify it mathematically.¹¹

We assume that the more complex mechanisms listed above (PDG, DDG or PG) could be leveraged to implement our transformation, but found it more natural to express ourselves in this language. We further believe that the way we compute the data dependencies is among the lightest, and with a very low memory footprint, as it requires only one pass on the source code to construct a matrix whose size is the number of variables in the program.

¹¹ This analysis also shares interesting links to a static analysis of values growth [10,9], as discussed more in-depth in a first draft [7].

4.2 Assessment of Existing Automated Loop Transformation and Parallelization Tools

While we conjecture that other mechanisms *could*, in theory, treat loops of any kind like we do, we now substantiate our claim that none of them do: in short, any loop with non-basic condition or update statement is excluded from the optimizations we now discuss. We limit this consideration to tools that support C language transformations, because it is our choice implementation language for experimental evaluation in Sect. 5. We also focus on presenting the kinds of loops that other “popular” [35] automatic loop transformation frameworks *do not* distribute, but that our algorithm can distribute. In particular, we do not discuss loops containing control-flow modifiers (such as `break`; or `continue`); neither our algorithm nor OpenMP nor the underlying dependency mechanisms of the discussed tools—to the best of our knowledge—can accommodate those.

Tools that fit the above specification include Cetus, a compiler infrastructure for the source-to-source transformation; Clava, a C/C++ source-to-source tool based on Clang; Par4All, an automatic parallelizing and optimizing compiler; Pluto, an automatic parallelizer and locality optimizer for affine loop nests; ROSE, a compiler-based infrastructure for building source-to-source program transformations and analysis tools; Intel’s C++ compiler (icc), and TRACO, an automatic parallelizing and optimizing compiler, based on the transitive closure of dependence graphs. While these tools perform various automatic transformations and optimizations, only ROSE and icc perform loop fission [35, Section 3.1].

Based on our assessment, most of these tools process only *canonical loops*:

Definition 15 (Canonical Loop [25, 4.4.1 Canonical Loop Nest Form]).
A canonical loop is a loop of the form

`for` (init-expr; test-expr; incr-expr) structured-block
for incr-expr a (single) increment or decrement by a constant or a variable, and
test-expr a single comparison between a variable and a variable or a constant.

Additional constraints on loop dependences are sometimes needed, e.g., the absence of loop-carried dependency for Cetus. It seems further that some tools cannot parallelize loops whose body contains e.g., `if` or `switch` statements [35, p. 18], but we have not investigated this claim further. However, our algorithm can handle `if`—and `switch` too, if it was part of our syntax—present in the body of the loop seamlessly.

It is always hard to infer the absence of support, but we evaluated the lack of formal discussion or example of e.g., `while` loop to be sufficient to determine that the tool cannot process `while` loops, unless of course they can trivially be transformed into `for` loops of the required form [39, p. 236]. We refer to a recent study [35, Section 2] for more detail on those notions and on the limitations of some of the tools discussed in Table 2.

Name	Fission	<code>for</code> loop	<code>while</code> loop	<code>do ...while</code> loop	ref.
Cetus	—	In canonical form	—	—	[17, p. 39], [11, p. 761]
Clava	—	In canonical form	—	—	[6]
icc	✓	Only if countable	—	—	[23, p. 2126]
Par4All	—	Unknown			[4,5]
Pluto	—	Only static control structures			[15]
ROSE	✓	In canonical form	—	—	[36, p. 124]
TRACO	—	In canonical form	—	—	[34]
OpenMP	—	In canonical form	—	—	[25]

Table 2. Feature support comparison of automated transformation and parallelization tools.

5 Evaluation

We performed an experimental evaluation of our loop fission technique on a suite of parallel benchmarks. Taking the sequential baseline, we applied the loop fission transformation and parallelization. We compared the result of our technique to the baseline and to an alternative loop fission method implemented in ROSE.

We conducted this experiment in C programming language because it naturally maps to the syntax of the imperative `while` language presented in Sect. 2. We implement the `parallel` command as OpenMP directives. For instance, the sequential baseline program on the left of Fig. 5 becomes the parallel version on right,¹² after applying our loop fission transformation and parallelization.

The evaluation experimentally substantiated two claims about our technique:

1. It can parallelize loops that are completely ignored by other automatic loop transformation tools, and results in appreciable gain, upper-bounded by the number of parallelizable loops produced by loop fission.
2. Concerning loops that other automatic loop transformation tools can distribute, it yields comparable results in speedup potential. We also demonstrate how insertion of parallel directives can be automated, which supports the practicality of our method.

These results combined confirm that our loop fission technique can easily be integrated into existing tools to improve the performances of the resulting code.

5.1 Benchmarks

Special consideration was necessary to prepare an appropriate benchmark suite for evaluation. We wanted to test our technique on a range of standard problems, across different domains and data sizes, and to include problems containing `while` loops. Because our technique is specifically designed for loop fission, we also needed to identify problems that offered potential to apply this transformation.

¹² This example is inspired by benchmark `bicg` from PolyBench/C and presented in our artifact.

```

j = 0;
while (j<M)
{
    s[j] += r[j]*A[j];
    q[j] += A[j]*p[j];
    j++;
}

#pragma omp parallel private(j)
{ // Each "pragma" block below
  // have its own copy of j.
  #pragma omp single nowait
  { // "nowait" lets the next
    // block start in parallel.
    j = 0;
    while (j<M) {
        s[j] += r[j]*A[j];
        j++;
    }
  }
  #pragma omp single
  {
    j = 0;
    while (j<M) {
        q[j] += A[j]*p[j];
        j++;
    }
  }
} // Both blocks must be terminated
// before passing this point.

```

Fig. 5. Code transformation example

Finding a suite to fit these parameters is challenging, because standard parallel programming benchmark suites offer mixed opportunity for various program optimizations and focus on loops in canonical form.

We resolved this challenge by preparing a curated set, pooling from three standard parallel programming benchmark suites. PolyBench/C is a polyhedral benchmark suite, representing e.g., linear algebra, data mining and stencils; and commonly used for measuring various loop optimizations. NAS Parallel Benchmarks are designed for performance evaluation of parallel supercomputers, derived from computational fluid dynamics applications. MiBench is an embedded benchmark suite, with everyday programming applications e.g., image-processing libraries, telecommunication, security and office equipment routines. From these suites, we extracted problems that offered potential for loop fission, or already assumed expected form, resulting in 12 benchmarks. We detail these benchmarks in Table 4. Because these three suites are not mutually compatible, we leveraged the timing utilities from PolyBench/C to establish a common and comparable measurement strategy. To assess performance of other kinds of loops that our algorithm can distribute, but which do not occur prevalently in these benchmarks, we converted a portion of problems to use **while** loops.

Comparison target We compared our approach to ROSE Compiler. It is a rich compiler architecture that offers various program transformations and auto-

matic parallelization, and supports multiple compilation targets. ROSE’s built-in LoopProcessor tool supports loop fission for C-to-C programs. This input/output specification was necessary to allow observation of the transformation results and fit with the measurement strategy we defined previously. To our knowledge, ROSE is the only tool that satisfies these evaluation requirements.

Experimental setup We ran the benchmarks using a Linux 5.10.0-18-amd64 #1 SMP Debian 5.10.140-1 (2022-09-02) x86_64 GNU/Linux machine, with 4 Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz processors, and gcc compiler version 7.5.0. The evaluation was performed in a containerized environment on Docker version 20.10.18, build b40c2f6. For each benchmark, we recorded the clock time 5 times, excluded min and max, and averaged the remaining 3 times to obtain the result. We constrained variance between recorded times not to exceed 5%. We ran experiments on 5 input data sizes, as defined in PolyBench/C: MINI, SMALL, MEDIUM, LARGE and EXTRALARGE (abbr. XS, S, M, L, XL). We also tested 4 gcc compiler optimization levels -O0 through -O3. Speedup is the ratio of sequential and parallel executions, $S = T_{\text{Seq}}/T_{\text{Par}}$, where a value greater than 1 indicates parallel is outperforming the sequential execution. In presentation of these results, the sequential benchmarks are always considered the baseline, and speedup is reported in relation to the transformed versions. Our open source benchmarks, and instructions for reproducing the results, are available online [8]. It should be noted that some results may be sensitive to the particular setup on which those experiments are run.

5.2 Results

In analyzing the results, we distinguish two cases: distributing and parallelizing loops with potentially unknown iterations, and loops with pre-determined iterations (typically `while` and `for` loops, respectively). The difficulty of parallelizing the former arises from the need to synchronize evaluation of the loop recurrence and termination condition. Improper synchronization results in overshooting the iterations [37], rendering such loops effectively sequential.

Loop fission addresses this challenge by recognizing independence between statements and producing parallelizable loops. Special care is needed when inserting parallelization directives for such loops. This remains a limitation of automated tools and is not natively supported by OpenMP. We resolved this issue by using the OpenMP `single` directive, to prevent overshooting the loop termination condition and need for synchronization between threads, enabling parallel execution by multiple threads on individual loop statements. The strategy is simple, implementable, and we show it to be effective. However, it is also upper-bounded in speedup potential by the number of parallelizable loops produced by the transformation. This is a syntactic constraint, rather than one based on number of available cores.

The results, presented in Table 3, show that our approach, paired with the described parallelization strategy, yields a gain relative to the number of

independent parallelizable loops in the transformed benchmark. We observe this e.g., for benchmarks `bicg`, `gesummv`, and `mvt`, as presented in Fig. 6. We also confirm that ROSE’s approach did not transform these loops, and report no gain for the alternative approach.

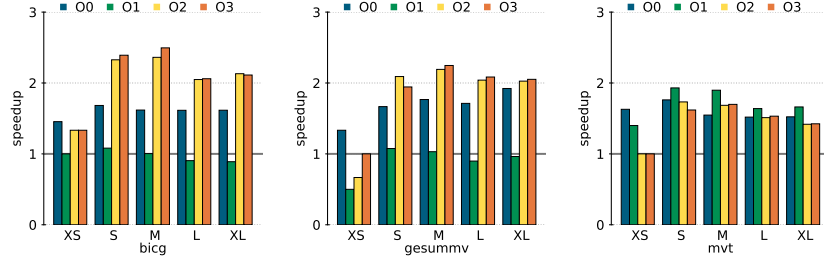


Fig. 6. Speedup of selected benchmarks implemented using `while` loops. Note the influence of various compiler optimization levels, -O0 to -O3 on each problem, and how parallelization overhead tends to decrease as input data size grows from MINI to EXTRALARGE. The gain is lower for `mvt` because it assumes fissioned form in the original benchmark. `bicg` and `gesummv` obtain higher gain from applied loop distribution.

Comparison with ROSE The remaining benchmarks, with known iteration spaces, can be transformed by both evaluated loop fission techniques: ours and ROSE’s LoopProcessor. In terms of transformation results, we observed relatively similar results for both techniques. We discovered one interesting transformation difference, with benchmark `gemm`, which ROSE handles differently from our technique.

After transformation, the program must be parallelized by inserting OpenMP directives. This parallelization step can be fully automatic and performed with e.g., ROSE or Clava, demonstrating that pipelining the transformed programs is feasible. For evaluations, we used manual parallelization for our technique and automatic approach for ROSE. However, we also noted that the automatic insertion of parallelization directives yielded, in some cases, suboptimal choices, such as parallelization of loop nests. This added unnecessary overhead to execution time, and negatively impacted the results obtained for ROSE, e.g., for benchmarks `fdtd-2d` and `gemm`, as observable in the results. It is possible this issue could be mitigated by providing annotations and more detailed instructions for applying the parallelization directives. In other experiments with alternative parallelization tools [7, Sect. 4.3], we have been successful at finding optimal parallelization directives automatically, and therefore conclude it is achievable. We again refer to Table 3 for a detailed presentation of the experimental evaluation results.

Benchmark		-O0		-O1		-O2		-O3	
Name	Size	ours	rose	ours	rose	ours	rose	ours	rose
3mm	XS	2.71	0.07	2.26	0.02	1.71	0.02	1.73	0.01
	S	2.80	0.22	3.78	0.09	3.49	0.05	3.35	0.05
	M	2.20	0.46	3.44	0.27	3.08	0.13	3.05	0.13
	L	2.85	1.92	3.11	1.16	2.89	0.66	2.97	0.66
	XL	2.16	2.31	3.13	1.83	2.24	1.05	2.25	1.04
bigc	XS	1.45	0.96	1.00	1.00	1.33	1.00	1.33	1.00
	S	1.68	0.98	1.08	1.00	2.33	1.01	2.39	1.02
	M	1.62	0.97	1.00	0.98	2.36	0.96	2.50	1.00
	L	1.61	0.96	0.90	0.94	2.05	0.95	2.06	0.95
	XL	1.62	0.96	0.89	0.95	2.13	0.93	2.11	0.94
colormap	XS	2.14	1.01	1.50	1.02	1.54	1.04	1.52	1.01
	S	2.08	0.97	1.57	1.00	1.54	1.02	1.43	0.99
	M	1.98	0.95	1.46	0.96	1.49	0.98	1.19	1.00
	L	1.93	1.03	1.42	0.98	1.44	0.98	1.20	1.01
	XL	1.82	1.00	1.53	0.97	1.55	0.99	1.16	1.00
conjgrad	XS	2.43	1.45	1.82	0.69	2.77	0.65	2.50	0.52
	S	2.50	2.39	1.91	2.03	2.84	1.88	2.96	1.65
	M	2.56	2.58	1.94	2.66	2.93	2.44	3.20	2.33
	L	2.38	2.62	1.73	2.96	2.92	2.92	3.24	2.91
	XL	2.29	2.61	1.59	2.55	2.72	2.57	2.99	2.39
cp50	XS	1.90	0.97	1.97	1.00	2.18	1.01	2.09	1.01
	S	1.94	0.95	2.00	1.02	2.08	1.00	2.07	1.00
	M	1.89	0.98	1.76	0.97	1.83	0.99	1.82	0.98
	L	1.74	0.98	1.49	0.96	1.51	0.96	1.50	0.96
	XL	1.63	0.99	1.16	0.96	1.07	0.98	1.11	0.96
deriche	XS	2.00	0.90	1.93	0.51	2.18	0.53	2.11	0.51
	S	2.30	1.49	2.16	1.05	2.17	1.04	2.14	1.03
	M	2.68	2.35	2.88	2.20	2.68	2.22	2.72	2.20
	L	1.79	1.75	2.08	2.03	2.05	2.05	2.07	2.04
	XL	1.12	1.12	1.65	1.61	1.67	1.67	1.60	1.64

Benchmark		-O0		-O1		-O2		-O3	
Name	Size	ours	rose	ours	rose	ours	rose	ours	rose
fdtd-2d	XS	2.34	0.27	1.48	0.05	1.81	0.06	1.15	0.03
	S	2.57	0.59	2.68	0.15	3.12	0.17	2.47	0.09
	M	2.23	0.82	2.01	0.29	2.47	0.30	2.60	0.24
	L	2.15	1.20	1.89	0.65	1.98	0.61	2.16	0.71
	XL	2.17	1.38	1.47	0.79	1.50	0.73	1.68	0.86
gemm	XS	2.73	0.09	2.33	0.02	2.43	0.02	1.20	0.01
	S	2.87	0.21	3.98	0.05	3.09	0.04	3.01	0.02
	M	2.57	0.56	3.42	0.12	3.40	0.12	2.73	0.05
	L	2.44	1.50	1.79	0.35	1.87	0.36	2.20	0.25
	XL	2.44	1.95	1.85	0.60	1.85	0.70	1.96	0.50
gesummv	XS	1.33	1.00	0.50	0.67	0.67	0.67	1.00	1.00
	S	1.67	0.95	1.08	1.03	2.09	1.03	1.94	1.01
	M	1.77	0.98	1.03	1.00	2.19	1.00	2.25	1.00
	L	1.71	0.94	0.90	0.93	2.04	0.93	2.08	0.97
	XL	1.92	0.98	0.96	0.98	2.03	0.99	2.05	0.98
mvt	XS	1.63	1.00	1.40	0.88	1.00	1.00	1.00	1.00
	S	1.76	1.01	1.93	1.01	1.73	1.02	1.62	1.00
	M	1.55	0.96	1.90	1.00	1.69	1.02	1.70	1.03
	L	1.52	0.98	1.64	0.97	1.51	0.98	1.53	1.00
	XL	1.52	0.98	1.66	0.99	1.42	1.00	1.42	1.00
remap	XS	1.43	0.97	0.54	1.00	0.54	1.00	0.64	1.00
	S	2.07	0.94	1.20	1.02	1.13	1.03	1.19	1.01
	M	2.43	0.99	3.13	0.96	3.36	0.98	2.89	0.97
	L	2.09	1.00	1.34	0.97	1.54	1.02	1.74	1.00
	XL	2.11	1.00	1.28	0.99	1.52	0.99	1.57	1.00
tblshft	XS	3.19	3.27	2.70	2.65	2.68	2.73	2.82	2.82
	S	3.37	3.45	2.82	2.84	2.89	2.86	3.05	3.08
	M	3.31	3.62	2.93	3.00	2.79	2.85	3.21	3.19
	L	3.05	3.40	2.17	2.32	2.38	2.32	2.40	2.39
	XL	3.08	3.48	1.91	1.85	1.64	1.69	1.96	1.96

Table 3. Speedup comparison between original sequential and transformed parallel benchmarks, comparing our loop fission technique with ROSE Compiler, for various data sizes and compiler optimization levels. We note that the problems containing only **while** loop (in **bold**) are not transformed by ROSE and therefore report no gain. The other results vary depending on parallelization strategy, but as noted with e.g., problems **conjgrad** and **tblshft**, we obtain similar speedup for both fission strategies when automatic parallelization yields optimal OpenMP directives.

Benchmark	Description	for loop	while loop	Source
3mm	3D matrix multiplication	✓		PolyBench/C
bigc	BiCG sub kernel of BiCGStab linear solver		✓	PolyBench/C
colormap	TIFF image conversion of photometric palette		✓	MiBench
conjgrad	Conjugate gradient routine	✓		NAS-CG
cp50	Ghostsript/CP50 color print routine	✓	✓	MiBench
deriche	Edge detection filter	✓		PolyBench/C
fdtd-2d	2-D finite different time domain kernel	✓		PolyBench/C
gemm	Matrix-multiply $C=\alpha.A.B+\beta.C$	✓		PolyBench/C
gesummv	Scalar, vector and matrix multiplication		✓	PolyBench/C
mvt	Matrix vector product and transpose		✓	PolyBench/C
remap	4D matrix memory remapping	✓		NAS-UA
tblshft	TIFF PixarLog compression main table bit shift	✓	✓	MiBench

Table 4. Descriptions of evaluated parallel benchmarks.

6 Conclusion

This work is only the first step in a very exciting direction. “Ordinary code”, and not only code that was specifically written for e.g., scientific calculation or other resource-demanding operations, should be executed in parallel to leverage our modern architectures. As a consequence, the much larger codebase concerned with parallelization is much less predictable and offers more diverse loop structures. Focusing on resource-demanding programs led previous efforts not only to focus on predictable loop structures, but to completely ignore other non-canonical loops. Our effort, based on an original dependency analysis, leads to re-integrate such loops in the realm of parallel optimization. This alone, in our opinion, justifies further investigation in integrating our algorithm into specialized tools.

As presented in Fig. 6, our experimental results offer some variability, but they need to be put in context: loop distribution is often only *the first step* in the optimization pipeline. Loops that have been split can then be vectorized, blocked, unrolled, etc., providing additional gain in terms of speed. Exactly as for loop fusion [31], a more global treatment of loops is needed to strike the right balance and find the optimum code transformation. Such a journey will be demanding and complex, but we believe this work enables it by reintegrating *all* loops in the realm of parallel optimization.

Acknowledgments The authors wish to express their gratitude to João Bispo for explaining how to integrate **AutoPar-Clava** in the first version of their benchmark, to Assya Sellak for her contribution to the first steps of this work, and to the reviewers for their insightful comments.

References

1. Abel, A., Altenkirch, T.: A predicative analysis of structural recursion. *Journal of Functional Programming* **12**(1), 1–41 (2002). <https://doi.org/10.1017/S0956796801004191>
2. Abu-Sufah, Kuck, Lawrie: On the performance enhancement of paging systems through program analysis and transformations. *IEEE Transactions on Computers* **C-30**(5), 341–356 (1981). <https://doi.org/10.1109/TC.1981.1675792>
3. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: *Compilers: Principles, Techniques, and Tools* (2nd Edition). Addison Wesley (Aug 2006)
4. Amini, M.: *Source-to-Source Automatic Program Transformations for GPU-like Hardware Accelerators*. Theses, Ecole Nationale Supérieure des Mines de Paris (Dec 2012), <https://pastel.archives-ouvertes.fr/pastel-00958033>
5. Amini, M., Creusillet, B., Even, S., Keryell, R., Goubier, O., Guelton, S., McMahon, J.O., Pasquier, F.X., Péan, G., Villalon, P.: Par4All: From Convex Array Regions to Heterogeneous Computing. In: *IMPACT 2012 : Second International Workshop on Polyhedral Compilation Techniques HiPEAC 2012*. Paris, France (Jan 2012), <https://hal-mines-paristech.archives-ouvertes.fr/hal-00744733>
6. Arabnejad, H., Bispo, J., Cardoso, J.M.P., Barbosa, J.G.: Source-to-source compilation targeting openmp-based automatic parallelization of C applications. *The Journal of Supercomputing* **76**(9), 6753–6785 (Sep 2020). <https://doi.org/10.1007/s11227-019-03109-9>
7. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: A Novel Loop Fission Technique Inspired by Implicit Computational Complexity (May 2022), <https://hal.archives-ouvertes.fr/hal-03669387v1>, draft
8. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: Loop fission benchmarks (Sep 2022). <https://doi.org/10.5281/zenodo.7080145>, <https://github.com/statycc/loop-fission>
9. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: mwp-analysis improvement and implementation: Realizing implicit computational complexity. In: Felty, A.P. (ed.) *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022)*. Leibniz International Proceedings in Informatics, vol. 228, pp. 26:1–26:23. Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2022). <https://doi.org/10.4230/LIPIcs.FSCD.2022.26>
10. Aubert, C., Rubiano, T., Rusch, N., Seiller, T.: pymwp: MWP analysis in Python (Sep 2022), <https://github.com/statycc/pymwp/>
11. Bae, H., Mustafa, D., Lee, J., Aurangzeb, Lin, H., Dave, C., Eigenmann, R., Midkiff, S.P.: The cetus source-to-source compiler infrastructure: Overview and evaluation. *Int. J. Parallel Program.* **41**(6), 753–767 (2013). <https://doi.org/10.1007/s10766-012-0211-z>
12. Baier, C., Katoen, J., Larsen, K.: *Principles of Model Checking*. MIT Press (2008)
13. Benabderrahmane, M., Pouchet, L., Cohen, A., Bastoul, C.: The polyhedral model is more widely applicable than you think. In: Gupta, R. (ed.) *Compiler Construction, 19th International Conference, CC 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, March 20–28, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6011, pp. 283–303. Springer (2010). https://doi.org/10.1007/978-3-642-11970-5_16
14. Bertolacci, I.J., Strout, M.M., de Supinski, B.R., Scogland, T.R.W., Davis, E.C., Olschanowsky, C.: Extending openmp to facilitate loop optimization. In: de Supinski, B.R., Valero-Lara, P., Martorell, X., Bellido, S.M., Labarta, J. (eds.) *Evolving*

- OpenMP for Evolving Architectures - 14th International Workshop on OpenMP, IWOMP 2018, Barcelona, Spain, September 26-28, 2018, Proceedings. Lecture Notes in Computer Science, vol. 11128, pp. 53–65. Springer (2018). https://doi.org/10.1007/978-3-319-98521-3_4
15. Bondhugula, U., Hartono, A., Ramanujam, J., Sadayappan, P.: A practical automatic polyhedral parallelizer and locality optimizer. *SIGPLAN Not.* **43**(6), 101–113 (jun 2008). <https://doi.org/10.1145/1379022.1375595>
 16. Chung, F.R.K.: On the coverings of graphs. *Discrete Mathematics* **30**(2), 89–93 (1980). [https://doi.org/10.1016/0012-365X\(80\)90109-0](https://doi.org/10.1016/0012-365X(80)90109-0)
 17. Dave, C., Bae, H., Min, S., Lee, S., Eigenmann, R., Midkiff, S.P.: Cetus: A source-to-source compiler infrastructure for multicores. *Computer* **42**(11), 36–42 (2009). <https://doi.org/10.1109/MC.2009.385>
 18. Ferrante, J., Ottenstein, K.J., Warren, J.D.: The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems* **9**(3), 319–349 (jul 1987). <https://doi.org/10.1145/24039.24041>
 19. gcc.gnu.org git - gcc.git/blob - gcc/tree-loop-distribution.c, <https://gcc.gnu.org/git/?p=gcc.git;a=blob;f=gcc/tree-loop-distribution.c;h=65aa1df4abae2c6acf40299f710bc62ee6bacc07;hb=HEAD#l139>
 20. Grosser, T.: Enabling Polyhedral Optimizations in LLVM. Master’s thesis, Universität Passau (4 2011), <https://polly.llvm.org/publications/grosser-diploma-thesis.pdf>
 21. Holewinski, J., Ramamurthi, R., Ravishankar, M., Fauzia, N., Pouchet, L.N., Rountev, A., Sadayappan, P.: Dynamic trace-based analysis of vectorization potential of applications. In: Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation. p. 371–382. PLDI ’12, Association for Computing Machinery, New York, NY, USA (2012). <https://doi.org/10.1145/2254064.2254108>
 22. Intel: oneTBB documentation (2022), <https://oneapi-src.github.io/oneTBB/>
 23. Intel Corporation: Intel C++ Compiler Classic Developer Guide and Reference, https://www.intel.com/content/dam/develop/external/us/en/documents/cpp_compiler_classic.pdf
 24. Karp, R.M., Miller, R.E., Winograd, S.: The organization of computations for uniform recurrence equations. *Journal of the ACM* **14**(3), 563–590 (1967). <https://doi.org/10.1145/321406.321418>
 25. Klemm, M., de Supinski, B.R. (eds.): OpenMP Application Programming Interface Specification Version 5.2. OpenMP Architecture Review Board (Nov 2021), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>
 26. Kristiansen, L., Jones, N.D.: The flow of data and the complexity of algorithms. In: Cooper, S.B., Löwe, B., Torenvliet, L. (eds.) *New Computational Paradigms, First Conference on Computability in Europe, CiE 2005, Amsterdam, The Netherlands, June 8-12, 2005, Proceedings. Lecture Notes in Computer Science*, vol. 3526, pp. 263–274. Springer (2005). https://doi.org/10.1007/11494645_33
 27. Laird, J., Manzonetto, G., McCusker, G., Pagani, M.: Weighted relational models of typed lambda-calculi. In: *LICS*. pp. 301–310. IEEE Computer Society (2013). <https://doi.org/10.1109/LICS.2013.36>
 28. Lattner, C., Adiv, V.S.: LLVM: A compilation framework for lifelong program analysis & transformation. In: 2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004), 20-24 March 2004, San Jose, CA, USA. pp. 75–88. IEEE Computer Society (2004). <https://doi.org/10.1109/CGO.2004.1281665>, <https://ieeexplore.ieee.org/xpl/conhome/9012/proceeding>

29. Lee, C.S., Jones, N.D., Ben-Amram, A.M.: The size-change principle for program termination. In: Hankin, C., Schmidt, D. (eds.) Conference Record of POPL 2001: The 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, London, UK, January 17-19, 2001. pp. 81–92. ACM (2001). <https://doi.org/10.1145/360204.360210>
30. [loopfission]: Loop fission interference graph (fig), <https://reviews.llvm.org/D73801>
31. Mehta, S., Lin, P., Yew, P.: Revisiting loop fusion in the polyhedral framework. In: Moreira, J.E., Larus, J.R. (eds.) ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '14, Orlando, FL, USA, February 15-19, 2014. pp. 233–246. ACM (2014). <https://doi.org/10.1145/2555243.2555250>
32. microsoft: Parallel patterns library (ppl) (2021), <https://docs.microsoft.com/en-us/cpp/parallel/concrt/parallel-patterns-library-ppl?view=msvc-170>
33. Moyén, J., Rubiano, T., Seiller, T.: Loop quasi-invariant chunk detection. In: D'Souza, D., Kumar, K.N. (eds.) Automated Technology for Verification and Analysis - 15th International Symposium, ATVA 2017, Pune, India, October 3-6, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10482. Springer (2017). https://doi.org/10.1007/978-3-319-68167-2_7
34. Palkowski, M., Klimek, T., Bielecki, W.: TRACO: an automatic loop nest parallelizer for numerical applications. In: Ganzha, M., Maciaszek, L.A., Paprzycki, M. (eds.) 2015 Federated Conference on Computer Science and Information Systems, FedCSIS 2015, Łódź, Poland, September 13-16, 2015. Annals of Computer Science and Information Systems, vol. 5, pp. 681–686. IEEE (2015). <https://doi.org/10.15439/2015F34>
35. Prema, S., Nasre, R., Jehadeesan, R., Panigrahi, B.: A study on popular auto-parallelization frameworks. Concurrency and Computation: Practice and Experience **31**(17), e5168 (Feb 2019). <https://doi.org/10.1002/cpe.5168>
36. Quinlan, D., Liao, C., Panas, T., Matzke, R., Schordan, M., Vuduc, R., , Yi, Q.: Rose user manual: A tool for building source-to-source translators draft user manual (version 0.9.11.115), <http://rosecompiler.org/uploads/ROSE-UserManual.pdf>
37. Rauchwerger, L., Padua, D.A.: Parallelizing while loops for multiprocessor systems. In: Proceedings of the 9th International Symposium on Parallel Processing. p. 347–356. IPPS '95, IEEE Computer Society, USA (1995)
38. Seiller, T.: Interaction graphs: Full linear logic. In: Grohe, M., Koskinen, E., Shankar, N. (eds.) Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16, New York, NY, USA, July 5-8, 2016. pp. 427–436. ACM (2016). <https://doi.org/10.1145/2933575.2934568>
39. Vitorović, A., Tomašević, M.V., Milutinović, V.M.: Manual parallelization versus state-of-the-art parallelization techniques. In: Hurson, A. (ed.) Advances in Computers, vol. 92, pp. 203–251. Elsevier (2014). <https://doi.org/10.1016/B978-0-12-420232-0.00005-2>

3 Unpublished Research

3.1 Polynomial Postconditions via mwp-Bounds

✓ *Implicit computational complexity & formal methods*

Neea Rusch

Research paper draft.

Abstract

Formal specifications are necessary to guarantee software meets critical safety properties, but retrofitting specifications to existing software requires considerable resources and expertise. Although inference eases discovery of specifications, existing inference techniques are limited in targeting different specification conditions. Particularly, inference of postconditions has rarely been the targeted focus of prior investigations. This paper presents a static program analysis that automatically synthesizes postconditions from pure program syntax. These postconditions represent final values of variables in numerical loops. For each variable, the analysis finds one approximative postcondition, of at most polynomial form, if it is expressible in the underlying theory. This theory is a sound and compositional flow calculus with complexity-theoretic origins. However, applying the flow calculus to postcondition inference required several enhancements. As technical contributions, we resolve multiple open problems about the flow calculus and increase its expressive power. Experiments and comparison study show the postcondition analysis is orthogonal to alternative techniques and generalizes to classic algorithms. These findings suggest the postcondition analysis can offer support in specification tasks even if full implementation details are still in flux.

3.1.1 Introduction

Software engineers have an aphorism that warns against publishing releases on Fridays. It is shared lightheartedly, but embeds serious commentary about the normalcy of software instability. Formal methods provides the techniques to improve and achieve rigorous software quality guarantees. Unfortunately, integrating formal methods to mainstream software development workflows remains challenging due to, e.g., lack of training and tool-specific issues (ter Beek et al., 2024). Continued research effort must be dedicated to reducing the entry barriers.

This paper aims to improve accessibility of formal methods. We do this by introducing a program analysis that synthesizes partial specification conditions. We develop an automatic inference of approximative postconditions in numerical loops. Postconditions are specialized assertions that must hold after the loop terminates. They are complementary to inductive loop invariants, i.e., conditions that must hold pre-iteration and be preserved by the loop (Sankaranarayanan, Sipma, & Manna, 2004). Although postconditions may be derivable from invariants, loop invariant inference is one of the hardest problems in verification (Dillig, Dillig, Li, & McMillan, 2013; Si, Dai, Raghothaman, Naik, & Song, 2018). Therefore, we suggest the opposite approach: starting with postconditions. It is well-known that having postconditions supports discovery of inductive invariants (Furia & Meyer, 2010). Moreover, as generalized assertions, postconditions assist in various software development activities (Nguyen, Nguyen, & Duong, 2022), like testing (Alagarsamy, Tantithamthavorn, & Aleti, 2024; Zhang & Mesbah, 2015) and code maintenance (Rosenblum, 1995). However, dedicated studies of postcondition inference are a novelty in research literature (Molina, Ponzio, Aguirre, & Frias, 2021; Popeea & Chin, 2007).

Specifications must be precise, consistent, and complete descriptions of program behavior. Making formal specifications mechanically verifiable requires they are expressed in a structured language. However, defining formal specifications is non-trivial and requires expertise. In the meantime, various resource analyses (Brockschmidt, Emmes, Falke, Fuhs, & Giesl, 2016; Jones & Kristiansen, 2009) already compute varied notions of postconditions as means to arrive to their primary result. Thus, it seems natural that resource analyses could be used to infer postconditions for formal specifications. This is precisely the key intuition we exploit in the paper. We present a complexity-theoretic flow calculus (Aubert, Rubiano, Rusch, & Seiller, 2022a; Jones & Kristiansen, 2009) that we use to automatically infer variable postconditions. In this view, the program implements the intended behavior, but misses a formal proof. Our goal is to assist software engineers in completing this missing step.

3.1.1.1 Problem formulation and solution overview

We analyze deterministic imperative numerical loops. The loop may be bounded or unbounded, with unknown termination behavior. The loop manipulates a fixed number of natural number variables. Beyond type, we make no assumptions about initial variable values. Our goal is to automatically infer postconditions of the variables occurring in the loop. A postcondition is an assertion about a variable's value that holds after iteration, if the loop terminates. The loop may represent a program fragment that is still under development. Reducing contextual information this way is practically motivated because such information is commonly absent in realistic unverified programs.

Listing 60 shows a canonical verification problem. Given a specification with a precondition (assume) and a loop command (for), the goal of formal verification is to prove that the postcondition (assert) is satisfiable. The program *LucidLoop*, in Listing 61, shows the problem variant we address in the paper. When precise variable values, precondition, and iteration count are unknown, what postconditions (⊙) can we infer from the syntax?

```
assume( $X_2 \leq 10 \wedge X_4 == 2 \wedge X_5 > 4$ );  
for( $i=0; i < 10; i++$ ) {  
     $X_3 = X_2 * X_2$ ;  
     $X_3 = X_3 + X_5$ ;  
     $X_4 = X_4 + X_5$ ; }  
assert( $X_3 \leq 100 + X_5 \wedge X_4 \geq 42$ );
```

Listing 60: Precise context (ideal).

```
for( $i=0; i < X_1; i++$ ) {  
     $X_3 = X_2 * X_2$ ;  
     $X_3 = X_3 + X_5$ ;  
     $X_4 = X_4 + X_5$ ; }  
assert(⊙);
```

Listing 61: Imprecise context (ours), *LucidLoop*.

Our analysis infers postconditions that express the growth of variable values in the loop. Adding postconditions supports discovery of other specification conditions, like invariants,

that then permit formal verification (the Duet analyzer in §3.1.6.2 is an example). The analysis is sound, guaranteeing that if a postcondition can be inferred, it is known to hold.

As result, our analysis generates mwp-bounds. These are symbolic expressions describing the value growth of the program variables. An *mwp-bound* (Jones & Kristiansen, 2009) represent a variable's *final value* in terms of the *initial values* and omitting constants. An mwp-bound is an approximation of the upper bound of the final value. A critical idea is that the **mwp-bounds are postconditions**. Since the problem formulation assumes no preconditions, a function in terms of the inputs is the most precise achievable postcondition in most cases. An mwp-bound is maximally polynomial in form. A variable value growth that is beyond a polynomial is not expressible as an mwp-bound. However, our experiments (§3.1.7) suggest that this is not a prohibitive limitation. The analysis successfully assigns postconditions to most variables in a set of diverse benchmarks.

Based on the mwp-bound form, we categorize the variables as linear, iteration-independent, iteration-dependent, or inconclusive (in increasing order). We discuss the semantics of the categories in §3.1.5.4; but briefly, they are behavioral descriptors. For example, a linear variable has only a light dependency on the containing loop because its value is updated by at most a constant. While our analysis is not complete for arbitrary programs, we can guarantee that the inferred mwp-bounds are optimal (§3.1.5.1) w.r.t. in the expressiveness of the flow calculus. In other words, we find the least upper bound that describes variable value growth.

At program point ②, our analysis gives the following result.

- Variables X_1 , X_2 , and X_5 values have grown at most linearly from the initials
- Variable X_3 value is iteration-independent and bounded by $\max(X_3, X_2 + X_5)$
- Variable X_4 value is iteration-dependent and bounded by $X_4 + X_1 \times X_5$

We can determine manually the precise postconditions for comparison. The linear variables never change from their initial values. The final value of X_3 is its initial value X_3 or $X_2^2 + X_5$, depending on if the loop iterates. For variable X_4 , the precise postcondition is $X_4 + X_1 \times X_5$.

3.1.1.2 Contributions

1. Our main result is an automatic analysis for postcondition inference in numerical loops (§3.1.5). The analysis is applicable in imprecise contexts and in absence of initial variable values and program annotations.
2. To obtain the analysis, we extend the flow calculus of mwp-bounds with two new capabilities: locating optimal bounds and bounding variables in presence of whole-program derivation failure (§3.1.4). This provides a strictly more expressive system than prior formulations of the flow calculus.
3. To materialize our theory, we implement mwp_ℓ for analyzing numerical loops in C (§3.1.5.5). mwp_ℓ is already integrated into a public static analyzer `pymwp`, extending the utility of our results beyond the paper presentation.
4. We demonstrate the relevance and effectiveness of the technique through analyzer comparisons (§3.1.6) and experiments (§3.1.7). The findings show our technique is orthogonal to alternatives, generalizes to natural algorithms, and infers postconditions for most variables in the evaluated benchmarks.

More broadly, our contributions are refreshing in three ways. They strengthen the existing connections (Nguyen, Antonopoulos, Ruef, & Hicks, 2017; Nguyen, Kapur, Weimer, & Forrest, 2014) between complexity theory and formal verification. We show how to transform a technique from complexity theory to a practical application, which is non-trivial and

rarely done (Aubert, Rubiano, Rusch, & Seiller, 2022a; Moyen, 2017). Finally, we show that constructing static analyses around small core languages can be beneficial, if we assume a compositional analysis that covers a sufficiently large class of programs.

3.1.2 A High-Level Picture

3.1.2.1 Conceptual primer of the flow calculus

Our postcondition inference is based on the *flow calculus of mwp-bounds* (Aubert, Rubiano, Rusch, & Seiller, 2022a; Jones & Kristiansen, 2009). It is a data flow analysis that tracks *variable value growth* between commands in imperative programs. The analysis aims to discover a polynomially bounded data-flow relation between the *initial values* $x_1, \dots, x_n$, for natural-number variables $X_1, \dots, X_n$, and the *final values* x'_i of X_i (for $i = 1, \dots, n$). More precisely, the flow calculus aims to guarantee variables are always polynomially bounded, including in every intermediate program state. For example, the program $X1=X2+X3; X1=X1+X1$ is satisfactory, because the final value of each variable grows at most polynomially w.r.t. inputs. Variables $X2$ and $X3$ do not change and final value of $X1$ is $2 \times (X2 + X3)$. In contrast, the program $X1=1; \text{while}(X2)\{X1=X1+X1\}$ is unsatisfactory, because variable $X1$ grows exponentially in 2^{X2} . When satisfactory value growth can be confirmed for all variables, the analysis assigns the program a *bound* that characterizes the value growth. The analysis result is derived statically, by applying inference rules to the commands of the program, in a compositional bottom-up manner.

Value growth as data flow graphs. As an internal bookkeeping procedure, the flow calculus collects information in matrices (§3.1.3). For enhance intuition, we represent these matrices also as data flow graphs (Fig. 26). A *data flow graph* (DFG) of program $\mathcal{C}$ is a

multigraph of n vertices where n is the number of variables involved in $\mathcal{C}$. The vertices are the variables of $\mathcal{C}$ and the edges denote dependence between variables.

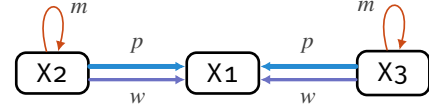
Flow coefficients. The edges of a DFG are quantified by flow coefficients. The coefficients characterize the weight of the dependence between variables. When no dependence exists, the coefficient is 0 (we omit 0-edges in DFGs). An m denotes a maximally linear dependence. Inside loops, a linear dependence permits direct data flows between variables, but prohibits arithmetic operations. The next stronger dependence, w , stands for weak polynomial. The w coefficient permits some arithmetic operations, but with restriction. Inside loops, the value growth must be eventually unaffected by changes in the loop iteration count. The strongest permissible dependence is polynomial. A p coefficient signals that a variable value grows at most polynomially throughout computation. Finally, other forms of dependence are either beyond polynomial or not expressible. The ∞ coefficient marks these cases as *failure*. The flow calculus is nondeterministic, therefore multiple coefficients may characterize one pair of variables.

```

X1=X2+X3;
X1=X1+X1;

```

(a) A polynomially bounded program.



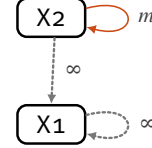
(b) The data flows of program 26a.

```

X1=1;
while(X2)
  X1=X1+X1

```

(c) An exponential program.



(d) The data flows of program 26c.

→ *max. linear*
 → *weak polynomial*
 → *polynomial*
 - - - - - ∞ failure

Figure 26: Simple imperative programs and their corresponding data flow graphs. Edges below ∞ represent acceptable value growth. In the left program, all variable values grow at most polynomially, i.e., satisfactorily. In the right program, variable X1 grows exponentially. This is “too fast” for a polynomial bound.

Analysis result and derivability. If all final values can be bounded by polynomials, then the input program is *derivable*. The flow calculus assigns an *mwp-bound* to every variable of a derivable program. An mwp-bound is an expression, in terms of the inputs, that characterizes the value growth of a single variable. If a derivable program terminates, the soundness theorem of the flow calculus (Jones & Kristiansen, 2009, p. 12) guarantees that the variable value growth is polynomially-bounded. The result is sound but not complete, i.e., not all satisfactory programs can be captured, but on success we can prove the result with a derivation. Since the analysis omits termination, this is a partial correctness guarantee.

The flow calculus offers no guarantee to programs that are not derivable; their variable value growth is *unknown*. A program always fails if some variable value grows “too fast”, e.g., exponentially. Failure may also arise from inability to express satisfactory behavior. This

is a built-in limitation of the flow calculus; and more broadly, a common limitation among related systems (Baillot, Barthe, & Dal Lago, 2015, p. 2). The flow calculus alleviates the issues with nondeterministic inference rules. The nondeterminism increases expressiveness, to capture a larger class of programs. But one program may admit multiple derivations, which in turn complicates the analysis procedure. A program is derivable if there exists a derivation without an ∞ coefficient.

3.1.2.2 Loop specifications for formal guarantees

Formally verifiable programs are defined as precise mathematical models through specifications. Verifying the program requires giving a proof that the program satisfies its specification. The motivation for using formal methods is the strength of guarantees it provides. In contrast to tests and inspection, a proof conclusively ensures behavioral correctness at the modeled level of abstraction.

Specifications. A specification is a formal contract with three components. 1. *Precondition* P , is the initial logic expressions assumed to hold before entering the loop. 2. Program, `loop b C`, performs command C until the expression b becomes false. 3. *Postcondition* Q , is the final logic expressions asserted to hold after the loop terminates.

Using a Hoare triple (Hoare, 1969), we can express the specification as $\{ P \} \text{ loop } b \ C \ \{ Q \}$. The implementation of the program is correct if we can construct a proof that shows the program satisfies its specification in all states. A correctness proof requires verifying that every computation terminates, every call to another procedure satisfies its preconditions, and the postcondition holds at program termination (Furia & Meyer, 2010). For an example of specifications, refer to Appendix 3.1.9.

Relating postconditions and loop invariants. A postcondition is a higher-level view describing the program goal. In case of loops, a *postcondition* is a conditions that must hold after the loop terminates. The postcondition inference problem can be expressed informally as follows. Given a precondition P (in our formulation a constant true, $\top$), and a program $\text{loop } b \ C$, the inference problem involves finding a postcondition Q that satisfies the inference rule $P = \top, \{ b \} C \{ \top \}, \neg b \rightarrow Q \vdash \text{loop } b \ C$. Postconditions inferred using the flow calculus are provable by the soundness theorem. However, pre- and postcondition alone are typically too weak to prove a program correct. Loop invariants are needed to make the specification provable.

An *invariant* describes an assertion about a program location that must be true for every program state reaching the location (Furia & Meyer, 2010; Nguyen, Nguyen, & Duong, 2022).¹ A loop invariant is a weakened form of the postcondition (Furia & Meyer, 2010). An invariant is inductive if it holds the first time the location is reached and is preserved in every cycle returning to the location (Sankaranarayanan, Sipma, & Manna, 2004). Verifying loops requires discovering sufficiently strong inductive loop invariants to prove the specification. For an invariant to be sufficient, it must be weak enough to be derived from the precondition and strong enough to conclude the postcondition.

Inductive loop invariants and postconditions are symbiotic: the discovery of one assists finding the other (Furia & Meyer, 2010). Every invariant is necessarily a (weak) postcondition, as it must hold at termination, but a postcondition must not be invariant. For example, given natural numbers i and n , the loop `for( $i=0; i < n; i++$ )  $i++$ ;` has an invariant $0 \leq i \leq n$ and a postcondition $i = n$. Invariant inference techniques may assume preconditions and postconditions are known. However, this assumption is impractical, since manually adding specifications to code fragments is non-trivial.

¹A postcondition is also an invariant; for clarity, we always refer to a postcondition as such and use ‘invariant’ to refer to an inductive loop invariant.

3.1.3 Technical Preliminaries of the Flow Calculus

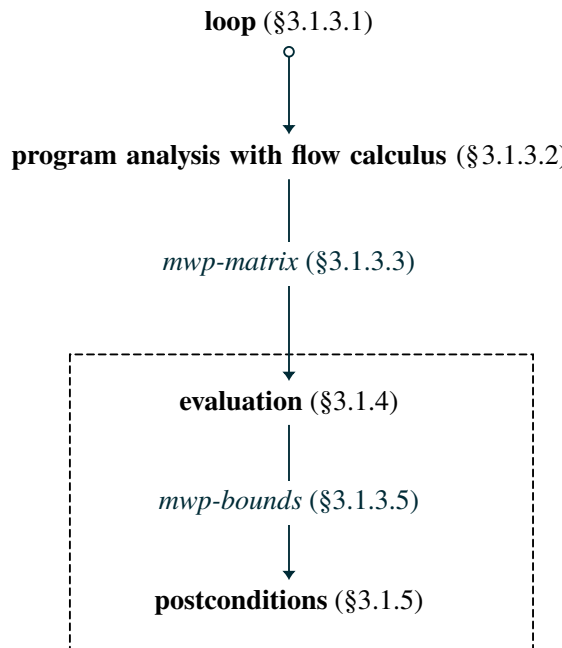


Figure 27: The mwp-analysis workflow.

The Fig. 27 illustrates the complete analysis workflow. The boxed region highlights the steps and enhancements introduced in this paper. The region outside the box refers to the existing techniques we build on.

The analysis takes as input an imperative loop (§3.1.3.1). We analyze the loop using the inference rules of the flow calculus (§3.1.3.2). As output, the analysis produces an *mwp-matrix* (§3.1.3.3). Our enhancements involve evaluating these mwp-matrices. We define first an evaluation procedure (§3.1.4.2) that enables extracting optimal mwp-bounds from mwp-matrices, and in more cases than previously (§3.1.4.4). This provides the postconditions that address the problem formulation of §3.1.1.1.

3.1.3.1 Imperative language of input programs

Definition 1 (Imperative language). *Letting natural number variables range over X and Y and Boolean expressions over b , we define expressions e , and commands C as follows*

$$e := X \parallel X - Y \parallel X + Y \parallel X * Y$$

$$C := \text{skip} \parallel X = e \parallel \text{if } b \text{ then } C \text{ else } C \parallel \text{while } b \text{ } C \parallel \text{loop } X \text{ } C \parallel C;C$$

The command $\text{loop } X \text{ } C$ means “do C X times” and the variable X is not allowed to occur in command C . Command $C;C$ is for sequencing. We write “program” for a series of commands composed sequentially. □

We implicitly convert between conventional for loops and loops of the imperative language in examples. The values of Boolean expressions do not matter. For emphasis, we substitute b with $*$ in control expressions. Although the base language is rudimentary, it captures a core fragment of conventional mainstream programming languages, like C and Java. Our technique applies to all languages sharing the same core, including intermediate representations. Similarly, the approach is applicable even if a programming language provides little structure.

3.1.3.2 About construction of mwp-matrices

How mwp-matrices are constructed is not crucial for the developments of this paper. The construction procedure is well-established in literature (Jones & Kristiansen, 2009, Sect. 5–6) and (Aubert, Rubiano, Rusch, & Seiller, 2022a, Sect. 2–3).² We give a brief overview,

²Our work assumes the variant described in (Aubert, Rubiano, Rusch, & Seiller, 2022a).

but make no adjustment to the construction procedure. Rather, mwp-matrices are our inputs of interest. We focus on *interpreting* the information they encode.

During matrix construction, the flow calculus assigns matrices to commands of the input program. The procedure resembled typing, except the “types” are complicated matrices. An *mwp-matrix* M captures data flow facts about the analyzed program $\mathcal{C}$. The matrices track, through flow coefficients, how data flows in variables between commands. What coefficients are assigned is determined by the *inference rules* of the flow calculus. The relevant inference rules are included in Figures 28 and 30.³ To trade precision for efficiency, the flow calculus omits evaluating loop iteration bounds and reasoning about termination and control expressions, overestimating their effect. At conclusion, the flow calculus assigns one mwp-matrix to the analyzed program, $\vdash \mathcal{C} : M$. It contains all derivations of the input program in one (complex) data structure.

$$\begin{array}{c}
 \frac{}{\vdash e : \{x_i^w \mid x_i \in \text{var}(e)\}} \text{ E2} \qquad \frac{\vdash x_i : V_1 \quad \vdash x_j : V_2}{\vdash x_i + x_j : pV_1 \oplus V_2} \text{ E3} \qquad \frac{\vdash x_i : V_1 \quad \vdash x_j : V_2}{\vdash x_i + x_j : V_1 \oplus pV_2} \text{ E4}
 \end{array}$$

Figure 28: A selection of flow calculus rules for assigning vectors to expressions. A variable is denoted as x_i where the i refers to an index of a matrix column (resp. x_j for the j th column). In rules E3 and E4, a vector of coefficients is denoted by V_n . In other words, the rules assign vectors of coefficients to variables. Three rules can be used to analyze a single expression. The nondeterminism of the flow calculus comes from these inference rules.

Nondeterminism is necessary to improve the expressive power of the flow calculus, but complicates the analysis procedure. There are the three rules (Fig. 28) to analyze an expression of binary addition,⁴ like $x_1 + x_2$. Informally, the rule E2 says “we can always assign a w coefficient to all variables in the expression”. Although such constant treatment seems potentially harmful, the inference rules for analyzing commands (Fig. 30) ensure only ac-

³To ease the presentation, we show rules of the original flow calculus (Jones & Kristiansen, 2009). These rules are consistent with the enhanced variant (Aubert, Rubiano, Rusch, & Seiller, 2022a) we build on.

⁴Similar rules exist for subtraction, and multiplication is always handled by E2.

ceptable data flow patterns are accepted as derivable. The rule E3 says “the vector of the left operand is multiplied by a polynomial coefficient.” The p -coefficient tracks the fact that certain data flow patterns might not be polynomially bounded (Jones & Kristiansen, 2009, p. 14). Applying the rules to a vector propagates the effect to all variables that are transitively dependent on the left operand. Rule E4 is similar to E3, except the p coefficient is applied to the right operand. Thus, three different derivation can arise from one expression, and from every command that contains the expression. Going forward, we omit the details of these rules and instead refer to them by a mapping $0 \mapsto E4, 1 \mapsto E3, 2 \mapsto E2$.

3.1.3.3 Our starting point of interest: interpreting mwp-matrices

An mwp-matrix represents the derivations of an analyzed program. Before diving into full mwp-matrices, this section describes its *elements*.

Basic terms. Flow coefficients—described in §3.1.2.1—are the core building blocks of mwp-matrices. The flow coefficients are elements of the finite set $\text{MWP}^\infty \triangleq \{0, m, w, p, \infty\}$. A domain $\mathcal{D} \triangleq \{0, 1, 2\}$ is a bidirectional map of the flow calculus inference rules. In other words, the domain members can be translated to flow calculus inference rules (E2–E4) and vice versa. The *degree* of choice, $k : \mathbb{N}$, is a counter of derivations. The degree represents the number of times a derivation choice must be made during program analysis. The degree is precisely equal to the count of binary arithmetic operations (where the operator is either $+$ or $-$).

Definition 2 (Derivation choice). *Letting $\mathcal{D}$ be a domain and $k \in \mathbb{N}$ be the degree of choice, we define a derivation choice as $\delta(i, j)$, where $i \subseteq \mathcal{D}$ and $j \leq k$.* □

A derivation choice, denoted by δ , represents the nondeterminism of the flow calculus. We call i the *value* and j the *index* of the derivation choice. Informally, it captures that an inference rule i is applied at program point j .

Definition 3 (Derivation choice sequence). *Letting k be the degree of choice, we define a derivation choice sequence as $\Delta = (\delta_1, \delta_2, \dots, \delta_{k'})$ where $0 \leq k' \leq k$.* $\square$

Since a program is a *series* of commands, correspondingly we must have *sequences* of derivation choices, denoted as Δ . Simple data flow patterns do not require making derivation choices; therefore Δ can be empty. There are two restrictions on Δ . First, an index j is allowed to occur at most once in a sequence, i.e., the indices in a sequence must be unique. Second, the sequence is sorted by the index in ascending order. These restrictions support efficient computation during matrix construction. Since we are only concerned with interpreting mwp-matrices, we assume every Δ satisfies the uniqueness and order properties by construction. By Fig. 28, it is evident that one variable can be assigned multiple flow coefficients. The fact that Δ reduces to a particular coefficient is represented in a *monomial*.

Definition 4 (Monomial). *Letting $\alpha \in \text{mwp}^\infty$ be a coefficient and Δ be a sequence of derivation choices, we define a monomial as (α, Δ) .* $\square$

A monomial in the flow calculus is disjoint from the classic notion of monomials. For example, both $m.(\delta(0, 0), \delta(2, 1))$ and 0 are valid monomials by definition. The former says “if we apply rule 0 then rule 2 (at indices 0 and 1, resp.) then we obtain an m coefficient”. The latter says “no matter the choice, we always obtain a 0 coefficient”. Finally, the fact that different Δ reduce to different coefficients is represented by a *polynomial structure*.

Definition 5 (Polynomial structure). *We define a polynomial structure as a sequence of monomials $(0, (\alpha_1, \Delta_1), (\alpha_2, \Delta_2), \dots, (\alpha_m, \Delta_m))$ where $m \geq 0$.* $\square$

The polynomial structure is prefixed by 0-monomial to ensure it is always non-empty. **The elements of an mwp-matrix are polynomial structures.**

3.1.3.4 Decoding an mwp-matrix by example

An mwp-matrix M associates variables with polynomial structures.⁵ This association is defined by position. The matrix size is determined by the number of program variables, such that for n variables, the matrix size is $n \times n$. The matrix is labelled by the variables. An mwp-matrix is interpreted column-wise. The data-flow facts about a variable at column j are collected in the polynomial structures at rows $i = (1, \dots, n)$ in M_{ij} .

Example 20 (mwp-matrix of LucidLoop). *Consider the mwp-matrix of LucidLoop.*

<pre>for(i=0;i<X1;i++) { X3=X2*X2; X3=X3+X5; ① X4=X4+X5; ② }</pre>	:	<table><tr><th></th><th>X1</th><th>X2</th><th>X3</th><th>X4</th><th>X5</th></tr><tr><th>X1</th><td>m</td><td>0</td><td>$p(0,0), p(1,0)$</td><td>$p(0,1), \infty(1,1), \infty(2,1)$</td><td>$0$</td></tr><tr><th>X2</th><td>0</td><td>m</td><td>$w(0,0), p(1,0), w(2,0)$</td><td>$\infty(1,1), \infty(2,1)$</td><td>0</td></tr><tr><th>X3</th><td>0</td><td>0</td><td>m</td><td>$\infty(1,1), \infty(2,1)$</td><td>0</td></tr><tr><th>X4</th><td>0</td><td>0</td><td>0</td><td>$m, \infty(1,1), \infty(2,1)$</td><td>$0$</td></tr><tr><th>X5</th><td>0</td><td>0</td><td>$p(0,0), m(1,0), w(2,0)$</td><td>$p(0,1), \infty(1,1), \infty(2,1)$</td><td>$m$</td></tr></table>		X1	X2	X3	X4	X5	X1	m	0	$p(0,0), p(1,0)$	$p(0,1), \infty(1,1), \infty(2,1)$	0	X2	0	m	$w(0,0), p(1,0), w(2,0)$	$\infty(1,1), \infty(2,1)$	0	X3	0	0	m	$\infty(1,1), \infty(2,1)$	0	X4	0	0	0	$m, \infty(1,1), \infty(2,1)$	0	X5	0	0	$p(0,0), m(1,0), w(2,0)$	$p(0,1), \infty(1,1), \infty(2,1)$	m
	X1	X2	X3	X4	X5																																	
X1	m	0	$p(0,0), p(1,0)$	$p(0,1), \infty(1,1), \infty(2,1)$	0																																	
X2	0	m	$w(0,0), p(1,0), w(2,0)$	$\infty(1,1), \infty(2,1)$	0																																	
X3	0	0	m	$\infty(1,1), \infty(2,1)$	0																																	
X4	0	0	0	$m, \infty(1,1), \infty(2,1)$	0																																	
X5	0	0	$p(0,0), m(1,0), w(2,0)$	$p(0,1), \infty(1,1), \infty(2,1)$	m																																	

We mark the program points where a derivation choice must be made as ① and ②. The program points correspond to indices 0 and 1, respectively. These points give the mwp-matrix a degree of 2. The addition expressions are analyzed by the inference rules of Fig. 28. The mwp-matrix contains derivations choices at columns X3 and X4 because they are the data-flow targets of the additions. Correspondingly, in Fig. 29, the variables are targets of dependence edges.

⁵For clarity and compactness, we omit needless 0's, δ -symbols, and delimiters when displaying mwp-matrices.

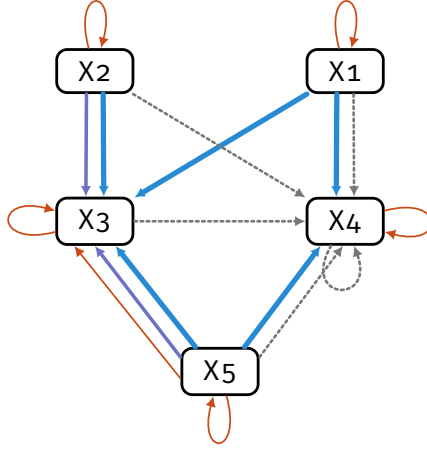


Figure 29: A data-flow graph of LucidLoop.

Variables X_1 , X_2 , and X_5 are assigned at most m coefficients. An m -flow at the mwp-matrix diagonal means the variable depends on its (own) initial value. Since the program has no commands that would introduce multiple derivations on these variables, their polynomial structures are simple coefficients. Variable X_3 shows varying dependencies on X_1 , X_2 , and X_5 at corresponding rows. The coefficients indicate that the dependencies are at most polynomial (similarly observable in the DFG). The absence of ∞ means the value growth of X_3 is acceptable in all derivations. Variable X_4 is assigned ∞ in some derivations. Intuitively, this is because the value of X_4 accumulates at each loop iteration and its value growth is potentially problematic. The ∞ -flow signals that some derivations fail and flags X_4 as the source of failure. However, this is the current extent of our capabilities to describe X_4 (Aubert, Rubiano, Rusch, & Seiller, 2023b). $\square$

This paper develops enhanced strategies to extract more precise information from mwp-matrices. poly_2 Although we show only compact cases, an mwp-matrix captures 3^k derivation choices where k is the degree. Clever solutions are necessary to explore and interpret the mwp-matrix data efficiently, but have been missed by previous flow calculus refinements (Aubert, Rubiano, Rusch, & Seiller, 2022a, 2023b).

3.1.3.5 Interpreting mwp-bounds

The mwp-matrix columns encode the variable value growth bounds. An mwp-bound is an expression of form $\max(\vec{x}, \text{poly}_1(\vec{y})) + \text{poly}_2(\vec{z})$. Variables characterized by m -flow are listed in $\vec{x}$; w -flows in $\vec{y}$, and p -flows in $\vec{z}$. Variables characterized by 0-flow do not occur in the expression. No bound exists if some variable is characterized by ∞ . The poly_1 and poly_2 are *honest polynomials*, build up from constants and variables by applying $+$ and $\times$. The variables with w -flow occur in poly_1 and variables with p -flow in poly_2 . Any of the three variable lists might be empty, and poly_1 and poly_2 may not be present. When characterizing value growth, an mwp-bound is approximative. It excludes precise constants and degrees of polynomials. An mwp-bound is a monotonic over-approximation, i.e., it does not decrease if variable value decrease over subtraction. A program *bound* is a conjunction of its variables' mwp-bounds. Ex. 21 shows illustrative cases of mwp-bounds. The differences between the bound forms are discussed in Sections 3.1.5.1 and 3.1.5.4.

Note 1. For variable X and an mwp-bound W , we write $X' \leq W$ to denote the mwp-bound of variable X . We use the overloaded notation X' to refer to the variable's final value. Variables in W always refer to initial values.

Example 21. The mwp-bound (on left) is interpreted as, the final value of...

$$\begin{aligned} X_2' &\leq X_2 && \dots X_2 \text{ is bounded linearly by its initial value.} \\ X_3' &\leq \max(X_3, X_2 + X_5) && \dots X_3 \text{ is bounded by a weak polynomial in } X_3 \text{ or } X_2 + X_5. \\ X_4' &\leq \max(X_4) + X_1 \times X_5 && \dots X_4 \text{ is bounded by a polynomial in } X_4 \text{ and } X_1 \times X_5. \end{aligned}$$

□

3.1.4 Variable-Guided Matrix Exploration

3.1.4.1 Addressed limitations

We introduce two solutions to address current limitations of the flow calculus.

1. An mwp-matrix *evaluation strategy*, to efficiently determine if an individual variable admits an mwp-bound of specific form.
2. Improved *derivation failure handling*, to identify variables that maintain acceptable value growth in presence of whole-program derivation failure.

The nondeterminism of the flow calculus permits capturing more programs, but creates a potential state explosion problem. Effectively handling this aspect becomes critical in the second analysis phase, where an mwp-matrix is *evaluated* to find mwp-bounds of variables. The first challenge is finding the derivation choices that do not produce failure. The second challenge is determining the coefficients assigned to each variable. The coefficients are not obvious from the derivation choices of an mwp-matrix; rather, they require an application. An *application* reduces the polynomial structures to simple coefficients, which then leads to derivation failure or yields a program bound.

A brute force solution iterates all derivations and applies all choices to observe the result. However, such naive solution is impractical, due to latency and a potential yield of exponentially many program bounds. The ideal solution should find the optimal bound (if one exists) and without the redundancy of iterating every derivation.

The evaluation strategy we present operates as follows. It takes the *unwanted* derivation choices then negates them. The evaluation result captures the *permissible* choices. The term permissible is abstract – the meaning depends on what is unwanted. For example, if the derivation choices of ∞ coefficients are unwanted, the permissible choices are those

that avoid failure, i.e., non- ∞ . The result of an evaluation is a *disjunction of choice vectors* that compactly capture the permissible derivation choices.

Definition 6 (Choice vector). *Letting $\mathcal{D}$ be a domain and $k \in \mathbb{N}$ be a choice degree, we define a choice vector as $\vec{C} = (c_1, c_2, \dots, c_k)$, where $c_i \subseteq \mathcal{D}$ and $c_i \neq \emptyset$ for all $i \in \{1, 2, \dots, k\}$.* □

We show choice vectors in Examples 22, 23, and 24.

3.1.4.2 Evaluation for identifying derivable programs

The precise mwp-matrix evaluation strategy is presented in Algorithm 4 and we describe it here informally. The evaluation is parametric on three inputs: (i) the degree of choice k , (ii) domain $\mathcal{D}$, and (iii) a set of unwanted derivation choice-sequences, $\mathcal{S} = \{\Delta_1, \dots, \Delta_n\}$ where $n \geq 0$. Observe that all parameters are obtained from an mwp-matrix, but the mwp-matrix or its coefficients are not used. The evaluation returns a list of choice vectors. In the maximally permissive case, the result is a single choice vector permitting everything. If no result exists or no choice is necessary, the result is an empty list. These outcomes are handled as base cases (Lines 3–7). All interesting evaluations fall between these extremes.

The evaluation operates in two steps. First it simplifies $\mathcal{S}$ to the minimal length, non-empty sequences while preserving its effect. Then, it generates the choice vectors by negating the remaining unwanted choices. The procedure is practically efficient because simplification eliminates redundancy before the choice vector are generated. Internally, it benefits from the finiteness of the domain and from having advance complete knowledge of all derivation choices.

Input: degree k ($\mathbb{N}$), domain $\mathcal{D}$ (set), derivation choice-sequences $\mathcal{S}$ (set)

Output: choice vectors $\mathcal{C}$ (list)

```

1   $\mathcal{C} = \varepsilon$ 
2  ▷ Handle base cases
3  if  $k = 0$  or  $|\mathcal{D}| \leq 1$  or  $\mathcal{S} = \emptyset$  then
4      if  $\mathcal{S} = \emptyset$  then
5          Create  $\vec{c} := \text{ChoiceVector}(k, \mathcal{D})$           ▷  $k$ -length vector of elements in  $\mathcal{D}$ 
6           $\mathcal{C} = \vec{c} :: \mathcal{C}$ 
7      return  $\mathcal{C}$ 
8  ▷ Step 1: Simplify  $\mathcal{S}$  until convergences
9  do Capture initial  $size := |\mathcal{S}|$           ▷  $\mathcal{O}(\mathcal{S}^3)$ 
10      $\mathcal{S} = \text{SIMPLIFY}(\mathcal{S})$ 
11     while  $size \neq |\mathcal{S}|$ 
12     ▷ Step 2: Generate choice vectors
13     Compute  $P :=$  the product of sequences in  $\mathcal{S}$ 
14     for all paths  $p$  in  $P$  do          ▷  $\mathcal{O}(\prod_{s \in \mathcal{S}} |s|)$ 
15         Create  $\vec{c} := \text{ChoiceVector}(k, \mathcal{D})$ 
16         for all  $(i, j)$  in  $p$  do          ▷  $\mathcal{O}(|\mathcal{S}|)$ 
17             Remove  $i$  at  $\vec{c}_j$ 
18         if  $\forall j, \vec{c}_j \neq \emptyset$  then
19              $\mathcal{C} = \vec{c} :: \mathcal{C}$ 
20     return  $\mathcal{C}$ 

```

Algorithm 4: mwp-matrix evaluation

The simplification step (Line 10) is crucial. It must be sound and complete to ensure all distinct derivation patterns are preserved in $\mathcal{S}$, but also reduces $\mathcal{S}$ to a minimal set. Different simplifications are applied on $\mathcal{S}$ iteratively until convergence, including the following.

Remove super-sequences. When a sequence leads to an unwanted outcome, every longer sequence producing the same outcome is redundant. For example, if $\mathcal{S}$ contains $((0, 0), (0, 1), (1, 2))$ and $((0, 1))$ we remove the longer sequence.

Head-elimination. If many non-singleton sequences differ only on their head element value, and the values cover the domain $\mathcal{D}$, then the head is redundant. The sequences can be replaced by one sequence without the head element. For example, the sequences $((0, 0)(1, 1)), ((1, 0)(1, 1)), ((2, 0)(1, 1))$ can be replaced by $((1, 1))$.

Tail-elimination. By symmetry, the same as above, but on the tail element.

The generation step (Lines 13–19) produces the choice vectors. They are constructed by computing the cross product of $\mathcal{S}$. We take one derivation choice from each sequence, then eliminating those choices. This prevents choosing any unwanted sequence completely. If the choice vector elements are non-empty after elimination, we appended the choice vector to the result (Line 19). We have annotated the computational costs of the iterative steps in Algorithm 4. The algorithm obviously terminates. Since the generation step involves a product, it is a source of potential inefficiency. However, we have not encountered natural programs where the simplification does not reduce $\mathcal{S}$ sufficiently to make the generation step problematic. A full implementation of the algorithm is included in our artifact, and described in the documentation of pymwp.⁶

Example 22 (LucidLoop is derivable). *In Ex. 20, the distinct monomials that cause failure are $\infty.\delta(1, 1)$ and $\infty.\delta(2, 1)$. The mwp-matrix degree is $k = 2$, the domain is $\mathcal{D} = \{0, 1, 2\}$,*

⁶See: <https://statycc.github.io/pymwp/choice>

and the unwanted derivation choice-sequences are $\mathcal{S} = \{((1, 1)), ((2, 1))\}$. The evaluation returns a choice vector $(\{0, 1, 2\}, \{0\})$ witnessing successful derivation choices. The choice vector communicates that we can select any derivation rule to analyze command ①, but must apply “inference rule 0” (E4) at command ②. $\square$

The application of a choice vector requires making a selection at each vector index, then applying the selections to the polynomial structures of the mwp-matrix. A monomial evaluates to $\delta(i, j) = \alpha$ if the j th choice is i , and 0 otherwise. A polynomial structure evaluates to its maximal coefficient. Thus, the application produces the maximal coefficient among the monomials (cf. Ex. 24).

3.1.4.3 Variable projection and querying mwp-bounds

Until now, the flow calculus of mwp-bounds has been restricted to deriving mwp-bounds for all variables concurrently. For postcondition inference, we want to obtain information about *individual* variables. Since Algorithm 4 does not require mwp-matrices or coefficients as input, it can be directly reused for variable-specific evaluations.

Analyzing variable X_j requires taking derivation choices only from column j , instead of the entire mwp-matrix. To determine if a variable admits a particular form of mwp-bound, we issue “queries” against the evaluation procedure, altering the derivation choices provided as parameter $\mathcal{S}$. For example, to find derivations with at most m coefficients (and whether they exist), we take the derivation choices from monomials whose coefficient are in $\{w, p, \infty\}$. If the evaluation returns a choice vector, it specifies the derivations where the variable is assigned an mwp-bound of at most m coefficients. Bounds with at most w or p can be queried similarly, by adjusting $\mathcal{S}$.

Example 23 (Variable X_3 is bounded by at most w coefficients). In Ex. 20, variable X_3 does not have a derivation with at most m coefficients. This can be determined by evaluation of the distinct choices in column X_3 with $\{w, p, \infty\}$ coefficients, i.e., $\mathcal{S} = \{((0,0)), ((1,0)), ((2,0))\}$. This evaluation does not return a choice vector. However, an mwp-bound of at most w exists, by the choice vector $(\{2\}, \{0, 1, 2\})$. $\square$

Variable query is safe for derivable programs, because all variables are guaranteed to have an mwp-bound by the soundness theorem of the flow calculus (Jones & Kristiansen, 2009, p. 12). Additional caution is needed when a program is not derivable.

3.1.4.4 Variable mwp-bounds in presence of failure

Derivation failure arises from the inference rules of commands `while` and `loop`, in Fig. 30.

$$\begin{array}{c} \forall i, M_{ii}^* = m \frac{\vdash C : M}{\vdash \text{loop } X_\ell \{C\} : M^* \oplus \{ \ell^p \rightarrow j \mid \exists i, M_{ij}^* = p \}} \text{ L} \\ \\ \forall i, M_{ii}^* = m \text{ and } \forall i, j, M_{ij}^* \neq p \frac{\vdash C : M}{\vdash \text{while } b \text{ do } \{C\} : M^*} \text{ W} \end{array}$$

Figure 30: The flow calculus inference rules for commands `while` and `loop`.

The interesting parts about these rules are the side conditions. For the `loop` rule (L), the side condition says a loop can be analyzed if the flow coefficients at the mwp-matrix diagonal are at most m . The `while` (W) rule is more restrictive. In addition to diagonal m , no p coefficients are allowed to occur anywhere in the mwp-matrix. These side conditions ensure variable values grow at most polynomially over repeated executions of the command body.

A program that does not satisfy the side conditions is not derivable. Then, variable value growth is inconclusive and no bound is assigned to its variables. The flow calculus offers no guarantees to programs that are not derivable. A single rogue variable can cause a whole-program derivation failure. This treatment of failure limits the utility of the analysis. Since our goal is to infer postconditions of loops, this restriction excludes many programs we want to analyze.

Improving the failure handling. Our intuition is that the mwp-matrices accumulate information that permits reasoning about certain variables in presence of failure. The main new idea is to *isolate the failing variables* from those with acceptable behavior. If such isolation is possible, then the remaining variables can be analyzed. The idea is conservative because it does not change the rules of the flow calculus. Rather, the improvement is based on leveraging fully the information already collected in mwp-matrices. We make only the analytical adjustment illustrated in Fig. 31. Our treatment of failure is sound by the guarantees of the original flow calculus – explained shortly after the motivating example.

Example 24 (Evaluating an always failing derivation). *The program in Fig. 31a is not derivable because variable X_4 fails in every derivation. The area of failure, that we want to isolate, is shaded in the mwp-matrix of Fig. 31b. Not all variables are problematic. There is no dependency from X_4 to variables X_2 and X_5 (row X_4). Variable X_3 is more complicated due to the ∞ coefficients in column X_3 . Since we want derivations where X_3 is derivable, we compute its choice vector: $(\{2\}, \{0, 1, 2\})$. At index 0, variable X_3 permits one derivation choice: 2. Applying $(2, 0)$ to the mwp-matrix, at column X_3 , yields coefficients: $X_2 \mapsto w$, $X_3 \mapsto m$, $X_4 \mapsto 0$, and $X_5 \mapsto w$. The 0 reveals that in every derivable case variable X_3 is independent of X_4 . Therefore, variables X_3 , X_2 and X_5 can be assigned mwp-bounds. $\square$*

```

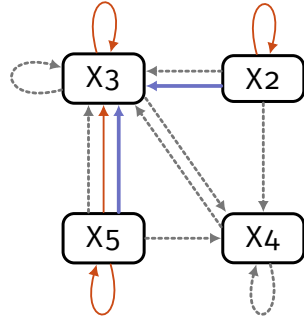
while(*) {
  X3=X2*X2;
  X3=X3+X5;
  X4=X4+X5; }

```

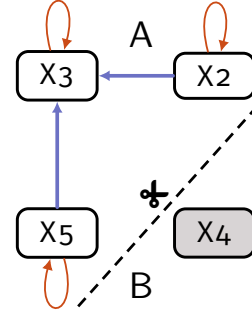
$$: \begin{array}{c} \begin{array}{ccccc} & X2 & & X3 & & X4 & & X5 \\ X2 & \left(\begin{array}{cccc} m & \infty(0,0), \infty(1,0), w(2,0) & \infty(0,1), \infty(1,1), \infty(2,1) & 0 \end{array} \right. \\ X3 & \left(\begin{array}{cccc} 0 & m, \infty(0,0), \infty(1,0) & \infty(0,1), \infty(1,1), \infty(2,1) & 0 \end{array} \right. \\ X4 & \left(\begin{array}{cccc} 0 & \infty(1,0) & \infty(0,1), \infty(1,1), \infty(2,1) & 0 \end{array} \right. \\ X5 & \left(\begin{array}{cccc} 0 & \infty(0,0), m(1,0), \infty(1,0), w(2,0) & \infty(0,1), \infty(1,1), \infty(2,1) & m \end{array} \right) \end{array} \end{array}$$

(b) mwp-matrix with failure

(a) Input program



(c) DFG of the mwp-matrix



(d) Grouping and independence

```

while(*) {
  X3=X2*X2;
  X3=X3+X5; }

```

```

while(*) {
  X4=X4+X5;
}

```

(e) Sub-programs

Figure 31: A demonstration of improved derivation failure handling. The program (31a) is a modified version of LucidLoop. The variable X_4 causes failure in every derivation (31b). Starting with 31c, we restrict the graph to derivations where non-failing variables can be bounded. This reduces the edges to those shown in 31d. The sub-program that corresponds to variables in set A (31e, above) is derivable.

Variable grouping (Fig. 31d). The whole-program derivability is first determined by §3.1.4.2. In the negative case, at least one variable must be failing. Crucially, a failing variable fails in every derivation. Therefore, it suffices to evaluate variables individually for at most p coefficients, by Algorithm 4. We group variables into two sets. Set A with variables that satisfy polynomial value growth, and set B with variables that fail. Which set each variable belongs to is determined by the existence of a choice vector.

Establishing independence from failure (Fig. 31d). Recall, the 0 coefficients track absence of data flows between variables. Thus, we can determine if sets A and B are data-flow independent from the 0s. We must ensure that there is no data flow *from* the failure set B to set A in the cases where variables in A are derivable. The condition can be determined by direct inspection of the mwp-matrix. The independence condition corresponds to a cut of the data flow graph.

Partitioning to sub-programs (Fig. 31e). When failure independence is affirmative, we partition the whole-program into two sub-programs.⁷ These sub-programs correspond to the data flows in sets A and B. No bound can be derived for the failing variables in set B. Variables in set A are analyzed by the standard rules of the flow calculus, which ensures soundness. *Derivability of this sub-program is the source of increase in the flow calculus expressivity.*

⁷This is a conceptual step; no concrete program transformation is necessary.

3.1.5 mwp-bounds as Postconditions

3.1.5.1 Optimal mwp-bounds by form

The enhanced flow calculus now provides a technique for postcondition inference. We aim to compute the optimal postconditions for each variable, if they exist. There are two unresolved concerns. First, mwp-bounds cannot be totally ordered since some forms are incomparable. Second, mwp-bounds of different form can evaluate to the same numeric polynomial. For example, the three mwp-bounds $W_1 \equiv \max(0, X_1 + X_2) + 0$ and $W_2 \equiv \max(X_1, 0) + X_2$ and $W_3 \equiv \max(X_2, 0) + X_1$ are all numerically equal to $X_1 + X_2$. Therefore, reasoning about optimality requires an alternative approach.

To establish an ordering, we leverage two built-in features of the flow calculus. The individual coefficient are ordered $0 < m < w < p < \infty$. Moreover, the mwp-bounds carry semantic meaning *by form*. The growth of a variable value is at most linear (resp. iteration-independent, iteration-dependent) if its mwp-bound contains at most m (resp. w , p) coefficients. This gives sufficient justification for our definition of optimality.

Definition 7 (Optimality). *We define an order on mwp-bounds by form, by the maximal coefficient it contains: $0 < m\text{-bound} < w\text{-bound} < p\text{-bound} < \infty(\text{none})$. A variable's mwp-bound is optimal if it is the least bound in this order.* $\square$

For example, among W_1 , W_2 , and W_3 , the w -bound $\max(0, X_1 + X_2) + 0$ is optimal because it contains no p coefficients. It supplies the evidence that the variable's value growth is eventually loop iteration independent (discussed in §3.1.5.4). The other candidates W_2 and W_3 are too weak to reach the same conclusion. Finding a single derivation that admits the optimal form is sufficient.

3.1.5.2 Variable postcondition search

When a program is derivable, or a variable is disjoint from failure, the general procedure for deriving a variable's optimal postcondition is as follows.

1. Run the mwp-matrix evaluation (Algorithm 4) iteratively. Construct the set $\mathcal{S}$ from monomials whose flow coefficient is greater than m (next w , then p).
2. Stop at the first choice vector. This solution is optimal.

Since the search focuses on individual variables, we may want to ask whether multiple postconditions can occur concurrently. To determine the answer, we take the intersection of their choice vectors (Def. 8). A non-empty intersection specifies the derivations in which both postconditions hold. Related questions about postconditions can be formulated similarly, as operations on choice vectors.

Definition 8 (Choice vector intersection). *Letting $\vec{C}_a = (a_1, a_2, \dots, a_k)$ and $\vec{C}_b = (b_1, b_2, \dots, b_k)$ be choice vectors of length k , we define the intersection of $\vec{C}_a$ and $\vec{C}_b$ as*

$$\vec{C}_a \cap \vec{C}_b = \begin{cases} (a_1 \cap b_1, a_2 \cap b_2, \dots, a_k \cap b_k), & \text{if } \nexists i \text{ such that } a_i \cap b_i = \emptyset, \\ \emptyset, & \text{otherwise.} \end{cases}$$

□

Example 25 (Successful and optimal postcondition of X_3). *By Ex. 22, `LucidLoop` is derivable by choice vector $(\{0, 1, 2\}, \{0\})$. By Ex. 23, variable X_3 is assigned its optimal postcondition in derivations $(\{2\}, \{0, 1, 2\})$. The whole-program is derivable and variable X_3 is assigned its optimal postcondition in derivations defined by choices $(\{0, 1, 2\}, \{0\}) \cap (\{2\}, \{0, 1, 2\}) = (\{2\}, \{0\})$.*

□

3.1.5.3 Program analysis for postcondition inference

Postcondition inference requires adjusting the imperative language (§3.1.3.1) to a language of loops. A *loop program* starts with a looping command (`while` or `loop`) whose body is any command in the imperative language. Using the mwp-matrix evaluation procedure, we compute postconditions for all variables that have expressible mwp-bounds. To avoid repeated analysis, we proceed from loop nests to parent and compose mwp-matrices in a bottom-up manner. Postcondition inference of loop $\mathcal{C}$ is defined as follows.

1. Extract all (possibly nested) loops from the program $\mathcal{C}$.
2. (bottom-up) For each loop l :
 - (i) Run the mwp analysis to derive the mwp-matrix, $l : M$.
 - (ii) Using Algorithm 4, evaluate M to determine if l is derivable.
 - ▷ If yes: mark every variable as satisfactory.
 - ▷ If no: mark the non-failing variables as satisfactory (§3.1.4.4).
 - (iii) Evaluate satisfactory variables for optimal postconditions (§3.1.5.2).
 - (iv) Record the postconditions of satisfactory variables.
3. Return the analysis result for $\mathcal{C}$.

3.1.5.4 Postcondition categories as descriptors of variable value behavior

A language of loops restricts the kind of computations we may encounter. The inferred postconditions can be described by the following categories.

Linear. A variable's mwp-bound is linear if its value does not change or it is a target of direct assignment (without arithmetic). Inside loops, other operations are “too strong” to retain linear behavior. In LucidLoop, variables X_1 , X_2 and X_5 are linear because their values never change.

Iteration-independent. A variable is iteration-independent if its final value depends on a fixed number of iterations (e.g., the first or last one) or eventually reaches a fixed point. Iteration independence means that, beyond the fixed point, the variable is unaffected by an increase in loop iteration count. Thus, iteration-independence is a quasi-invariance property. In LucidLoop, if the loop iterates at least once, the final value of X_3 is determined by X_2 and X_5 .

Iteration-dependent. Arithmetic computations involving multiple changing variables lead to iteration-dependent value growth. It may occur in bounded loops, but not otherwise. This is because a continuously increasing value may be boundable under finite iteration, but not if the loop is unbounded. In LucidLoop, changing the number of times the loop iterates changes the final value of X_4 respectively. This makes X_4 iteration-dependent.

Inconclusive. If no expressible postcondition exists, the variable is inconclusive. It characterizes cases where a variable's value growth is outside the previous three categories. In Ex. 24, since the loop does not terminate, the value of X_4 grows in perpetuity. This makes the variable's value growth inconclusive.

A note on numerical minimality. One variable may be assigned multiple incomparable mwp-bounds. The definition of optimality does not guarantee the selected postcondition is the numerical minimum by evaluation. For example, variable X_3 in LucidLoop is assigned three mwp-bounds $W_1 \equiv \max(X_3, X_2) + X_1 \times X_5$ and $W_2 \equiv \max(X_3, X_2 + X_5)$ and $W_3 \equiv \max(X_3, X_5) + X_1 \times X_2$. For purpose of this example, assume $X_1 > 0$ to consider the iterative case and ignore W_2 . This leaves two alternatives. These can evaluate to distinct numeric values, yet both are equally optimal by definition.

3.1.5.5 Implementing postcondition inference with mwp_ℓ

We implemented the analysis of §3.1.5.3 as an extension of the static analyzer `pymwp`. `pymwp` (Aubert, Rubiano, Rusch, & Seiller, 2023b) is an open source implementation of the flow calculus of mwp-bounds on a subset of C. `pymwp` takes as input a C file and analyzes variable value growth in each of its functions. Our implementation adds to `pymwp` a new loop analysis mode, mwp_ℓ . The loop analysis is complementary to the default function analysis mode, which we name mwp_f for distinction. The primary differences are that mwp_ℓ looks for optimal bounds by variable in a loop, and mwp_f finds existence of any bound for all variables in a function. Though applicable to both, the paper enhancements are implemented only in mwp_ℓ to enable evaluation.

During program analysis, the input file is mapped to the imperative language of Def. 1. Only loops that are fully expressible in the imperative language are analyzed. Analyzing program fragments is possible due to compositionality of the flow calculus. Stated differently, we can apply the analysis early, even if some program parts are missing. `pymwp` supports all loop constructs of C. The flow calculus treats the iteration space conservatively as an over-approximation. Keywords `break` and `continue` have no observable impact. Similarly, verification macros like `assert` may be present, but they do not impact the analysis.

When a C loop construct is obviously bounded,⁸ it is treated as a bounded loop in the flow calculus. Otherwise, the construct is treated as an unbounded loop. Detection of loop boundedness impacts the number of derivable variables, since a bounded loop permits iteration-dependency. The current detection is based on loop form and it is still rudimentary. In the future, improving the analyzer’s handling of richer loop forms would yield more bounded variables.

To use the analysis results as concrete verification assertions, two more steps are required. First, we must record the initial variable values because they are needed to express postconditions. Next, we must complete the parts that are left implicit in mwp-bounds, like constants. §3.1.9 demonstrates how we perform both steps. In general, the postconditions provide bounding expressions w.r.t. input variables, with some possible omissions. However, filling in the expression is easier than starting with no expression at all.

3.1.6 Comparing Related Techniques

3.1.6.1 Automatic inference of specification conditions

Our work relates primarily to approaches that aim to unify software verification and complexity. Whereas loop invariants are used in (Nguyen, Antonopoulos, Ruef, & Hicks, 2017) to obtain complexity results, we synthesize specification conditions starting from complexity analysis. The bidirectionality suggests that further investigations that connect the two topics is warranted. Narrowing down to specifications, automatic specification inference in general is a challenging problem (Dillig, Dillig, Li, & McMillan, 2013; Yu, Wang, & Wang, 2023). It is common to break down the problem into smaller parts: preconditions, postconditions, and (inductive) invariants. Inference of postconditions is the most relevant

⁸For example, in `for(int i=0; i<N; i++)` the iterator `i` must not occur in the body and the body operations are arithmetic. Without nesting, it is clearly a finite loop.

to our analysis. Although invariant inference is studied extensively in literature (Colón, Sankaranarayanan, & Sipma, 2003; Cousot & Halbwachs, 1978; Dillig, Dillig, Li, & McMillan, 2013; Karr, 1976; Nguyen, Antonopoulos, Ruef, & Hicks, 2017; Nguyen, Kapur, Weimer, & Forrest, 2014; Ryan, Wong, Yao, Gu, & Jana, 2020; Sankaranarayanan, Sipma, & Manna, 2004; Si, Dai, Raghothaman, Naik, & Song, 2018; Yao, Ryan, Wong, Jana, & Gu, 2020; Yu, Wang, & Wang, 2023), existing works that intentionally target postconditions are rare (Molina, Ponzio, Aguirre, & Frias, 2021; Popeea & Chin, 2007).

There are a few instances that infer postconditions statically. In (Popeea & Chin, 2007), show how abstract interpretation can be used to obtain a static technique for postcondition inference. Conceptually it is close to our goals, but abstract interpretation differs considerably from the flow calculus. Moreover, no implementation is available for comparison. The complexity analyzer KoAT (Giesl, Lommen, Hark, & Meyer, 2022) is a static analyzer with an open source implementation. However, it targets complexity-theoretic program properties, like time and size bounds, and is not specialized in verification.

Dynamic analyzers are complementary to static techniques. They inspect program traces to infer *likely* invariants at the traced program points. EvoSpex (Molina, Ponzio, Aguirre, & Frias, 2021) is a dynamic postcondition analyzer. It is designed for Java methods for reasoning about postconditions in classes (accessors, mutators, heap structures, etc.); and thus distant from numerical loop analysis. The invariant detector Daikon (Ernst et al., 2007) handles many program constructs, including numerical loops. Since Daikon is compatible with our problem formulation, we discuss it in §3.1.6.2). DIG (Nguyen, Kapur, Weimer, & Forrest, 2014) is another dynamic numeric invariant generator. DIG can perform invariant inference at arbitrary program points, which makes it usable as a postcondition detector. Although it is similar to Daikon, DIG mixes static and dynamic techniques to obtain more informative results. In general, dynamic program analysis differs notably from static program analysis. Since the results are based on traces, they are necessarily incomplete. The

quality of inferred results is directly related to the available analysis inputs. Importantly, a dynamic analyzer requires executing the input program, which implies that the program must be runnable. The same is not necessary for our analysis that can work with program fragments.

3.1.6.2 A Comparison of Alternative Approaches

Since we aim to support concrete verification tasks, we must compare our analysis to available implementations that can address the same problem. In this section, we compare⁹ mwp_ℓ to three mature¹⁰ analyzers: KoAT, Duet, and Daikon. These analyzers are designed for complexity analysis, verification, and invariant inference, resp. Each analyzer analyzes different program scopes (cf. Table 19 and §3.1.10.3) and has a different specification of output format. Therefore, a statistical comparison is insufficient for useful comparison. A better way is to present canonical cases that the analyzers handle differently. As comparison workloads we consider the loops of Fig. 32. They are instances from the benchmarks used in §3.1.7. To give a preview of our findings—which we summarize in Table 19—the alternative analyzers are orthogonal. The comparison shows mwp_ℓ is useful in cases the other tools ignore, and vice versa.

⁹Refer to §3.1.10 for the technical details of this comparison.

¹⁰Each analyzer is developmentally stable with 10+ years of history.

<pre> for(int i=0;i<X1;i++) { X3=X2*X2; X3=X3+X5; X4=X4+X5; } </pre>	<pre> while(nondet()) { X1=X2+X2; X2=X3+X3; X4=X5+X5; } </pre>	<pre> assume(y==0); while(y<10000) { x=x+y; y=y+1; } </pre>
---	--	--

Listing 62: LucidLoop

Listing 63: Function condition

Listing 64: Finite iteration

Figure 32: Loop cases for analyzer comparison. The loop in 62 is known to terminate and its guard variable X_1 does not occur in the body. In 63, the loop iteration space and termination are unknown since they are controlled by a nondeterministic function. The loop in 64 has a fixed iteration space, but the postcondition of x is difficult to infer. Assuming $y = 0$, the precise formula is $x' = x + (y' \times y' - y') \div 2$ where y' is iteration count.

Inference with mwp_ℓ . The results of mwp_ℓ give a baseline for comparison.

- Listing 62: As explained in §3.1.1.1.
- Listing 63: X_3 and X_5 are linear. The other are $X_2' \leq \max(X_2, X_3)$, $X_4' \leq \max(X_4, X_5)$, and $X_1' \leq \max(X_1, X_2 + X_3)$.
- Listing 64: The program converts to a bounded loop. The constants 1 and 10000 are lifted to inputs c_1 and c_2 , resp. The postconditions are $x' \leq x + c_2 \times (c_1 + y)$ and $y' \leq y + (c_1 \times c_2)$.

Complexity analyzer KoAT. KoAT (KoAT2 Developers, 2024) is a part of the automated termination and complexity prover AProVE (Giesl et al., 2016). KoAT infers complexity bounds: time, cost, size bounds, etc. The bound most relevant to our problem is the *size bound*, which indicate how large the absolute value of an integer variable may become (Lommen & Giesl, 2023). Applying KoAT on C programs requires compiling the C code (through LLVM bytecode) into an integer program. The transformed program is then analyzed by KoAT (Giesl, Lommen, Hark, & Meyer, 2022). The translation step renames the program

variables. Variables that do not contribute to the complexity result are discarded during pre-processing. This treatment has the following effects: (i) KoAT can distinguish between loops that differ only on iteration counts, and (ii) size bounds are inferred only for variables that impact the loop iteration. The latter is the complement of the flow calculus, where the iterator of a bounded loop is not allowed to occur in the body.

- Listing 62: We obtain $x_1' : 2 \cdot x_1$ and $i' : x_1 + 2$; and x_2 – x_5 are discarded.
- Listing 63: No size bounds are generated for variables x_1 – x_5 .
- Listing 64: Variable y has precise size bound $y : 1000$. Variable x is discarded.

Duet – the analyzer of unbounded concurrency. Duet is a static verifier for concurrent programs whose thread count cannot be statically bounded (Duet Developers, 2024). We include it in this comparison because it contains analysis techniques that relate to our problem. In particular, the implementation of transition ideals (Cyphert & Kincaid, 2024) computes loop summaries that produce over-approximations of a formula that describes the loop body. The summaries can capture non-linear invariants and generalize over arbitrary control flow. The theory of transition ideals is monotone. In other words, a program with more precise specifications yields a more informative loop summary. Then, Duet aims to prove program correctness using the invariants generated from transition ideals.

- Listings 62 – 64: In absence of assertions, Duet produces a single response, no errors and no unsafe assertions. This response is not meaningful for our use case. After adding postconditions (assertions) manually, Duet verifies them successfully. For example, if we add to Listing 64 the assertions $x' = x + (y' \times y' - y') \div 2$ and $y' = 1000$, Duet verifies the program with 0 errors, 2 safe assertions. When assertions are not available, mwp_ℓ could assist Duet toward obtaining the initial assertions.

Postconditions with Daikon. Daikon (Daikon developers, 2024; Ernst et al., 2007) is a dynamic invariant detector with front-ends to support many programming languages. Daikon predicts likely invariants at function entry and exit points. Critically, the function internals are opaque during invariant detection. The inference relies on execution traces, invariant templates, and configuration options. Daikon infers postconditions for a return variable and a single function may generate multiple postconditions. Since the invariants are likely, they must be checked for correctness, possibly manually. Daikon does not produce results for the displayed fragments, but after a modification to a whole-program, it infers the following.

- Listing 62: $x_3' > x_2$, $x_3' > x_4$ and $x_3' > x_5$. These either do not generalize or require additional assumptions to prove.
- Listing 63: No result.
- Listing 64: Precise numeric value of x or y , depending on which one is returned. Although the arithmetic formula is not recovered, Daikon is the only technique that can give a precise value for x .

3.1.7 Experimental Evaluation

We examine two questions about the implementation of our technique.

1. **How effective is mwp_ℓ at discovering postconditions in general?** We execute mwp_ℓ against four benchmark suites that include a rich set of benchmarks from complexity theory to loop invariant inference. Three of the suites are design-independent of the applied analysis technique.

Feature	Daikon	Duet	KoAT	mwp _ℓ
Analysis scope (§3.1.10.3)	function entry/exit	invariants	loop control	loop body
Input format	execution traces	program	program	program
Output format	likely invariants	SAT/error	size bounds	mwp-bounds
Numerical domain	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{N}$
Postcondition expressivity	>+	>+	>+	+
Program fragment analysis	○	●	●	●
Handles program divergence	○	●	●	●
Distinguishes iteration count	●	○	●	○
Body variables coverage	●	●	●	●
Results soundness	○	●	●	●
Postconditions for Fig. 32	● ○ ●	○ ○ ○	● ○ ●	● ● ●

Table 19: Summary of analyzer capabilities, behaviors, and assumptions. For input format, we write *program* to mean syntactical analysis written in a subset of the C language, though the subsets differ. A positive response is phrased as favorable: ● = yes, ● = partial, ○ = no.

2. **What is the concrete impact of the new theoretical enhancements?** We hypothesize that the introduced enhancements are critical for extending the utility of the flow calculus. To quantify this improvement, we compare performance of mwp_ℓ and mwp_f across the four benchmark suites.

Setup: benchmarks, metrics, and environment. We consider four benchmark suites, of numerical C loops, and 620 total problem (detailed in §3.1.11). The complexity suite is mainly linear time complexity problems whose termination behavior is unknown. The linear (Si, Dai, Raghothaman, Naik, & Song, 2018) and non-linear (Nguyen, Antonopoulos, Ruef, & Hicks, 2017; Yu, Wang, & Wang, 2023) suites come from loop invariant inference

literature. The non-linear suite includes classic numerical algorithms like geometric series and divisor computations. The mwp suite (Aubert, Rubiano, Rusch, & Seiller, 2023b) problems are designed pose challenges to analyses based on the flow calculus of mwp-bounds. All suites include branching statements and nondeterministic control expressions that simulate external function calls. The complexity and non-linear suites have problems with sequential and nested loops. As evaluation metrics, we recorded the count and kind of obtained postconditions (i.e., mwp-bounds). We also recorded all meta-data collected by pymwp like program statistics. We set the timeout to 10 seconds. Without timeouts the metrics are deterministic. We ran all experiments on commodity hardware; on a native 10-core macOS 15.3.2 M1 arm64 host with 16 GB of RAM. All experiment results are in Table 21.

Analyzer	Suite	Loops	Variables	Bounds	m	w	p	∞
mwp _{ℓ}	Complexity	623 (.84)	1407	988 (.70)	567	49	372	419 (.30)
(ours)	Linear	49 (1.0)	103	80 (.78)	22	7	51	23 (.22)
	Non-linear	43 (.90)	172	107 (.62)	48	8	51	65 (.38)
	mwp	30 (1.0)	105	77 (.73)	38	30	9	28 (.27)
	Suite	Functions	Variables	Bounds	m	w	p	∞
mwp _{f}	Complexity	399 (.79)	1153	616 (.53)	330	6	280	537 (.47)
(prior)	Linear	49 (1.0)	131	95 (.73)	38	1	57	36 (.27)
	Non-linear	29 (.78)	159	60 (.38)	14	3	43	99 (.62)
	mwp	30 (1.0)	105	66 (.63)	28	22	16	39 (.37)

Table 21: Experiment results for mwp _{f} and mwp _{ℓ} with totals and (mean). *Loops/functions* shows the number of analyzed instances and suite coverage (%). *Bounds* is the number of inferred postconditions, with the mean is relative to analyzed *variables*. The *mwp*-columns show a breakdown of postconditions by bound form. The number of unbounded variables is shown in column ∞ .

3.1.7.1 Inference generalizability

Across the four suites, mwp_ℓ succeeds at analyzing 84%-100% of the loops. It finds postconditions for 62%-78% of variables occurring in those loops. The postconditions are different in form from most complexity analyzers (refer to (Aubert, Rubiano, Rusch, & Seiller, 2023b; Lommen & Giesl, 2023)). Most complexity analyzers are essentially restricted to linear arithmetic (Lommen & Giesl, 2023). Encouragingly, mwp_ℓ shows success also on the non-linear suite. The results are positive because the suite contains complex arithmetic and natural algorithms. The linear suite expectedly yields more postconditions, since the suite is easier. On the complexity problems, that dominate in problem quantity, mwp_ℓ already bounds 70% of variables. The number depends largely on how loop boundedness is decided. We expect that, after improving the current mechanism, mwp_ℓ would produce even more postconditions.

We observe some limitations w.r.t. expressivity. First, loops with unsupported syntax are not analyzed. For example, some variables in the complexity suite are updated by a random integer generator. Such operations are inherently outside the guarantees that can be provided by the flow calculus. When the growth rate of some variables is truly beyond polynomial, it is not expressible as mwp -bounds. Finally, sometimes the polynomial describing the variable value growth may be too complicated to express. Based on the findings, further increase in expressivity should be one of the main directions of future research. In summary, mwp_ℓ succeeds at analyzing most loops, and it generates postconditions for most variables in those loops.

3.1.7.2 Impact of theoretical enhancements

We compare mwp_ℓ and mwp_f to quantify the impact of the enhancements introduced in this paper. As practical implementations of the flow calculus, mwp_f represents the state-of-the-

art in the analysis capabilities. Since the two analysis modes differ on program scopes, not all results are comparable. However, we pre-processed the mwp-suite such that it is fully comparable. The mwp-suite results confirm that the paper enhancements improve abilities to infer postconditions. On the mwp suite, mwp_ℓ bounds 73% of variables compared to 63% by mwp_f . Further, mwp_ℓ finds optimal bounds, which is reflected in the distribution of bound forms. The other suites are comparable through the statistical means. Particularly on the non-linear suite, mwp_f finds a bound for only 38% variables. This is because one failing variable causes failure of all variables. On the same suite, mwp_ℓ bounds 62% variables due to its enhanced failure handling.

3.1.8 Conclusions and Future Directions

Assistance in specification inference is paramount to increase adoption of formal methods in practice. The paper has presented how to repurpose a complexity-theoretic analysis to postcondition inference. This required four new enhancements:

- (i) projecting the analysis on individual variables,
- (ii) introducing an evaluation strategy to obtain optimal mwp-bounds,
- (iii) improving derivation failure-handling to increase analysis expressiveness,
- (iv) and adopting the analysis to a new use case, i.e., formal verification.

A comparison study shows our technique offers complementary strengths among the related inference approaches. The theory is materialized in an implementation, mwp_ℓ . Our experiments show that the paper enhancements improve the flow calculus and make it applicable toward uses in formal verification.

Future directions. Although we have extended the flow calculus capabilities, multiple directions for future work remain, and many emerge from this work. The two main directions concern enriching the analysis expressiveness and precision. E.g., leveraging assumptions (if available), tracking variable immutability, and accounting for control expression would generate more precise specification conditions. Currently, polynomial p -flows are not allowed inside unbounded loops. Discovering ways to relax this restriction would improve expressiveness and yield more postconditions. Due to the complexity-theoretic origins, the flow calculus targets polynomial *upper* bounds. For verification, it would be useful to extend the technique to also cover lower bounds, and assign bounds on exponential growth. On the practical side, our analysis does not cover division operator and requires expanding operations to binary form. These limitations can be resolved by refactoring the input program, but should be resolved at the theoretical level.

We are encouraged by the continued enhancements of the flow calculus and discovering its potential uses. The analysis could already be implemented as a developer plug-in to assist in writing specifications. In future research, we will consider extending the capabilities of the flow calculus. We are generally curious about whether similar solver-free syntactic analyses could be designed to infer *other* specification conditions. Another ongoing project is to formally verify the flow calculus theory. The results of this paper provide important justification to the formalization effort.

3.1.9 Appendix A: Application to Program Verification

The following example shows how to complete an implementation with specification conditions, in the verification-aware Dafny programming language. The postconditions are those inferred by our analysis.

```
0 method LucidLoop(X1: nat, X2: nat, X3: nat, X4: nat, X5: nat){
1
2   // for bookkeeping -- record initial values
3   var X1', X2', X3', X4', X5' := X1, X2, X3, X4, X5;
4
5   for i := 0 to X1'
6     // invariants (omitted)
7     {
8       X3' := X2' * X2' + X5';
9       X4' := X4' + X5';
10    }
11
12   // postconditions
13   assert X1' ≤ X1;           // linear
14   assert X2' ≤ X2;           // linear
15   assert X5' ≤ X5;           // linear
16   assert X3' ≤ Max(X3, X2*X2+X5); // weak polynomial
17   assert X4' ≤ X4+X1*X5;      // polynomial
18 }
```

Dafny

Listing 65: LucidLoop verified in Dafny.

Concrete program verification requires two additional steps. First, recording the initial variable values (L4). This is simply a matter of creating copies of variables. Recording the

initial values enables referring to them in the postconditions. Second, we add the constants (L14–16) omitted in the postconditions of our analysis. Although this step requires manual effort, is it significantly easier to “fill in” the constants, than to infer full assertion clauses. Maintaining human oversight in this step has an additional benefit. It forces to check that the assertions are sensible. If the implementation has a bug—e.g., variable grows exponentially when it should not—the bug becomes detectable during this step. A fully automatic technique would not alert to the issue.

After adding invariants, the Dafny verifier immediately constructs a proof, which confirms that the assertions always hold. Generating suitable inductive loop invariants is a challenge for the related works that specialize in invariant inference.

3.1.10 Appendix B: Technical Details of Analyzer Comparison

3.1.10.1 Executing the analyzers

We analyzed the programs in Listings 71–73 as follows.

mwp_ℓ. We run pymwp v0.6.0 in the loop analysis mode.

```
pymwp [program].c --mode L --strict
```

cmd

Listing 66: Running pymwp in loop analysis mode.

KoAT. The KoAT web interface is sufficient to confirm our findings. We applied the following options: ✓ control-flow refinement ✓ size bounds ✓ unsolvable loops, and default timeout. The web interface address is:

`https://aprove.informatik.rwth-aachen.de/interface/v-koat/c`

Duet. We ran Duet from source, git revision 1d36b05, with following options. The compositional recurrence analysis generates invariants for sequential programs. Transition ideals is implemented as linear and quadratic simulation modes.

```
duet.exe [program].c
  -cra                # compositional recurrence analysis
  -cra-refine         # enable loop refinement
  -monotone           # disable non-monotone analysis features
  -[theory]           # lirr-usp or lirr-sp-quad
```

cmd

Listing 67: Program analysis with Duet.

Daikon. Daikon requires multiple version of a program, then compiling and tracing them with Kvasir. The * means we trace multiple versions of input.

Supply options to Kvasir before the program name argument.

```
kvasir-dtrace
  --dtrace-file=[program].dtrace    # trace destination
  --decls-file=[program].decls      # declarations file
  [program]*.o                      # compiled program
```

cmd

Listing 68: Creating a trace with Daikon.

After tracing, we run Daikon with following options to infer postconditions.

```
java -cp $DAIKONDIR/daikon.jar daikon.Daikon
  --conf_limit=.50                  # confidence
  -o [program].inv.gz               # output
  [program]*.dtrace                 # path to traces
  [program].decls                   # path to declarations
```

cmd

Listing 69: Infer invariants using Daikon.

After inference, the invariants can be printed to a suitable display format.

```
java -cp $DAIKONDIR/daikon.jar daikon.PrintInvariants
  --wrap_xml --output_num_samples  # format options
  [program].inv.gz                  # source
  [program].txt                     # output path
```

cmd

Listing 70: Displaying invariants inferred by Daikon.

3.1.10.2 Comparison programs

```
int loop(int X1, int X2, int X3,  
        int X4, int X5) {  
    for(int i = 0; i < X1; i++) {  
        X3 = X2 * X2;  
        X3 = X3 + X5;  
        X4 = X4 + X5;  
    }  
    return X3;  
}
```

C

```
int loop(int X1, int X2, int X3,  
        int X4, int X5) {  
    while(__VERIFIER_nondet_int()){  
        X1 = X2 + X2;  
        X2 = X3 + X3;  
        X4 = X5 + X5;  
    }  
    return X4;  
}
```

C

Listing 71: mwp/example 3.4.

Listing 72: mwp/not infinite #4.

```
int loop(int x, int y) {  
    y = 0;  
    while(y < 10000) {  
        x = x + y;  
        y = y + 1;  
    }  
    return x;  
}
```

C

Listing 73: Linear #02.

These are programs of Fig. 32 expanded with headers and return statements. For Duet analysis, loop must be renamed to main. Having a main function raises an error with

KoAT. Daikon expects adding a separate main method with calls to loop. In Listing 73, `assume` is unsupported and should be omitted before analyzing it with KoAT.

3.1.10.3 Analyzer scopes

<code>int main(int X, int Y, int Z) {</code>	
<code> for(int i=0; i<Y; i++)</code>	<i>KoAT</i>
<code> X = X + Z;</code>	<i>mwp_ℓ</i>
<code> assert(...);</code>	<i>Duet</i>
<code> return X;</code>	<i>Daikon</i>
<code>}</code>	

Figure 33: Postcondition analysis scopes. The analyzers focus on complementary program regions identified by the different colors. KoAT tracks variables that control loop iteration. *mwp_ℓ* analyzes variables inside the loop body. Duet infers loop summaries based on available assertions. Daikon infers likely invariants of the return variable and abstracts function internals.

3.1.11 Appendix C: Details of Experimental Evaluation

The complexity suite is the “Complexity C Integer” suite from the Termination Problem Database version 11.3 (TPDB maintainers, 2022). This suite is used in the annual Termination and Complexity Competition.

The linear suite (Si, Dai, Raghothaman, Naik, & Song, 2018) contains inference problems for linear loop invariants. The problems are pre-annotated with assertions (these have no impact on our analysis). We excluded 9 benchmarks that are known to be invalid (Ryan, Wong, Yao, Gu, & Jana, 2020, Appendix G) as they violate the specified assertions. We also unified benchmarks that have the same precondition and loop, as

Suite	Linear	mwp	Complexity	Non-linear	Total
Benchmarks	49	30	504	37	620
Lines of code	652 (13.31)	270 (9.00)	6,066 (12.04)	710 (19.19)	7,698
Loops	49 (1.00)	30 (1.00)	740 (1.47)	48 (1.30)	867
Variables					
– loop	117 (2.39)	105 (3.50)	1,921 (3.81)	208 (5.62)	2,351
– functions	131 (2.67)	105 (3.50)	1,519 (3.01)	208 (5.62)	1,963

Table 23: Benchmark suite characteristics by count and (mean).

they are identical for the purpose of *postcondition* inference, ending with 49 benchmarks in total.

The nonlinear suite (Nguyen, Antonopoulos, Ruef, & Hicks, 2017) (also referred to as NLA-suite in literature) is an extended formulation of the suite, with the additional problems coming from (Yu, Wang, & Wang, 2023).

The mwp suite (Aubert, Rubiano, Rusch, & Seiller, 2023b) is designed specifically to be challenging for the flow calculus of mwp-bounds, with complex data flows and arithmetic operations. To obtain strictly comparable results between mwp_ℓ and mwp_f , as they scope variables differently, we exclude nested loops and loopless benchmarks.

The statistics of the suites are summarized in Table 23. A single benchmark can contain multiple functions, loops, and sequential and/or nested loops. Due to differences in the targeted program scopes between mwp_ℓ and mwp_f , the variable counts are specified by scope. Loop-scoped variables include loop guards and variables in the loop body. Function-scoped variables contain all parameters and variables in the function body. We modified the benchmarks by expanding n-ary expressions to binary form to match the input language

of pymwp. Since the boundedness check is still rudimentary, some loop conditions were rewritten in detectable form. A more robust approach would use a specialized compiler.

3.2 A Logic for Anytime Non-Interference

① *Implicit computational complexity & security*

Clément Aubert and Neea Rusch

Research paper draft.

An earlier version of this work appeared at the 19th workshop on Programming Languages and Analysis for Security (PLAS) 2024.

Abstract

Non-interference is an information flow policy for guaranteeing confidentiality, i.e., that effects of sensitive data are not exposed to lower-level users, even indirectly. When phrased in terms of programming languages, non-interference is studied by attaching security classes to variables, then analyzing the classes to determine if a violation, or a data “leak”, can occur. Security type systems are common controls for analyzing and enforcing non-interference. Unfortunately, they require inference algorithms, program-level security specifications, non-standard compilers, and are generally too restrictive or complex for practical implementation. In this paper, we present a program logic $\top_{\text{NI}}^*$ that guarantees the semantic security property of *anytime non-interference*. By anytime, we mean a malicious actor with low-level access cannot infer anything about higher-level values at any point of the program execution. The logic links non-interference violations precisely to the faulty commands and violations cannot be erased by program composition. We draw rich inspiration from complexity-theoretic flow calculi, but obtaining a logic for security analysis required significant adjustments. Finally, we share a prototype to demonstrate $\top_{\text{NI}}^*$ can be implemented as an automatic, annotation-free, static security analyzer to obtain confidentiality guarantees in practice.

3.2.1 Introduction

Verifying that data is handled securely during computation is challenging because it requires information beyond the program syntax. For example, consider a hash function that computes a checksum of its input. Assume there exists a malicious actor who can observe outputs of the hash function. If all inputs are public data, we can guarantee the function does not expose secrets to the actor. However, if we change the inputs to secret data, like social security numbers, we no longer have the same guarantee for the same function. The

hash function then “leaks” information that possibly enables the actor to recover the secret data.

Non-interference (Goguen & Meseguer, 1982) is a classical semantic security property that constrains information flow during computation. It is a mechanism to enforce *confidentiality*, i.e., concealment of information or resources from unauthorized parties (Bishop, 2019). Non-interference is an attractive target of study because it offers strong end-to-end guarantees for data protection, it is inherently compositional, and can be enforced with program logics or security type systems (Cecchetti, Myers, & Arden, 2017; Frumin, Krebbers, & Birkedal, 2021). Informally, a program is non-interfering when secret data does not affect calculation of its public outputs (Sabelfeld & Myers, 2003). In other words, data can only stay at the same security class or flow to higher classes. Although a desirable property, non-interference is in general undecidable by Rice’s Theorem (Rice, 1953) and constructing a system that completely adheres to non-interference is overly restrictive to capture real-world security requirements (Bossi, Macedonio, Piazza, & Rossi, 2005; Cecchetti, Myers, & Arden, 2017). However, unattainability of an ideal construction does not restrict analysis of imperfect systems. Program analysis enables detecting information flow issues, supports informed assessments of vulnerabilities, and identifies potential mitigations.

Our work extends the analysis of non-interference in theoretical directions while yielding practical advantages. In literature, terminology around non-interference is defined somewhat fluidly (Sabelfeld & Myers, 2003); admitting multiple different but related formal definitions (Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020), and generalizing to the informal description provided previously. In this paper, we introduce the notion of *anytime non-interference*. The anytime property is powerful because it accounts for intermediate states of computation. Thus, it is strictly stronger than the classic non-interference that is expressed in terms of inputs and outputs. To lift our theory toward the real world, we provide our main result: a program logic $\top_{\text{NI}}^*$ that enables lightweight automatic static

program analysis of anytime non-interference. We demonstrate practicality of $\top_{\text{NI}}^*$ through examples, a prototype implementation, and discussions of how anytime non-interference elegantly supports numerous static program analysis applications.

Anytime non-interference tracks potential information flow leaks in every legal program state. A non-interfering program can be interrupted arbitrarily without compromising its non-interference guarantee. Conversely, once a violating operation has occurred, it is impossible to erase it. We consider time as updates of public variable values. In other words, the latency between public variable updates is instant. A change between two secret values, that causes a loop to iterate longer according to a physical clock, has no observable side effect. In general, anytime non-interference models security at the abstraction level of programming languages and excludes lower-level execution details. However, we consider the approach justified because potential information flow issues are often detectable from syntax.

Anytime non-interference is furthermore *termination-insensitive*. Because information signaled through termination can leak secrets indirectly, termination handling is an ongoing design challenge for non-interference systems (Bay & Askarov, 2020). Untrusted programs, that pose high security risks, require strong *progress-sensitive* non-interference that considers both termination and I/O interactions (Hedin & Sabelfeld, 2012). Trusted programs, with predictable run-time behavior, permit weaker security checks and termination-insensitivity. Unfortunately, the binary situation provides no middle ground for programs that mix trusted and untrusted code, e.g., by dynamic code loading. In §3.2.6.2 we discuss how to address this limitation by partitioning computations based on security classes. This hybrid approach relaxes the limitations of termination-insensitivity and monolithic termination handling.

3.2.1.1 The Essential Security Terminology Decoded

Our work belongs to the domain of *language-based security* (Sabelfeld & Myers, 2003; Schneider, Morrisett, & Harper, 2001), where programming languages principles (semantics, analysis, type systems, rewriting, etc.) are used to strengthen application security. The $\top_{\text{NI}}^*$ logic draws rich inspiration from implicit computational complexity (refer to §3.2.6.3), and has applications in static program analysis; thus our work intersects many related fields. Although we assume prior familiarity with logic and programming languages, we define the relevant security concepts in this section.

Information flow denotes an observable action between two agents A and B . If an action performed by A is observable to B , then there exists an information flow from A to B . The flow is *explicit* if it is directly observable from a single action. The flow is *implicit* when it is not directly observable, but reveals deductively the initial performed action, after a sequence of other actions. To represent information flow, we manipulate the conventional lattice model à la Denning (Denning, 1976).

An *information flow policy* is a statement of what is, and what is not, permissible (Bishop, 2019) for a program C in terms of flow between its variables; formally defined as follows:

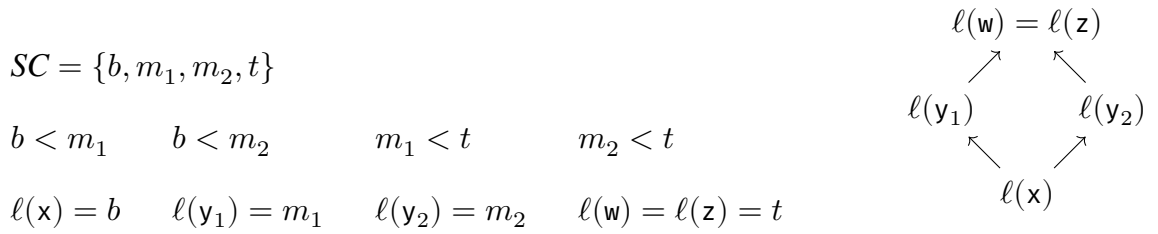
Definition 9 (Information Flow Policy (Volpano, Irvine, & Smith, 1996), Class Assignment). *An information flow policy is a lattice $SC = (SC, <)$ where SC is a partially <-ordered finite set of security classes.*

We write ℓ for the class assignment that assigns statically and definitely to each variable x occurring in a program C its security class $\ell(x) \in SC$. □

By abuse of notation, we assume that a class assignment always comes with an information flow policy, we write $c \in SC$, and for any two classes c_1, c_2 , we write $c_1 \leq c_2$ if $c_1 < c_2$

or $c_1 = c_2$, and $c_1 \perp c_2$ if $c_1 \not\leq c_2$ and $c_2 \not\leq c_1$ —in this case, we say that c_1 and c_2 are *orthogonal*.

A simple policy has two security classes, e.g., $LH = (\{l, h\}, \{l < h\})$ —for *low* and *high*; but a policy can be arbitrarily complex (refer to Ex. 29, located in Appendix, for a more concrete example). We generally use a Hasse diagram to represent the information flow policy and class assignment in a compact manner, as follows:



An *information flow control* (IFC) is a mechanism to enforce a policy (Bishop, 2019). Security type systems (presented in §3.2.6.3) are an example of a programming languages based IFCs. They enforce a policy by annotating a program with security types. Then, to be secure, a program must pass a compile-time type check. A sound IFC guarantees to find all policy violations and a precise IFC avoids raising excessive false alarms.

Formal security analysis uses the terms system model, security objective, and an attacker model (Bau & Mitchell, 2011; Bognar, Van Bulck, & Piessens, 2022). Our *system model*, i.e., the system we want to secure, is a sequential imperative program, as specified in §3.2.3.1, with effectful function calls discussed in §3.2.5. Our *security objective*, i.e., the system behaviors that are considered secure, is defined by anytime non-interference (Def. 22). An *adversary* is a malicious actor who poses a threat to the system. An *attacker model* specifies the capabilities and motivations of the adversary. We assume a *program-centric* attacker model (Hedin & Sabelfeld, 2012), where the adversary can (i) see the program syntax (ii) control public inputs and observe public outputs, and (iii) up to the attacker’s security class, access memory registers after updates.

3.2.1.2 Contributions

Our contributions are three-fold.

1. The main result is a lightweight syntactic IFC logic $\top_{\text{NI}}^*$ (§3.2.3), with a built-in automatable inference algorithm, that captures the semantic property of anytime non-interference.
2. We introduce the definition of anytime non-interference (Def. 22), and prove its correspondence with $\top_{\text{NI}}^*$, that follows from the fundamental theorem (Thm. 3).
3. We demonstrate the promising practical utility of $\top_{\text{NI}}^*$ (§3.2.6), to include by describing our prototype static analyzer `TYNI` for analysis of Java programs.

§3.2.5 furthermore discusses in detail how our approach can accommodate different treatments of functions with and without side effects, and §3.2.9 gathers additional examples to help understanding.

3.2.2 High-level Overview

We consider deterministic imperative programs, with conventional operational semantics, and variables of basic data types (integers, strings, etc.). Let our expository program be the one in Listing 74.

```
1 if (z==1) then  
2   if (x==1) then y = 1 else y = 0  
3 else x = y
```

While

Listing 74: Expository program.

In the program, certain data flows are potentially problematic. The assignment $x = y$ is an instances of an explicit flow, since there is a direct flow from variable y to x . The control expressions $z == 1$ and $x == 1$ represent implicit flows. They reveal information over execution paths, and indirectly expose values of the control statement variables. Admissibility of these data flows depends on the security classes of the variables, since non-interference forbids data flowing from higher classes to lower or between orthogonal classes. A sound IFC detects such issues and raises an alarm.

The logic $\top_{NI}^*$ produces a matrix of coefficients by applying inference rules to programs. In a single derivation, it captures dependencies between all the program variables, for all execution paths and security classes. The matrix is interpreted by matching the *in-variables*, v_{in} (rows), with the *out-variables*, v_{out} (columns). The coefficients indicate:

- – *no* (non-interference) *violation*, no dependency from v_{in} to v_{out} ,
- ♠ – a *violation*—or “leak”, hence the symbol—, if $\ell(v_{out}) < \ell(v_{in})$ or $\ell(v_{out}) \perp \ell(v_{in})$.

The matrix of the expository program,

$$\begin{array}{c} \begin{array}{ccc} & x & y & z \\ \begin{array}{c} x \\ y \\ z \end{array} & \left(\begin{array}{ccc} \cdot & \spadesuit & \cdot \\ \spadesuit & \cdot & \cdot \\ \spadesuit & \spadesuit & \cdot \end{array} \right) \end{array}$$

expectedly shows *potential* violations in from y to x (explicit, in gray here) and x to y , z to x , and z to y (implicit). The matrix gives a summary of potential violations for the program that induced it. Evaluating the matrix determines if the program is non-interfering; or if there exists a class assignment that makes the program non-interfering. The evaluation function is parametric on the policy, enabling evaluation against different policies.

3.2.3 The Non-interference Logic

3.2.3.1 A Simple Imperative While Language

We use a simple imperative while language, with semantics similar to C. The grammar is given in Fig. 34. The language supports arrays and we let `for` and `do...while` loops be represented using while loops. How function calls can be added is discussed in §3.2.5. The language subsumes (up to `letvar` construct) the “core block-structured language” (Volpano, Irvine, & Smith, 1996), and it maps easily to the core fragment of C, Java, and other imperative programming languages.

$$\begin{aligned} \text{var} &:= i \mid \dots \mid t \mid \dots \mid x_1 \mid \dots \mid \text{var}[\text{exp}] && \text{(Variable)} \\ \text{exp} &:= \text{var} \mid \text{val} \mid \text{op}(\text{exp}, \dots, \text{exp}) && \text{(Expression)} \\ \text{com} &:= \text{var} = \text{exp} \mid \text{skip} \mid \text{if } \text{exp} \text{ then } \text{com} \text{ else } \text{com} \mid \text{while } \text{exp} \text{ do } \text{com} \mid \text{com}; \text{com} && \text{(Command)} \end{aligned}$$

Figure 34: A simple imperative while language

A variable $x, y, z, \dots$ represents either an undetermined “primitive” data type, e.g., not a reference variable, or an array, whose indices are given by an expression. We reserve t for arrays. An expression is either a variable, a value (e.g., integer literal) or the application to expressions of some operator op , which can be e.g., relational ($=, <$, etc.) or arithmetic ($+, -, \dots$). We let e (resp. C) range over expressions (resp. commands). We also use compound assignment operators and write e.g., $x++$ for $x+=1$. We assume commands to be correct, e.g., with operators correctly applied to expressions, no out-of-bounds errors, etc. A *program* C is a sequence of commands, each command being either an *assignment*, a *skip*, a *branching*,

Command C	$\text{Out}(C)$	$\text{In}(C)$	$\text{Occ}(C) = \text{Out}(C) \cup \text{In}(C)$
$x = e$	x	$\text{Occ}(e)$	$x \cup \text{Occ}(e)$
$t[e_1] = e_2$	t	$\text{Occ}(e_1) \cup \text{Occ}(e_2)$	$t \cup \text{Occ}(e_1) \cup \text{Occ}(e_2)$
skip	$\emptyset$	$\emptyset$	$\emptyset$
if e then C_1 else C_2	$\text{Out}(C_1) \cup \text{Out}(C_2)$	$\text{Occ}(e) \cup \text{In}(C_1) \cup \text{In}(C_2)$	$\text{Occ}(e) \cup \text{Occ}(C_1) \cup \text{Occ}(C_2)$
while e do C	$\text{Out}(C)$	$\text{Occ}(e) \cup \text{In}(C)$	$\text{Occ}(e) \cup \text{Occ}(C)$
$C_1; C_2$	$\text{Out}(C_1) \cup \text{Out}(C_2)$	$\text{In}(C_1) \cup \text{In}(C_2)$	$\text{Occ}(C_1) \cup \text{Occ}(C_2)$

Table 24: Definition of Out, In and Occ for commands.

a while loop or the composition of two commands. A program C' is a sub-program of C , denoted $C' \subseteq C$, if C' occurs verbatim in C . We also define the following sets of variables.

Definition 10 (Occ, Out and In). We define the variables occurring in an expression e by:

$$\text{Occ}(x) = x \quad \text{Occ}(\text{op}(e_1, \dots, e_n)) = \bigcup_{i=1}^n \text{Occ}(e_i) \quad \text{Occ}(t[e]) = t \cup \text{Occ}(e) \quad \text{Occ}(\text{val}) = \emptyset$$

The set $\text{Occ}(C)$ (resp. $\text{Out}(C)$, $\text{In}(C)$) of variables occurring in (resp. modified by, used by) a program C is defined in Table 24. We let $|\text{Occ}(C)|$ be the cardinal of $\text{Occ}(C)$. $\square$

3.2.3.2 Security-Flow Matrices for Non-interference Violation

The $\top_{\text{NI}}^*$ logic relies fundamentally on its ability to analyze data-flow dependencies between variables occurring in commands. In this section, we define the principles of this depen-

dency analysis, founded on the theory of *security-flow matrices*, and how it maps to the presented language. This dependency analysis is reminiscent of the one we developed to distribute loops (Aubert, Rubiano, Rusch, & Seiller, 2023e). We assume familiarity with monoids and matrices addition.

A security-flow matrix $\mathbb{M}(C)$ for a command C is a hollow matrix (i.e., a matrix with only $\cdot$ on the diagonal¹¹) over a monoid with an implicit choice of a denumeration of $\text{Occ}(C)$ ¹²

Definition 11 (Security monoid). *The security monoid is $(\{\cdot, \blacklozenge\}, \max)$, with $\cdot < \blacklozenge$.* $\square$

This monoid is isomorphic to the two-element Boolean algebra with only the disjunction, with $\blacklozenge$ representing a possible (non-interference) violation that cannot be erased.

Definition 12 (Security-flow matrix). *C , its security-flow matrix $\mathbb{M}(C)$ is a $|\text{Occ}(C)| \times |\text{Occ}(C)|$ matrix over the security monoid, whose construction is the object of §3.2.3.3.*

For $x, y \in \text{Occ}(C)$, we write $\mathbb{M}(C)(x, y)$ for the coefficient in $\mathbb{M}(C)$ at the row corresponding to the in-variable x and column corresponding to the out-variable y . $\square$

Definition 13 (Violation). *Given C , its security-flow matrix $\mathbb{M}(C)$ and a class assignment ℓ , C has a violation if there exists x and y such that $\mathbb{M}(C)(x, y) = \blacklozenge$ and either $\ell(y) < \ell(x)$ or $\ell(y) \perp \ell(x)$:*

$$x \begin{pmatrix} \dots & y & \dots \\ \vdots & \ddots & \ddots \\ & \blacklozenge & \\ \vdots & \ddots & \ddots \end{pmatrix} \Rightarrow C \text{ has a violation if } \ell(y) < \ell(x) \text{ or } \ell(y) \perp \ell(x).$$

¹¹This choice is clarified after Def. 13.

¹²We will use the order in which the variables occur in the program as their implicit order.

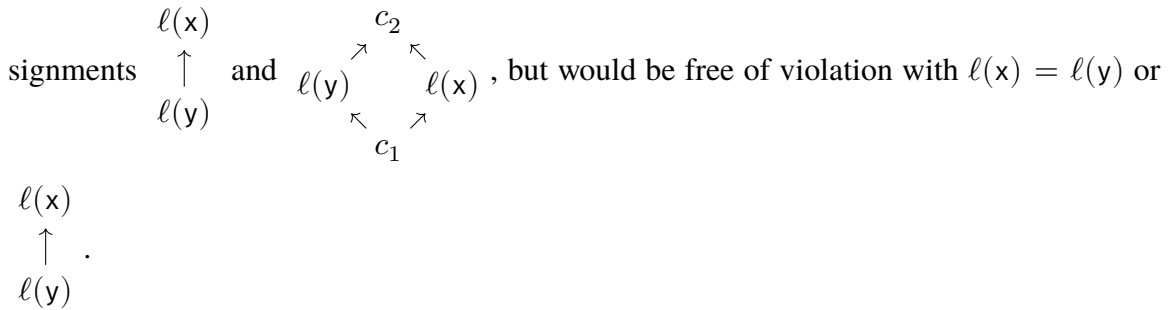
□

Since $\ell(x) < \ell(x)$ and $\ell(x) \perp \ell(x)$ are always false, there is no point keeping track of the values on the diagonal: this is why hollow matrices are enough. This is also confirmed by the intuition: it does not make sense to track data “leaking” from a variable to itself.

How a security-flow matrix is constructed by induction over the command is explained in §3.2.3.3. To avoid resizing matrices whenever additional variables are considered, we identify $\mathbb{M}(C)$ with its embedding in any larger matrix, i.e., we abusively call the security-flow matrix of C any matrix containing $\mathbb{M}(C)$ (up to rows swapping and columns swapping) and containing $\cdot$ otherwise, implicitly viewing the additional rows and columns as variables not occurring in C . Visually, this means that the following matrices are all viewed as $\mathbb{M}(C)$ with $\text{Occ}(C) = \{x, y\}$ and $\mathbb{M}(C)(x, y) = \blacklozenge$:

$$\begin{array}{ccc} \begin{array}{c} x \quad y \\ \begin{pmatrix} \cdot & \blacklozenge \\ \cdot & \cdot \end{pmatrix} \end{array} & \begin{array}{c} y \quad x \quad z \\ \begin{pmatrix} \cdot & \cdot & \cdot \\ \blacklozenge & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix} \end{array} & \begin{array}{c} w \quad x \quad y \\ \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \blacklozenge \\ \cdot & \cdot & \cdot \end{pmatrix} \end{array} \end{array}$$

Continuing this example and using our compact presentation of information flow policy and class assignment as single Hasse diagram, C would have a violation with the level as-



3.2.3.3 Constructing Security-Flow Matrices

The security-flow matrix of a command is constructed by induction, using the security monoid. §3.2.9 gathers additional examples with longer discussion.

C	$\text{Out}(C), \text{In}(C)$	$\mathbb{M}(C)$	C has violation(s) if ...
$w = 3$	$\text{Out}(C) = \{w\}$ $\text{In}(C) = \emptyset$	$w \begin{pmatrix} w \\ \cdot \end{pmatrix}$	(Impossible)
$y = x$	$\text{Out}(C) = \{y\}$ $\text{In}(C) = \{x\}$	$y \begin{pmatrix} y & x \\ \cdot & \cdot \end{pmatrix}$ $x \begin{pmatrix} \bullet & \cdot \end{pmatrix}$	$\ell(y) < \ell(x)$ or $\ell(y) \perp \ell(x)$.
$w = t[x + 1]$	$\text{Out}(C) = \{w\}$ $\text{In}(C) = \{t, x\}$	$w \begin{pmatrix} w & t & x \\ \cdot & \cdot & \cdot \end{pmatrix}$ $t \begin{pmatrix} \bullet & \cdot & \cdot \end{pmatrix}$ $x \begin{pmatrix} \bullet & \cdot & \cdot \end{pmatrix}$	$\ell(w) < \ell(t), \quad \ell(w) \perp \ell(t),$ $\ell(w) < \ell(x)$ or $\ell(w) \perp \ell(x)$.
$t[i] = u + j$	$\text{Out}(C) = \{t\}$ $\text{In}(C) = \{i, u, j\}$	$t \begin{pmatrix} t & i & u & j \\ \cdot & \cdot & \cdot & \cdot \end{pmatrix}$ $i \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix}$ $u \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix}$ $j \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix}$	$\ell(t) < \ell(i), \quad \ell(t) \perp \ell(i),$ $\ell(t) < \ell(u), \quad \ell(t) \perp \ell(u),$ $\ell(t) < \ell(j)$ or $\ell(t) \perp \ell(j)$.

Table 25: Statement Examples, Sets, Representations of their Possible Violation(s).

3.2.3.3.1 Base Cases: Assignment and Skip. The security-flow matrix for an assignment C simply tracks flows from $\text{In}(C)$ to $\text{Out}(C)$:

Definition 14 (Assignment). *Given an assignment C , we define $\mathbb{M}(C)$ by:*

$$\mathbb{M}(C)(x, y) = \begin{cases} \bullet & \text{if } x \in \text{In}(C), y \in \text{Out}(C) \text{ and } x \neq y \\ \cdot & \text{otherwise} \end{cases}$$

□

We illustrate in Table 25 some basic cases: we consider an array a single entity, and that changing one value in it means being able to access it completely. More precisely, $t[i]$ on the left-hand side of an assignment is a violation if $\ell(t) > \ell(i)$ (resp. $\ell(t) \perp \ell(i)$). Indeed, it

implies that a lower-class (resp. orthogonal-class) variable (i) can decide where to write in a higher-class (resp. orthogonal-class) variable (t). However, $t[i]$ as an expression (e.g., on the right-hand side of an assignment or in a condition, as discussed in paragraph 3.2.3.3.3) is acceptable as long as the variable(s) storing the result of this calculation or dependent on that condition's truth value have class higher or equal to t and i classes.

Definition 15 (Skip). We let $\mathbb{M}(\text{skip})$ be the matrix with 0 rows and columns. □

Identifying $\mathbb{M}(\text{skip})$ with its embeddings, it is the empty matrix of any size.

3.2.3.3.2 Composition as a Commutative Operation. The security-flow matrix for a composition of commands is an abstraction that allows manipulating a sequence of commands as one command with its own matrix.

Definition 16 (Composition). We let $\mathbb{M}(C_1; \dots; C_n)$ be $\mathbb{M}(C_1) + \dots + \mathbb{M}(C_n)$. □

$$\begin{array}{c}
 \begin{array}{c} C_1 \\
 \begin{array}{ccccc} & w & x & y & z \\
 w & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \\
 x & \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix} \\
 y & \begin{pmatrix} \cdot & \cdot & \cdot & \bullet \end{pmatrix} \\
 z & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}
 \end{array}
 \end{array}
 +
 \begin{array}{c}
 \begin{array}{c} C_2 \\
 \begin{array}{ccccc} & w & x & y & z \\
 w & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \\
 x & \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix} \\
 y & \begin{pmatrix} \cdot & \bullet & \cdot & \bullet \end{pmatrix} \\
 z & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \begin{array}{c} C_1; C_2 \\
 \begin{array}{ccccc} & w & x & y & z \\
 w & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \\
 x & \begin{pmatrix} \bullet & \cdot & \cdot & \cdot \end{pmatrix} \\
 y & \begin{pmatrix} \cdot & \bullet & \cdot & \bullet \end{pmatrix} \\
 z & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}
 \end{array}
 \end{array}$$

$$\begin{array}{l}
 w = w + x; \\
 z = y + z
 \end{array}$$

$$\begin{array}{l}
 x = y * 2; \\
 z = 0
 \end{array}$$

Figure 35: Security-Flow Matrix of compositions.

The composition of commands C_1 and C_2 —themselves already the result of compositions of assignments involving disjoint variables—is illustrated in Fig. 35. Two important observations:

1. Some existing approaches might consider $C_1;C_2$ as free of violation even if $\ell(z) < \ell(y)$, since $z = \circ$ will wipe out the content of z and “cancel” the violation introduced by $z = y$. The intuition is that an attacker observing the output (or even all the final values) cannot deduce anything about z ’s value (and, transitively, about the value of the higher-class y) once the computation is over. Our “once a violation, always a violation” approach ignores the fact that “ultimately”, this violation may be hidden—the anytime non-interference guarantee is discussed in §3.2.4.
2. Interestingly, $\mathbb{M}(C_1;C_2) = \mathbb{M}(C_2;C_1)$ since composition is interpreted as a sum of matrices over our *commutative* security monoid. While previous flow-based approaches (Aubert, Rubiano, Rusch, & Seiller, 2022a; Aubert, Rubiano, Rusch, & Seiller, 2023e; Jones & Kristiansen, 2009) require a semi-ring because composition was handled *via* product of matrices, the current set-up simplifies the machinery precisely to keep track of past violations.

3.2.3.3.3 A Correction for Implicit Flows. To account for implicit flows, branchings and loops require a *correction*. The main idea is that interpreting `if e then  $C_1$  else  $C_2$` (resp. `while e do C`) require to record that all the variables modified in C_1 and C_2 (resp. in C) depend on the variables *occurring* in e (as opposed to the assignment considering the variables *used* by C).

Definition 17 (Correction). *The correction $\text{Cr}(e)_C$ of an expression e on a program C is*

$$\text{Cr}(e)_C(x, y) = \begin{cases} \blacklozenge & \text{if } x \in \text{Occ}(e), y \in \text{Out}(C) \text{ and } x \neq y \\ \cdot & \text{otherwise} \end{cases}$$

□

Intuitively, the correction states that if the variable y is modified in the body of either branch of the branching or in the body of the loop and x occurs in the expression, then there is a violation if $\ell(y) < \ell(x)$ or $\ell(y) \perp \ell(x)$.

As an example, let us use Fig. 35 to construct $\text{Cr}(w > x)_{C_1;C_2}$, e.g., $w > x$'s correction for $C_1;C_2$. Variables w and x , through the expression $w > x$, control the values of w , x and z since C_1 and C_2 set those values, and their execution depend on it, giving:

$$\begin{array}{c} w \quad x \quad y \quad z \\ \begin{array}{c} w \\ x \\ y \\ z \end{array} \begin{pmatrix} \cdot & \bullet & \cdot & \bullet \\ \bullet & \cdot & \cdot & \bullet \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}.$$

Observe also that in $\text{Cr}(t[i] := x)_C$ the variables t , i and x would be marked as controlling the variables occurring in $\text{Out}(C)$. However, no constraint would be imposed between the classes of t , i and x , since they would all be required to flow into classes that are higher or equal to theirs.

3.2.3.3.4 Conditionals and Loops. Following our previous observation, branchings and loops are interpreted similarly.

Definition 18 (Branching). *We let $\mathbb{M}(\text{if } e \text{ then } C_1 \text{ else } C_2)$ be $\mathbb{M}(C_1;C_2) + \text{Cr}(e)_{C_1;C_2}$.*

□

Adding $\text{Cr}(w > x)_{C_1; C_2}$ to $\mathbb{M}(C_1) + \mathbb{M}(C_2)$ from Fig. 35, we obtain:

$$\mathbb{M} \left(\begin{array}{l} \text{if } (w > x) \text{ then} \\ \quad w = w + x; \\ \quad z = y + 2 \\ \text{else} \\ \quad x = y * 2; \\ \quad z = 0 \end{array} \right) = \begin{array}{c} w \quad x \quad y \quad z \\ \begin{array}{c} w \\ x \\ y \\ z \end{array} \begin{pmatrix} \cdot & \blacktriangledown & \cdot & \blacktriangledown \\ \blacktriangledown & \cdot & \cdot & \blacktriangledown \\ \cdot & \blacktriangledown & \cdot & \blacktriangledown \\ \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}.$$

Observe that there is a violation if $\ell(w) < \ell(x)$ or $\ell(w) \perp \ell(x)$ from the statement $w = w + x$, and that there is a violation if $\ell(x) < \ell(w)$ or $\ell(x) \perp \ell(w)$. The latter comes from the fact that the value of w will decide if $x = y * 2$ will execute through the expression. To be free of violations, such a program must be given a class assignment satisfying $\ell(w) = \ell(x)$ and the other constraints recorded in the matrix.

Definition 19 (Loop). We let $\mathbb{M}(\text{while } e \text{ do } C)$ be $\mathbb{M}(C) + \text{Cr}(e)_C$. □

$$\mathbb{M} \left(\begin{array}{l} \text{while}(t[i] \neq j) \{ \\ \quad s1[i] = j*j; \\ \quad s2[i] = 1/j; \\ \quad i++ \\ \} \end{array} \right) = \begin{array}{c} t \quad i \quad j \quad s1 \quad s2 \\ \begin{array}{c} t \\ i \\ j \\ s1 \\ s2 \end{array} \begin{pmatrix} \cdot & \blacktriangledown & \cdot & \blacktriangledown & \blacktriangledown \\ \cdot & \cdot & \cdot & \blacktriangledown & \blacktriangledown \\ \cdot & \blacktriangledown & \cdot & \blacktriangledown & \blacktriangledown \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \end{array}$$

Since $s1$ and $s2$ do not control any other variable, their rows are all $\cdot$ -s. On the other hand, t , i and j control the values of $s1$, $s2$ and i , since they determine how many times the body will execute.

3.2.4 Capturing Anytime Non-Interference

In the seminal work of Volpano et al. (Volpano, Irvine, & Smith, 1996, pg. 173), “[s]oundness [wa]s formulated as a kind of noninterference property. [...] If a variable v has security [class c], then one can change the initial values of any variables whose security levels are not dominated by $[c]$, execute the program, and the *final value* of v will be the same, *provided the program terminates successfully*.” (our emphasis). *Anytime non-interference*, defined below and captured by $\top_{\text{NI}}^*$, inspects the values *while the program is being executed*. It allows us to 1. avoid making assumptions of program termination, or avoid waiting for termination, and 2. model attackers who are capable of observing updates to variables at class c or lower.

First, we need to define a notion of *timed* execution, which captures the idea that an external observer can see updates on variables below a particular security class in “real time”.

Definition 20 (Timed command execution). *Given*

1. *a program C with variables $x_1, \dots, x_n$,*
2. *a class assignment $\ell : \text{Occ}(C) \rightarrow \text{SC}$,*
3. *a security class $c \in \text{SC}$,*
4. *a time (counter) $t \in \mathbb{N}$,*
5. *and a value list $\vec{v} = v_1, \dots, v_n$,*

we write

- $C[\vec{v}]_0$ *for the program C where the variable x_i was assigned v_i ,¹³ for $1 \leq i \leq n$,*

¹³Since arrays have a fixed size, we assume, for simplicity, that a variable x_i representing an array of size s is given a value $v_i = v_i^1, \dots, v_i^s$.

- $C[\vec{v} \rightarrow \vec{v'}]_t$ if, while executing the commands in $C[\vec{v}]_0$, x_i contains the value v'_i , for $1 \leq i \leq n$ after variables at class c or lower have been updated t times.

If, after t updates, variables at level c or lower stop being updated, then we let, for all $t' > t$, $C[\vec{v} \rightarrow \vec{v'}]_t = C[\vec{v} \rightarrow \vec{v'}]_{t'}$. $\square$

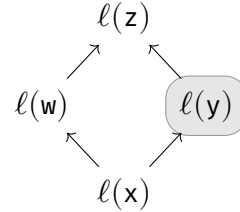
Deciding whether variables will be updated after t may be difficult in all generality, but simple checks as e.g., testing for membership in $\text{Out}(C')$ for $C' \subseteq C$ the subprogram of C that remains to be executed can give in some cases a rapid answer.

We give below a program along with a class assignment (where the security class c is grayed out) and two tables containing the value held by memory locations at time counter t . The initial value lists are $\vec{v}_1 = 1, 2, 3, 4$ and $\vec{v}_2 = 5, 2, 3, 4$. Observe that t is incremented only when values held by variables at or below c (with the grayed out background) are updated.

```

if (w > x) then
    w = w + x;
    z = y + 2
else
    x = y * 2;
    z = 0

```



t	w	x	y	z
0	1	2	3	4
1	1	6	3	4
1	1	6	3	0

t	w	x	y	z
0	5	2	3	4
0	7	2	3	4
0	7	2	3	5

Hence we have $C[\vec{v}_1 \rightarrow \vec{v}'_1]_1$ and $C[\vec{v}_2 \rightarrow \vec{v}'_2]_0 = C[\vec{v}_2 \rightarrow \vec{v}'_2]_1$ for $\vec{v}'_1 = 1, 6, 3, \circ$ and $\vec{v}'_2 = 7, 2, 3, 5$.

Definition 21 (Up-to c equivalence). *Given C , a class assignment $\ell : \text{Occ}(C) \rightarrow \text{SC}$ and $c \in \text{SC}$, two values lists $\vec{v}$ and $\vec{w}$ are up-to c equivalent, written $\vec{v} \stackrel{c}{\sim} \vec{w}$ if $\ell(x_i) \leq c \implies v_i = w_i$.* $\square$

Intuitively, two value lists are up-to c equivalent if they agree on the values of the variables of class c or lower: re-using the example above, we have $\vec{v}_1 \stackrel{\ell(y)}{\sim} \vec{v}_2$ but $\vec{v}_1 \not\stackrel{\ell(w)}{\sim} \vec{v}_2$.

We can now formally state the anytime non-interference property:

Definition 22 (Anytime non-interference). *A program C is anytime non-interfering for $\ell : \text{Occ}(C) \rightarrow \text{SC}$ if for all security class $c \in \text{SC}$, for all time $t \in \mathbb{N}$ and all $\vec{v}$ and $\vec{w}$,*

$$\vec{v} \stackrel{c}{\sim} \vec{w}, C[\vec{v} \rightarrow \vec{v}']_t, C[\vec{w} \rightarrow \vec{w}']_t \implies \vec{v}' \stackrel{c}{\sim} \vec{w}'.$$

$\square$

Note that we can conclude that the program above is *not* anytime non-interfering for the given class assignment, since $\vec{v}_1 \stackrel{\ell(y)}{\sim} \vec{v}_2$ but $\vec{v}'_1 \not\stackrel{\ell(y)}{\sim} \vec{v}'_2$: we had already noted, using $\top_{\text{NI}}^*$, that $\ell(w) = \ell(x)$ was required for this program to be anytime non-interfering. Indeed, this property has a natural equivalent in $\top_{\text{NI}}^*$, and can be established using it:

Definition 23 (Non-interfering class assignment). *Given $\mathbb{M}(C)$, a class assignment ℓ is anytime non-interfering for C iff C has no violation (Def. 13).* $\square$

Note that the trivial class assignment i that assigns to all values the same security class c_i is always non-interfering, since in that case $i(y) < i(x)$ and $i(y) \perp i(x)$ are always false. Conversely, any program is anytime non-interfering for i , since value lists are up-to c_i equivalent if and only if they are equal.

Theorem 3 (Correspondance). *A program C is anytime non-interfering for ℓ (Def. 22) if and only if ℓ is anytime non-interfering for C (Def. 23).*

The proof is detailed in §3.2.8, it leverages the idea that only assignments and corrections can introduce $\blacklozenge$ in security-flow matrices. This mirror the idea that only assignments, loops and branchings can trigger the update of an element in a value list, hence connecting the two definitions of anytime non-interference. An important assumption is that expressions are falsifiable, e.g., that if $x_i \in \text{Occ}(e)$, then there exists at least one value for v_i that will make e evaluate to `false`, and at least one value that will make it evaluate to `true`.

3.2.5 Interpreting Function Calls in an Anytime Non-Interfering Context

We detail below how function calls can be integrated into our analysis. The main challenge is to nail down the correct interpretation of anytime non-interference for functions that may have side effects or, conversely, that may return a value with a lower class than its inputs. We start, as a warm-up, by discussing how to add to $\top_{\text{NI}}^*$ pure functions, then functions with side effects, before finally discussing the meaning of anytime non-interference for functions.

3.2.5.1 Warm-Up: Pure Functions

First, let us discuss how *pure* functions can be integrated into $\mathbb{T}_{\text{NI}}^*$. The first steps are to add $\text{fun}(\text{exp}, \dots, \text{exp})$ to the expressions, let f and g range over function, and to let $\text{Occ}(f(e_1, \dots, e_n)) = f_{\bar{e}}$ for $f_{\bar{e}}$ a freshly introduced variable unique to f , $e_1, \dots, e_n$.¹⁴ Let us illustrate those first steps by interpreting two programs involving function calls. Simply using the definition of $\text{Occ}(f(e_1, \dots, e_n))$, and without changing Definitions 14 or 18, we have:

C	$\text{Out}(C), \text{In}(C)$	$\mathbb{M}(C)$
$x = f(y)$	$\text{Out}(C) = \{x\}$ $\text{In}(C) = \{f_y\}$	$\begin{matrix} & x & y & f_y \\ \begin{matrix} x \\ y \\ f_y \end{matrix} & \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \bullet & \cdot & \cdot \end{pmatrix} \end{matrix}$
$\text{if } (g(x, y) > x)$ $\quad \text{then } y = z$ else skip	$\text{Out}(C) = \{y\}$ $\text{In}(C) = \{g_{x,y}, x, z\}$	$\begin{matrix} & x & y & z & g_{x,y} \\ \begin{matrix} x \\ y \\ z \\ g_{x,y} \end{matrix} & \begin{pmatrix} \cdot & \bullet & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \bullet & \cdot & \cdot \\ \cdot & \bullet & \cdot & \cdot \end{pmatrix} \end{matrix}$

Table 27: Anytime non-interference with pure functions.

It may seem surprising that y does not occur in the $\text{In}(C)$ sets, considering that its value may affect the output of the function call, hence controlling indirectly the out-variables. This design choice *lets the class assignment handle this decision*. The core idea is that the level assignment $\ell : \text{Occ}(C) \rightarrow \text{SC}$ now additionally needs to assign a class (or a collection of constraints) to each $f_{\bar{e}}$ variable. Multiple design choices exist, e.g.,

- $\ell(f_{\bar{e}})$ can be a constant class c , reflecting the fact that all function outputs should be assigned the same security class regardless of the classes assigned to its inputs,

¹⁴This point is clarified at the end of this subsection.

- $\ell(f_{\vec{e}})$ can be a function of $\ell(x)$ for $x \in \text{Occ}(e_1) \cup \dots \cup \text{Occ}(e_n)$ such as the supremum, the infimum (written max and min), the first projection π_1 , etc.
- $\ell(f_{\vec{e}})$ could otherwise depends on the particular structure of $\vec{e}$, e.g., be the supremum if a variable whose class is above a particular threshold occurs, a constant otherwise, etc.

This adds “external” constraints to our definition of violation: *in addition* of having to provide a level assignment meeting the condition of Def. 13, one has to check that the constraints given on the classes of the $f_{\vec{e}}$ variables are met. As an additional benefit, this allows to handle functions with 0 parameters, since the flow from the argument(s, or lack thereof) to the function’s output need not to be tracked in the security-flow matrix.

Going back to our first example above, if we consider that f ’s output class must be strictly higher than its input class, then we have the additional requirement that $\ell(f_x) > \ell(x)$ for each variable x such that f_x occurs in $\mathbb{M}(C)$. Hence, our first program C would be free of violations

with $\begin{array}{c} \ell(x) \\ \uparrow \\ \ell(f_y) \\ \uparrow \\ \ell(y) \end{array}$ and $\begin{array}{c} \ell(f_y) = \ell(x) \\ \uparrow \\ \ell(y) \end{array}$, but it would have a violation with $\begin{array}{ccc} & \ell(x) & \\ & \nearrow \quad \nwarrow & \\ \ell(y) & & \ell(f_y) \\ \nwarrow \quad \nearrow & & \\ & c & \end{array}$, even

if this latter class assignment would have met the requirements of Def. 13.

The rest of the interpretation is the same, even $\mathbb{M}(C)$ remains a $|\text{Occ}(C)| \times |\text{Occ}(C)|$ matrix, since $f_{\vec{e}}$ variables are defined as occurring in C . The only tedious aspect is to handle the introduction of $f_{\vec{e}}$ variables elegantly. One would want e.g.,

$$\text{Occ}(f(x + y))) = \text{Occ}(f(x - y))) \quad \text{and} \quad \text{Occ}(f(x + 3))) = \text{Occ}(f(x - 5)))$$

but relying on *the set* $\text{Occ}(e_1) \cup \dots \cup \text{Occ}(e_n)$ would not be correct, as one would want e.g.,

$$\text{Occ}(f(x, y)) \neq \text{Occ}(f(y, x)) \quad \text{and} \quad \text{Occ}(f(x, x, y)) \neq \text{Occ}(f(x, y, y)).$$

A more precise definition of function type signature, capable of handling repetition and swapping in the argument list, would be required but presents no challenge.

3.2.5.2 Completing the Picture: Functions With Side Effects

Our development so far assumes that the only way a function can leak information is through its return value. Considering functions with side effects (such as `print`, `read`, accessing a non-local variable, passing argument by reference, etc.) increases expressivity of $\mathbb{T}_{\text{NI}}^*$, but requires to discuss more precisely what is meant by anytime non-interfering function calls.

We first focus on how effects can be integrated into $\mathbb{T}_{\text{NI}}^*$. Interestingly, the column $f_{\bar{e}}^{\downarrow}$ always remains empty, since the variable $f_{\bar{e}}$ will never be in an Out set (it never “receives” a value). Hence, we can use its out-variable to store information about its possible side effects. To this end, we now “split” $f_{\bar{e}}$ into $f_{\bar{e}}^{\text{in}}$ (on the row) and $f_{\bar{e}}^{\text{out}}$ (on the column) for each “signature” $f(e_1, \dots, e_n)$. This convention is illustrated in Ex. 31 with another example of *pure* functions.

To account for effects, the expression $\text{fun}(exp, \dots, exp)$ should now be treated as a *command* and an *expression*. We then edit the definition of the variables occurring in $f(e_1, \dots, e_n)$

C	$\mathbb{M}^e(C) = \dots$	$\text{Out}(C), \text{In}(C)$	$\mathbb{M}^e(C)$
$g(x + y)$	$\mathbb{M}(g_{x,y}^{\text{out}} = x + y)$ $+ \mathbb{M}(\text{skip})$	$\text{Out}(C) = \{g_{x,y}^{\text{out}}\}$ $\text{In}(C) = \{x, y\}$	$\begin{matrix} & x & y & g_{x,y}^{\text{out}} \\ \begin{matrix} x \\ y \\ g_{x,y}^{\text{in}} \end{matrix} & \begin{pmatrix} \cdot & \cdot & \bullet \\ \cdot & \cdot & \bullet \\ \cdot & \cdot & \cdot \end{pmatrix} \end{matrix}$
$x = f(y)$	$\mathbb{M}(f_y^{\text{out}} = y)$ $+ \mathbb{M}(x = f(y))$	$\text{Out}(C) = \{x, f_y^{\text{out}}\}$ $\text{In}(C) = \{y, f_y^{\text{in}}\}$	$\begin{matrix} & x & y & f_y^{\text{out}} \\ \begin{matrix} x \\ y \\ f_y^{\text{in}} \end{matrix} & \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \bullet \\ \bullet & \cdot & \cdot \end{pmatrix} \end{matrix}$
$\text{if } (g()=1)$ $\text{then } x = \emptyset$ else skip	$\text{Cr}(g()=1)_{x=\emptyset}; \text{skip}$ $+ \mathbb{M}(x = \emptyset)$ $+ \mathbb{M}(g_{\emptyset}^{\text{out}} =) + \mathbb{M}(\text{skip})$	$\text{Out}(C) = \{x, g_{\emptyset}^{\text{out}}\}$ $\text{In}(C) = \emptyset$	$\begin{matrix} & x & g_{\emptyset}^{\text{out}} \\ \begin{matrix} x \\ g_{\emptyset}^{\text{in}} \end{matrix} & \begin{pmatrix} \cdot & \cdot \\ \bullet & \bullet \end{pmatrix} \end{matrix}$

Table 28: Statement Examples, Interpretation and Sets – Involving Effects.

(henceforth simply denoted $f(\vec{e})$) and in $\vec{e}$ to get a more complete picture:

$$\text{Occ}(f(\vec{e})) = f_{\vec{e}}^{\text{in}} \quad \text{Occ}(\vec{e}) = \begin{cases} \text{Occ}(e_1) \cup \dots \cup \text{Occ}(e_n) & \text{if } n > 0 \\ f_{\emptyset}^{\text{in}} & \text{otherwise} \end{cases}$$

with the “otherwise” case above handling functions with 0 parameters. We then let

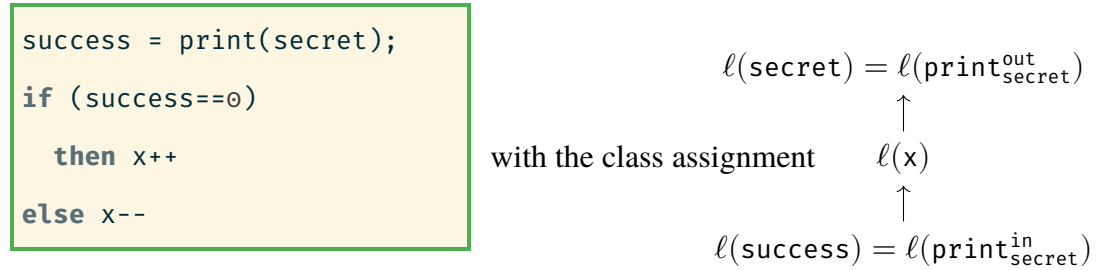
$$\mathbb{M}(f(\vec{e})) = \mathbb{M}(\text{skip}) \quad \mathbb{M}^e(C) = \sum_{f(\vec{e}) \subseteq \text{Occ}(C)} \mathbb{M}(f_{\vec{e}}^{\text{out}} = \vec{e}) + \mathbb{M}(C)$$

and study $\mathbb{M}^e(C)$ moving forward, where the above definition interprets all function calls as assignments and then interpret the rest of the program as before, skipping the commands calling a function without using their return value. Table 28 gathers examples of programs involving effectful functions. The last example follows our definitions, but may be hard to unpack: the critical point is to see that $g_{\emptyset}^{\text{in}}$ controlling the values of x and $g_{\emptyset}^{\text{out}}$ reflects the fact that g “on its own” (i.e., without any input) will decide of its output and hence if $x = \emptyset$ will execute.

This interpretation entails the following two principles:

- An effectful function is completely transparent: the first example of Table 28 requires $\ell(g_{x,y}^{\text{out}}) \geq \max(\ell(y), \ell(x))$, e.g., as if g is revealing all the data it is processing.
- A function can nevertheless have $\ell(f_e^{\text{in}})$ be less than or orthogonal to the level of its arguments. This means that a function can have a return value that is independent of its arguments, e.g., a success or failure code in displaying the arguments at the screen.

Those principles can be both desirable and are not incompatible. Indeed, a program



should be considered as anytime non-interfering: a user having access to `secret`'s class can see its value be displayed on the screen, but an attacker having access to at most `x`'s class cannot infer `secret`'s value, despite being able to access the return value of `print(↓ secret)`—which is constantly set to the lowest class available.

Three challenges remain:

- The intuitive reading of our security-flow matrices is lost. For example, since $y \notin \text{In}(x = f(y))$, $\mathbb{M}^e(x = f(y))(x, y) \neq \spadesuit$, and y is not recorded as impacting the value of x . This design choice is a *feature*, as it does not “force” y to control x 's value when processed through f : this allows finer constraints on the level of f_y^{in} .

- It rigidly assumes that functions with side effects will reveal all their data at all times. This conservative approach is also a feature, but could be tuned by refining how constraints for f_e^{out} class assignments are recorded.
- Developing a definition of anytime non-interference for programs with side effects (Def. 22) will require to develop a notion of external observer and of contextual equivalences, to correctly account for side effects and multiple communication channels.

We believe that those issues can be addressed by developing a richer theory that incorporates *external knowledge on functions*, but reserve it for future work.

3.2.6 Practical Applications and Comparison

3.2.6.1 Implementing the Anytime Non-interference Logic

We have created a prototype static analyzer `TYNI` that implements the $\mathbb{T}_{\text{NI}}^*$ logic. The way matrices are composed in $\mathbb{T}_{\text{NI}}^*$ is a key feature in making the analysis straightforward and efficient in practice. Because composition order is irrelevant (Ex. 27), it suffices to represent matrices as hash maps where composition is a union of maps.

The `TYNI` analyzer accepts as input a program file written in Java. It first translates the program into a parse tree (without optimizations), then analyzes the tree based on the rules of $\mathbb{T}_{\text{NI}}^*$. The analysis is recursive over the methods of a Java class. Obtaining a sound result requires a Java method fully expressible in the $\mathbb{T}_{\text{NI}}^*$ grammar (Fig. 34). Commands that are not covered are highlighted by `TYNI` and the analyzer outputs a partial result. This handling assists the continued development of $\mathbb{T}_{\text{NI}}^*$. Currently, it already handles all the examples from §3.2.9.

The outlined engineering choices have multiple advantages. Java is frequently used to implement taint analyzers, an instance of non-interference fixed to two security classes. Tyni is thus prepared for a similar use case. Since Java compiles into bytecode, a kind of intermediate stack language, it enables program analysis at multiple language representations. Although compiler optimizations could reduce the rate of false alarms, e.g., by eliminating dead-code, it would artificially inflate the analysis precision and thus we prefer our strategy. Currently, Tyni produces security-flow matrices for input programs. The security flow matrices serve as basis for the extended applications, including the directions presented next.

Preservation of anytime non-interference. The $\mathbb{T}_{\text{NI}}^*$ logic does not require much language structure; in particular, it assumes no language-specific syntactic features. It is possible to map its grammar to numerous language representations, including intermediate representations and bytecode. Comparing security-flow matrices of the same program at different representations enables analyzing preservation of security properties and detecting compilation issues.

Security class inference. When security classes of variables are known partially, it is possible to infer them for all variables. The inference requires a security-flow matrix, an information flow policy, and the known class assignments. The inference is then framed as a satisfiability problem. If a satisfactory assignment exists, it provides the security classes for all variables. This application is similar to type inference, but requires no program as input. Further, the same security-flow matrix can be easily evaluated against different information flow policies.

Taint analysis. Taint analysis detects information flow issues between a high source and a low sink. Aside a program, the sources and sinks are necessary, and analyzers commonly assume them as inputs. The analyzers then compete on precision along various axes: path-coverage, syntax-coverage, context-sensitivity, false alarm rate,

etc. Taint analysis can be formulated with security-flow matrices by analyzing source to sink connectivity.

3.2.6.2 Circumventing Termination-insensitivity via Distribution

Several real-world programs are non-terminating by design: web servers, embedded systems, and cyber-physical systems are among the examples. While the programs can terminate, the termination events are infrequent and uncharacteristic of standard behavior. Security analyses that can handle absence of termination are necessary to support such programs.

Anytime non-interference is termination-insensitive and compatible with the study of non-terminating programs. However, termination-insensitivity is too weak to guard against untrusted code, e.g., execution of the `eval` command. To offer an alleviation strategy, we present an approach to distribute security-sensitive computation. This way, computations that require elevated security checks are handled separately from trusted code.

The idea is to pair $\top_{NI}^*$ with a *distribution analysis* (Aubert, Rubiano, Rusch, & Seiller, 2023e) that detects disjoint program fragments. That program fragments are disjoint means there exists no exchange of variable data between program fragments. The judgement of disjointness is derived via a sound data flow analysis that guarantees the property. It is then permissible to execute the program fragments in separate execution contexts. Although the distribution analysis naturally fits parallel computations, it is not restricted to this use case. We conjecture it offers broader utility here, to ensure program security.

To combine the two analysis, we first analyze a program with $\top_{NI}^*$ to identify its information flow constraints. Then, we use the distribution analysis to identify the program's distribution potential. Merging the results, the disjoint program fragments are assigned appropriate security classes. The fragments are then allocated to different execution contexts, where

each fragment can have a different security class designation. This way, a program must not adhere to a monolithic security strategy, but can have a finer-grained strategy based on its content. Due to paper scope, we reserve a detailed treatment for an extended version.

3.2.6.3 Overview of Alternative and Related Approaches

In language-based security, non-interference is commonly achieved through security type systems. Type theoretic non-interference provides strong end-to-end confidentiality guarantees in a static and scalable way. Initiating from the seminal work of Volpano et al. (Volpano, Irvine, & Smith, 1996), security type systems have been extended to consider non-interference under numerous paradigms, including concurrency (Derakhshan, Balzer, & Yao, 2024; Frumin, Krebbers, & Birkedal, 2021; Volpano & Smith, 1998), formal reasoning (Frumin, Krebbers, & Birkedal, 2021; Nelson, Bornholt, Krishnamurthy, Torlak, & Wang, 2020), secure compilation (Barthe, Basu, & Rezk, 2004), etc. A major challenge among the security type systems is *declassification*, a kind of security downgrading operation (Cecchetti, Myers, & Arden, 2017). A *downgrading* mechanism permits elevating the security judgement around control-flow constructs then lowering it afterward. In other words, information is allowed to flow contrary to the policy (Cecchetti, Myers, & Arden, 2017). The mechanism is necessary to increase the expressive power of security type systems. However, downgrading is generally not safe (Derakhshan, Balzer, & Yao, 2024) and eliminates the strong compositional guarantees of non-interference (Cecchetti, Myers, & Arden, 2017). In practice, security type systems are challenging to use because they modify the programming language. A program must be annotated with security types and compiled with non-standard tools that can enforce the types (Lamba, Taylor, Beardsley, Bambeck, Bond, & Lin, 2024). There is also a stark contrast in expressiveness of theoretical and practical systems. E.g., (Huang, Dong, & Milanova, 2014) categorically excludes implicit flows.

The Dependency Core Calculus (DCC) (Abadi, Banerjee, Heintze, & Riecke, 1999) is conceptually related to $\top_{NI}^*$. The DCC is an extension of lambda-calculus, framed around the notion of data dependence, of which non-interference is an instance. Though similarly rooted in dependency analysis $\top_{NI}^*$ originates from works of implicit computational complexity (ICC). It is a refinement of (Aubert, Rubiano, Rusch, & Seiller, 2023e; Moyen, Rubiano, & Seiller, 2017a), but $\top_{NI}^*$ required significant adjustment, particularly around matrix composition and functions.

Implicit computational complexity studies machine-free characterizations of complexity classes by introducing *restrictions* in programming languages that in turn guarantee semantic properties (Dal Lago, 2011). A critical idea is that ICC techniques can benefit from, and offer support in, other analytic domains. The use of ICC techniques in such extended ways is an emerging research topic. Previously, a non-interference type system provided a foundation for a series of complexity-theoretic results (Hainry & P  choux, 2023; Marion, 2011). In the opposite direction, an ICC system was applied to cryptographic proofs (Baillot, Barthe, & Dal Lago, 2019). Although $\top_{NI}^*$ has transformed from its origins to not enforce complexity bounds, it reinforces the bidirectional connection between ICC and language-based security.

3.2.7 Conclusion: Strengths, Limitations and Future Directions

Anytime non-interference detects violations at any program point, enforcing a finer-grained security policy than classic non-interference that is defined in terms of inputs and outputs. We have presented $\top_{NI}^*$, a sound and compositional program logic, that captures the semantic security property of anytime non-interference in imperative programs. The logic assigns security flow matrices to commands where the matrices represent the program’s potentially interfering information flows. The logic is lightweight and does not require program anno-

tations, specialty compilers, and adds no run-time overhead. Beside the compelling theory, $\mathbb{T}_{\text{NI}}^*$ can be implemented to obtain automated security analysis in practice. We have constructed a prototype static analyzer `TYNI` to analyze Java programs. By extension, `TYNI` can support a range of applications, e.g., security class inference, taint analysis, and security preservation analysis.

Although the utility of $\mathbb{T}_{\text{NI}}^*$ is encouraging, the development is still mainly theoretical. Our immediate priority is enriching the syntax with effectful functions and object oriented constructs. For additional strength, we hope to mechanize the theory. On the practical side, the prototype analyzer has room for enhancements. It already computes security-flow matrices, but an extension to the applications requires additional engineering steps. With the current syntax coverage, experimental comparisons are still out of scope. In the meantime, $\mathbb{T}_{\text{NI}}^*$ provides an promising avenue for security analysis and future enhancements.

3.2.8 Appendix A: Proof of Thm. 3

A first useful observation is that if $C' \subseteq C$, then $\mathbb{M}(C')$ is included in $\mathbb{M}(C)$, in the sense that $\mathbb{M}(C')(x, y) = \blacklozenge \implies \mathbb{M}(C)(x, y) = \blacklozenge$. This simple observation comes from our “additive” interpretation of commands, and is useful in proving our theorem. One should also note that if ℓ is not anytime non-interfering for C' , then any class assignment extending ℓ to $\text{Occ}(C)$ is not anytime non-interfering for C .

Theorem 3 (Correspondance). *A program C is anytime non-interfering for ℓ (Def. 22) if and only if ℓ is anytime non-interfering for C (Def. 23).*

Proof. Let us assume given C , $\ell : \text{Occ}(C) \rightarrow \text{SC}$, and that $\mathbb{M}(C)$ has been computed.

For the if part Suppose that ℓ is anytime non-interfering for C , but that C is not anytime non-interfering for ℓ . Then there must exist a class $c \in \text{SC}$ and a counter t such that for some $\vec{v}$ and $\vec{w}'$,

$$\vec{v} \stackrel{c}{\sim} \vec{w} \quad (3.1) \qquad c[\vec{w} \rightarrow \vec{w}']_t \quad (3.3)$$

$$c[\vec{v} \rightarrow \vec{v}']_t \quad (3.2) \qquad \vec{v}' \stackrel{c}{\not\sim} \vec{w}' \quad (3.4)$$

For $\stackrel{c}{\not\sim}$ the negation of $\stackrel{c}{\sim}$, i.e., there must exists x_i such that

$$\ell(x_i) \leq c \quad (3.5) \qquad v'_i \neq w'_i \quad (3.6)$$

For Equation 3.6 to hold, it must be the case that C contains a statement of the form $x_i = e_1$,¹⁵ possibly guarded by while and if statements using the expressions

¹⁵Which can be $t[e_1^1] = e_1^2$, in which case we let $\text{Occ}(e_1) = \text{Occ}(e_1^1) \cup \text{Occ}(e_1^2)$ and carry out the same reasoning.

$e_2, \dots, e_n$. Let $x_1, \dots, x_m = \bigcup_{j=1}^n \text{Occ}(e_j)$, and observe that since by Equation 3.1 our input value lists are up-to c equivalent, it must be the case that there exists $j \in \{1, \dots, m\}$ such that

$$\ell(x_j) > c \text{ or } \ell(x_j) \perp c, \quad (3.7)$$

otherwise Equation 3.6 could not hold.¹⁶ Furthermore, thanks to Equation 3.5 we know that

$$j \neq i. \quad (3.8)$$

Let C' be the smallest sub-program of C where $x_i = e_1$ occurs and either $x_j \in \text{Occ}(e_1)$ or x_j occurs in the condition of a while or if command guarding the command $x_i \leftarrow e_1$. Intuitively, C' has one of the following forms:

```
x_i = ... x_j ...;
```

```
while(... x_j ...){
    ...
    x_i = ...;
    ...
}
```

```
if(... x_j ...){
    ...
    x_i = ...;
    ...
}
```

Listing 75: (A)ssignment Case

Listing 76:
(L)oop Case

Listing 77: (B)ranching Case

Hence, $x_j \in \text{In}(C')$, $x_i \in \text{Out}(C')$, and inspecting the rules of our interpretation allows us to conclude that $\mathbb{M}(C)(x_j, x_i) = \spadesuit$, since $\mathbb{M}(C')$ is included in $\mathbb{M}(C)$.¹⁷

¹⁶To be more rigorous, it could be the case that the classes of $x_1, \dots, x_m$ are c or below, but that *one of them is itself* impacted by a variable at a higher or incomparable class. To handle, this case, one simply replaces x_i and x_j with those “problematic” variables, decreases the counter t to when the value of the one with the lower class was changed, and carry out the same reasoning, possibly repeating this step again. Since t decreased, we are guaranteed to identify “the first” anytime non-interference violation and to reason about it.

¹⁷In brief terms, this comes from Table 24, remembering that x_j being in the condition in the (L) and (B) cases implies that it is in $\text{In}(C')$ and that a $\spadesuit$ was introduced between its in-variable and x_i ’s out-variable in $\mathbb{M}(C')$.

Hence, $x_j, x_i \in \text{Occ}(C)$ and $j \neq i$ by Equation 3.8, so we can use our assumption that ℓ is non-interfering for C to conclude that $\ell(x_j) \leq \ell(x_i)$ —which, in conjunction with Equation 3.5, contradicts Equation 3.7.

For the only if part Let us assume that C is anytime non-interfering for ℓ , we need to prove that ℓ is anytime non-interfering for C e.g., that C has no violation: for all i, j ,

$$\mathbb{M}(C)(x_j, x_i) = \spadesuit \quad (3.9)$$

implies

$$\ell(x_j) \leq \ell(x_i). \quad (3.10)$$

Note that if $i = j$, then Equation 3.9 cannot hold since security-flow matrices are hollow, hence we only have to prove the $i \neq j$ case. We prove it below, factoring-in as previously the remarks about x_i possibly being an array and having to “chase down” the exact pair of variables violating anytime non-interference.

Equation 3.9 implies that there is a sub-program C' of C such that $x_j \in \text{In}(C')$ and $x_i \in \text{Out}(C')$.¹⁸ By a reasoning similar to the previous case, it means that C' has one of the three forms (A), (L) or (B) presented in Listings 75–77.

Now, consider two values lists $\vec{v}$ and $\vec{w}$ that are up-to $\ell(x_i)$ equivalent, and assume by contradiction that Equation 3.10 does not hold. It means that $\vec{v}$ and $\vec{w}$ can diverge on the value of v_j that gets attributed to x_j , but that at any time counter t , we should have $C'[\vec{v} \rightarrow \vec{v'}]_t, C'[\vec{w} \rightarrow \vec{w'}]_t \implies \vec{v'} \sim \vec{w'}$. However, depending on the form of C' , the value held by x_j will impact directly (A) or indirectly ((L), (B)) the value held by x_i at a particular time, or the number of time it is updated,¹⁹ contradicting anytime non-interference of C' and hence of C .

¹⁸Remembering Table 24, if x_j occurs in the expression of a condition, it occurs in the In set of the overall program.

¹⁹This is where our “falsifiability of expressions” hypothesis is used.

□

3.2.9 Appendix B: Examples

To ease the presentation, we present the construction equations as inference rules, treating the inductive ones as inference rules with hypothesis, and the base cases (assignment, skip, but also computing the correction) as axioms.

Example 26 (Transitive information flow). *Consider a program with two commands:*

```

if (h==0) then y = 1 else skip    (C1)
if (y==0) then z = 1 else y = z    (C2)

```

While

Although no direct assignment exists from h to z, the variables are transitively dependent through y. The matrix labels are h y z but omitted for compactness.

The derivation (π_1) of command (C1) is

$$\frac{\frac{}{y==1 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn} \quad \frac{}{\text{Skip} : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{skip} \quad \frac{}{h==0 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cr}}{C1 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cond}$$

and the derivation (π_2) of command (C2) is

$$\frac{\frac{}{z==1 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn} \quad \frac{}{y=z : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn} \quad \frac{}{y==0 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cr}}{C2 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cond}$$

We conclude the derivation by composing the command matrices.

$$\frac{\frac{\vdots \pi_1}{C1 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cond} \quad \frac{\vdots \pi_2}{C2 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Cond}}{C1; C2 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Comp}$$

The concluding matrix captures the flows: z to y , h to y and y to z , but does not show (at top-right) the transitive flow from h to z . A violation depends on the assignment of security classes. Non-interference requires $\ell(h) \leq \ell(y)$ and $\ell(y) \leq \ell(z)$. This exposes the transitive flow $\ell(h) \leq \ell(z)$. A security-flow matrix contains more information than what is immediately visible. $\square$

Example 27 (Composition irrelevance). We derive a matrix for program

`z = 3; x = y; x = z`

While

by

$$\frac{\frac{\vdots}{z=3 : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn} \quad \frac{\vdots}{x=y : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn}}{z=3; x=y : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Comp} \quad \frac{\vdots}{x=z : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Asgn}}{z=3; x=y; x=z : \begin{pmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}} \text{Comp}$$

The matrix labels are $x \ y \ z$. If y holds secret data, and x is a public with $\ell(x) < \ell(y)$, the program violates anytime non-interference. Although x is overwritten in a later command, a violation cannot be erased once it has occurred. Also observe that composition is commutative – composing the commands in any order would yield the same program matrix. $\square$

Example 28 (Context sensitivity). *The following program (from Huang, Dong, and Milanova (2014) adjusted to Fig. 34 grammar) shows assignments to string buffers sb1 and sb2. The potentially sensitive request does not interfere with query. A context-sensitive analysis detects this and does not raise an unnecessary alarm.*

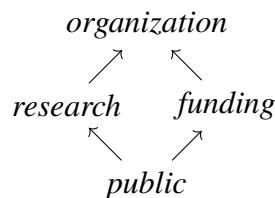
1	user = request["user"];	
2	sb1 = "SELECT * FROM Users WHERE name=";	
3	sb2 = "SELECT * FROM Users WHERE name=";	
4	sb1 += user;	
5	sb2 += "John";	
6	query=sb2;	
7	// execute query	While

	user	request	sb1	sb2	query
user	.	.	•	.	.
request	•	.	.	.	.
sb1	.	.	.	.	.
sb2	.	.	.	.	•
query	.	.	.	.	.

The program matrix is on the right. An anytime non-interference violation is avoided if $\ell(\text{request}) \leq \ell(\text{user})$, $\ell(\text{user}) \leq \ell(\text{sb1})$, and $\ell(\text{sb2}) \leq \ell(\text{query})$. Since request and query are disjoint in the matrix, the variables are anytime non-interfering for all security classes. □

Our next analysis example requires a policy with incomparable security classes, like the one we present now.

Example 29 (HMO information flow policy). *The HMO (for Health Maintenance Organization) information flow policy, represented as a Hasse diagram, is:*



□

Example 30 (Incomparable security classes). *The mvt-kernel, from the PolyBench/C (Pouchet & Yuki, 2016) parallel programming benchmark suite, calculates a matrix vector product and transpose.*

```

1 void kernel_mvt(...){
2   for (i=0; i<N; i++)
3     for (j=0; j<N; j++)
4       x1[i] = x1[i]+A[i][j]*y1[j];
5   for (i=0; i<N; i++)
6     for (j=0; j<N; j++)
7       x2[i] = x2[i]+A[j][i]*y2[j];
8 }

```

C

	i	j	N	x1	x2	y1	y2	A
i	•	•	•	•	•	•	•	•
j	•	•	•	•	•	•	•	•
N	•	•	•	•	•	•	•	•
x1	•	•	•	•	•	•	•	•
x2	•	•	•	•	•	•	•	•
y1	•	•	•	•	•	•	•	•
y2	•	•	•	•	•	•	•	•
A	•	•	•	•	•	•	•	•

Observe that x_1 and x_2 are disjoint, sharing no observable information flow in the matrix.

Their security classes may be incomparable. Using the HMO policy, the assignment

$$\{i, j, N, A\} \mapsto \text{public} \quad \{y_1, x_1\} \mapsto \text{research} \quad \{y_2, x_2\} \mapsto \text{funding}$$

is satisfactory. Similarly, assignment $\ell(x_1) = \text{organization}$ satisfies anytime non-interference. However, $\ell(A) = \text{research}$ is a violation because we have $\mathbb{M}(C)(A, x_2) = \bullet$. It would require that $\text{research} \leq \text{funding}$, but by the HMO policy the classes are incomparable.

□

Example 31 (Function calls and arrays). *The program references two functions (treated as pure), called inside the condition and inside the body of a while loop:*

```

1 while(y < f(b)){
2   t[f(b)] = x;
3   a = t[a] + b;
4   y = g(b, x);
5   x = f(a);
6 }

```

While

	t	a	b	x	y	f_a^{out}	f_b^{out}	$g_{b,x}^{\text{out}}$
t	.	•	.	.	.	.	.	.
a	.	.	.	.	.	.	.	.
b	.	•	.	.	.	.	.	.
x	•	.	.	.	.	.	.	.
y	•	•	.	•	.	.	.	.
f_a^{in}	.	.	.	•	.	.	.	.
f_b^{in}	•	•	.	•	•	.	.	.
$g_{b,x}^{\text{in}}$	.	.	.	.	•	.	.	.

Since variables introduced for (pure) function calls correspond to expressions, they do not belong to any Out set, and their columns in a security-flow matrix will always be empty. However, the class of function parameters can be considered with additional constraints, not reported in the security-flow matrix. Typically, one can require that $\ell(g_{b,x}^{\text{in}}) = \max(\ell(b), \ell(x))$ for all pairs x, b such that $g(b, x)$ occurs in the program. This would invalidate a level assignment with e.g., $\ell(t) < \ell(b)$, since $\ell(b) \leq \ell(g_{b,x}^{\text{in}})$ would be required by this condition, and $\ell(g_{b,x}^{\text{in}}) \leq \ell(y) \leq \ell(t)$ are required by the security-flow matrix. $\square$

3.3 Certifying Complexity Analysis

✓ *Implicit computational complexity & formal methods*

Neea Rusch

Workshop paper/ongoing research.

The workshop paper was presented at the Ninth International Workshop on Coq for Programming Languages (CoqPL) 2023.

Abstract

This work drafts a strategy that leverages the field of Implicit Computational Complexity to certify resource usage in imperative programs. This original approach sidesteps some of the most common—and difficult—obstacles “traditional” complexity theory face when implemented in Coq.

3.3.1 Motivation

The ability to statically infer resource bounds of programs offers numerous benefits, e.g., to insure safe memory usage. Even more preferable if those guarantees are established with the rigor of formal verification, because that increases confidence in the obtained analysis result and enables integration of complexity analyses into larger formal developments.

Unfortunately, computational complexity is notoriously difficult to represent formally for several reasons. In general, deriving a complexity bound for an arbitrary program is an undecidable problem. In the area of complexity theory, “formalisations of even basic complexity-theoretic results are not available” (Forster, Kunze, & Wuttke, 2020, p. 114), hindering certification attempts.

For practical complexity analyses, many existing techniques present methodological challenges if they require e.g., program termination or inlining functions (Carbonneaux, Hoffmann, & Shao, 2015). Therefore, a realistic pathway toward formal certification of a program’s resource usage is narrow. A few encouraging early results exist, and we discuss some of those in §3.3.3. In this proposal we will sketch how a different approach, founded on Implicit Computational Complexity, could sidestep some of the usual difficulties in implementing and verifying complexity analyses in Coq.

The field of Implicit Computational Complexity (ICC) (Dal Lago, 2011) drives better understanding of complexity classes, but it also guides the development of resources-aware languages and static source code analyzers. The core idea is to bound resources *while the program is being written (or type checked)* instead of measuring its resource usage afterwards on an abstract model of computation. This can be done through e.g., bounded recursion or using typing mechanisms. The goal is to find a syntactical restriction or a type system such that a program can be written or typed only if it belongs to a particular complexity class. ICC-based systems are often compositional, and they offer more natural tools to write programs than theoretical models of computation used in complexity theory. We speculate these combined properties could make ICC-approaches a conceivable pathway toward certified complexity and sketch a more detailed plan below.

3.3.2 Preliminary Action Plan

We plan to formalize in Coq an ICC-based complexity analysis technique, the *mwp-flow analysis* (Jones & Kristiansen, 2009).²⁰ We chose this method because its internal mechanics has been recently studied (Aubert, Rubiano, Rusch, & Seiller, 2022a), and by our assessment, it seems suitable for formalization in Coq. As for Coq, it seems like the ideal target language because of its existing libraries and preliminary work—some of which are discussed in §3.3.3—, most notably related to compilers (Leroy, 2009).

3.3.2.1 Overview of *mwp*-Flow Analysis

The *mwp*-flow analysis certifies polynomial bounds on the size of the values manipulated by an imperative program. While it does not ensure (or require) program termination, it

²⁰Where *mwp* stands for *maximum*, *weak polynomial* and *polynomial*, representing increasing growth rates of variables values.

provides a certificate guaranteeing that the program uses throughout its execution at most a polynomial amount of space, and as a consequence that if it terminates, it will do so in polynomial time in the size of its inputs.

The analysis computes, for each program variable, a vector tracking how it depends on other variables. The vector values are determined by applying the nondeterministic rules of the sound *mwp*-calculus to the commands of the program. Those vectors are collected in a matrix. A program is assigned a matrix only if all the values in it are bounded by a polynomial in the inputs sizes. This technique is compositional, abstracts away e.g., iteration bounds, and operates on a memory-less imperative language, reminiscent of the Imp language from Software Foundations (Pierce et al., 2025).

3.3.2.2 The Coq Formalization

Our goal is to certify the analysis as presented in the original paper (Jones & Kristiansen, 2009). Note that this does not mean that the bound is certified, but that *the mechanism to compute those bounds* is certified. Of course, this implies the correctness of the bounds as a by-product but constitutes a major difference w.r.t. the results discussed in §3.3.3. Preliminary explorations have led us to establish the following milestones.

The mathematical foundations Our first goal is to define the mathematical structure required to carry out the rest of the construction. This requires defining vectors, matrices and their operations, semi-rings, and honest polynomials²¹ that are needed to represent the *mwp*-bounds. The Mathematical Components library (Mahboubi & Tassi, 2022; Mathematical Components development team, 2024) will lay the foundations

²¹Which are “polynomial build up from constants in $\mathbb{N}$ and variables by applying the operations $+$ (addition) and $\times$ (multiplication).” (Jones & Kristiansen, 2009, p. 5)

for the linear algebra representations, but likely requires extensions to accommodate our specific analysis.

Implementing the language The analyzed language is a simple imperative language that manipulates natural numbers, held in a fixed number of program variables. Its syntax includes variables, expressions (operations $+$ and $\times$), Boolean expressions, and commands (e.g., assignment, loop and decision statements, command sequences, and skip), with their usual semantics. We expect implementing it and its small-steps semantics in Coq to be relatively simple, following the examples from Software Foundations (Pierce et al., 2024, 2025).

Implementing the typing system Even if it can be computationally expensive to run an automatic inference (Aubert, Rubiano, Rusch, & Seiller, 2023b), the typing system *in itself* is relatively simple. It contains only 10 rules, essentially one for each type of command, and except for the initial assignment of vectors to variables, is fully deterministic. We conjecture that standard methods (Chlipala, 2010, 2022) to implement simple type systems will be enough, but will require some care to scale to the matrix-as-type paradigm of this analysis.

Certifying the analysis This will be the most demanding part of our plan. The original paper contains all the required handwritten proofs, but the authors caution that “[t]hese proofs are long, technical and occasionally highly nontrivial” (Jones & Kristiansen, 2009, p. 2). The main result of the paper is the soundness proof of the analysis (Jones & Kristiansen, 2009, Theorem 5.3), i.e., the proof of the existence of a matrix typing the program implies the existence of an honest polynomial bounding the variables’ growth rates. The main result follows from 15 pages of proofs presented in section 7 of the paper. This section revolves around proving the soundness properties of the calculus, and we expect the most substantial effort to be spent on formalizing these proofs. Some of them are quite intricate but with a satisfactory level of detail. The

cases concerning soundness of loops are the most difficult on paper, but their inductive nature *should* (we hope!) be processed by Coq rather easily.

We leave for future work the possibility of creating a formally verified, automatic static analyzer founded on the proof of correctness of this method: as we discussed in other works (Aubert, Rubiano, Rusch, & Seiller, 2022a, 2023b), care is required to implement a typing strategy that does not rapidly become intractable.

3.3.3 Related Work

A few prior results exist that combine formalization of complexity and Coq. They range from practical analyses to proofs in computational complexity theory.

For practical application, Coq has been used to verify stack bounds for assembly code (Carbonneaux, Hoffmann, Ramananandro, & Shao, 2014) and to obtain WCET loop-bound estimation (Blazy, Maroneze, & Pichardie, 2013). Carbonneaux et al. (Carbonneaux, Hoffmann, Reps, & Shao, 2017) presented an automatic static analyzer for imperative programs, and although the analyzer itself is not verified, it generates bounds with machine-checkable certificates, to guarantee that the computed bound holds. For functional paradigm, McCarthy et al. (McCarthy, Fetscher, New, Feltey, & Findler, 2018) developed a Coq library, with a monad that counts abstract steps, which enabled running time analysis of programs written using the monad. An ICC-based characterization was introduced by Férée et al. (Férée, Hym, Mayero, Moyen, & Nowak, 2018), in the form of a Coq library, that allows for readily proving that a function is computable in polynomial time.

Coq has also been used to formalize some of the foundations of modern complexity theory. Ciaffaglione (Ciaffaglione, 2016) proved the undecidability of the halting problem. Guéneau et al. (Guéneau, Charguéraud, & Pottier, 2018) formalize the $\mathcal{O}$ notation. Forster

et al. (Forster, Kunze, & Wuttke, 2020) implemented a multi-tape to single-tape compiler, and introduced the first formalized Universal Turing Machine verified w.r.t. time and space complexity, for any model of computation, in any proof assistant. More recently, Gähler and Kunze formalized the Cook-Levin Theorem in Coq (Gähler & Kunze, 2021). Despite these advances, formalization of complexity is in early stages and basic complexity-theoretic results e.g., time and space hierarchy theorems, remain unavailable.

Our proposed project differs from these earlier results primarily in its intent. We plan to formalize the complexity analysis mechanism itself—not its computed result, as was done previously. In their work with the Turing Machines, Forster et al. (Forster, Kunze, & Wuttke, 2020) were explicit in emphasizing the challenge they experienced in formalizing complexity. We hypothesize that our ICC-based approach, with e.g., its built-in abstractions, will help reduce this challenge. It is our hope that CoqPL will welcome our proposal for a certified complexity analysis in Coq, and will be keen on indicating any library, tool or resource that could help.

4 Discussion

4.1 Observations About Specific Techniques

The specific techniques are the flow calculus of mwp-bounds and the QI Framework.

4.1.1 Advancements of the Flow Calculus

The original flow calculus of mwp-bounds (Jones & Kristiansen, 2009) provides a purely syntactic, sound and compositional theoretical technique for analyzing value growth of variables in imperative programs. Although promising, the technique poses various application challenges. In particular, issues arise from the nondeterministic inference rules and handling of potential derivation failure. The developments presented in this dissertation resolve or ease those tensions in several directions. At conclusion, we have arrived to an *enhanced* variant—i.e., the mwp^∞ calculus—with richer capabilities and utility than the original system.

From principles to practical efficiency. Two critical changes—in §7.1.3—were keeping track of derivation choices through *polynomial structures*, and introducing the flow coefficient ∞ that tracks derivation failure. These adjustments enables exploring all derivation paths concurrently, without needing to back-track during the analysis procedure. In the new formulation every input program is assigned precisely one *complex* mwp-matrix, instead of up to exponentially many *simple* mwp-matrices.

Although the technical adjustments make program analysis automatable, they also introduce a new problem. Critically, the *mwp-bounds*—that represent variable value growth and are the output of the analysis—become “hidden” in the complex matrix. It is necessary to invent ways to *interpret* the new complex data structure to mitigate the gap.

How to completely solve this new problem is described in a second “phase” of technical developments, in §3.1. The manuscript provides an evaluation procedure that enables recovering mwp-bounds from a complex mwp-matrix. With this enhancement, the MWP^∞ calculus computes mwp-bounds that are comparable to the original calculus, but in a *practically efficient* way.

The surpassing capabilities. Because the complex mwp-matrix contains information about *all* derivations,¹ the MWP^∞ calculus enables certain analysis features that are not possible in the original calculus.

For example, it is possible to determine the optimal mwp-bounds of each program variable. In case of derivation failure, it is possible to precisely identify the variable(s) involved, and the program point that induces the failure. It may also be possible to bound the value growth of certain variables in presence of whole-program derivation failure. This increases expressiveness, because it enables bounding more variables than what was previously possible (refer to §3.1.7 for comparison). A critical step to obtain these capabilities is projecting the complex mwp-matrices on individual variables (described in §3.1.4.3).

Extracting new information from programs. A side effect of the derivation-history tracking is that it enables extracting *new information* about the analyzed program. Although a complex mwp-matrix encapsulates the information, a clever strategy is required to extract it. A bit surprisingly, the evaluation procedure—developed in §3.1.4.2 to determine mwp-bounds—serves as a kind of “oracle” method. We can issue “queries” against it to answer a variety of questions about the analyzed program. For example, the following questions can be answered.

¹Essentially, the complex mwp-matrix encodes a derivation history.

1. Does a (particular) variable always fail the derivation?
2. What variables are the sources of derivation failure?
3. By what sequence of derivation choices is a program derivable (or not derivable)?
4. What is the optimal mwp-bound assigned to a (particular) variable?
5. What is the maximal flow coefficient between a pair of variables?
6. Is a pair of variables always independent w.r.t. data flow?
7. Does a derivation exist where multiple variables obtain optimal mwp-bounds?

All of these questions can be answered in the MWP^∞ calculus. In the original flow calculus, answering any of them is either not possible or would require exhaustive (infeasible) enumeration.

Enhanced insights about the calculus. The developments reveal deeper insights of the flow calculus. For example, we have now discovered that the 0-coefficients are interesting, because they track data flow independence between variables.²

A second insight concerns the w -coefficient. The original calculus describes it as iteration-independent, but the significance of that term remained mysterious until recently. We now know that, inside loops, the w -coefficients corresponds to a quasi-invariance property. During iteration, it will (at some point) become unaffected by increase in loop iteration count (cf. §3.1.5.4).

Finally, we also know that focusing the analysis to loops—instead of functions as in the original design—is beneficial to obtain practical analysis. In some cases, a loop-scoped

²Refer to §1.2.4.3 and §1.2.5.4 on the relevance of this property.

analysis enables extracting results where function-based analysis fails (cf. §3.1.7). This reinforces the view that it is justifiable to focus complexity analyses on loop constructs (Ben-Amram & Hamilton, 2020).

Resolution of related research questions. In response to the research questions posed in §1.1.4.1, we can now confirm a positive answer to (RQ1). It is possible to obtain automatic program analysis from the flow calculus of mwp-bounds. However, the result is not immediate, and substantial adjustment was required to the underlying theory. The enhanced technique is applicable to automatic resource analysis of imperative programs, as demonstrated by pymwp in §2.1. We can also confirm the analysis results a complementary to alternative state-of-the-art techniques (Aubert, Rubiano, Rusch, & Seiller, 2023b, p. 5).

Relating to (RQ3), a possible application of the flow calculus is to inference specification conditions (cf. §3.1) in numerical loops. Such use case is unconventional for complexity-based analyses, but it should be explored more because the findings of §3.1 are encouraging. Currently, the mwp^∞ calculus can derive results for many kinds of loops, including several natural algorithms (cf. §3.1.7).

Answering (RQ2)—a formal proof of soundness—still remains an outstanding task (cf. §3.3). However, the advancements in other areas are important to motivate the formalization effort. There is now strong evidence to justify the relevance and need for formal verification.

4.1.2 Extending the QI Framework

In a second research direction, we applied the QI-framework to two new program analysis applications: distributing loops for parallelization and tracking the security property of non-interference.

4.1.2.1 Loop Distributing Program Optimization

In “Distributing and Parallelizing Non-canonical Loops” (§2.2), we used the QI framework to design a program optimization that improves runtime execution profile of programs. The optimization seeks to increase parallelization potential in initially sequential imperative loops. The analysis evaluates *dependence* between commands. It aims to discover data flow *independent* sub-computations. If such independence exists, we replace the original loop with multiple new loops. The new loops can be distributed between the available processors for parallel execution. The analysis is practically useful as a compiler optimization technique.

The benefits of ICC. One uncommon aspect of the analysis is that it enables reasoning about loops whose iteration space is unknown. Such loops are beyond the capabilities of traditional parallelizing transformations. However, many ICC systems—like the QI framework and flow calculus of mwp-bounds—over-approximate iteration space. By assuming iteration is *unbounded*, a loop that does terminate is still within the over-approximation. This way, the guarantees apply to all loop iteration spaces.

The cost of flexibility is reduced precision. However, sometimes the trade is beneficial. For the parallelizing optimization, the property of interest is data flow independence in and between commands. The focus is binary – either a data flow exists or does not exist. Beyond existence, *recurrences* of data flows are irrelevant. This enables relaxing the need to know precise loop iteration space.³

³Though, determining whether loop optimization is *beneficial* requires knowledge of the loop iterations.

Moving beyond complexity. Although the goal of the analysis is to improve running time, the optimization is no longer defined in terms of complexity classes⁴ The application thus moves the technique beyond complexity theory. The original complexity result is lost, but in return, we gain a practical program optimization technique.

Resolution of related research questions. The loop distribution technique addresses (RQ4) from §1.1.4.1. Aligned with the main hypothesis, it demonstrates a new application of ICC systems. The work is complete, but follow-up directions are discussed in §4.3.

4.1.2.2 Detecting Non-Interference

In subsequent work (§3.2), we explored an application of the QI framework to track non-interference. Non-interference is a security property to establish that secret information does not “leak” to lower security classes (cf. §1.2.5). Although this property *seems* different from parallelizing loops, the two applications share a common origin. Both analyses consider data flow independence. This makes the QI framework suitable for both analyses.

Derived insights. Our starting point was the earlier loop parallelization formulation. One surprising discovery was that we were able to relax that mathematical constraints that were needed previously (refer to paragraph 3.2.3.3.2). For example, the order of command composition was no longer important. For a loop transformation, the order is important to preserve functional behavior. However, the read-only non-interference analysis is composition-insensitive, because it aims is to detect an information flow policy violation *anywhere* in the program. Once such leak is detected, it is not possible to erase it.

⁴This is different from the prior use of the QI framework (Moyen, Rubiano, & Seiller, 2017a) that aimed to alter the program’s overall complexity by modifying nested loops.

The non-interference analysis allows us to evaluate generalizability of ICC systems. Our long-held assumption is that, because the QI framework targets a core imperative language, it can be applied to any language representation that shares that same core. The non-interference analysis allows us to challenge this assumption. Our plan is map our non-interference logic to two different language representations, at different stages of compilation, to study preservation of the non-interference property.

Resolution of related research questions. Our investigation into (RQ5) from §1.1.4.1 is still ongoing. Completing the work requires addressing certain theoretical questions (e.g., refining the definition on anytime non-interference (§3.2.4) and precisely identifying its ideal application. Based on the prior success with the QI framework, we are confident that the outstanding steps are achievable.

4.1.2.3 New Insights About the QI Framework

Our investigations of the QI framework provide a case example showing that ICC systems can be flexible enough to track various non-functional program properties. Although the research problems are phrased in terms of ‘properties’, the actual insight comes from better understanding the underlying theory. For example, to track a complexity property, we must compute fixed points and preserve the program command order in transformations. In the loop distribution work and the non-interference analysis, it was possible to gradually relax these requirements.

Although the QI framework appears similar to the flow calculus, there are notable differences. Critically, the QI framework enables tracking information in control statements. The programming language also includes arrays. These capabilities are critical to support the adjusted analyses studied in this dissertation.

Generalizing further, our investigations have revealed that the QI framework is suitable for tracking program properties that can be modeled as *instances of data dependence* (or independence). This is reminiscent of the Dependency Core Calculus (Abadi, Banerjee, Heintze, & Riecke, 1999). Based on this observation, the QI framework likely encapsulates even richer utility than uncovered thus far.

4.2 Broader Research Findings

Moving beyond specific techniques, our investigation revealed observations about automatic resource analysis and ICC, and the challenges that emerge from applying implicit computational complexity.

4.2.1 Utilities in Automatic Resource Analysis

In the most rudimentary form, ICC-based program analyses evaluate a binary membership problem. The task is to decide whether an input program belongs to a targeted complexity class. Like in safe recursion (§ 1.2.1.3), no explicit bound needs to be provided (Moyen, 2017, p. 13).

Although such binary result is coarse in the amount of information it provides, it does have applications. For example, Karp (Zhang, Hartline, & Dimoulas, 2022) is a domain-specific language (DSL) for programming and testing Karp reductions. A Karp reduction must be polynomial time, but no explicit bounds are required. For applications in cryptography, the situation is reversed. A critical step is showing that secret communications have desired properties despite of the best efforts of an untrusted party (Rivest, 1990). A guarantee of secret communication then requires showing that no feasible (polynomial-time) algorithm

exists for breaking the security mechanism. Yet, these are isolated use cases; a coarse binary analysis has limited utility in general.

The practical relevance of an analysis increases if we can show it can support software engineering or verification tasks. Such analyses produce quantitative information that helps engineers understand the program behavior, and detect and repair issues early. Ideally, an analysis should express resource bounds and flag resource-intensive procedures. Certain ICC systems—like the flow calculus of mwp-bounds—can target such the practically-oriented applications up to an extent. However, if some computation exceeds polynomial bounds, the analysis should still yield an informative result. This last requirement is often beyond capabilities of ICC systems (Baillot, Dal Lago, & Moyen, 2012).

On the analysis input side, the utility improves with expressive power. Essentially, we would want the analyzer to handle the rich constructs that occur in real-world programs. ICC systems are typically defined on core languages (“toy” languages). However, it is possible to work around this limitation by focusing on interesting sub-classes of programs, e.g., integer programs or numerical loops. Through compositional analysis of program *fragments*, ICC systems extend to analysis of general programming languages. This approach is demonstrated in pymwp (§2.1). Having a restricted syntax is thus not an insurmountable hurdle to applications, in our experience. To make an analysis comparable with many existing resource analyzers (Montoya, 2017c), it suffices to target the C Integer Programs grammar (Termination Portal, 2015) used in the Termination Competition (Giesl, Rubio, Sternagel, Waldmann, & Yamada, 2019).

4.2.2 Motivating Analyzer Developments

A second line of observations concerns the automatic resources analyzers. There are various challenges with locating, reusing, and running experiments with the analyzers.

First, resource analyzers differ greatly in input/output formats (cf. § 1.2.2.4). For example, the ones that are based on cost equation systems are different from the analyzers of C programs. This is problematic because it is difficult to faithfully compare new techniques with the existing ones (on the same workloads), or replace an analyzer with an alternative. Although some concerted efforts—like the Termination Competition—push to establish a shared I/O format, there is no widely adopted standard.

Since resource analyzers are primarily developed as “standalone” tools, their continued maintenance and development is not strongly incentivized. Certain analyzers, although they regularly appear in scientific literature, are inaccessible (Sinn, Zuleger, & Veith, 2017), difficult to locate (Carbonneaux, Hoffmann, & Shao, 2015), or deprecated (Gulwani, Mehra, & Chilimbi, 2009; Srikanth, Sahin, & Harris, 2017).

These issues extend beyond resource analysis. Scientific software is general differs from “mainstream” software (Hannay, MacLeod, Singer, Langtangen, Pfahl, & Wilson, 2009; Joppa et al., 2013). For example, scientific software aims to materialize ideas that correspond to publishable research results. After presentation, the software is archived (ACM, 2020), and authors move on to the next prototype. This pattern does not encourage solving the issues identified earlier.

One community, that operates differently, is formal methods (see as evidence Table 8). Many formal verification tools are developed with continuity in mind (Beyer, 2022; TPTP.org, 2025). Moreover, the SMT-LIB Standard (Barrett, Clark and Fontaine, Pascal and Tinelli, Cesare, 2025) is a model for promoting the adoption of common languages interface. It would be beneficial for resource analyzers to follow similar practices.

4.2.3 The *Implicit Costs* of Applied ICC

The dissertation manuscripts follow a pattern of pairing ICC with a secondary research domain. While such intersectional strategy comes with rich application potential, a substantial challenge involved identifying suitable research domains where ICC techniques could offer meaningful benefit. There are thus added “costs” with this research strategy, as suggested in §1.1.2.

One cost that became particularly pronounced was the learning overhead. This is reflected in §1.2. Making a meaningful contribution requires first having sufficient familiarity with the secondary domain to identify research gaps. It also requires learning the terminology and implicit assumptions of the secondary domain. For example, in presenting developments of the flow calculus, we learned through trial-and-error that experiments were necessary to support our findings.

The second cost is about understanding different measures of significance. While we might hold the theoretical origins in high esteem, it becomes minutiae when targeting applications in other domains. Using an ICC technique to solve a problem in ways that is comparable to existing techniques is not enough for a scientific contribution. The significance of the application must be standalone and independent of the theoretical origin. In other words, a technique becomes interesting only after we can show that it can solve an interesting problem.

This described research strategy has repeated a few times throughout the dissertation, thus multiplying these costs. Following the strategy requires persistence, and due to the learning overhead, is quite challenging for dissertation research.

4.3 Open Problems for Future Work

4.3.1 Questions About the Flow Calculus

The following questions remain unanswered about the flow calculus of mwp-bounds.

- *Have the developments changed the complexity of the derivability problem?*⁵ The manuscripts in §7.1, §2.1, and §3.1 alter the inference procedure, therefore the answer is not obvious.
- *How to obtain concrete bounds from mwp-bounds?* The honest polynomials of mwp-bounds are composed of variables, constants, and operators, but how to extract the exact combination is unclear.
- *Could the flow-calculus also capture lower bounds?* A lower bounds analysis would complement the existing capabilities by detecting variables that have certainly exponential value growth, which is useful e.g., for bug finding.
- *Issues around translating real-world programming languages to the imperative language of the flow calculus.* Although the flow calculus inference rules are phrased as *while* and *loop* commands, they should be interpreted as *unbounded* and *bounded* loops. An ideal translation should compile the input program to the language of the flow calculus, while accounting for loop boundedness. A solution might be already available in the compilers literature.
- *How to account for various rich program constructs, like arrays?* Looping over arrays is a frequently occurring programming pattern. Although this is a natural extension to the flow calculus, there is no existing approach to this problem.

⁵The problem is NP-complete by (Jones & Kristiansen, 2009, p. 40)).

As evidenced in these open programs, the flow calculus of mwp-bounds is interesting because it provides an extensible and rich logical framework for reasoning. Its full potential still remains to be explored.

4.3.2 Other Emerging Research Questions

Generalizing the loop distribution. The solution of §2.2 assumes a setting where loop distribution should be applied. Relaxing this assumption raises at least two new important questions relating to the loop transformation strategy. First, loop distribution and parallelization introduce computational overhead. The overhead comes from replicating loop headers across distributed loops and instrumenting parallel runtimes. It is therefore necessary to first determine the profitability of the operation. Only if the operation is beneficial by some metric, should we “accept” the overhead of loop distribution. Second, compiler loop transformations do not occur in isolation. We must consider combinations of loop transformations—like fusion, tiling, and interchange, etc.—and their combined effect on the input program. Investigating these questions is a natural extension of the prior work.

Complexity-by-Construction. The X-by-Construction paradigm (ter Beek, Cleophas, Schaefer, & Watson, 2018) is concerned with ensuring non-functional properties by construction. The XbC aim is to automatically generate error-free software from specifications (ter Beek, Cleophas, Legay, Schaefer, & Watson, 2020). ICC systems define the programming language-based mechanics to construct polytime programs, therefore it is conceivable that ICC systems could extend the XbC approach in cases where X stands for complexity. It would require pairing ICC techniques with program synthesis. Conceptually, this is the “bottom-up” dual of the “top-down” approach implemented in pymwp.

Formally verifying complexity. ICC systems are based on programming languages, making them naturally suited to formalization. However, the number of works that support this claim is still modest—refer to e.g., (Atkey, 2024; Férée, Hym, Mayero, Moyen, & Nowak, 2018; Heraud & Nowak, 2011). The investigation is necessary to have formal guarantees of complexity. There are two separate questions (i) the formalization of logical systems that provide guarantees, and (ii) formalizing analyzers that provide complexity guarantees. The existing works address the first question. The closes solution to the second is giving guarantees to the results computed by an analyzer (Carbonneaux, Hoffmann, Reys, & Shao, 2017).

Extending beyond traditional programming paradigms. ICC has been widely studied in the context of “traditional” programming paradigms, including imperative and functional programs.⁶ However, there are many more important programming paradigms, like probabilistic programs and quantum computing, that warrant investigation. These paradigms change the notion of programming languages and thus require different analytical approaches. Representative works exploring this direction include (Avanzini, Moser, Péchoux, & Perdrix, 2024; Avanzini, Moser, & Schaper, 2020; Colledan & Dal Lago, 2024; Dal Lago, Masini, & Zorzi, 2010).

4.4 Completion of Dissertation Goals

As part of the dissertation aims, Section §1.1.3 outlined four goals. We can now assess the completion status of the goals.

⁶For example, the works building on Resource Aware ML (Hoffmann, Aehlig, & Hofmann, 2012) have been extended to many paradigms.

✓ **Goal 1.** *Extend applied capabilities in automatic program analysis and verification.* The works extending the flow calculus of mwp-bounds—in Sections §7.1, §2.1, and §3.1—and the loop distribution technique of §2.2, immediately support the goal. Each manuscript takes lifts implicit computational complexity beyond its theoretical origins, and demonstrates its uses in an applied context. The complementary nature of these techniques is discussed in §2.1.2.3 (see in particular Table 10), §2.2.4 and §3.1.6. Therefore, the goal was met satisfactorily.

▣ **Goal 2.** *Take ICC techniques closer to integration with real-world software development workflows.* The dissertation work is consistently supported by software artifacts (cf. §8). While the artifacts demonstrate the practical relevance of the investigations, and have driven advancements in the underlying theories, the results so far are still a step in “isolation”. Integrating techniques based on implicit computational complexity into other software development tools (compilers, verifiers, etc.) is important to promote relevance and continue advancement of ICC. Such ambitions encourage thinking about ICC from new (more practical) perspectives. However, concrete integration remains as an outstanding goal. The work in using the flow calculus to infer specification conditions (Rusch, 2025a) is one step in that direction.

▣ **Goal 3.** *Initiate conversations about the relevance of ICC applications.* The third goal is “internal” to those who are already familiar with ICC. The dissertation introduction (i.e., “Addressed Problem” in §1.1.2), and the introduction to ICC (“A Snapshot of Theoretical Results” in §1.2.1.4), both highlighted the strong preference for pure theoretical development. Taking inspiration from (Moyen, 2017), the motivation is to push theoretical developments beyond this pure framing. There was some progress in initiating conversations about applications—via (Aubert, Rubiano, Rusch, & Seiller, 2022a) and a presentation at the Seminar on Semantic and Formal Approaches to Complexity (Rusch, 2023)—but the overall effort toward the goal was limited. Al-

though this outcome is less than ideal, the motivations presented in the published works, and this dissertation, will remain discoverable to others in perpetuity.

- ✓ **Goal 4.** *Expose ideas from implicit computational complexity to broader research communities.* The dissemination of the dissertation work involved many written works and giving presentations to diverse audiences. The “themes” of the presentations venues included type theory (the TYPES’22 conference (Aubert, Rubiano, Rusch, & Seiller, 2022d)), interactive theorem proving (CoqPL’23 (Aubert, Rubiano, Rusch, & Seiller, 2023c)), language-based security (PLAS’24 (Aubert & Rusch, 2024)), and programming languages (at the doctoral symposiums of SPLASH’22 (Rusch, 2022) and ECOOP’25 (Rusch, 2025b)), to name a few. The number of attendees at these events was in the magnitude of hundreds. Based on observation, implicit computational complexity was largely an unfamiliar topic to the audiences. Conversely, through the different modes of dissemination, a substantial effort was made to increase exposure and awareness of implicit computational complexity. Therefore, the final goal was satisfactorily met.

Although the goals were completed with a mixed degree of success, there was progress in every direction. The goals are intentionally open-ended. They need not be fully achieved within the timespan of doctoral studies. It is always possible to push further, even in the cases that were completed satisfactorily.

4.5 Research Conclusions

We can now summarize the key takeaways of this dissertation – in Fig. 36.

The first finding recognizes the fact that Implicit computational complexity offers complementary techniques to perform automatic resource analysis (cf. §7.1 and §2.1). However,

*** Primary findings.**

1. Implicit computational complexit (ICC) provides complementary techniques of automatic resource analysis.
2. ICC systems are flexible and can be modified to track other non-functional properties, beyond complexity.
3. Although this dissertation has provided early evidence, continued exploration is needed to unlock further application potential.

Figure 36: Primary research findings.

such applications may require substantial adjustment to the base theory. While designing ICC systems, it is therefore important to consider practical challenges ahead of time.

The second finding emphasizes *extensibility*. Starting with an ICC system, it is possible to adjust it to track alternate properties. The value of such exploration is two-fold. Due to the unusual starting point, the obtained analysis may be complementary to existing techniques (as in §2.2). Moreover, the adjustment provides new insights of the technique itself.

The third finding is reflective. The dissertation investigation has explored many applications of ICC. However, investigation has been far from exhaustive and many more applications remain to be discovered. In particular, the guarantees ICC can offer by construction should be investigated further. Yet, such exploration necessitates a viewpoint shift and seeing ICC as more than “just” a complementary approach to complexity theory. When viewed broadly, ICC systems become rich reasoning frameworks that allow analyzing and verifying wide varieties of program properties.

5 Summary

5.1 Applied Implicit Computational Complexity:

Extended Abstract

Neea Rusch

Symposium paper.

The summary was presented at the Doctoral Symposium of the European Conference on Object-Oriented Programming (ECOOP) 2025.

5.1.1 Seeking Strong Software Quality Guarantees

The gold standard in formal verification is showing that an implementation of a program is totally correct. Total correctness requires proving two conditions about the program: that the program meets its specification, and that the computation terminates (Leino, 2023). The former ensures the program computes the expected output for all inputs. The latter ensures that the computed result will become observable eventually. Thus, a proof of total correctness gives us strong assurance that the program behaves expectedly at runtime, i.e., when the program is executed.

An additional consideration in reasoning about program correctness concerns non-functional properties. Non-functional properties characterize the program's quality features like execution latency, power consumption, and information flow security (ter Beek, Cleophas, Schaefer, & Watson, 2018). Pure functional correctness is *practically unsatisfactory* if a program requires exorbitant time to compute its result; aggressively drains a device battery, or reveals secret information (Aubert, Rubiano, Rusch, & Seiller, 2022a; Heraud & Nowak, 2011). Engineering truly high-quality programs thus requires additional techniques that enable us to formally verify non-functional properties. Unfortunately, analyzing non-functional properties is non-trivial and generally undecidable (Rice, 1953). Developing specification formalisms and reasoning methods, that can provide some guarantees despite the undecidability, is an active area of research (Kaminski, 2025).

The field of *Implicit Computational Complexity (ICC)* (Dal Lago, 2011) offers a conceivable pathway toward formal verification of certain non-functional properties. ICC is complementary to the classic complexity theory because it uses programming languages as the mechanism to reason about runtime behaviors. Broadly, complexity theory is concerned with resource requirements of computations (Goldreich, 2008). Those requirements are organized in a hierarchy of complexity classes. The classes describe resource requirements

w.r.t. an abstract (mathematical) machine model. The classes correspond roughly to resources like running time and memory consumption of programs that are executed on real-world computers. To streamline this reasoning, *implicit* computational complexity aims to discover *machine-independent* characterizations of the complexity classes. ICC systems are designed by introducing a *restriction*, at the level of a programming language, that guarantees a complexity property. Then, every program satisfying the restriction is known to belong to a particular complexity class. This in turn provides a guarantee about the program’s runtime resource consumption.

Exchanging machine models for syntactic criteria is powerful in several ways. For example, it facilitates formal verification of programs. ICC enables defining syntactical specifications of resource consumption, and developing fully automatable analyses to verify whether a program satisfies such a specification (Heraud & Nowak, 2011). ICC systems have been designed for numerous combinations of programming languages (imperative, lambda-calculus, function algebras, quantum, term rewrite systems, π -calculus, ...), restriction-techniques (linear logic, data flow analysis, type systems, category theory, ...), and complexity classes (P, PSPACE, L, PP, ...) (Moyen, 2017; Péchoux, 2020).

5.1.2 Addressed Problem and Goals

Besides uses in formal verification, there are many good motivations for implicit reasoning about computational complexity. ICC allows guaranteeing program properties *by construction*, before any program exists. ICC drives better understanding of complexity classes and yields more natural definitions and proofs of central results than classical complexity theory (Kristiansen, 2017). For example, ICC systems show that complexity classes are intrinsic mathematical entities that do not depend on a particular machine model. Multiple ICC systems, e.g., (Jones & Kristiansen, 2009; Marion, 2011), are rich enough to express natu-

ral algorithms. Since the reasoning techniques differ from the existing alternatives, the results ICC systems produce are often orthogonal (Aubert, Rubiano, Rusch, & Seiller, 2022a). Finally, many ICC systems—e.g., (Atkey, 2024; Hainry & Péchoux, 2023; Jones & Kristiansen, 2009; Marion, 2011)—are *compositional*. This means they use modular techniques that enable bottom-up reasoning about subprograms. Compositionality is critical for scalability (Carbonneaux, Hoffmann, & Shao, 2015) and useful for proving soundness (Keidel, 2021), i.e., that the analysis technique produces correct results. However, compositionality is missed by many prominent program analysis techniques (Carbonneaux, Hoffmann, & Shao, 2015; Schiebel, Sattler, Schubert, Apel, & Bodden, 2024).

Despite this rich inventory of compelling features, ICC has remained largely a theoretical novelty, with a few exceptions (Avanzini & Dal Lago, 2017; Avanzini, Moser, & Schnabl, 2008; Férée, Hym, Mayero, Moyen, & Nowak, 2018; Hainry, Jeandel, Péchoux, & Zeyen, 2021; Hoffmann, Aehlig, & Hofmann, 2012; Moyen, Rubiano, & Seiller, 2017a). A theory in absence of applications means the power, limitations, and utilities of the theory are difficult to recognize. In our view (Bishop, 2019, p. xxxv) (Moyen, 2017, p. 75), theory and application are symbiotic. Investigations of both are needed for scientific advancement.

The embedded manifesto of the dissertation research is that *applications are necessary* to push forward our collective understanding of implicit computational complexity. It is conceivable that potentially great power is locked inside the theoretical ICC systems. It remains an open problem to release that power. Within the ICC community, more investigations are needed to explore the capabilities of the existing systems. Externally, the ICC-based program analyses should be exposed to broader research communities. These steps would motivate enhancements in ICC systems and facilitate discovery of new applications.

The work in the dissertation is only a small step toward these ambitious long-term goals. The realistic expectations of the dissertation are about technical and social advancement. Overall, there are four high-level goals.

1. Extend applied capabilities of ICC in automatic program analysis and verification.
2. Take ICC a few steps closer to integration into real-world software development workflows.
3. Initiate conversations on the relevance of ICC applications.
4. Expose ICC to broader research communities.

A guiding intuition behind these goals is that, if applied, implicit computational complexity could provide us several new practical program analyses. The use cases of such analysis include formal verification of many non-functional properties.

5.1.3 Research Questions and Approaches

The following *main hypothesis* guides the dissertation investigation.

Implicit computational complexity offers applied utilities when lifted from the theoretical domain.

Since the hypothesis is broad, the research projects that challenge it must be more refined. We focus on two paths. The first applies ICC in automatic analysis of *resource consumption*. This application is natural because it parallels the theoretical use of ICC. The second path takes the investigation to a more unconventional direction. It asks whether we can adjust ICC systems to track *other* program properties, beyond resource consumption. Stated differently, by swapping the property of interest, it allows us to assess the technical flexibility of ICC systems. To summarize, we investigate whether we can (i) apply the systems in the originally designed ways, and (ii) discover unexpected applications.

5.1.3.1 Automatic Resource Analysis

This series of work focuses on an ICC system called the *flow calculus of mwp-bounds* (Jones & Kristiansen, 2009). The flow calculus is a canonical example of a theoretical ICC system. It focuses on tracking change in data size during computation. For imperative programs, it aims to determine if it is possible to bound the growth of variable values by polynomials in inputs. The choice to focus on this particular ICC system is based on intuition. As a syntactical and compositional data-flow analysis, it seems like a good candidate for automatic resource analysis.

Several research questions arise from the flow calculus.

- (RQ1) Can we develop an *automatic* program analysis based on the theory?
- (RQ2) Given its paper proofs, is the theory correct, i.e., can we prove formally the soundness of the flow calculus?
- (RQ3) Assuming the theory can be automated, what are the use cases of such analysis?

These questions are still conventional because they target resource analysis. In other words, given a complexity-theoretic technique, we should reasonably expect that it can be applied in the designed way. If successful, the investigation should minimally produce two research artifacts—a program analyzers and mechanized proof—corresponding to RQ1 and RQ2. Additionally, it should produce new insights of the flow calculus and its possible applications. Since ICC is not the only technique for analyzing growth in data size, having an automatic analyzer would enable comparing it to the alternative approaches.

5.1.3.2 Analyzing Extended Properties

The second series of work applies implicit computational complexity to track other non-functional properties. The idea is to select an ICC system and adjust it to capture a different non-functional property. The motivation is multifaceted. It requires developing deep understanding of the mechanics of the chosen technique. Conceptually, it pushes to think about ICC outside the usual frame of computational complexity. Lastly, investigations in this direction are important to expose ICC to broader research communities, since the application is shifted away from (or beyond) resource consumption.

In this series of work, the starting point is an ICC system based on quasi-invariants (QI), as formulated in (Moyen, Rubiano, & Seiller, 2017a). Similar to the flow calculus, the QI framework is a data-flow analysis of imperative programs. However, the QI framework supports a richer input language and is founded on a seemingly adjustable mathematical framework. In the prior formulation (Moyen, Rubiano, & Seiller, 2017a), it was used to obtain a compile-time program optimization. The optimization lifts nested loops outside their containing loop, such that post-transformation the program complexity is reduced.

We ask the following questions about the extended uses of the QI framework.

(RQ4) How to develop a program transformation to increase parallelization potential?

(RQ5) How to use it to analyze security properties, specifically non-interference?

Research questions 4 and 5 are posed here with an air of obviousness. However, arriving to those questions was not obvious. A substantial hidden challenge of the dissertation research plan involves identifying suitable research domains—like parallel programming or language-based security (Sabelfeld & Myers, 2003)—where ICC-based techniques could offer meaningful benefit.

Another consideration is that adjusting an ICC system may lead to losing the original guarantee about resource consumption. In return, we gain an alternative guarantee about program behavior. Whether this trade is beneficial depends on the application. Therefore, besides tracking properties, this investigation is also about studying the mechanics of ICC systems, and uncovering how exactly they provide behavioral guarantees.

Finally, this direction of research contains a bonus challenge. Simply showing that an ICC-based technique can solve a problem in another application domain is not enough. It is necessary to show that the solution is *independently* relevant in the target domain, regardless of the origins of the solution. Based on experience, this criterion is strongly enforced by the scientific peer review process.

5.1.4 Methodology and Research Deliverables

Following the approaches of §5.1.3, we conducted a series of investigations. These resulted in several publications (Aubert, Rubiano, Rusch, & Seiller, 2022a, 2023b; Aubert, Rubiano, Rusch, & Seiller, 2023e), workshop presentations (Aubert, Rubiano, Rusch, & Seiller, 2022d, 2023c; Rusch, 2022), and ongoing work. We can summarize the main contributions as follows.

For RQ1, we enhanced the theory to develop an automatic resource analysis in (Aubert, Rubiano, Rusch, & Seiller, 2022a, 2023b; Rusch, 2025a). Due to inherent nondeterminism, developing an efficient and useful practical analysis was surprisingly difficult, but eventually achievable. The enhanced flow calculus can answer the same questions as the original formulation, but automatically and in practice. Moreover, through the enhancements we discovered how to make the calculus more expressive, allowing it to cover a larger class of programs than the original theory (Rusch, 2025a). The flow calculus is implemented

as an open source static analyzer, pymwp^{1,2} for analyzing a subset of C programs (Aubert, Rubiano, Rusch, & Seiller, 2023b). Based on this work, we discovered that the analysis can support inference of specification conditions (Rusch, 2025a) that are needed for formal verification. This finding corresponds to RQ3. Proving the soundness of the flow calculus (RQ2) is still ongoing work (Aubert, Rubiano, Rusch, & Seiller, 2023e). However, given the other developments, a formalization effort is now even more strongly motivated and justified.

Along the second direction, for RQ4, we applied the QI framework to develop a program optimization for loop parallelization (Aubert, Rubiano, Rusch, & Seiller, 2023e). This is useful, because the optimization can be applied to loops whose iteration count is unknown. Parallelizing such loops is outside the capabilities of prior techniques. In an ongoing investigation, for RQ5, we apply the QI framework to tracking programs non-interference (Goguen & Meseguer, 1982), a kind of confidentiality property. Non-interference ensures a program does not reveal secrets; and if it does, we can identify where the violation occurs. Our working intuition is that, since the analysis can be adjusted to different programming languages, it enables tracking *preservation* of the non-interference property at different compilation stages, while a program is transformed from source code to an executable. This is different from many existing techniques that analyze programs only at one stage of representation.

5.1.5 Research Findings and Conclusions

On automatic resource analysis. We can now confirm that it is possible to obtain automatic program analysis from the flow calculus of mwp-bounds, but only after substantial

¹Pronounced *pai em double-you pē*.

²<https://statycc.github.io/pymwp/>

adjustments. An efficient program analysis required inventing ways to manage costly sub-computations and solving additional problems that were exposed in the process. At conclusion, the enhanced technique is not just automatable, but can analyze more programs than the original theory. We can now confirm the analysis results are complementary to alternative state-of-the-art techniques (Aubert, Rubiano, Rusch, & Seiller, 2023b, p. 5). Thus, it enables us to analyze different questions about resource consumption. The implemented flow calculus can also be used in specification inference. There are potentially many more applications of the flow calculus remaining to be discovered.

On analysis of extended program properties. The QI framework provides a case example showing that ICC systems can be flexible enough to track other program properties. While adjusting the system, we made a surprising discovery in that changing the property of interest simplified the mathematical analysis. For example, to track a complexity property, we must compute fixed points and preserve the program command order. In the adjusted analysis, it was possible to relax these conditions. More generally, that the QI framework can be used to track various properties is reminiscent of the Dependency Core Calculus (DCC) (Abadi, Banerjee, Heintze, & Riecke, 1999). In DCC, different program properties are modeled as *instances* of a central notion of *dependency*. This mirrors our observations about the QI framework. This suggests that the QI framework encapsulates even wider utility than uncovered so far during this dissertation work.

On broader dissertation goals. Zooming out of the specific projects, we can reflect on the main hypothesis and the research goals. We have successfully gathered evidence of the application potential of ICC. The evidence supports the main hypothesis, but is still too limited to draw generalizing conclusions. However, our experiences of developing the ICC systems strongly reinforced the symbiotic view of theory and application. In the process, we faced many scientific and engineering challenges, and identified compelling motivations to

justify the investigations. At conclusion, we have strengthened the theories of interest and clarified their practical relevance.

In §5.1.2, we crafted four goals to define the dissertation expectations. The first was about extending the applied capabilities of ICC, which was met successfully. The second was about introducing ICC in development workflows. Although we have developed a static analyzer, it is a step in “isolation.” Integrating ICC analyses into other software development tools (compilers, verifiers, etc.) remains as an outstanding task. Such integration is important to promote relevance and continue development of ICC techniques. Our work in using the flow calculus to infer specification conditions (Rusch, 2025a) is a first step in that direction. The last two goals were social and about disseminating ideas across research communities. Within the ICC community, our efforts materialized in seminar talks and informal conversations. Although this record is below ideal, the ideas in the published works will remain discoverable to others in written form, asynchronously and in perpetuity, like (Moyen, 2017). In regard to other communities, we were successful at exposing the work at verification, programming languages, and security venues. Thus, the fourth goal was satisfactorily met.

Final thoughts & future perspectives. The ICC approach of guaranteeing program properties demonstrates great applied potential. For example, it offers a pathway toward formal verification of many non-functional properties. One research direction, that deserves more attention, is applying ICC systems to ensure properties by construction. This would provide guarantees before any program exists and eliminates the need for *aposteriori* analysis. Although the dissertation research did not extend that far, exploring this direction is an intriguing future goal. Whether it is attainable depends crucially on a viewpoint shift. We should regard *implicit computational complexity* in the broader context it offers—including as a rich toolbox of techniques for static program analysis—and continue the exploration of

its applied potential.

6 References

- Aaronson, S., Kuperberg, G., & Habryka, O. (2025). Complexity Zoo [Accessed: 2025-06-25]. https://complexityzoo.net/Complexity_Zoo
- Abadi, M., Banerjee, A., Heintze, N., & Riecke, J. G. (1999). A core calculus of dependency. *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 147–160. <https://doi.org/10.1145/292540.292555>
- Abel, A., & Altenkirch, T. (2002). A predicative analysis of structural recursion. *Journal of Functional Programming*, 12(1), 1–41. <https://doi.org/10.1017/S0956796801004191>
- Abu-Sufah, Kuck, & Lawrie. (1981). On the Performance Enhancement of Paging Systems Through Program Analysis and Transformations. *IEEE Transactions on Computers*, C-30(5), 341–356. <https://doi.org/10.1109/TC.1981.1675792>
- Aceto, L., Gorla, D., & Lybech, S. (2024). A Sound Type System for Secure Currency Flow. *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, 313, 1:1–1:27. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.1>
- ACM. (2020, August). *Artifact Review and Badging - Current* (Artifact Review and Badging Version 1.1 - August 24, 2020) [Accessed: 2025-03-22]. ACM. <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- Affeldt, R. (2025, January). *An Introduction to MathComp-Analysis* (Version 9 - January 13, 2025) [Accessed: 2025-05-07]. Lecture notes. <https://staff.aist.go.jp/reynald.affeldt/documents/karate-coq.pdf>
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006, August). *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers Principles, Techniques & Tools (2nd Edition)*. Pearson Education.
- Alagarsamy, S., Tantithamthavorn, C., & Aleti, A. (2024). A3Test: Assertion-Augmented Automated Test case generation. *Information and Software Technology*, 176, 107565. <https://doi.org/10.1016/j.infsof.2024.107565>
- Albert, E., Arenas, P., Genaim, S., & Puebla, G. (2008). Automatic Inference of Upper Bounds for Recurrence Relations in Cost Analysis. In *Static Analysis* (pp. 221–237). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-69166-2_15
- Albert, E., Arenas, P., Genaim, S., & Puebla, G. (2010). Closed-Form Upper Bounds in Static Cost Analysis. *Journal of Automated Reasoning*, 46(2), 161–203. <https://doi.org/10.1007/s10817-010-9174-1>
- Albert, E., Arenas, P., Genaim, S., Puebla, G., & Zanardini, D. (2007). COSTA: Design and Implementation of a Cost and Termination Analyzer for Java Bytecode. In F. S. de Boer, M. M. Bonsangue, S. Graf, & W. P. de Roever (Eds.), *Formal Methods for Components and Objects* (pp. 113–132, Vol. 5382). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-92188-2_5

- Albert, E., Arenas, P., Genaim, S., Puebla, G., & Zanardini, D. (2012). Cost analysis of object-oriented bytecode programs. *Theoretical Computer Science*, 413(1), 142–159. <https://doi.org/10.1016/j.tcs.2011.07.009>
- Albert, E., Bofill, M., Borralleras, C., Martin-Martin, E., & Rubio, A. (2019). Resource Analysis driven by (Conditional) Termination Proofs. *Theory and Practice of Logic Programming*, 19(5–6), 722–739. <https://doi.org/10.1017/s1471068419000152>
- Alias, C., Darté, A., Feautrier, P., & Gonnord, L. (2010). Multi-dimensional Rankings, Program Termination, and Complexity Bounds of Flowchart Programs. *Static Analysis*, 117–133. https://doi.org/10.1007/978-3-642-15769-1_8
- Allen, F. E. (1970). Control flow analysis. *ACM SIGPLAN Notices*, 5(7), 1–19. <https://doi.org/10.1145/390013.808479>
- Amadio, R. M., Ayache, N., Bobot, F., Boender, J., Campbell, B., Garnier, I., Madet, A., McKinna, J., Mulligan, D. P., Piccolo, M., Pollack, R., Régis-Gianas, Y., Coen, C. S., Stark, I., & Tranquilli, P. (2013). Certified Complexity (CerCo). In U. D. Lago & R. Peña (Eds.), *Foundational and Practical Aspects of Resource Analysis* (pp. 1–18, Vol. 8552). Springer International Publishing. https://doi.org/10.1007/978-3-319-12466-7_1
- Amini, M. (2012, December). *Source-to-Source Automatic Program Transformations for GPU-like Hardware Accelerators* [PhD thesis]. Ecole Nationale Supérieure des Mines de Paris. <https://pastel.archives-ouvertes.fr/pastel-00958033>
- Amini, M., Creusillet, B., Even, S., Keryell, R., Goubier, O., Guelton, S., McMahon, J. O., Pasquier, F.-X., Péan, G., & Villalon, P. (2012). Par4All: From Convex Array Regions to Heterogeneous Computing. *IMPACT 2012 : Second International Workshop on Polyhedral Compilation Techniques HiPEAC 2012*. <https://hal-mines-paristech.archives-ouvertes.fr/hal-00744733>
- Arabnejad, H., Bispo, J., Cardoso, J. M. P., & Barbosa, J. G. (2020). Source-to-source compilation targeting OpenMP-based automatic parallelization of C applications. *The Journal of Supercomputing*, 76(9), 6753–6785. <https://doi.org/10.1007/s11227-019-03109-9>
- Ariane 501 Inquiry Board. (1996). *ARIANE 5 Flight 501 Failure* (Technical report). European Space Agency. <https://esamultimedia.esa.int/docs/esa-x-1819eng.pdf>
- Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Outeau, D., & McDaniel, P. (2014). FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. *ACM SIGPLAN Notices*, 49(6), 259–269. <https://doi.org/10.1145/2666356.2594299>
- Atkey, R. (2024). Polynomial Time and Dependent Types. *Proceedings of the ACM on Programming Languages*, 8(POPL), 2288–2317. <https://doi.org/10.1145/3632918>
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2021, March). *LQICM On C Toy Parser* (Version 3.0.0) [git rev e21a3c9]. https://github.com/statycc/LQICM_On_C_Toy_Parser
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2022a). mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity. *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022)*, 228, 26:1–26:23. <https://doi.org/10.4230/LIPIcs.FSCD.2022.26>

- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2022b, May). *A Novel Loop Fission Technique Inspired by Implicit Computational Complexity* [Draft]. <https://hal.archives-ouvertes.fr/hal-03669387v1>
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2022d, June). *Realizing Implicit Computational Complexity* [At the 28th International Conference on Types for Proofs and Programs (TYPES)]. https://types22.inria.fr/files/2022/06/TYPES_2022_paper_14.pdf
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2023a). *pymwp documentation* [Accessed: 2025-06-23]. <https://statycc.github.io/pymwp/>
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2023b). *pymwp: A Static Analyzer Determining Polynomial Growth Bounds*. In *Automated Technology for Verification and Analysis* (pp. 263–275). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-45332-8_14
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2023c, January). *Certifying Complexity Analysis* [At the Ninth International Workshop on Coq for Programming Languages (CoqPL)]. <https://hal.science/hal-04083105v1/file/main.pdf>
- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2025, March). *pymwp: MWP analysis in Python* (Version 0.5.4). Zenodo. <https://doi.org/10.5281/zenodo.7879822>
- Aubert, C., & Rusch, N. (2024, October). Presentation of “An Information Flow Calculus for Non-Interference” [Accessed: 2025-09-14]. <https://plas24.github.io>
Presentation at the PLAS 2024 Workshop.
- Aubert, C., & Seiller, T. (2016). Logarithmic space and permutations. *Information and Computation*, 248, 2–21. <https://doi.org/10.1016/j.ic.2014.01.018>
- Aubert, C. A., Rubiano, T., Rusch, N., & Seiller, T. (2023e). Distributing and Parallelizing Non-canonical Loops. In *Verification, Model Checking, and Abstract Interpretation* (pp. 1–24). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-24950-1_1
- Avanzini, M., & Dal Lago, U. (2017). Automating Sized-Type Inference for Complexity Analysis. *Proceedings of the ACM on Programming Languages*, 1(ICFP), 43:1–43:29. <https://doi.org/10.1145/3110287>
- Avanzini, M., Moser, G., Péchoux, R., & Perdrix, S. (2024). On the hardness of analyzing quantum programs quantitatively. In *Programming languages and systems* (pp. 31–58). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-57267-8_2
- Avanzini, M., Moser, G., & Schaper, M. (2016). TcT: Tyrolean Complexity Tool. In *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 407–423). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-49674-9_24
- Avanzini, M., Moser, G., & Schaper, M. (2020). A modular cost analysis for probabilistic programs. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), 1–30. <https://doi.org/10.1145/3428240>
- Avanzini, M., Moser, G., & Schnabl, A. (2008). Automated Implicit Computational Complexity Analysis (System Description). In *Automated Reasoning* (pp. 132–138). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-71070-7_10
- Bae, H., Mustafa, D., Lee, J., Aurangzeb, Lin, H., Dave, C., Eigenmann, R., & Midkiff, S. P. (2013). The Cetus Source-to-Source Compiler Infrastructure: Overview and

- Evaluation. *International Journal of Parallel Programming*, 41(6), 753–767. <https://doi.org/10.1007/s10766-012-0211-z>
- Baier, C., & Katoen, J.-P. (2008). *Principles of Model Checking* [Contributor Kim Guldstrand Larsen]. The MIT Press.
- Baillot, P., Barthe, G., & Dal Lago, U. (2015). Implicit Computational Complexity of Subrecursive Definitions and Applications to Cryptographic Proofs. In *Logic for Programming, Artificial Intelligence, and Reasoning* (pp. 203–218). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-48899-7_15
- Baillot, P., Barthe, G., & Dal Lago, U. (2019). Implicit Computational Complexity of Subrecursive Definitions and Applications to Cryptographic Proofs. *Journal of Automated Reasoning*, 63(4), 813–855. <https://doi.org/10.1007/s10817-019-09530-2>
- Baillot, P., Dal Lago, U., & Moyén, J.-Y. (2012). On quasi-interpretations, blind abstractions and implicit complexity. *Mathematical Structures in Computer Science*, 22(4), 549–580. <https://doi.org/10.1017/s0960129511000685>
- Baillot, P., & Terui, K. (2004). Light Types for Polynomial Time Computation in Lambda-Calculus. *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004*, 266–275. <https://doi.org/10.1109/LICS.2004.1319621>
- Banerjee, U. (1997). *Dependence analysis* (Vol. 3). Springer Science and Business Media. <https://doi.org/10.1007/b102376>
- Barbosa, M., Barthe, G., Grégoire, B., Koutsos, A., & Strub, P.-Y. (2021). Mechanized Proofs of Adversarial Complexity and Application to Universal Composability. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2541–2563. <https://doi.org/10.1145/3460120.3484548>
- Barbosa, M., Barthe, G., Grégoire, B., Koutsos, A., & Strub, P.-Y. (2023). Mechanized proofs of adversarial complexity and application to universal composability. *ACM Transactions on Privacy and Security*, 26(3), 1–34. <https://doi.org/10.1145/3589962>
- Barrett, Clark and Fontaine, Pascal and Tinelli, Cesare. (2025). *The SMT-LIB Standard, Version 2.7* (Technical report). Department of Computer Science, The University of Iowa. <https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-02-05.pdf>
- Barthe, G., Basu, A., & Rezk, T. (2004). Security Types Preserving Compilation. *Verification, Model Checking, and Abstract Interpretation*, 2937, 2–15. https://doi.org/10.1007/978-3-540-24622-0_2
- Barthe, G., Demange, D., & Pichardie, D. (2014). Formal Verification of an SSA-Based Middle-End for CompCert. *ACM Transactions on Programming Languages and Systems*, 36(1), 4:1–4:35. <https://doi.org/10.1145/2579080>
- Barthe, G., Dupressoir, F., Grégoire, B., Kunz, C., Schmidt, B., & Strub, P.-Y. (2014). Easy-crypt: A tutorial. In *Foundations of security analysis and design vii* (pp. 146–166). Springer International Publishing. https://doi.org/10.1007/978-3-319-10082-1_6
- Barthe, G., Grégoire, B., & Zanella Béguelin, S. (2009). Formal certification of code-based cryptographic proofs. *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 90–101. <https://doi.org/10.1145/1480881.1480894>
- Barthe, G., Pichardie, D., & Rezk, T. (2007). A Certified Lightweight Non-interference Java Bytecode Verifier. In *Programming Languages and Systems* (pp. 125–140). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-71316-6_10

- Bau, J., & Mitchell, J. C. (2011). Security Modeling and Analysis. *IEEE Security & Privacy Magazine*, 9(3), 18–25. <https://doi.org/10.1109/msp.2011.2>
- Bay, J., & Askarov, A. (2020). Reconciling progress-insensitive noninterference and declassification. *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, 95–106. <https://doi.org/10.1109/csf49147.2020.00015>
- Bell, D. E., & La Padula, L. J. (1976). *Secure Computer System: Unified Exposition and Multics Interpretation* (Technical report). Mitre Corporation. <https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/bell76.pdf>
- Bellantoni, S., & Cook, S. (1992a). A new recursion-theoretic characterization of the polytime functions. *Computational Complexity*, 2(2), 97–110. <https://doi.org/10.1007/bf01201998>
- Bellantoni, S. J., & Cook, S. A. (1992b). A New Recursion-Theoretic Characterization of the Polytime Functions (Extended Abstract). In S. R. Kosaraju, M. Fellows, A. Wigderson, & J. A. Ellis (Eds.), *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing - STOC '92* (pp. 283–93). ACM. <https://doi.org/10.1145/129712.129740>
- Benabderrahmane, M., Pouchet, L., Cohen, A., & Bastoul, C. (2010). The Polyhedral Model Is More Widely Applicable Than You Think. In R. Gupta (Ed.), *Compiler Construction* (pp. 283–303, Vol. 6011). Springer. https://doi.org/10.1007/978-3-642-11970-5_16
- Ben-Amram, A. M. (2010). On Decidable Growth-Rate Properties of Imperative Programs. In P. Baillot (Ed.), *Electronic Proceedings in Theoretical Computer Science* (pp. 1–14, Vol. 23). <https://doi.org/10.4204/EPTCS.23.1>
- Ben-Amram, A. M., & Hamilton, G. (2020). Tight Polynomial Worst-Case Bounds for Loop Programs. *Logical Methods in Computer Science*, 16(2), 4:1–4:39. [https://doi.org/10.23638/LMCS-16\(2:4\)2020](https://doi.org/10.23638/LMCS-16(2:4)2020)
- Ben-Amram, A. M., Jones, N. D., & Kristiansen, L. (2008). Linear, Polynomial or Exponential? Complexity Inference in Polynomial Time. In A. Beckmann, C. Dimitracopoulos, & B. Löwe (Eds.), *Logic and Theory of Algorithms* (pp. 67–76, Vol. 5028). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-69407-6_7
- Bendersky, E. (2024, March). *pycparser - Complete C99 parser in pure Python* (Version 2.22). <https://github.com/eliben/pycparser>
- Beringer, L., & Hofmann, M. (2007). Secure information flow and program logics. *20th IEEE Computer Security Foundations Symposium (CSF'07)*, 233–248. <https://doi.org/10.1109/csf.2007.30>
- Bertolacci, I., Strout, M. M., de Supinski, B. R., Scogland, T. R. W., Davis, E. C., & Olschanowsky, C. (2018). Extending OpenMP to Facilitate Loop Optimization. In B. R. de Supinski, P. Valero-Lara, X. Martorell, S. M. Bellido, & J. Labarta (Eds.), *Evolving OpenMP for Evolving Architectures* (pp. 53–65). Springer International Publishing. https://doi.org/10.1007/978-3-319-98521-3_4
- Bertot, Y., & Castéran, P. (2004). *Interactive theorem proving and program development: Coq'Art: the calculus of inductive constructions* (1st ed.). Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-662-07964-5>

- Beyer, D. (2022). Progress on Software Verification: SV-COMP 2022. In *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 375–402). Springer International Publishing. https://doi.org/10.1007/978-3-030-99527-0_20
- Biba, K. J., et al. (1977). *Integrity considerations for secure computer systems* (Technical report). Mitre Corporation. <https://apps.dtic.mil/sti/pdfs/ADA039324.pdf>
- Bishop, M. (2019). *Computer security: art and science* (P. Education, Ed.). Addison-Wesley Professional.
- Blazy, S., Maroneze, A., & Pichardie, D. (2013). Formal Verification of Loop Bound Estimation for WCET Analysis. In *Verified Software: Theories, Tools, Experiments* (pp. 281–303). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-54108-7_15
- Board, O. A. R. (2021, November). *OpenMP Application Programming Interface Specification Version 5.2* (M. Klemm & B. R. de Supinski, Eds.) [Accessed: 2025-06-23]. <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>
- Bognar, M., Van Bulck, J., & Piessens, F. (2022). Mind the Gap: Studying the Insecurity of Provably Secure Embedded Trusted Execution Architectures. *2022 IEEE Symposium on Security and Privacy (SP)*, 1638–1655. <https://doi.org/10.1109/sp46214.2022.9833735>
- Bondhugula, U., Hartono, A., Ramanujam, J., & Sadayappan, P. (2008). A Practical Automatic Polyhedral Parallelizer and Locality Optimizer. *ACM SIGPLAN Notices*, 43(6), 101–113. <https://doi.org/10.1145/1379022.1375595>
- Bonfante, G., Kahle, R., Marion, J.-Y., & Oitavem, I. (2016). Two function algebras defining functions in NC^k boolean circuits. *Information and Computation*, 248, 82–103. <https://doi.org/10.1016/j.ic.2015.12.009>
- Bossi, A., Macedonio, D., Piazza, C., & Rossi, S. (2005). Information flow in secure contexts. *Journal of Computer Security*, 13(3), 391–422. <https://doi.org/10.3233/jcs-2005-13303>
- Brockschmidt, M., Emmes, F., Falke, S., Fuhs, C., & Giesl, J. (2016). Analyzing Runtime and Size Complexity of Integer Programs. *ACM Transactions on Programming Languages and Systems*, 38(4), 1–50. <https://doi.org/10.1145/2866575>
- Campbell, C., & Miller, A. (2011). *A Parallel Programming with Microsoft Visual C++ Design Patterns for Decomposition and Coordination on Multicore Architectures*. Microsoft Press.
- Carbonneaux, Q. (2018). *Modular and Certified Resource-Bound Analyses* [PhD thesis]. Yale University. <https://www.proquest.com/dissertations-theses/modular-certified-resource-bound-analyses/docview/2088887534/se-2>
- Carbonneaux, Q., Hoffmann, J., Ramananandro, T., & Shao, Z. (2014). End-to-End Verification of Stack-Space Bounds for C Programs. *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 270–281. <https://doi.org/10.1145/2594291.2594301>
- Carbonneaux, Q., Hoffmann, J., Reps, T., & Shao, Z. (2017). Automated Resource Analysis with Coq Proof Objects. In *Computer Aided Verification* (pp. 64–85). Springer International Publishing. https://doi.org/10.1007/978-3-319-63390-9_4
- Carbonneaux, Q., Hoffmann, J., & Shao, Z. (2015). Compositional Certified Resource Bounds. *Proceedings of the 36th ACM SIGPLAN Conference on Programming*

- Language Design and Implementation*, 467–478. <https://doi.org/10.1145/2737924.2737955>
- Cassez, F., Fuller, J., & Quiles, H. M. A. (2022). Deductive Verification of Smart Contracts with Dafny. In *Formal Methods for Industrial Critical Systems* (pp. 50–66). Springer International Publishing. https://doi.org/10.1007/978-3-031-15008-1_5
- Cecchetti, E., Myers, A. C., & Arden, O. (2017). Nonmalleable Information Flow Control. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1875–1891. <https://doi.org/10.1145/3133956.3134054>
- Chakarov, A., Geldenhuys, J., Heck, M., Hicks, M., Huang, S., Jaloyan, G. A., Joshi, A., Leino, R., Mayer, M., McLaughlin, S., Mritunjai, A., Claudel, C. P., Porncharoenwase, S., Rabe, F., Rapoport, M., Reger, G., Roux, C., Rungta, N., Salkeld, R., ... Yuan, Y. (2025). Formally Verified Cloud-Scale Authorization. *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2508–2521. <https://doi.org/10.1109/ICSE55347.2025.00166>
- Chang, B.-Y. E., Rustan, K., & Leino, M. (2005). Inferring Object Invariants: Extended Abstract [Proceedings of the First International Workshop on Abstract Interpretation of Object-oriented Languages (AIOOL 2005)]. *Electronic Notes in Theoretical Computer Science*, 131, 63–74. <https://doi.org/10.1016/j.entcs.2005.01.023>
- Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press.
- Chatterjee, K., Goharshady, A. K., Meggendorfer, T., & Žikelić, Đ. (2024). Quantitative Bounds on Resource Usage of Probabilistic Programs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1), 362–391. <https://doi.org/10.1145/3649824>
- Chlipala, A. (2010). An Introduction to Programming and Proving with Dependent Types in Coq. *Journal of Formalized Reasoning*, 3(2), 1–93. <https://doi.org/10.6092/issn.1972-5787/1978>
- Chlipala, A. (2013). *Certified programming with dependent types: a pragmatic introduction to the Coq proof assistant*. The MIT Press.
- Chlipala, A. (2022). *Formal Reasoning About Programs*. The MIT Press. http://adam.chlipala.net/frap/frap_book.pdf
- Chung, F. R. K. (1980). On the coverings of graphs. *Discrete Mathematics*, 30(2), 89–93. [https://doi.org/10.1016/0012-365X\(80\)90109-0](https://doi.org/10.1016/0012-365X(80)90109-0)
- Ciaffaglione, A. (2016). Towards Turing computability via coinduction. *Science of Computer Programming*, 126, 31–51. <https://doi.org/10.1016/j.scico.2016.02.004>
- Clarkson, M. R., & Schneider, F. B. (2010). Hyperproperties. *Journal of Computer Security*, 18(6), 1157–1210. <https://doi.org/10.3233/jcs-2009-0393>
- Cobham, A. (1965). The Intrinsic Computational Difficulty of Functions. In Y. Bar-Hillel (Ed.), *Logic, methodology and philosophy of science* (pp. 24–30). North-Holland Publishing Company. https://www.cs.toronto.edu/~sacook/homepage/cobham_intrinsic.pdf
- Colledan, A., & Dal Lago, U. (2024). Circuit width estimation via effect typing and linear dependency. In *Programming languages and systems* (pp. 3–30). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-57267-8_1

- Colón, M. A., Sankaranarayanan, S., & Sipma, H. B. (2003). Linear Invariant Generation Using Non-linear Constraint Solving. *Computer Aided Verification*, 420–432. https://doi.org/10.1007/978-3-540-45069-6_39
- Cook, B. (2024). An unexpected discovery: Automated reasoning often makes systems more efficient and easier to maintain [Accessed: 2025-04-17]. <https://aws.amazon.com/blogs/security/an-unexpected-discovery-automated-reasoning-often-makes-systems-more-efficient-and-easier-to-maintain/> AWS Security Blog, 17 October 2024.
- Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the third annual ACM symposium on Theory of computing - STOC '71*, 151–158. <https://doi.org/10.1145/800157.805047>
- Coquand, T., & P. Huet, G. (1988). The Calculus of Constructions. *Information and Computation*, 76(2/3), 95–120. [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3)
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd Ed.)* The MIT press.
- Cousot, P. (2021). *Principles of Abstract Interpretation*. The MIT Press.
- Cousot, P., & Cousot, R. (1977). Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In R. M. Graham, M. A. Harrison, & R. Sethi (Eds.), *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages - POPL '77* (pp. 238–252). ACM. <https://doi.org/10.1145/512950.512973>
- Cousot, P., & Cousot, R. (1978). Static Determination of Dynamic Properties of Recursive Procedures. In E. J. Neuhold (Ed.), *IFIP Conference on Formal Description of Programming Concepts* (pp. 237–278). North-Holland Publishing Company.
- Cousot, P., & Halbwachs, N. (1978). Automatic discovery of linear restraints among variables of a program. *Proceedings of the 5th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 84–96. <https://doi.org/10.1145/512760.512770>
- Crary, K., & Weirich, S. (2000). Resource bound certification. *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 184–198. <https://doi.org/10.1145/325694.325716>
- Cyphert, J., & Kincaid, Z. (2024). Solvable Polynomial Ideals: The Ideal Reflection for Program Analysis. *Proc. ACM Program. Lang.*, 8(POPL). <https://doi.org/10.1145/3632867>
- Dafny Contributors. (2025, February). *Dafny Documentation* (Version 4.10.0) [Accessed: 2025-04-05]. Technical documentation. Microsoft Research. <https://dafny.org/dafny>
- Dafny Language Developers. (2025, February). *Dafny Language* (Version 4.10.0). <https://github.com/dafny-lang/dafny>
- Daikon developers. (2024, May). *Daikon* (Version 5.8.20). <https://plse.cs.washington.edu/daikon>
- Dal Lago, U. (2011). A Short Introduction to Implicit Computational Complexity. In N. Bezhanishvili & V. Goranko (Eds.), *Lectures on Logic and Computation* (pp. 89–109, Vol. 7388). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-31485-8_3

- Dal Lago, U. (2022). Implicit computation complexity in higher-order programming languages: A Survey in Memory of Martin Hofmann. *Mathematical Structures in Computer Science*, 32(6), 760–776. <https://doi.org/10.1017/S0960129521000505>
- Dal Lago, U., Kahle, R., & Oitavem, I. (2021). A Recursion-Theoretic Characterization of the Probabilistic Class PP. *46th International Symposium on Mathematical Foundations of Computer Science (MFCS 2021)*, 202, 35:1–35:12. <https://doi.org/10.4230/LIPIcs.MFCS.2021.35>
- Dal Lago, U., Masini, A., & Zorzi, M. (2010). Quantum implicit computational complexity. *Theoretical Computer Science*, 411(2), 377–409. <https://doi.org/10.1016/j.tcs.2009.07.045>
- Das, A., Balzer, S., Hoffmann, J., Pfenning, F., & Santurkar, I. (2021). Resource-aware session types for digital contracts. *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, 1–16. <https://doi.org/10.1109/csf51468.2021.00004>
- Das, A., Hoffmann, J., & Pfenning, F. (2018). Work analysis with resource-aware session types. *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, 305–314. <https://doi.org/10.1145/3209108.3209146>
- Dash, S., Kaddar, Y., Paquet, H., & Staton, S. (2023). Affine Monads and Lazy Structures for Bayesian Programming. *Proceedings of the ACM on Programming Languages*, 7(POPL), 1338–1368. <https://doi.org/10.1145/3571239>
- Dash, S., Kaddar, Y., Paquet, H., & Staton, S. (2024, September). *LazyPPL: Lazy Probabilistic Programming Library* (Version git rev de9bdf6). <https://github.com/lazyppl-team/lazyppl>
- Dave, C., Bae, H., Min, S., Lee, S., Eigenmann, R., & Midkiff, S. P. (2009). Cetus: A Source-to-Source Compiler Infrastructure for Multicores. *Computer*, 42(11), 36–42. <https://doi.org/10.1109/MC.2009.385>
- de Moura, L., & Bjørner, N. (2008). Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 337–340). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-78800-3_24
- Denning, D. E. (1976). A Lattice Model of Secure Information Flow. *Communications of the ACM*, 19(5), 236–243. <https://doi.org/10.1145/360051.360056>
- Derakhshan, F., Balzer, S., & Yao, Y. (2024). Regrading Policies for Flexible Information Flow Control in Session-Typed Concurrency. *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, 313, 11:1–11:29. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.11>
- Dijkstra, E. W. (1970). *Notes on structured programming* (Technical report). Department of Mathematics, Technological University Eindhoven, The Netherlands. <https://dl.acm.org/doi/pdf/10.5555/1243380.1243381>
- Dijkstra, E. W. (1972). The humble programmer. *Communications of the ACM*, 15(10), 859–866. <https://doi.org/10.1145/355604.361591>
- Dijkstra, E. W. (1979a). A more formal treatment of a less simple example. In *Program Construction* (pp. 1–20). Springer Berlin Heidelberg. <https://doi.org/10.1007/bfb0014650>
- Dijkstra, E. W. (1979b). On the foolishness of ”natural language programming”. In *Program Construction* (pp. 51–53). Springer Berlin Heidelberg. <https://doi.org/10.1007/bfb0014656>

- Dillig, I., Dillig, T., Li, B., & McMillan, K. (2013). Inductive invariant generation via abductive inference. *ACM SIGPLAN Notices*, 48(10), 443–456. <https://doi.org/10.1145/2544173.2509511>
- Ding, S., & Zhang, Q. (2023). Witnessability of Undecidable Problems. *Proceedings of the ACM on Programming Languages*, 7(POPL), 982–1002. <https://doi.org/10.1145/3571227>
- Du, W. (2017). *Computer security: a hands-on approach*. CreateSpace Independent Publishing Platform.
- Dudina, I., & Stark, I. (2025). Static analysis to make the most of CHERI C/C++ for existing code: improving memory safety at scale. *International Journal on Software Tools for Technology Transfer*, 27(2), 225–237. <https://doi.org/10.1007/s10009-025-00781-6>
- Duet Developers. (2024, October). *Duet static analyzer* [git rev 5b8630f]. <https://github.com/zkincaid/duet>
- Edmonds, J. (1965). Paths, trees, and flowers. *Canadian Journal of mathematics*, 17, 449–467.
- Eggert, S. (2014). *Security via Noninterference - Analyzing Information Flows* [PhD thesis]. University of Kiel. http://macau.uni-kiel.de/receive/dissertation%5C_diss%5C_00015837
- Eilers, M., Dardinier, T., & Müller, P. (2023). CommCSL: Proving Information Flow Security for Concurrent Programs using Abstract Commutativity. *Proceedings of the ACM on Programming Languages*, 7(PLDI), 1682–1707. <https://doi.org/10.1145/3591289>
- Ernst, M. D., Perkins, J. H., Guo, P. J., McCamant, S., Pacheco, C., Tschantz, M. S., & Xiao, C. (2007). The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1), 35–45. <https://doi.org/10.1016/j.scico.2007.01.015>
- Falcone, Y., Fernandez, J.-C., & Mounier, L. (2009). Runtime Verification of Safety-Progress Properties. In S. Bensalem & D. A. Peled (Eds.), *Runtime Verification* (pp. 40–59). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-04694-0_4
- Feiertag, R. J., Levitt, K. N., & Robinson, L. (1977). Proving multilevel security of a system design. *ACM SIGOPS Operating Systems Review*, 11(5), 57–65. <https://doi.org/10.1145/1067625.806547>
- Férée, H., Hym, S., Mayero, M., Moyen, J., & Nowak, D. (2018). Formal proof of polynomial-time complexity with quasi-interpretations. *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 146–157. <https://doi.org/10.1145/3167097>
- Ferrante, J., Ottenstein, K. J., & Warren, J. D. (1987). The Program Dependence Graph and Its Use in Optimization. *ACM Transactions on Programming Languages and Systems*, 9(3), 319–349. <https://doi.org/10.1145/24039.24041>
- Flores-Montoya, A. (2016a). CoFloCo: system description. *15th International Workshop on Termination*, 20.
- Flores-Montoya, A. (2016b). Upper and Lower Amortized Cost Bounds of Programs Expressed as Cost Relations. In J. S. Fitzgerald, C. L. Heitmeyer, S. Gnesi, & A. Philippou (Eds.), *FM 2016: Formal Methods* (pp. 254–273, Vol. 9995). Springer International Publishing. https://doi.org/10.1007/978-3-319-48989-6_16

- Flores-Montoya, A., & Hähnle, R. (2014). Resource Analysis of Complex Programs with Cost Equations. In *Programming Languages and Systems* (pp. 275–295). Springer International Publishing. https://doi.org/10.1007/978-3-319-12736-1_15
- Focardi, R., & Gorrieri, R. (1997). The Compositional Security Checker: a tool for the verification of information flow security properties. *IEEE Transactions on Software Engineering*, 23(9), 550–571. <https://doi.org/10.1109/32.629493>
- Forster, Y., Kunze, F., & Wuttke, M. (2020). Verified programming of Turing machines in Coq. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 114–128. <https://doi.org/10.1145/3372885.3373816>
- Frumin, D., Krebbers, R., & Birkedal, L. (2021). Compositional Non-Interference for Fine-Grained Concurrent Programs. *2021 IEEE Symposium on Security and Privacy (SP)*, 1416–1433. <https://doi.org/10.1109/sp40001.2021.00003>
- Furia, C. A. (2014). Software Verification Assertion Inference[†] [Accessed: 2025-04-19]. https://se.inf.ethz.ch/courses/2014b_fall/sv/slides/06-AssertionInference.pdf
Lecture notes of Software Verification course, ETH Zürich, Fall 2014.
- Furia, C. A., Meyer, B., & Velder, S. (2014). Loop invariants: Analysis, classification, and examples. *ACM Computing Surveys*, 46(3), 1–51. <https://doi.org/10.1145/2506375>
- Furia, C. A., & Meyer, B. (2010). Inferring Loop Invariants Using Postconditions. In A. Blass, N. Dershowitz, & W. Reisig (Eds.), *Fields of Logic and Computation* (pp. 277–300). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-15025-8_15
- Gäher, L., & Kunze, F. (2021). Mechanising Complexity Theory: The Cook-Levin Theorem in Coq. *12th International Conference on Interactive Theorem Proving (ITP 2021)*, 193, 20:1–20:18. <https://doi.org/10.4230/LIPIcs.ITP.2021.20>
- Garg, D., & Pfenning, F. (2006). Non-Interference in Constructive Authorization Logic. *19th IEEE Computer Security Foundations Workshop (CSFW'06)*, 283–296. <https://doi.org/10.1109/csfw.2006.18>
- Silber, G.-A., & Pop, S. (2022). gcc.gnu.org Git - gcc.git/blob - gcc/tree-loop-distribution.c [Accessed: 2025-06-23]. <https://gcc.gnu.org/git/?p=gcc.git;a=blob;f=gcc/tree-loop-distribution.c;h=65aa1df4abae2c6acf40299f710bc62ee6bacc07;hb=HEAD#l39>
- Georges, A. L., Peters, B., Elbeheiry, L., White, L., Dolan, S., Eisenberg, R. A., Casinghino, C., Pottier, F., & Dreyer, D. (2025). Data race freedom à la mode. *Proceedings of the ACM on Programming Languages*, 9(POPL), 656–686. <https://doi.org/10.1145/3704859>
- Giallorenzo, S., Montesi, F., & Peressotti, M. (2024). Choral: Object-oriented Choreographic Programming. *ACM Transactions on Programming Languages and Systems*, 46(1), 1–59. <https://doi.org/10.1145/3632398>
- Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, C., Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., & Thiemann, R. (2016). Analyzing Program Termination and Complexity Automatically with AProVE. *Journal of Automated Reasoning*, 58(1), 3–31. <https://doi.org/10.1007/s10817-016-9388-y>
- Giesl, J., Lommen, N., Hark, M., & Meyer, F. (2022). Improving Automatic Complexity Analysis of Integer Programs. In *The Logic of Software. A Tasting Menu of Formal Methods: Essays Dedicated to Reiner Hähnle on the Occasion of His 60th Birthday*

- (pp. 193–228). Springer International Publishing. https://doi.org/10.1007/978-3-031-08166-8_10
- Giesl, J., Rubio, A., Sternagel, C., Waldmann, J., & Yamada, A. (2019). The termination and complexity competition. In D. Beyer, M. Huisman, F. Kordon, & B. Steffen (Eds.), *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 156–166, Vol. 11429). Springer International Publishing. https://doi.org/10.1007/978-3-030-17502-3_10
- Goguen, J. A., & Meseguer, J. (1982, April). Security policies and security models. In *1982 IEEE Symposium on Security and Privacy* (pp. 11–20). IEEE. <https://doi.org/10.1109/SP.1982.10014>
- Goldreich, O. (2008). *Computational Complexity: A Conceptual Perspective*. Cambridge University Press.
- Gonthier, G. (2008). The Four Colour Theorem: Engineering of a Formal Proof. In *Computer Mathematics* (pp. 333–333). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-87827-8_28
- Gonthier, G., Asperti, A., Avigad, J., Bertot, Y., Cohen, C., Garillot, F., Le Roux, S., Mahboubi, A., O’Connor, R., Ould Biha, S., Pasca, I., Rideau, L., Solovyev, A., Tassi, E., & Théry, L. (2013). A Machine-Checked Proof of the Odd Order Theorem. In *Interactive Theorem Proving* (pp. 163–179). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-39634-2_14
- Grodin, H., Niu, Y., Sterling, J., & Harper, R. (2023, October). *Decalf: A Directed, Effectful Cost-Aware Logical Framework* (Version v2.0.0-rc.1). Zenodo. <https://doi.org/10.5281/zenodo.8423788>
- Grodin, H., Niu, Y., Sterling, J., & Harper, R. (2024). Decalf: A Directed, Effectful Cost-Aware Logical Framework. *Proceedings of the ACM on Programming Languages*, 8(POPL), 273–301. <https://doi.org/10.1145/3632852>
- Grosser, T. (2011, April). *Enabling Polyhedral Optimizations in LLVM* [Diploma Thesis]. Universität Passau. <https://polly.llvm.org/publications/grosser-diploma-thesis.pdf>
- Guéneau, A. (2019, December). *Mechanized Verification of the Correctness and Asymptotic Complexity of Programs*. (*Vérification mécanisée de la correction et complexité asymptotique de programmes*) [PhD thesis]. Université de Paris. <https://tel.archives-ouvertes.fr/tel-02437532>
- Guéneau, A., Charguéraud, A., & Pottier, F. (2018). A Fistful of Dollars: Formalizing Asymptotic Complexity Claims via Deductive Program Verification. In *Programming Languages and Systems* (pp. 533–560). Springer International Publishing. https://doi.org/10.1007/978-3-319-89884-1_19
- Gulwani, S., Mehra, K. K., & Chilimbi, T. (2009). SPEED: Precise and Efficient Static Estimation of Program Computational Complexity. *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 127–139. <https://doi.org/10.1145/1480881.1480898>
- Hague, M., Jež, A., & Lin, A. W. (2024). Parikh’s Theorem Made Symbolic. *Proceedings of the ACM on Programming Languages*, 8(POPL), 1945–1977. <https://doi.org/10.1145/3632907>

- Hähnle, R., & Huisman, M. (2019). Deductive Software Verification: From Pen-and-Paper Proofs to Industrial Tools. In *Computing and Software Science* (pp. 345–373). Springer International Publishing. https://doi.org/10.1007/978-3-319-91908-9_18
- Hainry, E., Jeandel, E., Péchoux, R., & Zeyen, O. (2021). Complexityparser: An automatic tool for certifying poly-time complexity of Java programs. In A. Cerone & P. C. Ölveczky (Eds.), *Theoretical Aspects of Computing – ICTAC 2021* (pp. 357–365). Springer International Publishing. https://doi.org/10.1007/978-3-030-85315-0_20
- Hainry, E., Kapron, B., Marion, J.-Y., & Péchoux, R. (2024). Declassification Policy for Program Complexity Analysis. *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*. <https://doi.org/10.1145/3661814.3662100>
- Hainry, E., Kapron, B. M., Marion, J.-Y., & Péchoux, R. (2020). A tier-based typed programming language characterizing Feasible Functionals. *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*, 535–549. <https://doi.org/10.1145/3373718.3394768>
- Hainry, E., Marion, J.-Y., & Péchoux, R. (2013). Type-Based Complexity Analysis for Fork Processes. In *Foundations of Software Science and Computation Structures* (pp. 305–320). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-37075-5_20
- Hainry, E., & Péchoux, R. (2015). Objects in Polynomial Time. In *Programming Languages and Systems* (pp. 387–404). Springer International Publishing. https://doi.org/10.1007/978-3-319-26529-2_21
- Hainry, E., & Péchoux, R. (2018). A type-based complexity analysis of Object Oriented programs. *Information and Computation*, 261, 78–115. <https://doi.org/10.1016/j.ic.2018.05.006>
- Hainry, E., & Péchoux, R. (2023). A General Noninterference Policy for Polynomial Time. *Proceedings of the ACM on Programming Languages*, 7(POPL), 806–832. <https://doi.org/10.1145/3571221>
- Hamann, T., Herda, M., Mantel, H., Mohr, M., Schneider, D., & Tasch, M. (2018a). A Uniform Information-Flow Security Benchmark Suite for Source Code and Bytecode. In *Secure IT Systems* (pp. 437–453). Springer International Publishing. https://doi.org/10.1007/978-3-030-03638-6_27
- Hamann, T., Herda, M., Mantel, H., Mohr, M., Schneider, D., & Tasch, M. (2018b). *IFSPEC benchmark suite* (Version 1.0) [Refactored and updated online archive]. <https://github.com/statycc/ifspec>
- Hannay, J. E., MacLeod, C., Singer, J., Langtangen, H. P., Pfahl, D., & Wilson, G. (2009). How do scientists develop and use scientific software? *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, 1–8. <https://doi.org/10.1109/secse.2009.5069155>
- Hedin, D., & Sabelfeld, A. (2012). A perspective on information-flow control. In *Software safety and security* (pp. 319–347). IOS Press. <https://doi.org/10.3233/978-1-61499-028-4-319>
- Heraud, S., & Nowak, D. (2011). A Formalization of Polytime Functions. In *Interactive Theorem Proving* (pp. 119–134). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-22863-6_11

- Heraud, S., & Nowak, D. (2018, September). *The “BellantoniCook” formalization library* (Version 1.0.0) [git rev de94ed6]. <https://github.com/davidnowak/bellantonicook>
- Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576–580. <https://doi.org/10.1145/363235.363259>
- Hoare, T. (2006). The Ideal of Verified Software. In *Computer Aided Verification* (pp. 5–16). Springer Berlin Heidelberg. https://doi.org/10.1007/11817963_4
- Hoare, T., Misra, J., Leavens, G. T., & Shankar, N. (2021, October). The verified software initiative: A manifesto. In *Theories of Programming: The Life and Works of Tony Hoare* (pp. 81–92). ACM. <https://doi.org/10.1145/3477355.3477361>
- Hoffmann, J., Aehlig, K., & Hofmann, M. (2012). Resource Aware ML. In P. Madhusudan & S. A. Seshia (Eds.), *Computer Aided Verification* (pp. 781–786). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-31424-7_64
- Hoffmann, J., Das, A., & Weng, S.-C. (2017). Towards automatic resource bound analysis for OCaml. *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, 359–373. <https://doi.org/10.1145/3009837.3009842>
- Hoffmann, J., & Jost, S. (2022). Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science*, 32(6), 729–759. <https://doi.org/10.1017/s0960129521000487>
- Hoffmann, J., & Shao, Z. (2015). Automatic static cost analysis for parallel programs. In *Programming languages and systems* (pp. 132–157). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-46669-8_6
- Hofmann, M. (1999). Linear types and non-size-increasing polynomial time computation. *Proceedings. 14th Symposium on Logic in Computer Science (Cat. No. PR00158)*, 464–473. <https://doi.org/10.1109/lics.1999.782641>
- Holewinski, J., Ramamurthi, R., Ravishankar, M., Fauzia, N., Pouchet, L.-N., Rountev, A., & Sadayappan, P. (2012). Dynamic trace-based analysis of vectorization potential of applications. *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 371–382. <https://doi.org/10.1145/2254064.2254108>
- Hritcu, C., Hughes, J., Pierce, B. C., Spector-Zabusky, A., Vytiniotis, D., Azevedo de Amorim, A., & Lampropoulos, L. (2013). Testing noninterference, quickly. *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming*, 455–468. <https://doi.org/10.1145/2500365.2500574>
- Huang, W., Dong, Y., & Milanova, A. (2014). Type-Based Taint Analysis for Java Web Applications. In *Fundamental Approaches to Software Engineering* (pp. 140–154). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-54804-8_10
- Ileri, A. M., Zeldovich, N., Chlipala, A., & Kaashoek, F. (2024). Probability from Possibility: Probabilistic Confidentiality for Storage Systems Under Nondeterminism. *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, 96–111. <https://doi.org/10.1109/csf61375.2024.00041>
- Intel Corporation. (2022). *Intel C++ Compiler Classic Developer Guide and Reference* [Accessed: 2025-06-23]. https://www.intel.com/content/dam/develop/external/us/en/documents/cpp_compiler_classic.pdf

- Jones, N. D. (1999). LOGSPACE and PTIME characterized by programming languages. *Theoretical Computer Science*, 228(1–2), 151–174. [https://doi.org/10.1016/s0304-3975\(98\)00357-0](https://doi.org/10.1016/s0304-3975(98)00357-0)
- Jones, N. D. (2001). The expressive power of higher-order types or, life without CONS. *Journal of Functional Programming*, 11(1), 55–94. <https://doi.org/10.1017/s0956796800003889>
- Jones, N. D., & Kristiansen, L. (2009). A flow calculus of *mwp*-bounds for complexity analysis. *ACM Transactions on Computational Logic*, 10(4), 28:1–28:41. <https://doi.org/10.1145/1555746.1555752>
- Jones, N. D., & Nielson, F. (1995). Abstract Interpretation: A Semantics-Based Tool for Program Analysis. In S. Abramsky, D. M. Gabbay, & T. S. E. Maibaum (Eds.), *Semantic Modelling* (pp. 527–636, Vol. 4). Oxford University Press. <https://doi.org/10.1093/oso/9780198537809.003.0005>
- Joppa, L. N., McInerney, G., Harper, R., Salido, L., Takeda, K., O’Hara, K., Gavaghan, D., & Emmott, S. (2013). Troubling trends in scientific software use. *Science*, 340(6134), 814–815. <https://doi.org/10.1126/science.1231535>
- Jourdan, J.-H., Laporte, V., Blazy, S., Leroy, X., & Pichardie, D. (2015). A formally-verified C static analyzer. *ACM SIGPLAN Notices*, 50(1), 247–259. <https://doi.org/10.1145/2775051.2676966>
- Kaminski, B. (2025, March). Ina Schaefer on X-by-Construction [Accessed: 2025-04-27]. <https://etaps.org/blog/031-ina-schaefer/> ETAPS Blog, Published on 25 March 2025.
- Kammüller, F. (2008). Formalizing non-interference for a simple bytecode language in Coq. *Formal Aspects of Computing*, 20(3), 259–275. <https://doi.org/10.1007/s00165-007-0055-2>
- Karbyshev, A., Svendsen, K., Askarov, A., & Birkedal, L. (2018). Compositional Non-interference for Concurrent Programs via Separation and Framing. In *Principles of Security and Trust* (pp. 53–78). Springer International Publishing. https://doi.org/10.1007/978-3-319-89722-6_3
- Karp, R. M., Miller, R. E., & Winograd, S. (1967). The Organization of Computations for Uniform Recurrence Equations. *Journal of the ACM*, 14(3), 563–590. <https://doi.org/10.1145/321406.321418>
- Karr, M. (1976). Affine relationships among variables of a program. *Acta Informatica*, 6(2), 133–151. <https://doi.org/10.1007/BF00268497>
- Keidel, S. (2021, March). *Modular Specification and Compositional Soundness of Abstract Interpreters* [Doctoral dissertation, Johannes Gutenberg-Universität Mainz]. <https://doi.org/10.25358/openscience-6384>
- Kennedy, K., & Allen, J. R. (2001). *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc.
- KoAT2 Developers. (2024, February). *KoAT2* (Version koat2-twn-journal) [git rev 9c511e3]. <https://github.com/aprove-developers/KoAT2-Releases>
- Kristiansen, L. (2005). Neat function algebraic characterizations of logspace and linspace. *Computational Complexity*, 14(1), 72–88. <https://doi.org/10.1007/s00037-005-0191-0>

- Kristiansen, L. (2017, April). On Resource Analysis of Imperative Programs[†] [Accessed: 2025-01-06].
Presentation slides at DICE-FOPARA 2017, Uppsala, Sweden.
- Kristiansen, L., & Jones, N. D. (2005). The Flow of Data and the Complexity of Algorithms. In S. B. Cooper, B. Löwe, & L. Torenvliet (Eds.), *New Computational Paradigms* (pp. 263–274, Vol. 3526). Springer Berlin Heidelberg. https://doi.org/10.1007/11494645_33
- Lafont, Y. (2004). Soft linear logic and polynomial time. *Theoretical Computer Science*, 318(1), 163–180. <https://doi.org/10.1016/j.tcs.2003.10.018>
- Laird, J., Manzonetto, G., McCusker, G., & Pagani, M. (2013). Weighted Relational Models of Typed Lambda-Calculi. *2013 28th Annual ACM/IEEE Symposium on Logic in Computer Science*, 301–310. <https://doi.org/10.1109/LICS.2013.36>
- Lamba, A., Taylor, M., Beardsley, V., Bambeck, J., Bond, M. D., & Lin, Z. (2024). Cocoon: Static Information Flow Control in Rust. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1), 166–193. <https://doi.org/10.1145/3649817>
- Lamport, L. (1977). Proving the Correctness of Multiprocess Programs. *IEEE Transactions on Software Engineering*, SE-3(2), 125–143. <https://doi.org/10.1109/tse.1977.229904>
- Lattner, C., & Adve, V. S. (2004). LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, 75–88. <https://doi.org/10.1109/CGO.2004.1281665>
- Lawson, N. (2009). Side-channel attacks on cryptographic software. *IEEE Security & Privacy Magazine*, 7(6), 65–68. <https://doi.org/10.1109/msp.2009.165>
- Lee, C. S., Jones, N. D., & Ben-Amram, A. M. (2001). The size-change principle for program termination. *Proceedings of the 28th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 81–92. <https://doi.org/10.1145/360204.360210>
- Leino, K. R. M. (2008). *This is boogie 2* (Technical report). Microsoft. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/krml178.pdf>
- Leino, K. R. M. (2010a). Dafny: An automatic program verifier for functional correctness. *Logic for Programming, Artificial Intelligence, and Reasoning*, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20
- Leino, K. R. M. (2010b, April). Dafny An automatic program verifier for functional correctness[†] [Accessed: 2025-04-14]. <https://www.microsoft.com/en-us/research/publication/dafny-automatic-program-verifier-functional-correctness/>
Presentation slides of LPAR-16, on 27 April 2010.
- Leino, K. R. M. (2023). *Program proofs*. The MIT Press.
- Leivant, D. (1993). Stratified functional programs and computational complexity. In M. S. Van Deusen & B. Lang (Eds.), *Proceedings of the 20th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '93* (pp. 325–333). ACM Press. <https://doi.org/10.1145/158511.158659>
- Leivant, D., & Marion, J.-Y. (1995). Ramified recurrence and computational complexity II: Substitution and poly-space. In *Computer Science Logic* (pp. 486–500). Springer Berlin Heidelberg. <https://doi.org/10.1007/bfb0022277>

- Leivant, D., & Marion, J.-Y. (2013). Evolving Graph-Structures and Their Implicit Computational Complexity. In *Automata, Languages, and Programming* (pp. 349–360). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-39212-2_32
- Leroy, X. (2009). Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 107–115. <https://doi.org/10.1145/1538788.1538814>
- Leroy, X. (2018). Le logiciel, entre l’esprit et la matière [Accessed: 2025-06-30]. <https://www.college-de-france.fr/fr/agenda/lecon-inaugurale/le-logiciel-entre-esprit-et-la-matiere/le-logiciel-entre-esprit-et-la-matiere>
Lecture notes of Collège de France, 15 Novembre 2018.
- Leveson, N., & Turner, C. (1993). An investigation of the Therac-25 accidents. *Computer*, 26(7), 18–41. <https://doi.org/10.1109/mc.1993.274940>
- Levin, L. A. (1973). Universal sequential search problems [English translation]. *Problems of Information Transmission*, 9(3), 265–266.
- Lichtman, B., & Hoffmann, J. (2017). Arrays and References in Resource Aware ML. In D. Miller (Ed.), *2nd International Conference on Formal Structures for Computation and Deduction (FSCD 2017)* (26:1–26:20, Vol. 84). Schloss Dagstuhl - Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.FSCD.2017.26>
- Livshits, B., Sridharan, M., Smaragdakis, Y., Lhoták, O., Amaral, J. N., Chang, B.-Y. E., Guyer, S. Z., Khedker, U. P., Møller, A., & Vardoulakis, D. (2015). In defense of soundness: a manifesto. *Communications of the ACM*, 58(2), 44–46. <https://doi.org/10.1145/2644805>
- LLVM project contributors. (2020). Loop Fission Interference Graph (FIG) [Accessed: 2025-06-23]. <https://reviews.llvm.org/D73801>
- LLVM project contributors. (2025, May). *LLVM Loop Terminology (and Canonical Forms) – LLVM Compiler Infrastructure* (Version last updated on 2025-05-06) [Accessed: 2025-03-06]. Technical documentation. LLVM Project. <https://llvm.org/docs/LoopTerminology.html>
- Lommen, N., & Giesl, J. (2023). Targeting Completeness: Using Closed Forms for Size Bounds of Integer Programs. In *Frontiers of Combining Systems* (pp. 3–22). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-43369-6_1
- Mahboubi, A., & Tassi, E. (2022, September). *Mathematical Components*. Zenodo. <https://doi.org/10.5281/zenodo.7118596>
- Marion, J.-Y. (2011). A Type System for Complexity Flow Analysis. *2011 IEEE 26th Annual Symposium on Logic in Computer Science*, 123–132. <https://doi.org/10.1109/lics.2011.41>
- Marion, J.-Y., & Moyen, J.-Y. (2000). Efficient First Order Functional Program Interpreter with Time Bound Certifications. In *Logic for Programming and Automated Reasoning* (pp. 25–42). Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-44404-1_3
- Mars Climate Orbiter Mishap Investigation Board. (1999). *Mars Climate Orbiter Mishap Investigation Board, Phase I Report, November 10, 1999* (Technical report). NASA. https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf
- Mastroeni, I., & Pasqua, M. (2019). Statically analyzing information flows: an abstract interpretation-based hyperanalysis for non-interference. *Proceedings of the 34th*

- ACM/SIGAPP Symposium on Applied Computing, 2215–2223. <https://doi.org/10.1145/3297280.3297498>
- Mathematical Components development team. (2024, December). *The Mathematical Components Library* (Version 2.3.0). <https://github.com/math-comp/math-comp>
- McCarthy, J. A., Fetscher, B., New, M. S., Feltey, D., & Findler, R. B. (2018). A Coq library for internal verification of running-times. *Science of Computer Programming*, 164, 49–65. <https://doi.org/10.1016/j.scico.2017.05.001>
- Mehta, S., Lin, P., & Yew, P. (2014). Revisiting loop fusion in the polyhedral framework. In J. E. Moreira & J. R. Larus (Eds.), *Proceedings of the 19th ACM SIGPLAN symposium on Principles and practice of parallel programming* (pp. 233–246). ACM. <https://doi.org/10.1145/2555243.2555250>
- Meyer, B. (1988). *Object-oriented software construction* (Vol. 1). Prentice Hall International.
- Microsoft. (2025, March). *Boogie* (Version 3.5.1). <https://github.com/boogie-org/boogie>
- Mogbil, V. (2012, November). *Complexité en logique linéaire, et logique linéaire en complexité implicite* [Habilitation à diriger des recherches en informatique (HDR)]. Université Paris XIII. <http://lipn.univ-paris13.fr/~mogbil/hdr/mogbil-hdr-memoire.pdf>
- Molina, F., Ponzio, P., Aguirre, N., & Frias, M. (2021). EvoSpex: An Evolutionary Algorithm for Learning Postconditions. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 1223–1235. <https://doi.org/10.1109/ICSE43902.2021.00112>
- Møller, A. (2024, August). Static Program Analysis[†] [Accessed: 2025-02-08]. Lecture notes from Marktoberdorf Summer School 2024, Herrsching am Ammersee, Germany.
- Møller, A., & Schwartzbach, M. I. (2024). *Static Program Analysis*. Department of Computer Science, Aarhus University. <https://cs.au.dk/~amoeller/spa/spa.pdf>
- Montoya, A. F. (2017a). *Cost Analysis of Programs Based on the Refinement of Cost Relations* [PhD thesis]. Technische Universität Darmstadt. <http://tuprints.ulb.tu-darmstadt.de/6746/>
- Montoya, A. F. (2017b). Experimental evaluation of Cost Analysis of Programs Based on the Refinement of Cost Relations [Accessed: 2025-03-08]. <https://aeflores.github.io/CoFloCo/experimentsPhD>
- Montoya, A. F. (2017c). Experimental evaluation of Cost Analysis of Programs Based on the Refinement of Cost Relations [Accessed: 2025-09-15]. <https://aeflores.github.io/CoFloCo/experimentsPhD/>
Results of the experimental evaluation performed in the PhD thesis.
- Moser, G., & Schneckenreither, M. (2018). Automated amortised resource analysis for term rewrite systems. *Functional and Logic Programming (FLOPS 2018)*, 214–229. https://doi.org/10.1007/978-3-319-90686-7_14
- Moyen, J.-Y. (2017, February). *Implicit Complexity in Theory and Practice* [Habilitation à Diriger des Recherches (HDR)]. University of Copenhagen. https://lipn.univ-paris13.fr/~moyen/papiers/Habilitation_JY_Moyen.pdf
- Moyen, J.-Y., & Rubiano, T. (2016, April). *Detection of Non-Size Increasing Programs in Compilers* [At Developments in Implicit Computational Complexity (DICE 2016)]. https://lipn.univ-paris13.fr/DICE2016/Abstracts/paper_2.pdf

- Moyen, J.-Y., Rubiano, T., & Seiller, T. (2017a). Loop Quasi-Invariant Chunk Detection. In *Automated Technology for Verification and Analysis* (pp. 91–108). Springer International Publishing. https://doi.org/10.1007/978-3-319-68167-2_7
- Moyen, J., Rubiano, T., & Seiller, T. (2017b). Loop Quasi-Invariant Chunk Motion by peeling with statement composition. In G. Bonfante & G. Moser (Eds.), *Electronic Proceedings in Theoretical Computer Science* (pp. 47–59, Vol. 248). Open Publishing Association. <https://doi.org/10.4204/EPTCS.248.9>
- Müller, P. (2024, August). Ownership in program verification – from separation logic to Rust and back[†] [Accessed: 2025-04-19].
Lecture notes from Marktoberdorf Summer School 2024, Herrsching am Ammersee, Germany.
- Myers, A. C. (1999). JFlow: practical mostly-static information flow control. *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 228–241. <https://doi.org/10.1145/292540.292561>
- Nelson, L., Bornholt, J., Krishnamurthy, A., Torlak, E., & Wang, X. (2020). Noninterference specifications for secure systems. *ACM SIGOPS Operating Systems Review*, 54(1), 31–39. <https://doi.org/10.1145/3421473.3421478>
- Ngo, V. C., Carbonneaux, Q., & Hoffmann, J. (2018). Bounded Expectations: Resource Analysis for Probabilistic Programs. *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 496–512. <https://doi.org/10.1145/3192366.3192394>
- Nguyen, T., Antonopoulos, T., Ruef, A., & Hicks, M. (2017). Counterexample-guided approach to finding numerical invariants. *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 605–615. <https://doi.org/10.1145/3106237.3106281>
- Nguyen, T., Kapur, D., Weimer, W., & Forrest, S. (2014). DIG: A dynamic invariant generator for polynomial and array invariants. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 23(4), 1–30. <https://doi.org/10.1145/2556782>
- Nguyen, T., Nguyen, K., & Duong, H. (2022). SymInfer: inferring numerical invariants using symbolic states. *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 197–201. <https://doi.org/10.1145/3510454.3516833>
- Nielson, F., Nielson, H. R., & Hankin, C. (2010). *Principles of program analysis*. Springer.
- Niggl, K.-H., & Wunderlich, H. (2006). Certifying Polynomial Time and Linear/Polynomial Space for Imperative Programs. *SIAM Journal on Computing*, 35(5), 1122–1147. <https://doi.org/10.1137/s0097539704445597>
- Niu, Y., Sterling, J., Grodin, H., & Harper, R. (2022). A cost-aware logical framework. *Proceedings of the ACM on Programming Languages*, 6(POPL), 1–31. <https://doi.org/10.1145/3498670>
- Ölveczky, P. C. (2017). *Designing Reliable Distributed Systems*. Springer London. <https://doi.org/10.1007/978-1-4471-6687-0>
- oneTBB Contributors. (2025, March). *oneAPI Threading Building Blocks (oneTBB)* (oneTBB 2022.1.0) [Accessed: 2025-03-09]. Technical documentation. Intel. <https://uxlfoundation.github.io/oneTBB>

- OpenMP Architecture Review Board. (2024, November). *OpenMP Application Programming Interface* (Version 6.0 November 2024) [Accessed: 2025-03-08]. API documentation. OpenMP Architecture Review Board. <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>
- Palkowski, M., Klimek, T., & Bielecki, W. (2015). TRACO: An automatic loop nest parallelizer for numerical applications. In M. Ganzha, L. A. Maciaszek, & M. Paprzycki (Eds.), *Proceedings of the 2015 Federated Conference on Computer Science and Information Systems* (pp. 681–686, Vol. 5). IEEE. <https://doi.org/10.15439/2015F34>
- Patrignani, M., & Garg, D. (2017). Secure Compilation and Hyperproperty Preservation. *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 392–404. <https://doi.org/10.1109/csf.2017.13>
- Péchoux, R. (2020, December). *Complexité implicite : bilan et perspectives* [Habilitation à Diriger des Recherches (HDR)]. Université de Lorraine. <https://hal.univ-lorraine.fr/tel-02978986v3/file/HDR-RP.pdf>
- Pierce, B. C., Azevedo de Amorim, A., Casinghino, C., Gaboardi, M., Greenberg, M., Hrițcu, C., Sjöberg, V., Tolmach, A., & Yorgey, B. (2024). *Programming Language Foundations* (B. C. Pierce, Ed.; Version 6.7, Vol. 2). <https://softwarefoundations.cis.upenn.edu/plf-current/index.html>
- Pierce, B. C., Azevedo de Amorim, A., Casinghino, C., Gaboardi, M., Greenberg, M., Hrițcu, C., Sjöberg, V., & Yorgey, B. (2025). *Logical Foundations* (B. C. Pierce, Ed.; Version 6.7, Vol. 1). <https://softwarefoundations.cis.upenn.edu/lf-current/index.html>
- Piessens, F. (2024, August). Software security across abstraction layers[†] [Accessed: 2025-02-15].
Lecture notes from Marktoberdorf Summer School 2024, Herrsching am Ammersee, Germany.
- Popeea, C., & Chin, W.-N. (2007). Inferring Disjunctive Postconditions. *Advances in Computer Science - ASIAN 2006. Secure Software and Related Issues*, 331–345. https://doi.org/10.1007/978-3-540-77505-8_26
- Popeea, C., & Chin, W.-N. (2010). Dual analysis for proving safety and finding bugs. *Proceedings of the 2010 ACM Symposium on Applied Computing*, 2137–2143. <https://doi.org/10.1145/1774088.1774538>
- Pophale, S. (2023). Introduction to OpenMP Offload: Part I [Accessed: 2025-03-08]. https://www.olcf.ornl.gov/wp-content/uploads/OpenMP_Introduction_to_Offload_Part1.pdf
OpenMP training series at the National Energy Research Scientific Computing Center (NERSC).
- Pouchet, L.-N., & Yuki, T. (2016, February). *PolyBench/C 4.2* (Version 4.2). <https://sourceforge.net/projects/polybench/files/>
- Prema, S., Nasre, R., Jehadeesan, R., & Panigrahi, B. (2019). A study on popular auto-parallelization frameworks. *Concurrency and Computation: Practice and Experience*, 31(17), e5168. <https://doi.org/10.1002/cpe.5168>
- Pugh, W. (1991). The Omega test: a fast and practical integer programming algorithm for dependence analysis. *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 4–13. <https://doi.org/10.1145/125826.125848>

- Aubert, C., Rubiano, T., Rusch, N., & Seiller, T. (2023f). pymwp at Python Package Index [Accessed: 2025-06-23]. <https://pypi.org/project/pymwp/>
- Quinlan, D., Liao, C., Panas, T., Matzke, R., Schordan, M., Vuduc, R., & Yi, Q. (2019, September). *ROSE User Manual: A Tool for Building Source-to-Source Translators Draft User Manual (version 0.9.11.115)* [Accessed: 2025-06-23]. Lawrence Livermore National Laboratory. <http://rosecompiler.org/uploads/ROSE-UserManual.pdf>
- Rauchwerger, L., & Padua, D. A. (1995). Parallelizing While Loops for Multiprocessor Systems. *Proceedings of the 9th International Symposium on Parallel Processing*, 347–356.
- Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society*, 74(2), 358–366. <https://doi.org/10.1090/S0002-9947-1953-0053041-6>
- Rinard, M. C., & Diniz, P. C. (1996). Commutativity analysis: a new analysis framework for parallelizing compilers. *ACM SIGPLAN Notices*, 31(5), 54–67. <https://doi.org/10.1145/249069.231390>
- Rivest, R. L. (1990). Cryptography. In *Algorithms and complexity* (pp. 717–755). Elsevier. <https://doi.org/10.1016/b978-0-444-88071-0.50018-7>
- Roscoe, A., & Dathi, N. (1987). The pursuit of deadlock freedom. *Information and Computation*, 75(3), 289–327. [https://doi.org/10.1016/0890-5401\(87\)90004-6](https://doi.org/10.1016/0890-5401(87)90004-6)
- Roscoe, A., & Goldsmith, M. (1999). What is intransitive noninterference? *Proceedings of the 12th IEEE Computer Security Foundations Workshop*, 228–238. <https://doi.org/10.1109/csfw.1999.779776>
- Rosenblum, D. S. (1995). A practical approach to programming with assertions. *IEEE Transactions on Software Engineering*, 21(1), 19–31. <https://doi.org/10.1109/32.341844>
- Rosendahl, M. (1989). Automatic complexity analysis. *Proceedings of the fourth international conference on Functional programming languages and computer architecture - FPCA '89*, 144–156. <https://doi.org/10.1145/99370.99381>
- Rubiano, T. (2017, December). *Implicit Computational Complexity and Compilers. (Complexité Implicite et compilateurs)* [PhD thesis]. Université Sorbonne Paris Cité. <http://tel.archives-ouvertes.fr/tel-02362912>
- Rusch, N. (2022). Formally Verified Resource Bounds Through Implicit Computational Complexity. *Companion Proceedings of the 2022 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 17–20. <https://doi.org/10.1145/3563768.3565545>
- Rusch, N. (2023, March). Presentation of “mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity” [Accessed: 2025-09-14]. <https://www.cs.unibo.it/~dallago/SCOSEM/>
Presentation at the SCOT Seminar on Semantic and Formal Approaches to Complexity.
- Rusch, N. (2025a). *Polynomial Postconditions via mwp-Bounds* [Preprint].
- Rusch, N. (2025b, July). *Applied Implicit Computational Complexity (Extended Abstract)* [At the ECOOP 2025 Doctoral Symposium]. <https://nkrusch.github.io/dissertation/abstract.pdf>
- Rusch, N. (2025d, September). *Source code of "Applied Implicit Computational Complexity"*. GitHub. <https://github.com/nkrusch/dissertation>

- Rushby, J. (1992). *Noninterference, transitivity, and channel-control security policies* (Technical report). SRI International, Computer Science Laboratory Menlo Park. <https://www.csl.sri.com/~rushby/papers/csl-92-2-long.pdf>
- Ryan, G., Wong, J., Yao, J., Gu, R., & Jana, S. (2020). CLN2INV: Learning Loop Invariants with Continuous Logic Networks. *8th International Conference on Learning Representations, ICLR 2020*. <https://openreview.net/forum?id=HJlfuTEtvB>
- Sabelfeld, A., & Myers, A. C. (2003). Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1), 5–19. <https://doi.org/10.1109/JSAC.2002.806121>
- Sabelfeld, A., & Sands, D. (2009). Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5), 517–548. <https://doi.org/10.3233/jcs-2009-0352>
- Sankaranarayanan, S., Sipma, H. B., & Manna, Z. (2004). Non-linear loop invariant generation using Gröbner bases. *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 318–329. <https://doi.org/10.1145/964001.964028>
- Schiebel, F., Sattler, F., Schubert, P. D., Apel, S., & Bodden, E. (2024). Scaling Interprocedural Static Data-Flow Analysis to Large C/C++ Applications: An Experience Report. *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, 313, 36:1–36:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.36>
- Schneider, F. B., Morrisett, G., & Harper, R. (2001). A language-based approach to security. In *Informatics* (pp. 86–101). Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-44577-3_6
- Seiller, T. (2016). Interaction Graphs: Full Linear Logic. In M. Grohe, E. Koskinen, & N. Shankar (Eds.), *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science* (pp. 427–436). ACM. <https://doi.org/10.1145/2933575.2934568>
- Sergey, I. (2014, August). *Programs and Proofs: Mechanizing Mathematics with Dependent Types* [Lecture notes with exercises]. Zenodo. <https://doi.org/10.5281/zenodo.4996238>
- Shankar, N. (2023, May). Speaking Logic [Accessed: 2025-04-19]. <https://fm.csl.sri.com/SSFT23/speaklogicV12.pdf>
Lecture notes from the Twelfth Summer School on Formal Techniques, Menlo College, California.
- Si, X., Dai, H., Raghothaman, M., Naik, M., & Song, L. (2018). Learning Loop Invariants for Program Verification. *Advances in Neural Information Processing Systems 31*, 31, 7762–7773. <https://papers.nips.cc/paper/8001-learning-loop-invariants-for-program-verification>
- Singal, G., Gopalani, D., Kushwaha, R., & Badal, T. (2019). Automatic Parallelization of C Code Using OpenMP. In *Emerging Technologies in Computer Engineering: Microservices in Big Data Analytics* (pp. 298–309). Springer. https://doi.org/10.1007/978-981-13-8300-7_25
- Sinn, M., Zuleger, F., & Veith, H. (2017, January). *Complexity and Resource Bound Analysis of Imperative Programs Using Difference Constraints* (Vol. 59). Springer Science and Business Media. <https://doi.org/10.1007/s10817-016-9402-4>
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

- Slot, C. F., & van Emde Boas, P. (1984). On Tape Versus Core; An Application of Space Efficient Perfect Hash Functions to the Invariance of Space. In R. A. DeMillo (Ed.). ACM. <https://doi.org/10.1145/800057.808705>
- Smolka, G. (2025). *Modeling and Proving in Computational Type Theory Using the Coq Proof Assistant* [Textbook under Construction, version of April 7, 2025]. <https://www.ps.uni-saarland.de/~smolka/drafts/mpcctt.pdf>
- Spivey, J. M., & Abrial, J.-R. (1992). *The Z notation* (Vol. 29). Prentice Hall.
- Spiwack, A., Kiss, C., Bernardy, J.-P., Wu, N., & Eisenberg, R. A. (2022). Linearly qualified types: Generic inference for capabilities and uniqueness. *Proceedings of the ACM on Programming Languages*, 6(ICFP), 137–164. <https://doi.org/10.1145/3547626>
- Srikanth, A., Sahin, B., & Harris, W. R. (2017). Complexity verification using guided theorem enumeration. *ACM SIGPLAN Notices*, 52(1), 639–652. <https://doi.org/10.1145/3093333.3009864>
- Stefan, D., Russo, A., Buiras, P., Levy, A., Mitchell, J. C., & Mazières, D. (2012). Addressing covert termination and timing channels in concurrent information flow systems. *ACM SIGPLAN Notices*, 47(9), 201–214. <https://doi.org/10.1145/2398856.2364557>
- Steffen, M. (2020, September). *Static analysis and all that[†]* (Autumn 2020) [Accessed: 2025-02-04]. University of Oslo. <https://www.uio.no/studier/emner/matnat/ifi/IN5440/h20/script/intro-script.pdf>
- Lecture notes for course IN5440 “Selected topics in static analysis”.
- Suomela, J., Laine, S., Lehtinen, J., Laaksonen, A., Salmi, R., Lievonen, H., & Schultheis, E. (2025). *Programming Parallel Computers* (Version of 2025) [Accessed: 2025-05-07]. Lecture notes of the Aalto University course CS-E4580 “Programming Parallel Computers”. Aalto University. <https://ppc.cs.aalto.fi>
- Sutcliffe, G. (2024). Stepping Stones in the TPTP World. In C. Benz Müller, M. Heule, & R. Schmidt (Eds.), *Proceedings of the 12th International Joint Conference on Automated Reasoning* (pp. 30–50).
- ter Beek, M. H., Chapman, R., Cleaveland, R., Garavel, H., Gu, R., ter Horst, I., Keiren, J. J. A., Lecomte, T., Leuschel, M., Rozier, K. Y., Sampaio, A., Seceleanu, C., Thomas, M., Willemse, T. A. C., & Zhang, L. (2024). Formal Methods in Industry. *Formal Aspects of Computing*, 37(1). <https://doi.org/10.1145/3689374>
- ter Beek, M. H., Cleophas, L., Legay, A., Schaefer, I., & Watson, B. W. (2020). X-by-construction: Correctness meets probability. In *Leveraging applications of formal methods, verification and validation: Verification principles* (pp. 211–215). Springer International Publishing. https://doi.org/10.1007/978-3-030-61362-4_11
- ter Beek, M. H., Cleophas, L., Schaefer, I., & Watson, B. W. (2018). X-by-Construction. In *Leveraging Applications of Formal Methods, Verification and Validation. Modeling* (pp. 359–364). Springer International Publishing. https://doi.org/10.1007/978-3-030-03418-4_21
- Termination Portal. (2015). C Integer Programs [Accessed: 2025-05-07]. https://termination-portal.org/wiki/C_Integer_Programs
- The Coq development team. (2022). *Coq Documentation* [Accessed: 2025-06-23]. <https://web.archive.org/web/https://coq.github.io/doc/>
- The Rocq Community. (2025). Awesome Coq [git rev 0ceca5b]. <https://github.com/rocq-community/awesome-coq>

- The Rocq Development Team. (2025a). The Rocq Community [Accessed: 2025-04-14]. <https://rocq-prover.org/community>
- The Rocq Development Team. (2025b, March). *The Rocq Prover Reference Manual* (Version 9.0.0) [Accessed: 2025-04-16]. Technical documentation. The Rocq Core Team. <https://rocq-prover.org/doc/V9.0.0/refman/index.html>
- Torlak, E. (2015, May). Static Analysis [Accessed: 2025-02-09]. <https://courses.cs.washington.edu/courses/cse403/15sp/lectures/L19.pdf>
Lecture notes from CSE 403: Software Engineering, Spring 2015.
- TPDB maintainers. (2022, August). *The Termination Problem Database (TPDB)* (Version 11.3). <https://github.com/TermCOMP/TPDB>
- TPTP.org. (2025). The CADE ATP System Competition [Accessed: 2025-04-15]. <https://tptp.org/CASC/>
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 58(345-363), 5. <https://doi.org/10.1112/plms/s2-42.1.230>
- University System of Georgia (USG). (2024, April). Notice of Data Breach [Accessed: 2025-04-17]. https://www.usg.edu/news/release/notice_of_data_breach
Communications of External Affairs Division, Atlanta - April 15, 2024.
- van den Heuvel, B., Derakhshan, F., & Balzer, S. (2024). Information Flow Control in Cyclic Process Networks. *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, 313, 40:1–40:30. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.40>
- van der Hoeven, J., & Larrieu, R. (2019). Fast Gröbner basis computation and polynomial reduction for generic bivariate ideals. *Applicable Algebra in Engineering, Communication and Computing*, 30(6), 509–539. <https://doi.org/10.1007/s00200-019-00389-9>
- van der Meyden, R., & Zhang, C. (2007). A Comparison of Semantic Models for Noninterference. In *Formal Aspects in Security and Trust* (pp. 235–249). Springer. https://doi.org/10.1007/978-3-540-75227-1_16
- van der Meyden, R., & Zhang, C. (2010). A comparison of semantic models for noninterference. *Theoretical Computer Science*, 411(47), 4123–4147. <https://doi.org/10.1016/j.tcs.2010.08.013>
- Vanoni, G. (2022, June). *On Reasonable Space and Time Cost Models for the λ -Calculus* [PhD thesis]. Università di Bologna. <https://doi.org/10.48676/unibo/amsdottorato/10276>
- Vitorović, A., Tomašević, M. V., & Milutinović, V. M. (2014). Manual Parallelization Versus State-of-the-Art Parallelization Techniques. In A. Hurson (Ed.), *Advances in Computers* (pp. 203–251, Vol. 92). Elsevier. <https://doi.org/10.1016/B978-0-12-420232-0.00005-2>
- Volpano, D., & Smith, G. (1998). Probabilistic noninterference in a concurrent language. *Proceedings. 11th IEEE Computer Security Foundations Workshop (Cat. No. 98TB100238)*, 34–43. <https://doi.org/10.1109/csfw.1998.683153>
- Volpano, D. M., Irvine, C. E., & Smith, G. (1996). A Sound Type System for Secure Flow Analysis. *Journal of Computer Security*, 4(2/3), 167–188. <https://doi.org/10.3233/JCS-1996-42-304>

- von Oheimb, D. (2004). Information Flow Control Revisited: Noninfluence = Noninterference + Nonleakage. In *Computer Security – ESORICS 2004* (pp. 225–243). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-30108-0_14
- Wagner, L., & McLaughlin, S. (2024, June). Proving the correctness of AWS authorization [Accessed: 2025-04-14]. https://reinforce.awsevents.com/content/dam/reinforce/2024/slides/IAM401_Proving-the-correctness-of-AWS-authorization.pdf
Slides of presentation at AWS re:Inforce, Philadelphia, PA 10–12 June 2024.
- Wang, D., Kahn, D. M., & Hoffmann, J. (2020). Raising expectations: Automating expected cost analysis with types. *Proceedings of the ACM on Programming Languages*, 4(ICFP), 1–31. <https://doi.org/10.1145/3408992>
- Wayne, H. (2018). *Let’s Prove Leftpad*. <https://github.com/hwayne/lets-prove-leftpad>
- Wegbreit, B. (1975). Mechanical program analysis. *Communications of the ACM*, 18(9), 528–539. <https://doi.org/10.1145/361002.361016>
- Whitney, T., Sharkey, K., Jones, M., Blome, M., Hogenson, G., & Cai, S. (2021, August). *Parallel Patterns Library (PPL)* (Revision of 2 Aug 2021) [Accessed: 2025-03-09]. Technical article. Microsoft. <https://learn.microsoft.com/en-us/cpp/parallel/concrt/parallel-patterns-library-ppl>
- Wiedijk, F. (2025). Formalizing 100 Theorems [Accessed: 2025-04-15]. <https://www.cs.ru.nl/~freek/100/>
- Wikipedia contributors. (2024). 2024 CrowdStrike-related IT outages [Accessed: 2025-06-23]. https://en.wikipedia.org/wiki/2024_CrowdStrike-related_IT_outages
- Wikipedia contributors. (2025, January). Classified information in the United States – Wikipedia, The Free Encyclopedia [Accessed: 2025-03-03]. https://en.wikipedia.org/wiki/Classified_information_in_the_United_States
- Yao, J., Ryan, G., Wong, J., Jana, S., & Gu, R. (2020). Learning nonlinear loop invariants with gated continuous logic networks. *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 106–120. <https://doi.org/10.1145/3385412.3385986>
- Yu, S., Wang, T., & Wang, J. (2023). Loop Invariant Inference through SMT Solving Enhanced Reinforcement Learning. *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 175–187. <https://doi.org/10.1145/3597926.3598047>
- Zave, P., & Nelson, T. (2023a, May). No More Garbage In: Validating Formal Models [Accessed: 2025-04-19]. <https://fm.csl.sri.com/SSFT23/aLecture1.pdf>
Lecture notes from the Twelfth Summer School on Formal Techniques, Menlo College, California.
- Zave, P., & Nelson, T. (2023b, May). No More Garbage In: Validating Formal Models - A Short Course (Reference) [Accessed: 2025-04-19]. <https://fm.csl.sri.com/SSFT23/reference.pdf>
Lecture notes from the Twelfth Summer School on Formal Techniques, Menlo College, California.
- Zdancewic, S. (2004). Challenges for information-flow security [The 1st International Workshop on the Programming Language Interference and Dependence], 6. <https://www.cis.upenn.edu/~stevez/papers/Zda04.pdf>

- Zhang, C., Hartline, J. D., & Dimoulas, C. (2022). Karp: a language for NP reductions. *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 762–776. <https://doi.org/10.1145/3519939.3523732>
- Zhang, Y., & Mesbah, A. (2015). Assertions are strongly correlated with test suite effectiveness. *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. <https://doi.org/10.1145/2786805.2786858>
- Zhao, B., Li, Y., Han, L., Zhao, J., Gao, W., Zhao, R., & Yahyapour, R. (2018). A Practical and Aggressive Loop Fission Technique. In *Algorithms and Architectures for Parallel Processing* (pp. 66–75). Springer International Publishing. https://doi.org/10.1007/978-3-030-05234-8_9
- Zhao, J., Nagarakatte, S., Martin, M. M. K., & Zdancewic, S. (2013). Formal verification of SSA-based optimizations for LLVM. In H. Boehm & C. Flanagan (Eds.), *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 175–186). ACM. <https://doi.org/10.1145/2491956.2462164>
- Zima, H., & Chapman, B. (1990). *Supercompilers for parallel and vector computers*. ACM.

7 Additional Published Manuscripts

7.1 mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity

 *Implicit computational complexity & static analysis*

Clément Aubert, Thomas Rubiano, Neea Rusch, Thomas Seiller

The 7th International Conference on Formal Structures for Computation and Deduction
(FSCD), 2022

“mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity” © by Clément Aubert, Thomas Rubiano, Neea Rusch, Thomas Seiller.

This work is licensed under a Creative Commons Attribution 4.0 International License.

You should have received a copy of the license along with this work.

If not, see <https://creativecommons.org/licenses/by/4.0/>.

mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity

Clément Aubert ✉ 🏠 

School of Computer and Cyber Sciences, Augusta University, GA, USA

Thomas Rubiano ✉ 🏠

LIPN – UMR 7030 Université Sorbonne Paris Nord, France

Neea Rusch ✉ 🏠 

School of Computer and Cyber Sciences, Augusta University, GA, USA

Thomas Seiller ✉ 🏠 

LIPN – UMR 7030 Université Sorbonne Paris Nord, France
CNRS, Paris, France

Abstract

Implicit Computational Complexity (ICC) drives better understanding of complexity classes, but it also guides the development of resources-aware languages and static source code analyzers. Among the methods developed, the *mwp-flow analysis* [23] certifies polynomial bounds on the size of the values manipulated by an imperative program. This result is obtained by bounding the transitions between states instead of focusing on states in isolation, as most static analyzers do, and is not concerned with termination or tight bounds on values. Those differences, along with its built-in compositionality, make the mwp-flow analysis a good target for determining how ICC-inspired techniques diverge compared with more traditional static analysis methods. This paper's contributions are three-fold: we fine-tune the internal machinery of the original analysis to make it tractable in practice; we extend the analysis to function calls and leverage its machinery to compute the result of the analysis efficiently; and we implement the resulting analysis as a lightweight tool to automatically perform data-size analysis of C programs. This documented effort prepares and enables the development of certified complexity analysis, by transforming a costly analysis into a tractable program, that furthermore decorrelates the problem of deciding if a bound exist with the problem of computing it.

2012 ACM Subject Classification Software and its engineering → Automated static analysis; Theory of computation → Complexity theory and logic; Theory of computation → Logic and verification

Keywords and phrases Static Program Analysis, Implicit Computational Complexity, Automatic Complexity Analysis, Program Verification

Digital Object Identifier 10.4230/LIPICs.FSCD.2022.26

Related Version *Technical report*: <https://hal.archives-ouvertes.fr/hal-03596285> [4]

Supplementary Material *Software (Source Code)*: <https://github.com/statycc/pymwp>

archived at `swb:1:dir:22a4ab0cfad49138981ed25fc2abfe830fb7ccdf`

Software (Documentation and Demo): <https://statycc.github.io/pymwp>

Funding This research is supported by the Th. Jefferson Fund of the Embassy of France in the United States and the FACE Foundation, and has benefited from the research meeting 21453 “Static Analyses of Program Flows: Types and Certificate for Complexity” in Schloss Dagstuhl. Th. Rubiano and Th. Seiller are supported by the Île-de-France region through the DIM RFSI project “CoHOp”.

Acknowledgements The authors wish to express their gratitude to Assya Sellak for her contribution to this work, to the reviewers of previous versions for their comments, and to the FSCD community: in particular, the reviews we received were extremely interesting, and generated new directions and questions, for which we are thankful.



© Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller;
licensed under Creative Commons License CC-BY 4.0

7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022).

Editor: Amy P. Felty; Article No. 26; pp. 26:1–26:23



Leibniz International Proceedings in Informatics
LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

1 Introduction: letting ICC drive the development of static analyzers

Certifying program resource usages is possibly as crucial as the specification of program correctness, since a guaranteed correct program whose memory usage exceeds available resources is, in fact, unreliable. The field of Implicit Computational Complexity (ICC) theory [15] pioneers in “embedding” in the program itself a guarantee of its resource usage, using e.g., bounded recursion [8, 27] or type systems [6, 26]. This field initiated numerous distinct and original approaches, primarily to characterize complexity classes in a machine-independent way, with increasing expressivity, but these approaches have rarely materialized into concrete programming languages or program analyzers: even if, as opposed to traditional complexity, its models are generally expressive enough to write down actual algorithms [30, p. 11], they rarely escape the sphere of academia or extend beyond toy languages, with a few exceptions [5, 22]. However, by abstracting away constant factors and insignificant orders of magnitude, it is frequently conjectured that ICC will allow sidestepping some of the difficult issues one usually has to face when inferring the resource usage of a concrete program.

This work reinforces this conjecture by adjusting, improving and implementing an existing ICC technique, the *mwp-bounds analysis* [23], which certifies that the values computed by an imperative program will be bounded by polynomials in the program’s input. This flow analysis is elegant but computationally costly, and it missed an opportunity to leverage its built-in compositionality: we address both issues by revisiting and expanding the original flow calculus, and further make our point by implementing it on a subset of the C programming language. While the theory has been improved to allow analysis of function definitions and calls – including recursive ones, a feature not widely supported [21, p. 359] –, its integration into the implementation is underway, as we placed primary focus on developing an efficient and implementable technique for program analysis. Implementing a tool along the theory enabled testing improvements in real-life, which in return drove adjustments to the theory.

Our enhanced technique answers positively two questions asked by the authors of the original analysis [23, Section 1.2], namely

1. Can the method be extended to richer languages?
2. Can it lead to powerful and convenient tools?

It also supports the conjecture that ICC can be used to construct concrete tools, but highlights that doing so requires adjusting the theory to make it tractable in practice. This work also provides better insight into the original analysis, by e.g., separating the algorithm to decide the existence of a bound from its evaluation into a concrete bound; and by illustrating its plasticity: while our analysis conservatively extends the original one, it nevertheless greatly alters its internal machinery to ease its implementability. Last but not least, our technique is orthogonal to most static analysis methods, which focus on worst-case resource-usage complexity or termination, while ours establishes that the growth rate of variables values is at most polynomially related to their inputs.

Our paper starts by recalling the “original” *mwp-bounds analysis* [23] – to which we refer for a more gentle introduction – and discuss its limitations (Sect. 2). In Sect. 3, we motivate, introduce and justify two modifications to this original analysis, and state that this calculus can be reduced to the original one. We then extend this analysis along two axis (Sect. 4): we detail how functions calls can be analyzed, and how the structures we implemented allowed to speed up some very costly operations. Finally, Sect. 5 presents and discuss our implementation, and Sect. 6 concludes. The proofs, some additional details on semi-rings and the detail of our benchmarks are in appendix, with the exception of some tedious proofs relative to semi-rings that are only in our technical report [4].

2 Background: the original flow analysis

The original analysis [23] computes a polynomial bound – if it exists – on the sizes (of the value itself) of variables in an imperative **while** programming language, extended with a **loop** operator, by computing for each variable a vector that tracks how it depends on other variables – and the program itself gets assigned a matrix collecting those vectors. While this does not ensure termination, it provides a certificate guaranteeing that the program uses throughout its execution at most a polynomial amount of space, and as a consequence that if it terminates, it will do so in polynomial time.

2.1 Language analyzed: fragments of imperative language

► **Definition 1** (Imperative Language). *Letting natural number variables range over X and Y and boolean expressions over b , we define expressions e and commands C as follows:*

$$\begin{aligned} e &:= X \parallel X - Y \parallel X + Y \parallel X * Y \\ C &:= X = e \parallel \text{if } b \text{ then } C \text{ else } C \parallel \text{while } b \text{ do } \{C\} \parallel \text{loop } X \{C\} \parallel C ; C \end{aligned}$$

where **loop** $X \{C\}$ means “do C X times” and $C;C$ is used for sequentiality (“do C , then C ”). We write “program” for a series of commands composed sequentially.

This language assumes that the program’s inputs are the only variables, and that assigning a value to a variable inside the program is not permitted. Extending flow calculi to those operations has been discussed [23, p. 3] and proven possible [9], but we leave this for future work – in particular, our **C** examples will be of **foo** functions with their variables listed as parameters¹. However, we disallow w.l.o.g. composed expressions of the form $X + Y * Y$, which can always be dealt with in the style of three-address code.

2.2 A flow calculus of mwp-bounds for complexity analysis

Flows characterize controls from one variable to another, and can be, in increasing growth rate, of type 0 – the absence of any dependency – maximum, weak polynomial and polynomial. The bounds on programs written in the syntax of Sect. 2.1 are represented and calculated thanks to vectors and matrices whose coefficients are elements of the mwp semi-ring.

► **Definition 2** (The mwp semi-ring and matrices over it). *Letting $\text{MWP} = \{0, m, w, p\}$ with $0 < m < w < p$, and α, β, γ range over MWP , the mwp semi-ring $(\text{MWP}, 0, m, +, \times)$ is defined with $+$ = max, $\alpha \times \beta = \max(\alpha, \beta)$ if $\alpha, \beta \neq 0$, and 0 otherwise.*

We denote $\mathbb{M}(\text{MWP})$ the matrices over MWP , and, fixing $n \in \mathbb{N}$, M for $n \times n$ matrices over MWP , M_{ij} for the coefficient in the i th row and j th column of M , $\oplus$ for the componentwise addition, and $\otimes$ for the product of matrices defined in a standard way. The 0-element for addition is $0_{ij} = 0$ for all i, j , and the 1-element for product is $1_{ii} = m$, $1_{ij} = 0$ if $i \neq j$, and the resulting structure $(\mathbb{M}(\text{MWP}), 0, 1, \otimes, \oplus)$ is a semi-ring that we simply write $\mathbb{M}(\text{MWP})$. The closure operator $\cdot^$ is $M^* = 1 \oplus M \oplus (M^2) \oplus \dots$, for $M^0 = 1$, $M^{m+1} = M \otimes M^m$.*

¹ Our implementation allows to relax this condition, as exemplified in `inline_variable.c`, without losing any of the results expressed in this paper. Assuming a fixed number of variables, known ahead of time, is mostly a theoretical artifact used to simplify the analysis.

26:4 mwp-Analysis Improvement and Implementation: Realizing Implicit Complexity

$$\begin{array}{c}
 \frac{}{\vdash_{\text{JK}} \mathbf{x}_i : \{1^m\}} \text{E1} \qquad \frac{}{\vdash_{\text{JK}} \mathbf{e} : \{1^w \mid \mathbf{x}_i \in \text{var}(\mathbf{e})\}} \text{E2} \\
 \star \in \{+, -\} \quad \frac{\vdash_{\text{JK}} \mathbf{x}_i : V_1 \quad \vdash_{\text{JK}} \mathbf{x}_j : V_2}{\vdash_{\text{JK}} \mathbf{x}_i \star \mathbf{x}_j : pV_1 \oplus V_2} \text{E3} \qquad \star \in \{+, -\} \quad \frac{\vdash_{\text{JK}} \mathbf{x}_i : V_1 \quad \vdash_{\text{JK}} \mathbf{x}_j : V_2}{\vdash_{\text{JK}} \mathbf{x}_i \star \mathbf{x}_j : V_1 \oplus pV_2} \text{E4}
 \end{array}$$

(a) Rules for assigning vectors to expressions.

$$\begin{array}{c}
 \frac{\vdash_{\text{JK}} \mathbf{e} : V}{\vdash_{\text{JK}} \mathbf{x}_j = \mathbf{e} : 1 \stackrel{j}{\leftarrow} V} \text{A} \qquad \frac{\vdash_{\text{JK}} \mathbf{c}_1 : M_1 \quad \vdash_{\text{JK}} \mathbf{c}_2 : M_2}{\vdash_{\text{JK}} \mathbf{c}_1; \mathbf{c}_2 : M_1 \otimes M_2} \text{C} \\
 \frac{\vdash_{\text{JK}} \mathbf{c}_1 : M_1 \quad \vdash_{\text{JK}} \mathbf{c}_2 : M_2}{\vdash_{\text{JK}} \text{if } \mathbf{b} \text{ then } \mathbf{c}_1 \text{ else } \mathbf{c}_2 : M_1 \oplus M_2} \text{I} \\
 \forall i, M_{ii}^* = m \quad \frac{\vdash_{\text{JK}} \mathbf{c} : M}{\vdash_{\text{JK}} \text{loop } \mathbf{x}_1 \{ \mathbf{c} \} : M^* \oplus \{1^p \rightarrow j \mid \exists i, M_{ij}^* = p\}} \text{L} \\
 \forall i, M_{ii}^* = m \text{ and } \forall i, j, M_{ij}^* \neq p \quad \frac{\vdash_{\text{JK}} \mathbf{c} : M}{\vdash_{\text{JK}} \text{while } \mathbf{b} \text{ do } \{ \mathbf{c} \} : M^*} \text{W}
 \end{array}$$

(b) Rules for assigning matrices to commands.

■ **Figure 1** Original non-deterministic (“Jones-Kristiansen”) flow analysis rules.

Although not crucial to understand our development, details about (strong) semi-rings and the mwp semi-ring, and the construction of a semi-ring whose elements are matrices with coefficients in a semi-ring – so, in particular, $\mathbb{M}(\text{MWP})$ – are given in our technical report [4, A.1 and A.2] and sketched in appendix Appendix A.

Below, we let V_1, V_2 be column vectors with values in MWP, αV_1 be the usual scalar product, and $V_1 \oplus V_2$ be defined componentwise. We write $\{\alpha_i\}$ for the vector with 0 everywhere except for α in its i th row, and $\{\alpha_i^\beta\}$ for $\{\alpha_i\} \oplus \{\beta_j\}$.

Replacing in a matrix M the j th column vector by V is denoted $M \stackrel{j}{\leftarrow} V$. The matrix M with $M_{ij} = \alpha$ and 0 everywhere else is written $\{1^p \rightarrow j\}$, and the set of variables in the expression $\mathbf{e}$ is written $\text{var}(\mathbf{e})$. The assumption is made that exactly n different variables are manipulated throughout the analyzed program, so that n -vectors are assigned to expressions – in a non-deterministic way, to capture larger classes of programs [23, Section 8] – and $n \times n$ matrices are assigned to commands using the rules presented Fig. 1 [23, Section 5].

The intuition is that if $\vdash_{\text{JK}} \mathbf{c} : M$ can be derived, then all the values computed by $\mathbf{c}$ will grow at most polynomially w.r.t. its inputs [23, Theorem 5.3], e.g., will be bounded by $\max(\vec{x}, p_1(\vec{y})) + p_2(\vec{z})$, where p_1 and p_2 are polynomials and $\vec{x}$ (resp. $\vec{y}, \vec{z}$) are m -(resp. w -, p -)annotated variables in the vector for the considered output. Since the derivation system is non-deterministic, multiple matrices and polynomial bounds – that sometimes coincide – may be assigned to the same program. Furthermore, the coefficient at M_{1j} carries quantitative information about the way $\mathbf{x}_i$ depends on $\mathbf{x}_j$, knowing that 0- and m -flows are harmless and without constraints, but that w - and p -flows are more harmful w.r.t. polynomial bounds and need to be handled with care, particularly in loops – hence the condition on the L and W rules. The derivation may fail – some programs may not be assigned a matrix – if at least one of the variables used in the body of a loop depends “too strongly” upon another, making it impossible to ensure polynomial bounds on the loop itself. We will use the following example as a common basis to discuss possible failure, non-determinism, and our improvements.

► **Example 3.** Consider `loop X3 {X2 = X1 + X2}`. The body of the `loop` command admits three different derivations, obtained by applying A to one of the three derivation of the expression $X1 + X2$, that we name π_0 , π_1 and π_2 :

$$\frac{\frac{}{\vdash_{JK} X1 : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash_{JK} X2 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} E1}{\vdash_{JK} X1 + X2 : \begin{pmatrix} p \\ m \\ 0 \end{pmatrix}} E3 \quad \frac{\frac{}{\vdash_{JK} X1 : \begin{pmatrix} m \\ 0 \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash_{JK} X2 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} E1}{\vdash_{JK} X1 + X2 : \begin{pmatrix} m \\ p \\ 0 \end{pmatrix}} E4 \quad \frac{}{\vdash_{JK} X1 + X2 : \begin{pmatrix} w \\ 0 \\ 0 \end{pmatrix}} E2$$

From π_0 , the derivation of `loop X3 {X2 = X1 + X2}` can be completed using A and L, but since L requires having only m coefficients on the diagonal, π_1 cannot be used to complete the derivation, because of the p coefficient in a box below:

$$\frac{\frac{\vdots \pi_0}{\vdash_{JK} X1 + X2 : \begin{pmatrix} p \\ m \\ 0 \end{pmatrix}} A \quad \frac{}{\vdash_{JK} X2 = X1 + X2 : \begin{pmatrix} m & p & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{pmatrix}} L}{\vdash_{JK} \text{loop } X3 \{X2 = X1 + X2\} : \begin{pmatrix} m & p & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{pmatrix}} L \quad \frac{\frac{\vdots \pi_1}{\vdash_{JK} X1 + X2 : \begin{pmatrix} m \\ p \\ 0 \end{pmatrix}} A}{\vdash_{JK} X2 = X1 + X2 : \begin{pmatrix} m & m & 0 \\ 0 & [p] & 0 \\ 0 & 0 & m \end{pmatrix}} A$$

Similarly, using A after π_2 gives a w coefficient on the diagonal and makes it impossible to use L, hence only one derivation for this program exists.

2.3 Limitations and inefficiencies of the mwp analysis

Even if the proof techniques are far from trivial, with only 9 rules and skipping over boolean expressions (observe that the condition b has no impact in the rules I or W), the analysis is flexible and easy to carry out – at least mathematically. It also has inherent limitations: while the technique is sound, it is not complete and programs such as greatest common divisor fail to be assigned a matrix. We will discuss in Sect. 5.2, the benefits and originality of this analysis, but we would now like to stress how it is computationally inefficient, since the non-determinacy makes the analysis costly to carry out and can lead to memory explosions.

Abstracting Example 3, one can see that the base case of non-determinism – e.g., to assign a vector to $X1 \star X2$ – yields vectors $\begin{pmatrix} p \\ m \end{pmatrix}$ (using E1 then E3), $\begin{pmatrix} m \\ p \end{pmatrix}$ (using E1 then E4) and $\begin{pmatrix} w \\ 0 \end{pmatrix}$ (using E2). Since none of those vectors is less than the others, only two strategies are available to analyze a larger program containing $X1 \star X2$: either the derivations for this base case are considered one after the other, or they are all stored in memory at the same time. Considering the derivations for the base case one after the other can lead to a time explosion, as a program of n lines can have 3^n different derivations – as exemplified by `explosion.c`, a simple series of applications – and it is possible that only one of them can be completed, so all must be explored. On the other hand, storing those three vectors and constructing all the matrices in parallel leads to a memory explosion: the analysis for two commands involving 6 variables, with 3 choices – which cannot be simplified as explained previously – would result in 9 matrices of size 6×6 , i.e., 324 coefficients. All in all, a program of n lines with x different variables can require c_1^n different derivations, which can produce up to $(c_2 \times x)^2$ coefficients to store for some constants c_1, c_2 .

Beyond inefficiency, there are additional limitations: while the analysis is naturally compositional, this feature is not leveraged in the original system; furthermore, an occurrence of non-polynomial flows in the matrix causes the analysis to simply stop, thus not capturing failure in a meaningful way. We will discuss our solutions to these deficiencies next.

26:6 mwp-Analysis Improvement and Implementation: Realizing Implicit Complexity

$$\begin{array}{c}
\star \in \{+, -\} \quad \frac{}{\vdash \mathbf{x}_i \star \mathbf{x}_j : (0 \mapsto \{\frac{m}{i}, \frac{p}{j}\}) \oplus (1 \mapsto \{\frac{p}{i}, \frac{m}{j}\}) \oplus (2 \mapsto \{\frac{w}{i}, \frac{w}{j}\})} E^A \\
\frac{}{\vdash \mathbf{x}_i \star \mathbf{x}_j : \{\frac{w}{i}, \frac{w}{j}\}} E^M \quad \frac{}{\vdash \mathbf{x}_i : \{\frac{m}{i}\}} E^S \\
\text{(a) Rules for assigning vectors to expressions.} \\
\frac{}{\vdash \mathbf{e} : V} A \quad \frac{}{\vdash \mathbf{c}_1 : M_1 \quad \vdash \mathbf{c}_2 : M_2}{\vdash \mathbf{c}_1; \mathbf{c}_2 : M_1 \otimes M_2} C \quad \frac{}{\vdash \mathbf{c}_1 : M_1 \quad \vdash \mathbf{c}_2 : M_2}{\vdash \text{if } \mathbf{b} \text{ then } \mathbf{c}_1 \text{ else } \mathbf{c}_2 : M_1 \oplus M_2} I \\
\frac{}{\vdash \mathbf{x}_j = \mathbf{e} : 1 \leftarrow V} V \\
\frac{}{\vdash \mathbf{c} : M}{\vdash \text{loop } \mathbf{x}_1 \{ \mathbf{c} \} : M^* \oplus \{\frac{\infty}{j} \rightarrow j \mid M_{jj}^* \neq m\} \oplus \{\frac{p}{1} \rightarrow j \mid \exists i, M_{ij}^* = p\}} L^\infty \\
\frac{}{\vdash \mathbf{c} : M}{\vdash \text{while } \mathbf{b} \text{ do } \{ \mathbf{c} \} : M^* \oplus \{\frac{\infty}{j} \rightarrow j \mid M_{jj}^* \neq m\} \oplus \{\frac{\infty}{i} \rightarrow j \mid M_{ij}^* = p\}} W^\infty \\
\text{(b) Rules for assigning matrices to commands.}
\end{array}$$

■ **Figure 2** Deterministic improved flow analysis rules.

3 A deterministic, always-terminating, declension of the mwp analysis

The problem of finding a derivation in the original calculus is in NP [23, Theorem 8.1]. But since all the non-determinism is in the rules to assigning a vector, the potentially exponential number of derivations are actually extremely similar. Hence, instead of having the analysis stop when failing to establish a derivation and re-starting from scratch, storing the different vectors and constructing the derivation while keeping all the options open seems to be a better strategy, but, as we have seen, this causes a memory blow-up. We address it by fine-tuning the internal machinery: to represent non-determinism, we let the matrices take as values either functions from choices to coefficients in MWP or coefficients in MWP, so that instead of mapping choices to derivations, all the derivations are represented by the same matrix that internalizes the different choices. Sect. 3.1 discusses this improvement, which results in a notable gain: getting back to the example of Sect. 2.3, a program involving 6 variables, with 3 choices, would now be assigned a (unique) 6×6 matrix that requires 66 coefficients instead of the 324 we previously had – this is because 30 coefficients are “simple” values in MWP, and 6 are functions from a set of choices $\{0, 1, 2\}$ to values in MWP, each represented with 6 coefficients.

For the choices that give coefficients fulfilling the side condition of L or W, the derivation can proceed as usual, but when a particular choice gives a coefficient that violates it, we decided against simply removing it. Instead, to guarantee that all derivations always terminate, we mark that choice by indicating that it would not provide a polynomial bound. This requires extending the MWP semi-ring with a special value ∞ that represents failure in a local way, marking non-polynomial flows, and is detailed in Sect. 3.2. As a by-product, this enables fine-grained information on programs that *do not* have polynomially bounded growth, since the precise dependencies that break this growth rate can be localized.

Taken together (Sect. 3.3), our improvements ensure that exactly one matrix will always be assigned to a program while carrying over the correctness of the original analysis. We give in Fig. 2 the deterministic system we are introducing in full, but will gently introduce it though the remaining parts of this section: note that the rules A, C and I are unchanged, up to the fact that the matrices, sum and product are in a different semi-ring.

3.1 Internalizing non-determinism: the choice data flow semi-rings

Internalizing the choice requires altering the semi-ring used in the analysis: we want to replace the three vectors over MWP that can be assigned to an expression by a single vector over $\{0, 1, 2\} \rightarrow \text{MWP}$ that captures the same three choices. For a program needing to decide p times between the 3 available choices, this means replacing the $3 \times p$ different matrices in $\mathbb{M}(\text{MWP})$ by a single matrix in $\mathbb{M}(\{0, 1, 2\}^p \rightarrow \text{MWP})$. For any strong semi-ring $\mathbb{S}$ and family of sets $(A_i)_{i=1, \dots, p}$, both $A_i \rightarrow \mathbb{S}$ and $\mathbb{M}(\prod_{i=1}^p A_i \rightarrow \mathbb{S})$ are semi-rings, using the usual cartesian product of sets, and there exists an isomorphism $\mathbb{M}(\prod_{i=1}^p A_i \rightarrow \mathbb{S}) \cong \prod_{i=1}^p A_i \rightarrow \mathbb{M}(\mathbb{S})$ [4, A.3]. This dual nature of the semi-ring considered is useful:

- the analysis will now assign an element M of $\mathbb{M}(\prod_{i=1}^p A_i \rightarrow \text{MWP})$ to a program;
- representing M as an element of $\prod_{i=1}^p A_i \rightarrow \mathbb{M}(\text{MWP})$ allows one to use an *assignment* $\vec{a} = (a_1, \dots, a_p) \in \prod_{i=1}^p A_i$ to produce a matrix $M[\vec{a}] \in \mathbb{M}(\text{MWP})$, recovering the mwp-flow that would have been computed by making the choices $a_1, \dots, a_p$ in the derivation.

► **Remark 4.** As the unique degree of non-determinism to assign a matrix to commands is 3, our modification of the analysis flow consists simply of recording the different choices by letting $A_i = \{0, 1, 2\}$ for all $i = 1, \dots, p$ where p is the number of times a choice had to be taken. Starting with Sect. 4, function calls will require potentially different sets A_i .

► **Notation 5.** In the following and in the implementation alike, we will denote a function $(a_1^0 \times \dots \times a_p^0 \mapsto \alpha_0) + \dots + (a_1^k \times \dots \times a_p^k \mapsto \alpha_k)$ in $A^p \rightarrow \text{MWP}$ with $\text{Card}(A) = k$ by, omitting the product, $(\alpha_0 \delta(a_1^0, 0) \dots \delta(a_p^0, p)) + \dots + (\alpha_k \delta(a_1^k, 0) \dots \delta(a_p^k, p))$, with $\delta(i, j) = m$ if the j th choice is i , 0 otherwise. Example 8 will justify and explain this choice.

Our derivation system replaces the E3 and E4 rules with a single rule E^A (“additive”), and splits E2 in two exclusive rules, E^M for “multiplicative” and E^S for “simple” (atomic) expressions – Theorem 11 will prove how they are equivalent.

► **Example 6.** We represent the vectors $\begin{pmatrix} p \\ m \\ 0 \end{pmatrix}$, $\begin{pmatrix} m \\ p \\ 0 \end{pmatrix}$ and $\begin{pmatrix} w \\ w \\ 0 \end{pmatrix}$ from Example 3 with a single vector $\begin{pmatrix} p\delta(0,0)+m\delta(1,0)+w\delta(2,0) \\ m\delta(0,0)+p\delta(1,0)+w\delta(2,0) \\ 0 \end{pmatrix}$, that can be read as $\begin{pmatrix} \{0 \rightarrow p, 1 \rightarrow m, 2 \rightarrow w\} \\ \{0 \rightarrow m, 1 \rightarrow p, 2 \rightarrow w\} \\ 0 \end{pmatrix}$, where we write 0 for $\{0 \mapsto 0, 1 \mapsto 0, 2 \mapsto 0\}^2$. Since in particular³, $\mathbb{M}(\{0, 1, 2\} \rightarrow \text{MWP}) \cong \{0, 1, 2\} \rightarrow \mathbb{M}(\text{MWP})$, the obtained vector can be rewritten as $0 \mapsto \begin{pmatrix} p \\ m \\ 0 \end{pmatrix}, 1 \mapsto \begin{pmatrix} m \\ p \\ 0 \end{pmatrix}, 2 \mapsto \begin{pmatrix} w \\ w \\ 0 \end{pmatrix}$.

3.2 Internalizing failure: de-correlating derivations and bounds

The original analysis stops when detecting a non-polynomial flow, puts an end to the chosen strategy (i.e., set of choices) and restarts from scratch with another one. We adapt the rules so that every derivation can be completed even in the presence of non-polynomial flows, thanks to a new top element, ∞ , representing failure in a local way.

Ignoring our previous modification in this subsection, the semi-ring MWP^∞ we need to consider is $(\text{MWP} \cup \{\infty\}, 0, m, +^\infty, \times^\infty)$, with $\infty > \alpha$ for all $\alpha \in \text{MWP}$, $+^\infty = \max$ as before, and $\alpha \times^\infty \beta = 0$ if $\alpha, \beta \neq \infty$ and α or β is 0, $\max(\alpha, \beta)$ otherwise. This different condition in the definition of $\times^\infty$ ensures that once non-polynomial flows have been detected, they cannot be erased (as $\infty \times^\infty 0 = \infty$).

² The implementation supports both coefficients from MWP and coefficients from $\{0, 1, 2\}^p \rightarrow \text{MWP}$; cf. e.g., a simple assignment example `assign_expression.c`.

³ This is a variant of Lemma 21 [4, A.3]. While the latter lemma applies to algebras of square matrices, a similar result holds for rectangular matrices of a fixed size; the algebraic structure is no longer that of a semi-ring as rectangular matrices do not possess a proper multiplication, but the proof can be adapted to show the existence of an isomorphism of modules between the considered spaces.

The only cases where the original analysis may fail is if the side conditions of L or W (Fig. 1) are not met. We replace those by L^∞ and W^∞ (Fig. 2), which replace the problematic coefficients with ∞ , marking non-polynomial dependencies, and carry on the analysis.

► **Example 7.** The program from Example 3 would now receive three derivations (omitting the one obtained from π_0 , as the resulting matrix is identical):

$$\frac{\frac{\vdots \pi_1}{\vdash X1 + X2 : \begin{pmatrix} m \\ p \\ 0 \end{pmatrix}} \text{ A} \quad \frac{\vdash X2 = X1 + X2 : \begin{pmatrix} m & m & 0 \\ 0 & p & 0 \\ 0 & 0 & m \end{pmatrix}}{\vdash \text{loop } X3 \{X2 = X1 + X2\} : \begin{pmatrix} m & p & 0 \\ 0 & \infty & 0 \\ 0 & p & m \end{pmatrix}} L^\infty}{\vdash \text{loop } X3 \{X2 = X1 + X2\} : \begin{pmatrix} m & p & 0 \\ 0 & \infty & 0 \\ 0 & p & m \end{pmatrix}} L^\infty \quad \frac{\frac{\vdash X1 + X2 : \begin{pmatrix} w \\ w \\ 0 \end{pmatrix}}{\vdash X2 = X1 + X2 : \begin{pmatrix} m & w & 0 \\ 0 & w & 0 \\ 0 & 0 & m \end{pmatrix}} \text{ A} \quad \frac{\vdash X2 = X1 + X2 : \begin{pmatrix} m & w & 0 \\ 0 & w & 0 \\ 0 & 0 & m \end{pmatrix}}{\vdash \text{loop } X3 \{X2 = X1 + X2\} : \begin{pmatrix} m & w & 0 \\ 0 & \infty & 0 \\ 0 & 0 & m \end{pmatrix}} L^\infty}{\vdash \text{loop } X3 \{X2 = X1 + X2\} : \begin{pmatrix} m & w & 0 \\ 0 & \infty & 0 \\ 0 & 0 & m \end{pmatrix}} L^\infty$$

Of course, neither of those two derivations would yield polynomial bound – since they contain ∞ coefficients – but it becomes possible to determine that the last one is “better” – since $\begin{pmatrix} p \\ \infty \\ p \end{pmatrix} > \begin{pmatrix} w \\ \infty \\ 0 \end{pmatrix}$ – and to observe how their “failure” would propagate in larger programs, possibly establishing that one fares better than the other in terms of non-polynomial growths. This could imply, for instance, that particular programs without polynomial bounds could still be considered “reasonable” if they are exponential only in some variables that are known to have smaller values in input.

3.3 Merging the improvements: illustrations and proofs

We prove that our system captures the original system in the sense that set aside ∞ coefficients, both systems agree (Theorem 11), but also that exactly one matrix is produced per program (Theorem 10) – i.e., that we can analyze as many programs as originally, and still be correct regarding the bounds. Before doing so, we would like to give more specifics on our system, by combining the semi-rings and intuitions from the previous two subsections. We have discussed our “axiomatic” (E^A , E^M , E^S) and “loop” rules (L^∞ and W^∞), but remain to discuss the rules for assignment (A), **if** (I) and composition (C) – which is where both improvements meet. Mathematically speaking, adopting the semi-ring defined over matrices with coefficients in $\{0, 1, 2\}^p \rightarrow \text{MWP} \cup \{\infty\}$ is straightforward, and we simply write $\oplus$ and $\otimes$ the operations resulting from merging the two transformations. We discuss in Sect. 4.3 how, however, those operations are computationally costly and how we address this challenge.

► **Example 8.** Using our deterministic system presented in Fig. 2, consider the following:

$$\frac{\frac{\vdash X1 + X2 : V}{\vdash X1 = X1 + X2 : 1 \stackrel{1}{\leftarrow} V} \text{ A} \quad \frac{\frac{\vdash X1 - X3 : V'}{\vdash X1 = X1 - X3 : 1 \stackrel{1}{\leftarrow} V'} \text{ A}}{\vdash \text{if } b \text{ then } \{X1 = X1 + X2\} \text{ else } \{X1 = X1 - X3\} : (1 \stackrel{1}{\leftarrow} V) \oplus (1 \stackrel{1}{\leftarrow} V')} \text{ I}$$

with

$$\begin{aligned} V &= 0 \mapsto \begin{pmatrix} m & p \\ 1 & 2 \end{pmatrix} \oplus 1 \mapsto \begin{pmatrix} p & m \\ 1 & 2 \end{pmatrix} \oplus 2 \mapsto \begin{pmatrix} w & w \\ 1 & 2 \end{pmatrix} \\ V' &= 0 \mapsto \begin{pmatrix} m & p \\ 1 & 3 \end{pmatrix} \oplus 1 \mapsto \begin{pmatrix} p & m \\ 1 & 3 \end{pmatrix} \oplus 2 \mapsto \begin{pmatrix} w & w \\ 1 & 3 \end{pmatrix} \\ 1 \stackrel{1}{\leftarrow} V &\cong \begin{pmatrix} (0 \rightarrow m) \oplus (1 \rightarrow p) \oplus (2 \rightarrow w) & 0 & 0 \\ (0 \rightarrow p) \oplus (1 \rightarrow m) \oplus (2 \rightarrow w) & m & 0 \\ 0 & 0 & m \end{pmatrix} = \begin{pmatrix} m\delta(0,0) \oplus p\delta(1,0) \oplus w\delta(2,0) & 0 & 0 \\ p\delta(0,0) \oplus m\delta(1,0) \oplus w\delta(2,0) & m & 0 \\ 0 & 0 & m \end{pmatrix} \\ 1 \stackrel{1}{\leftarrow} V' &\cong \begin{pmatrix} (0 \rightarrow m) \oplus (1 \rightarrow p) \oplus (2 \rightarrow w) & 0 & 0 \\ (0 \rightarrow p) \oplus (1 \rightarrow m) \oplus (2 \rightarrow w) & m & 0 \\ 0 & 0 & m \end{pmatrix} = \begin{pmatrix} m\delta(0,1) \oplus p\delta(1,1) \oplus w\delta(2,1) & 0 & 0 \\ p\delta(0,1) \oplus m\delta(1,1) \oplus w\delta(2,1) & m & 0 \\ 0 & 0 & m \end{pmatrix} \end{aligned}$$

Some care is needed to perform the addition for the I rule: the choices in the left and right branches are independent, so we must use coefficients in $\{0, 1, 2\}^2 \rightarrow \text{MWP}$ for the 2^3 choices. While the mapping notation would require to use positions to describe which choice is being refereed to, the δ notation makes it immediate, as it encodes in the second value of δ that two choices are considered, numbering the choice in the left branch 0. Hence we can sum the coefficients and obtain the matrix that can be observed in our implementation by analyzing `example7.c`.

► **Example 9.** Our deterministic system now assigns to `loop X3 {X2 = X1 + X2}` from Example 3 the unique matrix

$$\begin{pmatrix} m & (0 \rightarrow p) \oplus (1 \rightarrow m) \oplus (2 \rightarrow w) & 0 \\ 0 & (0 \rightarrow m) \oplus (1 \rightarrow \infty) \oplus (2 \rightarrow \infty) & 0 \\ 0 & (0 \rightarrow p) \oplus (1 \rightarrow 0) \oplus (2 \rightarrow 0) & m \end{pmatrix} = \begin{pmatrix} m & p\delta(0,0) \oplus m\delta(1,0) \oplus w\delta(2,0) & 0 \\ 0 & m\delta(0,0) \oplus \infty\delta(1,0) \oplus \infty\delta(2,0) & 0 \\ 0 & p\delta(0,0) \oplus 0\delta(1,0) \oplus 0\delta(2,0) & m \end{pmatrix}$$

where we observe that

1. only one choice, one assignment, 0, gives a matrix without ∞ coefficient, corresponding to the fact that, in the original system, only π_0 could be used to complete the proof,
2. the choice impacts the matrix locally, the coefficients being mostly the same, independently from the choice,
3. the influence of `X2` on itself is where possible non-polynomial growth rates lies, as the ∞ coefficient are in the second column, second row.

We are now in possession of all the material and intuitions needed to state the correspondence between our system and the original one of Jones and Kristiansen.

► **Theorem 10 (Determinacy and termination).** *Given a program P , there exists unique $p \in \mathbb{N}$ and $M \in \mathbb{M}(\{0, 1, 2\}^p \rightarrow \text{MWP}^\infty)$ such that $\vdash P : M$.*

Proof. The existence of the matrix is guaranteed by the completeness of the rules, as any program written in the syntax presented in Sect. 2.1 can be typed with the rules of Fig. 2. The uniqueness of the matrix is given by the fact that no two rules can be applied to the same command. Details are provided in Appendix B. ◀

► **Theorem 11 (Adequacy).** *If $\vdash P : M$, then for all $\vec{a} \in A^p$, $\vdash_{\text{JK}} P : M[\vec{a}]$ iff $\infty \notin M[\vec{a}]$.*

Proof. The proof uses that P cannot be assigned a matrix in the original calculus iff the deterministic calculus introduce a ∞ coefficient, and from the fact that both calculus coincide in all the other cases. Details are provided in Appendix B. ◀

► **Corollary 12 (Soundness).** *If $\vdash P : M$ and there exists $\vec{a} \in A^p$ such that $\infty \notin M[\vec{a}]$, then every value computed by P is bounded by a polynomial in the inputs.*

Proof. This is an immediate corollary of the original soundness theorem [23, Theorem 5.3] and of Theorem 11. ◀

This proves that the two analyses coincide, when excluding ∞ , and that we can re-use the original proofs. However, our alternative definition should be understood as an important improvement, as it enables a better proof-search strategy while optimizing the memory usage, and hence enables the implementation (Sect. 5). It also lets the programmer gain more fine-grained feedback, and illustrates the flexibility of the analysis: the latter will also be demonstrated by the improvements we discuss in the next section.

4 Extending and improving the analysis: functions and efficiency

To improve this analysis, one could try to extract a tight bound, to certify it, or to port it to a compiler's intermediate representation. Adding constant values is arguably immediate [23, p. 3] but handling pointers, even if technically possible, would probably require significant work. This illustrates at the same time the flexibility of the analysis, and the distance separating ICC-inspired techniques from their usage on actual programs. We decided to narrow this gap along two axes: the first one consists of allowing function definitions and calls in our syntax. It is arguably a small improvement, but illustrates nicely the compositionality of the analysis, and includes recursively defined functions. The second extension intersects the theory and the implementation: it details how our semi-ring structure can be leveraged to maintain a tractable algorithm to compute costly operations on our matrices, and to separate the problem of deciding if a bound exists from computing its form.

4.1 Leveraging compositionality to analyze function calls

Thanks to its compositionality, this analysis can easily integrate functions and procedures, by re-using the matrix and choices of a program implementing the function called. We begin by adding to the syntax the possibility of defining multiple functions and calling them:

► **Definition 13** (Functions). *Letting $\mathbf{R}$ (resp. f) range over variables (resp. function names), we add function calls⁴ to the commands (Def. 1) and allow function declarations:*

$$C := \mathbf{x}i = f(\mathbf{x}1, \dots, \mathbf{x}n) \qquad F := f(\mathbf{x}1, \dots, \mathbf{x}n)\{C; \text{return } R\}$$

In a function declaration, $f(\mathbf{x}1, \dots, \mathbf{x}n)$ is called the header, and the body is simply C (i.e., $\text{return } R$ is not part of the body). A program is now a series of function declarations such that all the function calls refer to previously declared functions – we deal with recursive calls in Sect. 4.2 – and a chunk is a series of commands.

Now, given a function declaration computing f , we can obtain the matrix M_f by analyzing the body of f as previously done. It is then possible to store the assignments $\vec{a}_0, \dots, \vec{a}_k$, for which no ∞ coefficients appear⁵, and to project the resulting matrices to only keep the vector at $\mathbf{R}$ that provides quantitative information about all the possible dependencies of the output variable $\mathbf{R}$ w.r.t. input values, possibly merging choices leading to the same result. After this, we are left with a family $(M_f[\vec{a}_0])|_{\mathbf{R}}, \dots, (M_f[\vec{a}_k])|_{\mathbf{R}}$ of vectors – as the syntax here is restricted to functions with a single output value, even if accommodating multiple return values would be dealt with the same way – that we can re-use when calling the function.

The analysis of the command calling f is then dealt with the F rule below:

$$\frac{}{\vdash \mathbf{x}i = F(\mathbf{x}1, \dots, \mathbf{x}n) : 1 \stackrel{i}{\leftarrow} (((M_f[\vec{a}_0])|_{\mathbf{R}})\delta(0, c) \oplus \dots \oplus ((M_f[\vec{a}_k])|_{\mathbf{R}})\delta(k, c))} \text{F}$$

This rule introduces a choice c over k possible matrices, and it is possible that $k \neq 3$, but this is not an issue, since our semi-ring construction can accommodate any set of choice A .

⁴ Function calls that discard the output – procedures – could also be dealt with easily, but are vacuous in our effect-free, in particular pointer-free, language

⁵ Allowing ∞ coefficients would not change the method described nor its results, but it does not seem relevant to allow calling functions that are not polynomially bounded.

► **Example 14.** Consider the following two programs Q and P :

```

int f(X1, X2){
  while b do {X2=X1+X1};
  return X2;
}
Q =
}

int foo(X1, X2){
  X2=X1+X1;
  X1=f(X2, X2);
}
P =
}

```

We first have $\vdash X2 = X1 + X1 : V$ for $V = \begin{pmatrix} m & p\delta(0,0) \oplus p\delta(1,0) \oplus w\delta(2,0) \\ 0 \end{pmatrix}$, and since $V^* = \begin{pmatrix} m & p\delta(0,0) \oplus p\delta(1,0) \oplus w\delta(2,0) \\ 0 \end{pmatrix}$, applying W^∞ gives $\vdash Q : \begin{pmatrix} m & \infty\delta(0,0) \oplus \infty\delta(1,0) \oplus w\delta(2,0) \\ 0 \end{pmatrix}$. Noting that only one choice gives an ∞ -free matrix, we can now carry on the analysis of P :

$$\begin{array}{c}
\vdots \\
\vdash X2 = X1 + X1 : V \quad \vdash X1 = f(X2, X2) : 1 \xleftarrow{1} ((\frac{w}{m})\delta(0, c)) \quad \text{F} \\
\hline
\vdash P : V \otimes 1 \xleftarrow{1} ((\frac{w}{m})\delta(0, c)) \quad \text{C}
\end{array}$$

In this particular case, the c choice can be discarded, since only one option is available.

Now, to prove that the F rule faithfully extends the analysis (Theorem 17), i.e., preserves Corollary 12, we prove that the analysis of the program “inlining” the function call – as defined below – is, up to some bureaucratic variable manipulation and ignoring some ∞ coefficients, the same as the analysis resulting from using our rule. Intuitively, this mechanism provides the expected result because the choices in the function *do not* affect the program calling it, and because their sets of variables are disjoint – except for the return variable.

► **Definition 15** (In-lining function calls). *Let P be a chunk containing a call to the function f , and F be the function declaration computing the function f . The context $P[\cdot]$, a chunk containing a slot $[\cdot]$, is obtained by replacing in P the function call $Xi=f(X1, \dots, Xn)$, with $X'i=X1; \dots; X'n=Xn; [\cdot] \quad Xi=R$, for $R, X'1, \dots, X'n$ fresh variables added to the header containing the chunk.*

The chunk $\tilde{F}$ is obtained from the body of F by renaming the input variables to $X'1, \dots, X'n$, and the variable returned by F to R . The code $P[F]$ is finally obtained by computing the chunk $\tilde{F}$, and inserting it in place of the symbol $[\cdot]$ in $P[\cdot]$.

That P and $P[F]$ have, at the end of their executions, the same values stored in the variables of P is straightforward in our imperative programming language.

► **Example 16.** The in-lining of Q in P from Example 14 would give the following chunk $\tilde{Q}$ and context $P[\cdot]$, $P[Q]$ being obtained by replacing in the latter $[\cdot]$ with the former:

```

int foo(X1, X2, X'1, R){
  X2=X1+X1;
  X'1=X2;
  [ ]
  X1=R;
}
Q = while b do {R=X'1+X'1};  P[ ] =
}

```

The analysis of P (excluding the function call) and Q is implemented at `example15a.c`, and of $P[Q]$ at `example15b.c`: this latter diverges with Example 14 only up to projection and ∞ -coefficients that are removed by F but not when in-lining the function call.

Now, we need to prove that the matrices $M(P)$ – obtained by analyzing P and using the F rule for $Xi=f(X1, \dots, Xn)$; – and $M(P[F])$ – obtained by analyzing the inlined $P[F]$ – are the same. However, to avoid conflict with the variables and to project the matrices on the relevant values, some bureaucracy is needed: we write $\Pi_P(M(P[F]))$ (resp. $(1 - \Pi_P)(M(P[F]))$) the projection of $M(P[F])$ onto the variables in (resp. *not* in) P . Some non-deterministic choices may appear within the (modified) chunk $\tilde{F}$ inside $P[F]$, i.e.,

26:12 mwp-Analysis Improvement and Implementation: Realizing Implicit Complexity

- the coefficients of $M(P)$ are elements of the semi-ring $\prod_{i=1}^{p+1} A_i \rightarrow \mathbb{M}(\text{MWP})$, with one particular choice corresponding to the F rule – we write the corresponding index i_0 ;
- the coefficients of $M(P[F])$ are elements of the semi-ring $\prod_{i=1}^{p+k} B_i \rightarrow \mathbb{M}(\text{MWP})$, where k choices are made within the chunk $\bar{F}$ – we write the corresponding indexes $j_1, j_2, \dots, j_k$ (note these are in fact consecutive indexes).

We note $\pi : \{1, \dots, p+k\} \rightarrow \{1, \dots, p+1\}$ the projection of the choices in $P[F]$ onto

the corresponding choices in P , i.e., $\pi(j) = \begin{cases} j & \text{if } j < j_1 \\ i_0 & \text{if } j_1 \leq j < j_k \\ j - k + 1 & \text{if } j_k < j \end{cases}$. We note that

each matrix used as axiom in the function call corresponds to a specific assignment on indexes $j_1, \dots, j_k$. We write $\Psi : A_{i_0} \rightarrow \prod_{i=j_1}^{j_k} B_i$ the corresponding injection, extended to $\bar{\Psi} : \prod_{i=1}^{p+1} A_i \rightarrow \prod_{i=0}^{p+k} B_i$ straightforwardly.

► **Theorem 17.** *For all $\vec{a}$ in $\prod_{i=1}^{p+1} A_i$, $(M(P))[\vec{a}] = (1 - \Pi_P)(M(P[F]))[\bar{\Psi}(\vec{a})]$, and for all β in $\prod_{i=0}^{p+k} B_i$ not in the image of $\bar{\Psi}$, $(1 - \Pi_P)(M(P[F]))[\beta]$ contains ∞ .*

Proof. It is sufficient to prove it for the simplest chunk P containing only one command $\mathbf{X1} = \mathbf{f}(\mathbf{X1}, \dots, \mathbf{Xn})$. This comes from the compositional nature of the analysis, as a sequence of commands is assigned the product of the matrices of each individual command. Then, checking the theorem in this case is a straightforward, though tedious (due to keeping track of all indices), computation. ◀

4.2 Integrating recursive calls, the easy way

The question of dealing with self-referential, or recursive, calls, naturally arises when extending to function calls. It turns out that our approach makes such cases easy to handle.

A program implementing a function **rec** calling itself cannot use the F rule presented above as is, since the result of the analysis of **rec** is precisely what we are trying to establish. However, if **rec** takes two input variables $\mathbf{X1}$ and $\mathbf{X2}$ and its return value is assigned to a third variable $\mathbf{X3}$, then we already know that the vector at 3 will need to be replaced by the vector capturing the dependency between $\mathbf{X1}$, $\mathbf{X2}$, and the return variable of **rec** (which we will take to be $\mathbf{X3}$ in our example). The solution consists in replacing the actual values in this vector by variables α, β ranging over values in MWP^∞ , terminating the analysis with those variables, and then to resolve the equation – which is easy given the small size of the MWP^∞ semiring.

As an example⁶, consider the following program and compute the corresponding matrix:

```
int rec(X1, X2){
  X1 = X1 + X2;
  X3 = rec(X1, X2);
  return X3;
}
```

$$= \begin{pmatrix} m\delta(0,0) \oplus p\delta(1,0) \oplus w\delta(2,0) & 0 & 0 \\ p\delta(0,0) \oplus m\delta(1,0) \oplus w\delta(2,0) & m & 0 \\ 0 & 0 & m \end{pmatrix} \otimes 1 \leftarrow \begin{pmatrix} \alpha \\ \beta \\ 0 \end{pmatrix}$$

$$= \begin{pmatrix} m\delta(0,0) \oplus p\delta(1,0) \oplus w\delta(2,0) & 0 & \alpha m\delta(0,0) \oplus \alpha p\delta(1,0) \oplus \alpha w\delta(2,0) \\ p\delta(0,0) \oplus m\delta(1,0) \oplus w\delta(2,0) & m & \alpha p\delta(0,0) \oplus \alpha m\delta(1,0) \oplus \alpha w\delta(2,0) \oplus \beta \\ 0 & 0 & 0 \end{pmatrix}$$

Using the assignments 0, 1 and 2 gives $\begin{pmatrix} m & 0 & \alpha m \\ p & m & \alpha p \oplus \beta \\ 0 & 0 & 0 \end{pmatrix}$, $\begin{pmatrix} p & 0 & \alpha p \\ m & m & \alpha m \oplus \beta \\ 0 & 0 & 0 \end{pmatrix}$ and $\begin{pmatrix} w & 0 & \alpha w \\ 0 & m & \alpha w \oplus \beta \\ 0 & 0 & 0 \end{pmatrix}$, and since the third vector should be equal to $\begin{pmatrix} \alpha \\ \beta \\ 0 \end{pmatrix}$, this gives three systems of equations:

⁶ Where we use variables that are not parameters, following footnote 1, and where our recursive call does not terminate: we are focusing on growth rates and not on termination, and keep the example compact.

$$\left\{ \begin{array}{l} \alpha m = \alpha \\ \alpha p \oplus \beta = \beta \end{array} \right. \quad \left\{ \begin{array}{l} \alpha p = \alpha \\ \alpha m \oplus \beta = \beta \end{array} \right. \quad \left\{ \begin{array}{l} \alpha w = \alpha \\ \alpha w \oplus \beta = \beta \end{array} \right.$$

The smaller solution to the first (resp. second, third) equational system is $\{\alpha = m; \beta = p\}$ (resp. $\{\alpha = p; \beta = p\}$, $\{\alpha = w; \beta = w\}$), and as a consequence, we find two meaningful solutions (all others being larger than those): $\begin{pmatrix} m \\ p \\ 0 \end{pmatrix}$ and $\begin{pmatrix} w \\ w \\ 0 \end{pmatrix}$.

4.3 Taking advantage of polynomial structure to compute efficiently

Ensuring that the analysis is tractable is an important part of our contribution. For a program accepting n different derivations and having k different derivations that cannot be completed, the original flow calculus must run at most $k + 1$ times to find *one* derivation, while our analysis outputs the $k + n$ different derivations in one run, and then sorts them – as discussed next – by listing all the evaluations and looking for ∞ values. In this task, the C rule, that lets building programs from commands, is obviously crucial and consists simply in multiplying two matrices: however, since we are internalizing the choices, those matrices contain a mixture of functions from choices to coefficients in MWP^∞ and of coefficients in MWP . Multiplying such matrices is more costly, but also essential: an 8-line program such as `explosion.c` requires to multiply elements of its matrix 34,992 times⁷. This forces to represent and manipulate the elements of $\prod_{i=1}^p A_i \rightarrow \mathbb{M}(\text{MWP})$ – setting aside ∞ coefficients for a moment – cleverly: simple comparison showed that the improved algorithm presented below made the analysis roughly *five times* faster (Sect. C.3).

As discussed in Notation 5, elements of this semi-ring are represented as *polynomials* w.r.t. the generating set given by the functions $\delta(i, j) : \prod_{i=1}^p A_i \rightarrow \text{MWP}$ defined by $\delta(i, j)(a_1, \dots, a_p) = m$ if $a_j = i$ and $\delta(i, j)(a_1, \dots, a_p) = 0$ otherwise, i.e., an element of $\prod_{i=1}^p A_i \rightarrow \text{MWP}$ is represented as a polynomial $\sum_{i=1}^n \alpha_i \prod_{j=1}^{k_i} \delta(a_{i,j}, b_{i,j})$ with $\alpha_i \in \text{MWP}$.

This basis has an important property: the *monomials* $\alpha_i \prod_{j=1}^{k_i} \delta(a_{i,j}, b_{i,j})$ in a polynomial can be ordered so that the product with another monomial is ordered, i.e., if $\alpha \leq \beta$ and both $\alpha \times \gamma$ and $\beta \times \gamma$ are non-zero, then $\alpha \times \gamma \leq \beta \times \gamma$. This order is leveraged to obtain efficient algorithms, similar to what is done using Gröbner bases for computation of standard polynomials [35]. For instance, the algorithm for multiplication of polynomials uses this property to compute the product of an ordered polynomial P with $\sum_{i=1}^n \alpha_i \prod_{j=1}^{k_i} \delta(a_{i,j}, b_{i,j})$:

1. compute the products $P_i = P \times \alpha_i \prod_{j=1}^{k_i} \delta(a_{i,j}, b_{i,j})$ for all i ;
2. compare and order a list L of all the first elements of those polynomials;
3. append the smallest element to the result and remove it from the corresponding P_i ;
4. insert the (new) first element of P_i to the list L if it exists;
5. if L is non-empty, go back to step 3.

When adding or multiplying polynomials, which consist of monomials, we check if a monomial is contained or included by another, and exclude all redundant cases (cf. **contains** or **includes**). This is also done when inserting monomials. Thus we keep polynomials free of implementation choices that we would otherwise have to handle during evaluation.

⁷ The need to optimize functions is made even more obvious when we discuss benchmarking in Sect. 5.1.

4.4 Deciding the existence of a bound faster thanks to delta graphs

Adopting the $\prod_{i=1}^p A_i \rightarrow \text{MWP}^\infty$ semi-ring permits to complete all derivations simultaneously, but remains to determine if there exists an assignment $\vec{a} \in \prod_{i=1}^p A_i$ s.t. the resulting matrix is ∞ -free, to decide whenever a program accepts a polynomial bound: this is the *evaluation* step. Despite the optimizations detailed above that simplifies the task, this phase remains particularly costly, since the number of assignment grows exponentially w.r.t. the number of choice, which is linear in the number of variables. While this step is necessary (in one form or another) if one wishes to produce the actual mwp matrices certifying polynomial bounds, we implemented a specific data structure to keep track of assignments resulting in ∞ coefficients on the fly, thus allowing the analysis to provide a qualitative answer quickly. This section details how those *delta graphs* allow to immediately determines whenever a polynomial bound exists without having to compute the corresponding matrix, something that was not possible in the original, non-deterministic, calculus.

A delta graph is a graph whose vertices are monomials. The graph is populated during the analysis by adding those monomials that appear with an infinite coefficient – i.e., possible choices leading to ∞ in the resulting matrix. This graph is structured in layers: each layer corresponds to the size of the monomials (the number of deltas) it contains. The intuition is that a monomial – or rather a list of deltas $\delta(_, _)$ – defines a subset of the space $\prod_{i=1}^p A_i$; the less deltas in the monomial, the greater the subspace represented⁸. As we populate the delta graph, we create edges within a given layer to keep track of differences between monomials: we add an edge labeled i between two monomials if and only if they differ only on one delta $\delta(_, i)$ (i.e., one is obtained from the other by replacing the first index of $\delta(_, i)$). This is used to implement a *fusion* method on delta graphs, which simplifies the structure: as soon as a monomial m in layer n has $\text{Card}(A_i) - 1$ outgoing edges labelled i , we can remove all these monomials and insert a shorter monomial in layer $n - 1$, obtained from m by simply removing $\delta(_, i)$. This implements the fact that $\sum_{k=0}^{\text{Card}(A_i)-1} m\delta(k, j) = m$.

Now, remember the delta graph represents the subspace of assignments for which an ∞ appears. If at some point the delta graph is completely simplified (i.e., “fusions” to the graph with a unique monomial consisting in an empty list of $\delta(_, _)$), it means the whole space of assignments is represented and no mwp-bounds can be found. On the contrary, if the analysis ends with a delta graph different from the completely simplified one, at least one assignment exists for which no infinite coefficients appear, and therefore at least one mwp-bound exists. This allows one to answer the question “Is there at least one mwp-bound?” *without actually computing said bounds*. Based on the information collected in the delta graph and the matrix with polynomial coefficients, one can however recover all possible matrix assignments by going through all possible valuations.

This last part is implemented with a specific iterator that leverages the information collected in the delta graph to skip large sets of valuations in a single step. For instance, suppose the monomial $\delta(1, 1)$ lies in the delta graph – i.e., that an infinite coefficient will be reached if the second index is equal to 1. When asked the valuation after $(0, 0, 2, 2)$ (and supposing that $\text{Card}(A_i) = 3$ for all i), our `delta_iterator` will jump directly to $(0, 2, 0, 0)$, skipping all intermediate valuation of the form $(0, 1, a, b)$ in a single step. Similarly, it will jump from $(1, 0, 2, 2)$ to $(1, 2, 0, 0)$, again skipping several valuations at a time, providing a

⁸ Our intuitions here come from the standard topological structure of spaces of infinite sequences, where such a monomial represents a “cylinder set”, i.e., an element of the standard basis for open sets.

faster analysis. Note that the implementation required care, to correctly jump when given additional informations from the delta graph, e.g., to produce $(2, 0, 1, 0)$ as the successor of $(0, 0, 2, 2)$ if $\delta(0, 0)$, $\delta(1, 1)$ and $\delta(0, 2)$ all belong to the delta graph.

5 Implementing, testing and comparing the analysis

Demonstrating the implementability of the improved and extended mwp-bounds analysis requires an implementation. Our open-source solution, packaged through Python Package Index (PyPI) as `pymwp`, is a standalone command line tool, written in `Python`, that automatically performs growth-rate analysis on programs written in a subset of the `C` programming language. For programs that pass the analysis, it produces a matrix corresponding to the input program and a list of valid derivation choices; and for programs that do not have polynomial bounds, it reports infinity. Our motivation for choosing `C` as the language of analysis resulted from its central role and similarity with the original `while` language. `Python` was an ideal choice for the implementation because of its plasticity, collection of libraries, and because it allowed partial reuse of a previous flow analysis tool [3, 31, 32]. The source code is available on Github, along with an online demo, and detailed documentation [33] describing its current supported features and functionality. We now discuss how we tested and assessed it, and how it compares (or, rather *does not* compare) to other similar approaches.

5.1 Experimental evaluation

We allocated extensive focus and effort on testing and profiling our implementation, to ensure the correctness and efficiency of the analysis, and with the terminal objective of obtaining a usable tool. The test suite includes 42 `C` programs, carefully designed to exercise different aspects of the analysis, ranging from basic derivations, to ones producing worst-case behavior (by yielding e.g., dense matrices or exponential number of derivations), and classical examples such as computing the greatest common divisor or exponentiation.

We refer to our benchmarks (presented in Appendix C) for measured analysis results for each program. The most salient aspect is that our analysis is extremely fast (the time is measured in *milliseconds*) despite important numbers of function calls (in the 10k range, excluding builtin `Python` language calls, for 10-lines programs). Even examples tailored to stress our implementation cannot make the analysis go over *4 seconds*. We cannot compare our implementation with implementations of the original analysis, since it has never been implemented, and (according to our attempts) cannot be implemented in any realistic manner.

5.2 Related tools and incompatible metrics

This work was inspired by the series of works of the flow analysis from the “Copenhagen school” [11, 24]. The overall flow analysis approach is related in spirit to abstract interpretation [13, 14]; that bounds *transitions* between states (e.g., commands) instead of states [24]. This approach shaped the implementation of tools detecting loop quasi-invariants [31, 32].

Other communities share a similar goal of inferring resource-usage. Complexity analyzers such as SPEED [19] for `C++`, COSTA [1] for `Java` bytecode, ComplexityParser [21] for `Java`, Resource Aware ML for `OCaml` [29] or Cerco [2] and Verasco [25] for `C` generate (certified) cost or runtime analysis on (subsets of) imperative programming languages. Embracing such a large diversity is difficult, but our technique is different from existing implementations and tools: most of them focus on worst-case resource-usage complexity or termination, while we

26:16 mwp-Analysis Improvement and Implementation: Realizing Implicit Complexity

are interested in upper-bounds on the final values of program variables, i.e., we focus on *growth* instead of actual values. This makes the comparison with our approach difficult, but highlights at the same time its uniqueness in today's landscape of static analyzers.

Further, our approach provides other desirable properties:

1. it is compositional, which allows one to “hot-plug” bounds of previously analyzed functions without additional work,
2. it is modular, as the internal machinery can be altered – as in this paper – without having to re-develop the theory,
3. it is language-independent, as it reasons abstractly on imperative languages, but can be applied to real programs, as our implementation illustrates, and should extend to more complex languages,
4. it is lightweight and programmer-friendly, as it is fast, does not require annotations or to record value ranges,
5. it studies growth independently from e.g., iteration bounds, thus sidestepping difficult cases that worst-case analysis has to tackle, and
6. it may enable tight bounds on programs, as it has been done recently [10] for a similar analysis [11].

In particular compositionality is a highly desirable property – because otherwise the analysis needs to be re-run on programs or API whenever embedded into different pieces of software – yet difficult to achieve by most other approaches, as discussed and partially remedied recently [12]. While we suppose one approach could be used to derive the result obtained by the other, we do believe the originality of our pioneering ICC-based approach may inspire new and original directions in static program analysis.

6 Conclusion: limitations, strengths and future work

This work attempts to illustrate the usefulness and applicability of ICC results, but also the need to refine and adapt them. We showed that the mwp-flow analysis as originally described cannot scale to programs in a real programming language: while the considered analysis is definitely powerful and elegant, its mathematical nature let some costly operations go unchecked. However we have shown that, extended and coupled to optimizations techniques, its result enable the development of a novel and original static analysis technique on imperative programs, focused on *growth* rather than on termination or worst-case bounds.

This work is a proof of concept and it has limitations, both theoretical and practical: the theory is missing memory uses, pointers, and arrays and the supported feature set of the implementation could be extended. But instead of focusing on what this analysis *cannot* perform, we would like to stress that all the tools are in place to perform similar analysis on intermediate representations of code in compilers, which will naturally simplify the task of fitting richer program syntax to our analysis, and brings this technique yet another step closer to practical use cases.

One of our next steps include certifying the analysis using the Coq proof assistant [34], and implementing the analysis in certified tools such as the CompCert compiler [28] (or, more precisely, its static single assignment version [7]) or certified-llvm [36]. The plasticity of both compilers and of the implemented analysis should facilitate porting our results and approaches to support further programming languages in addition to C. As complexity analysis is notably difficult in Coq [20], we believe a push in this direction would be welcome, and that ICC provides all the needed tools for it.

Another direction is to explore the possibility for our analysis to focus on the *final* values of variables instead of tracking them throughout the whole program. Indeed, recall from Sect. 3.2 that our semi-ring is such that $\infty \times^\infty 0 = \infty$, but another valid choice would have been to pick $\infty \times^\infty 0 = 0$ [4, A.4]. In this case, it seems that if some non-polynomial growth is caused by a variable that is then “thrown away” (overridden), then a program could still pass the analysis: whether this lead gives relevant results is yet to determine, but it would be another nice illustration of the plasticity of this analysis.

Last but not least, working on finer comparison with other static analyzers [18] could be useful. We have stressed in Sect. 5.2 how such comparison was uneasy, as the finality of our tool is not directly comparable with any other analyzer we know of. However, some tools such as AProVE [17] or CoFloCo [16] provide polynomial upper bounds for C programs, and we could assess e.g., on the Termination Problems Data Base whether `pymwp` can analyze as many problems and how often all three analyzers agree. The motivation for the original mwp-analysis was to develop resource analysis for distinguishing feasible problems and to work at the boundary of undecidability, but this could actually be one of `pymwp`’s strength as a *pre-processor* to other static analyzers, to save them from running costly analysis on programs known to be unfeasible. This fast analysis and the compositionality of our tool could also, on longer term, be useful to construct IDE plug-ins that provide low-latency feedback to the programmer.

References

- 1 Elvira Albert, Puri Arenas, Samir Genaim, German Puebla, and Damiano Zanardini. COSTA: design and implementation of a cost and termination analyzer for java bytecode. In Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem P. de Roever, editors, *Formal Methods for Components and Objects, 6th International Symposium, FMCO 2007, Amsterdam, The Netherlands, October 24-26, 2007, Revised Lectures*, volume 5382 of *LNCS*, pages 113–132. Springer, 2007. doi:10.1007/978-3-540-92188-2_5.
- 2 Roberto M. Amadio, Nicholas Ayache, François Bobot, Jaap Boender, Brian Campbell, Ilias Garnier, Antoine Madet, James McKinna, Dominic P. Mulligan, Mauro Piccolo, Randy Pollack, Yann Régis-Gianas, Claudio Sacerdoti Coen, Ian Stark, and Paolo Tranquilli. Certified complexity (cerco). In Ugo Dal Lago and Ricardo Peña, editors, *Foundational and Practical Aspects of Resource Analysis - Third International Workshop, FOPARA 2013, Bertinoro, Italy, August 29-31, 2013, Revised Selected Papers*, volume 8552 of *LNCS*, pages 1–18. Springer, 2013. doi:10.1007/978-3-319-12466-7_1.
- 3 Clément Aubert, Thomas Rubiano, Neeva Rusch, and Thomas Seiller. Lqicm on c toy parser. URL: https://github.com/statycc/LQICM_On_C_Toy_Parser.
- 4 Clément Aubert, Thomas Rubiano, Neeva Rusch, and Thomas Seiller. mwp-analysis improvement and implementation: Realizing implicit computational complexity. Preliminary technical report, March 2022. URL: <https://hal.archives-ouvertes.fr/hal-03596285>.
- 5 Martin Avanzini and Ugo Dal Lago. Automating sized-type inference for complexity analysis. *Proc. ACM Program. Lang.*, 1(ICFP):43:1–43:29, 2017. doi:10.1145/3110287.
- 6 Patrick Baillot and Kazushige Terui. Light types for polynomial time computation in lambda-calculus. In *LICS*, pages 266–275. IEEE Computer Society, 2004. doi:10.1109/LICS.2004.1319621.
- 7 Gilles Barthe, Delphine Demange, and David Pichardie. Formal verification of an SSA-based middle-end for compcert. *ACM Trans. Program. Lang. Syst.*, 36(1):4:1–4:35, 2014. doi:10.1145/2579080.
- 8 Stephen J. Bellantoni and Stephen Arthur Cook. A new recursion-theoretic characterization of the polytime functions (extended abstract). In S. Rao Kosaraju, Mike Fellows, Avi Wigderson, and John A. Ellis, editors, *STOC*, pages 283–93. ACM, 1992. doi:10.1145/129712.129740.

- 9 Amir M. Ben-Amram. On decidable growth-rate properties of imperative programs. In Patrick Baillot, editor, *Proceedings International Workshop on Developments in Implicit Computational Complexity, DICE 2010, Paphos, Cyprus, 27-28th March 2010*, volume 23 of *EPTCS*, pages 1–14, 2010. doi:10.4204/EPTCS.23.1.
- 10 Amir M. Ben-Amram and Geoff W. Hamilton. Tight polynomial worst-case bounds for loop programs. *Log. Meth. Comput. Sci.*, 16(2), 2020. doi:10.23638/LMCS-16(2:4)2020.
- 11 Amir M. Ben-Amram, Neil D. Jones, and Lars Kristiansen. Linear, polynomial or exponential? complexity inference in polynomial time. In Arnold Beckmann and Costas Dimitracopoulos and Benedikt Löwe, editors, *Logic and Theory of Algorithms, 4th Conference on Computability in Europe, CiE 2008, Athens, Greece, June 15-20, 2008, Proceedings*, volume 5028 of *LNCS*, pages 67–76. Springer, 2008. doi:10.1007/978-3-540-69407-6_7.
- 12 Quentin Carbonneaux, Jan Hoffmann, and Zhong Shao. Compositional certified resource bounds. In David Grove and Stephen M. Blackburn, editors, *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, pages 467–478. ACM, 2015. doi:10.1145/2737924.2737955.
- 13 Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In Robert M. Graham, Michael A. Harrison, and Ravi Sethi, editors, *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*, pages 238–252. ACM, 1977. doi:10.1145/512950.512973.
- 14 Patrick Cousot and Radhia Cousot. Static determination of dynamic properties of recursive procedures. In Erich J. Neuhold, editor, *Formal Description of Programming Concepts: Proceedings of the IFIP Working Conference on Formal Description of Programming Concepts, St. Andrews, NB, Canada, August 1-5, 1977*, pages 237–278. North-Holland, 1977.
- 15 Ugo Dal Lago. A short introduction to implicit computational complexity. In Nick Bezhanishvili and Valentin Goranko, editors, *ESSLLI*, volume 7388 of *LNCS*, pages 89–109. Springer, 2011. doi:10.1007/978-3-642-31485-8_3.
- 16 Antonio Flores-Montoya. Upper and lower amortized cost bounds of programs expressed as cost relations. In John S. Fitzgerald, Constance L. Heitmeyer, Stefania Gnesi, and Anna Philippou, editors, *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, volume 9995 of *LNCS*, pages 254–273, 2016. doi:10.1007/978-3-319-48989-6_16.
- 17 Jürgen Giesl, Cornelius Aschermann, Marc Brockschmidt, Fabian Emmes, Florian Frohn, Jera Fuhs, Carsten Hensel, Carsten Otto, Martin Plücker, Peter Schneider-Kamp, Thomas Ströder, and René Swiderski, Stephanie and Thiemann. Analyzing program termination and complexity automatically with aprove. *J. Autom. Reasoning*, 58(1):3–31, 2017. doi:10.1007/s10817-016-9388-y.
- 18 Jürgen Giesl, Albert Rubio, Christian Sternagel, Johannes Waldmann, and Akihisa Yamada. The termination and complexity competition. In Dirk Beyer, Marieke Huisman, Fabrice Kordon, and Bernhard Steffen, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 25 Years of TACAS: TOOLympics, Held as Part of ETAPS 2019, Prague, Czech Republic, April 6-11, 2019, Proceedings, Part III*, volume 11429 of *LNCS*, pages 156–166. Springer, 2019. doi:10.1007/978-3-030-17502-3_10.
- 19 Sumit Gulwani, Krishna K. Mehra, and Trishul Chilimbi. Speed: Precise and efficient static estimation of program computational complexity. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '09*, pages 127–139, New York, NY, USA, 2009. Association for Computing Machinery. doi:10.1145/1480881.1480898.
- 20 Armaël Guéneau. *Mechanized Verification of the Correctness and Asymptotic Complexity of Programs. (Vérification mécanisée de la correction et complexité asymptotique de programmes)*. PhD thesis, Inria, Paris, France, 2019. URL: <https://tel.archives-ouvertes.fr/tel-02437532>.
- 21 Emmanuel Hainry, Emmanuel Jeandel, Romain Péchoux, and Olivier Zeyen. Complexityparser: An automatic tool for certifying poly-time complexity of Java programs. In Antonio Cerone and

- Peter Csaba Ölveczky, editors, *Theoretical Aspects of Computing - ICTAC 2021 - 18th International Colloquium, Virtual Event, Nur-Sultan, Kazakhstan, September 8-10, 2021, Proceedings*, volume 12819 of *LNCS*, pages 357–365. Springer, 2021. doi:10.1007/978-3-030-85315-0_20.
- 22 Jan Hoffmann, Klaus Aehlig, and Martin Hofmann. Resource aware ML. In P. Madhusudan and Sanjit A. Seshia, editors, *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings*, volume 7358 of *LNCS*, pages 781–786. Springer, 2012. doi:10.1007/978-3-642-31424-7_64.
 - 23 Neil D. Jones and Lars Kristiansen. A flow calculus of *mwp*-bounds for complexity analysis. *ACM Trans. Comput. Log.*, 10(4):28:1–28:41, 2009. doi:10.1145/1555746.1555752.
 - 24 Neil D. Jones and Flemming Nielson. Abstract interpretation: A semantics-based tool for program analysis. In Samson Abramsky, Dov M. Gabbay, and Thomas Stephen Edward Maibaum, editors, *Semantic Modelling*, volume 4 of *Handbook of Logic in Computer Science*, pages 527–636. Oxford University Press, 1995.
 - 25 Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. A formally-verified C static analyzer. In Sriram K. Rajamani and David Walker, editors, *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, pages 247–259. ACM, 2015. doi:10.1145/2676726.2676966.
 - 26 Yves Lafont. Soft linear logic and polynomial time. *Theor. Comput. Sci.*, 318(1):163–180, 2004. doi:10.1016/j.tcs.2003.10.018.
 - 27 Daniel Leivant. Stratified functional programs and computational complexity. In Mary S. Van Deusen and Bernard Lang, editors, *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 325–333. ACM Press, 1993. doi:10.1145/158511.158659.
 - 28 Xavier Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, 2009. doi:10.1145/1538788.1538814.
 - 29 Benjamin Lichtman and Jan Hoffmann. Arrays and references in resource aware ML. In Dale Miller, editor, *2nd International Conference on Formal Structures for Computation and Deduction, FSCD 2017, September 3-9, 2017, Oxford, UK*, volume 84 of *LIPICs*, pages 26:1–26:20. Schloss Dagstuhl, 2017. doi:10.4230/LIPICs.FSCD.2017.26.
 - 30 Jean-Yves Moyen. *Implicit Complexity in Theory and Practice*. Habilitation thesis, University of Copenhagen, 2017. URL: https://lipn.univ-paris13.fr/~moyen/papiers/Habilitation_JY_Moyen.pdf.
 - 31 Jean-Yves Moyen, Thomas Rubiano, and Thomas Seiller. Loop quasi-invariant chunk detection. In Deepak D'Souza and K. Narayan Kumar, editors, *ATVA*, volume 10482 of *LNCS*. Springer, 2017. doi:10.1007/978-3-319-68167-2_7.
 - 32 Jean-Yves Moyen, Thomas Rubiano, and Thomas Seiller. Loop quasi-invariant chunk motion by peeling with statement composition. In Guillaume Bonfante and Georg Moser, editors, *Proceedings 8th Workshop on Developments in Implicit Computational Complexity and 5th Workshop on Foundational and Practical Aspects of Resource Analysis, DICE-FOPARA@ETAPS 2017, Uppsala, Sweden, April 22-23, 2017*, volume 248 of *EPTCS*, pages 47–59, 2017. doi:10.4204/EPTCS.248.9.
 - 33 pymwp's documentation, 2021. URL: <https://statycc.github.io/pymwp/>.
 - 34 Coq Team. Coq documentation, 2022. URL: <https://coq.github.io/doc/>.
 - 35 Joris van der Hoeven and Robin Larrieu. Fast Gröbner basis computation and polynomial reduction for generic bivariate ideals. *Applicable Algebra in Engineering, Communication and Computing*, 30(6):509–539, December 2019. doi:10.1007/s00200-019-00389-9.
 - 36 Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewicz. Formal verification of SSA-based optimizations for LLVM. In Hans-Juergen Boehm and Cormac Flanagan, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 175–186. ACM, 2013. doi:10.1145/2491956.2462164.

A Technical appendix on semi-rings (abridged)

This is an abridged version of the technical development on semi-ring that is exposed in full details in our technical report [4, A.1 and A.2].

► **Lemma 18** (mwp semi-ring). *The tuple $(\{0, m, w, p\}, 0, m, +, \times)$, with*

- $0 < m < w < p$,
- $\alpha + \beta = \begin{cases} \alpha & \text{if } \alpha \geq \beta \\ \beta & \text{otherwise} \end{cases}$
- $\alpha \times \beta = \begin{cases} \alpha + \beta & \text{if } \alpha \neq 0 \text{ and } \beta \neq 0 \\ 0 & \text{otherwise} \end{cases}$

is a strong semi-ring.

► **Lemma 19** (Matrix semi-ring). *Given a strong semi-ring $\mathbb{S} = (S, 0, 1, +, \times)$, the tuple $\mathbb{M} = (M, 0, 1, \oplus, \otimes)$, with*

- M the set of all $n \times n$ matrices over S , for all $n \in \mathbb{N}$,
- 0 defined by $M = 0$ iff $M_{ij} = 0$ for all i and j ,
- 1 defined by $M = 1$ iff $M_{ij} = 1$ for $i = j$, $M_{ij} = 0$ otherwise,
- $\oplus$ defined by $C = A \oplus B$ iff $C_{ij} = A_{ij} + B_{ij}$,
- $\otimes$ defined by $C = A \otimes B$ iff $C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$,

is a strong semi-ring.

For simplicity, we will write $\mathbb{M}$ as $\mathbb{M}(\mathbb{S}) = (M(S), 0, 1, \oplus, \otimes)$.

► **Lemma 20** (Choices semi-ring). *Given a strong semi-ring $\mathbb{S} = (S, 0, 1, +, \times)$ and a set A , the tuple $\mathbb{F} = (F, 0, 1, \boxplus, \boxtimes)$, with*

- F the set of functions from A to S ,
- 0 the constant function $0(a) = 0$ for all $a \in A$,
- 1 the constant function $1(a) = 1$ for all $a \in A$,
- $\boxplus$ defined componentwise: $(f \boxplus g)(a) = (f(a)) + (g(a))$, for all f, g in F and $a \in A$,
- $\boxtimes$ defined componentwise: $(f \boxtimes g)(a) = (f(a)) \times (g(a))$, for all f, g in F and $a \in A$,

is a strong semi-ring.

For simplicity, we will write $\mathbb{F}$ as $A \rightarrow \mathbb{S} = (A \rightarrow S, 0, 1, +, \times)$.

► **Lemma 21.** *For all set A and strong semi-ring $\mathbb{S}$, $\mathbb{M}(A \rightarrow \mathbb{S}) \cong A \rightarrow \mathbb{M}(\mathbb{S})$.*

► **Lemma 22.** *Given a strong semi-ring $\mathbb{S} = (S, 0, 1, +, \times)$ and an element $\perp \notin S$, $\mathbb{S}^\perp = (S \cup \{\perp\}, 0, 1, +^\perp, \times^\perp)$ with, for all $a, b \in S \cup \{\perp\}$,*

$$a +^\perp b = \begin{cases} a + b & \text{if } a, b \neq \perp \\ \perp & \text{otherwise} \end{cases}$$

$$a \times^\perp b = \begin{cases} a \times b & \text{if } a, b \neq \perp \\ \perp & \text{otherwise} \end{cases}$$

is a semi-ring.

Proof. The proof is immediate, but note that $\mathbb{S}^\perp$ is not strong, as $\perp \times 0 = \perp$. ◀

B

 Omitted Proofs

► **Theorem 10** (Determinancy and termination). *Given a program P , there exists unique $p \in \mathbb{N}$ and $M \in \mathbb{M}(\{0, 1, 2\}^p \rightarrow \text{MWP}^\infty)$ such that $\vdash P : M$.*

Proof. The proof proceeds by induction on the length of the program P , expressed in number of commands. We let p be the number of variables in P , but observe that any program P can be treated as manipulating $p' > p$ different variables, by simply adding $p' - p$ additional rows and columns to the matrix, and leaving them unchanged by the derivation of P . While a complete proof would need to constantly account for the number of actual and potential variables used by P , we will simply assume that the reader understands that accounting for this technicality obfuscate more than it clarifies the proof, and we will freely resize the matrices to account for additional variables when needed.

If P is of length 1 Then we know P is of the form $\mathbf{x} = \mathbf{e}$, and only the rule A can be applied.

But then we need to prove that all expression $\mathbf{e}$ can be typed with exactly one vector.

An expression $\mathbf{e}$ is either a variable $\mathbf{x}$, or a composed expression $\mathbf{x} * \mathbf{y}$, $\mathbf{x} - \mathbf{y}$, or $\mathbf{x} + \mathbf{y}$.

But then, respectively, only E^S , E^M or E^A (for addition and subtraction) can be applied, and this case is proven.

If P is of length $n > 1$ Then we proceed by case on the structure of the command:

- If P is of the form **if $\mathbf{b}$ then P_1 else P_2** , then by induction we know for $i \in \{1, 2\}$ there exists p_i and M_i of size $p_i \times p_i$ such that $\vdash P_i : M_i$. If $p_1 \neq p_2$, then letting M_j being the smaller matrix, it is easy to rewrite P_j 's derivation to account for $|p_1 - p_2|$ additional variables, and as $\oplus$ is uniquely defined, we know that $M_1 \oplus M_2$ results in a unique matrix of size $\max(p_1, p_2)$.
- If P is of the form **while $\mathbf{b}$ do P'** , this is immediate by induction hypothesis on P' , considering that only W^∞ can be applied, and that this rule produces a unique matrix.
- If P is of the form **loop $\mathbf{x}$ { P' }**, this case is similar to the previous one, using L^∞ instead of W^∞ .
- If P is of the form $P_1; P_2$, this case is similar to the **if** case, with the possible need to resize one of the matrix obtained by induction, and using that $\otimes$ is uniquely defined. ◀

► **Theorem 11** (Adequacy). *If $\vdash P : M$, then for all $\vec{a} \in A^p$, $\vdash_{JK} P : M[\vec{a}]$ iff $\infty \notin M[\vec{a}]$.*

Proof. The proof proceeds by induction on the length of the program P , expressed in number of commands.

If P is of length 1 Then we know P is of the form $\mathbf{x} = \mathbf{e}$, and only the rule A can be applied,

in both systems. Hence, we need to prove that all expression $\mathbf{e}$ can be typed the same way in both systems. A careful comparison of Figures 1 and 2 shows that if $\mathbf{e}$ is of the form $\mathbf{x}_i$, then there is a small mismatch. In the original system, we can use either E_2 , and obtain $\vdash_{JK} \mathbf{x}_i : \{\frac{w}{1}\}$, or E_1 , and obtain $\vdash_{JK} \mathbf{x}_i : \{\frac{m}{1}\}$, while the only derivation in the deterministic system is using E^S to get $\vdash_{JK} \mathbf{x}_i : \{\frac{m}{1}\}$. As $m < w$, we argue that the deterministic system cannot obtain a derivation that is not useful anyway, and hence that it can be ignored.

As for the other cases, if $\mathbf{e}$ is a composed expression $\mathbf{x} * \mathbf{y}$, $\mathbf{x} - \mathbf{y}$, or $\mathbf{x} + \mathbf{y}$, it is easy to observe that E^A and E^M encapsulates all the possible combinations of E_2 and of E_1 followed by E_3 or E_4 that can be used.

If P is of length $n > 1$ Then the result holds by induction, once we observed that L^∞ and W^∞ are introducing ∞ coefficients *only if* L and W cannot be applied. ◀

C Benchmarks

C.1 Descriptions of program groups

- *Basics* – C programs performing operations corresponding to simple derivation trees.
- *Implementation paper* – example programs presented in this paper.
- *Original paper* – examples taken from or inspired by the original analysis [23].
- *Infinite* – programs whose matrices always contain infinite coefficients.
- *Polynomial* – programs whose matrices do not always contain infinite coefficients.
- *Other* – other C programs of interest.

C.2 Results

The benchmarks are categorized and grouped to distinguish the type of system behavior they exercise. For each program we capture in Table 1

1. program variable count
2. the lines of code in the source program (LOC column)
3. clock time taken by the full analysis (excluding saving result to file, which is otherwise default behavior),
4. number of function calls excluding builtin Python language calls, and
5. the result of the analysis.

Collectively the LOC, time, and function calls columns provide insight into the behavior of the analysis as different aspects of the system are being stress-tested. From the results column we report expected results on each benchmarked program. In the benchmarks table a passing result is represented with $\checkmark$ and ∞ otherwise. We do not report manually computed bounds as comparison, because the analysis is carried out on individual variables, thus calculating them on multivariate programs is tedious and futile. However, for simple programs such as `while_2.c`, it is straightforward through visual inspection to verify the obtained 2×2 -matrix is indeed the correct result.

These benchmarks were obtained using Python’s built-in `cProfile` utility, extended in `pymwp` implementation to enable batch profiling. The clock times are slight overestimates because the utility adds minor runtime overhead. The number of function calls includes primitive calls, but exclude built-in Python language calls. Full detailed results are viewable in the source code repository: <https://github.com/statycc/pymwp/releases/tag/profile-latest>

C.3 Comparison

It is not really meaningful or possible to compare those results with any other static analyzer, and impossible to compare it with any other implementation of this type of flow analysis. While we could, in theory, analyze our examples with other static analyzers, their results would be incomparable, as they would produce guarantees on termination or worst case resource usage, which are both orthogonal to our polynomial bounds on value growth. To our knowledge, the only static analyzer using similar metrics [5] was developed only for functional languages, thus preventing comparison. As for implementations of the original analysis, our first attempts showed that a naive implementation would likely fail to handle the memory or time explosions. We did, however, compare the gains resulting from the optimizations described in Sect. 4.3. In a nutshell, our improved algorithm for adding and multiplying polynomials resulted in the analysis being roughly *five times faster* for two programs that we estimate to be representative.

■ **Table 1** Benchmark results produced by `pymwp` on C programs.

Program name	Variables	LOC	Time (ms)	Function calls	Bound
<i>Basics</i>					
assign_expression	2	8	133	81614	✓
assign_variable	2	9	115	81238	✓
if	2	9	118	82046	✓
if_else	2	7	118	82928	✓
inline_variable	2	9	118	81979	✓
while_1	2	7	117	82934	✓
while_2	2	7	117	83964	✓
while_if	3	9	122	91572	✓
<i>Implementation paper</i>					
example7	3	10	122	86898	✓
example15_a	2+2	25	122	88763	✓
example15_b	4	16	137	122016	✓
<i>Original paper</i>					
example3_1_a	3	10	110	85286	✓
example3_1_b	3	10	120	87637	✓
example3_1_c	3	11	121	89173	✓
example3_1_d	2	12	116	80002	∞
example3_2	3	12	118	83182	∞
example3_4	5	18	134	108890	∞
example5_1	2	10	116	81185	✓
example7_10	3	10	119	86053	✓
example7_11	4	11	139	119379	✓
<i>Infinite</i>					
exponent_1	4	16	127	99893	∞
exponent_2	4	13	123	92846	∞
infinite_2	2	6	143	128275	∞
infinite_3	3	9	120	89880	∞
infinite_4	5	9	3274	5924420	∞
infinite_5	5	11	369	529231	∞
infinite_6	4	14	1624	2836726	∞
infinite_7	5	15	631	964189	∞
infinite_8	6	23	880	1444782	∞
<i>Polynomial</i>					
notinfinite_2	2	4	119	86174	✓
notinfinite_3	4	9	131	104826	✓
notinfinite_4	5	11	169	168242	✓
notinfinite_5	4	11	174	176179	✓
notinfinite_6	4	16	195	215765	✓
notinfinite_7	5	15	1161	1961806	✓
notinfinite_8	6	22	1893	3172293	✓
<i>Other</i>					
dense	3	16	157	151428	✓
dense_loop	3	17	269	353068	✓
explosion	18	23	1296	2327071	✓
gcd	2	12	114	84914	∞
simplified_dense	2	9	118	85098	✓

8 Index of Artifacts

Published artifacts. The artifacts in Table 30 correspond to the published manuscripts in Chapters §2 and §7 of this dissertation.

	Year	Description	Version
1.	2022	pymwp (Aubert, Rubiano, Rusch, & Seiller, 2022c) <i>Companion artifact for published manuscript in §7.1.</i> Archive: <code>swh:1:dir:22a4ab0cfad49138981ed25fc2abfe830fb7ccdf</code> Repository: <code>statycc/pymwp/releases/tag/FSCD22</code>	0.1.6
2.	2022	Loop fission evaluation (Aubert, Rubiano, Rusch, & Seiller, 2022e) <i>Companion artifact for published manuscript in §2.2.</i> Archive: <code>10.5281/zenodo.7080144</code> Repository: <code>statycc/loop-fission</code>	1.0
3.	2023	pymwp (Aubert, Rubiano, Rusch, & Seiller, 2023d) <i>Companion artifact for published manuscript in §2.1.</i> Archive: <code>10.5281/zenodo.7908484</code> Repository: <code>statycc/pymwp/releases/tag/0.4.2</code>	0.4.2
4.	2025	Applied Implicit Computational Complexity (Rusch, 2025c) <i>An artifact to complement this dissertation.</i> Archive: <code>10.5281/zenodo.17148077</code> Repository: <code>nkrusch/dissertation</code>	1.0.0

Table 30: Published and archived artifacts.

Artifacts in development. Some software artifacts remain under development. They correspond to the unpublished works of Chapter §3.

1. **Postconditions inference evaluation** (*companion artifact for §3.1*). The artifact includes the analysis implementation and supports the experimental evaluation claims.
2. **TYNI security analyzer** (*companion artifact for §3.2*). The artifact is an implementation of the described program analysis technique.
3. **Formalization of the flow calculus of mwp-bounds** (*companion artifact for §3.3*). The artifact aims to formalizes the paper proofs from Jones and Kristiansen (2009), to prove the soundness theorem (Thm. 2) of the mwp-calculus.

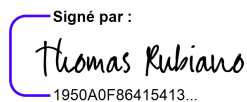
9 Co-author Statements

Co-author Declaration

A summary of work performed by Thomas Rubiano on “pymwp: A Static Analyzer Determining Polynomial Growth Bounds.”

Thomas Rubiano is the original software architect of the pymwp analyzer and responsible for much of the groundwork. His contributions included crafting the overall analysis workflow, and designing the core data structures. For example, the Relation data structure (discussed in Sect. 4.1), and Polynomial structures, that track coefficients in the matrices, were originally implemented by Thomas. The starting point of the pymwp analyzer is based on earlier work on Loop peeling with Thomas Seiller and Jean-Yves Moyer (https://github.com/statycc/LQICM_On_C_Toy_Parser). The software development contributions to pymwp are detailed at:

<https://github.com/statycc/pymwp/graphs/contributors>.

Signé par :

1950A0F86415413...

Thomas Rubiano

5/28/2025 | 06:48:38 EDT

Date signed

Co-author Declaration

A summary of work performed by Thomas Rubiano on “Distributing and Parallelizing Non-canonical Loops.”

Thomas’s Rubiano’s doctoral work (Rubiano, 2017) on compositional data flow static analysis in compilers is a grounding motivation behind this work. Thomas Rubiano and Thomas Seiller collaborated on the initial concept of splitting loops by dependencies, and the mathematical analysis behind it, particularly during our seminar in Dagstuhl in 2021 (<https://www.dagstuhl.de/en/seminars/seminar-calendar/seminar-details/21453>). Thomas Rubiano created a prototype analyzer implementing the theory, a test environment and benchmarks to test the theory. Although the analyzer and those benchmarks did not make it to the final paper (<https://github.com/statycc/loop-fission/graphs/contributors>), their development facilitated the development of the end result. For example, the loop fission algorithm (Sect. 3) refined original ideas behind the prototype.

Signé par :

1950A0F86415413...

Thomas Rubiano

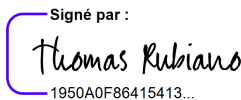
5/28/2025 | 06:48:38 EDT

Date signed

Co-author Declaration

A summary of work performed by Thomas Rubiano on “mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity.”

All authors actively discussed ideas on how to improve and implement the flow calculus during the research meetings. Thomas Rubiano implemented the delta graph technique (Sect. 4.4) and contributed to the implementation (Sect. 5 and Appendix C). Precise contributions to the source code can be traced using <https://github.com/statycc/pymwp/graphs/contributors>.

Signé par :

1950A0F86415413...

Thomas Rubiano

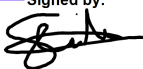
5/28/2025 | 06:48:38 EDT

Date signed

Co-author Declaration

A summary of work performed by Thomas Seiller on “pymwp: A Static Analyzer Determining Polynomial Growth Bounds.”

Thomas Seiller contributed greatly to the theory implemented in pymwp. Specifically, wrt. the paper, Thomas was involved in discussion about techniques to expose derivation failure (Sect 4.2). His contributions also helped improve the paper presentation (for example by suggesting to develop Table 2) and ground the soundness result (for example by leveraging coverings).

Signed by:

C0EB208754CE442...

Thomas Seiller

5/28/2025 | 07:25:33 EDT

Date signed

Co-author Declaration

A summary of work performed by Thomas Seiller on “Distributing and Parallelizing Non-canonical Loops.”

Thomas Seiller worked on the theory of the loop parallelization. The parallelization idea follows from earlier work with Thomas Rubiano. Thomas Seiller and Thomas Rubiano developed the dependency analysis needed to split loops. In addition, Thomas Seiller actively discussed the scope of the work and strategies of parallelization with OpenMP directives.

Signed by:

C0EB208754CE442...

Thomas Seiller

5/28/2025 | 07:25:33 EDT

Date signed

Co-author Declaration

A summary of work performed by Thomas Seiller on “mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity.”

Thomas Seiller contributed greatly to the adjusted theory of the flow calculus (Sect. 3) and is the main architect behind the treatment of function calls (Sect. 4.1). The efficient computation with polynomial structures (Sect. 4.3) was written almost entirely by Thomas Seiller. Introducing ∞ coefficient, the deterministic rules, and how to compute with the extended set of coefficients was defined in collaboration by Thomas Seiller and Clément Aubert.

Signed by:

C0EB208754CE442...

Thomas Seiller

5/28/2025 | 07:25:33 EDT

Date signed

10 Index of Acronyms

AARA	Automatic Amortized Resource Analysis
ACM	Association for Computing Machinery
AI	Abstract Interpretation
API	Application Programming Interface
ARB	Architecture Review Board
AST	Abstract Syntax Tree
ATP	Automatic Theorem Proving
BC	the Bellantoni-and-Cook class
BFF	Basic Feasible Functionals
BNF	Backus–Naur form
C99	the C language standard defined by ISO/IEC 9899:1999
CADE	Conference on Automated Deduction
CASC	CADE ATP System Competition
CE	Cost Equation
CES	Cost Equation System
CFG	Control Flow Graph
CLI	Command Line Interface
CPU	Central Processing Unit
DCC	Dependency Core Calculus
DDG	Data Dependence Graph
DFG	Data Flow Graph
DSL	Domain Specific Language
FIG	Loop Fission Interference Graph
FP	complexity class FP
GCC	the GNU Compiler Collection
GPU	Graphical Processing Unit
HMO	Health Maintenance Organization
HoTT	Homotopy Type Theory
ICC	Implicit Computational Complexity
icc	Intel’s C++ Compiler
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IFC	Information Flow Control
IFP	Information Flow Policy
IP	Integer Program
IR	Intermediate Representation
ITP	Interactive Theorem Proving
ITS	Integer Transition System
L	Line
LoC	Lines of Code

LP	Linear Programming
mathcomp	Mathematical Components library
MILP	Mixed Integer Linear Programming
ML	Meta Language
MLS	Multi Level Security
NI	Non-Interference
NP	Non-deterministic Polynomial time
oneTBB	oneAPI Threading Building Blocks
OOP	Object Oriented Programming
OpenMP	Open Multi-Processing
OS	Operating System
PDG	Program Dependence Graph
PG	Partition dependence Graph
PL	Programming Language
PPL	(Microsoft) Parallel Patterns Library
PyPI	Python Package Index
QI	quasi-invariant
QTT	Quantitative Type Theory
RG	Reduced dependence Graph
RR	Recurrence Relation
SAT	Boolean SATisfiability problem
SC	Security Class
SCC	Strongly Connected Components
SFM	Security-Flow Matrix
SIMD	Single Instruction Multiple Data
SMT	Satisfiability Modulo Theories
SSA	Static Single Assignment
SSReflect	Small Scale Reflection
STR	Stratified programs
SV-Comp	Competition on Software Verification
TM	Turing Machine
TPTP	Thousands of Problems for Theorem Provers
TRS	Term Rewrite System
URL	Uniform Resource Locator
USG	University System of Georgia
UTM	Universal Turing Machine
WCET	Worst-Case Execution Time
www	world wide web
XbC	X-by-Construction

11 Symbol Index

$\cdot$	absence of non-interference violation 264, 267–270, 272, 273, 278, 281, 292–296
$\cdot^*$	closure operator 229, 365, 366, 368, 373
$:=$	definition or assignment (alternatively $::=$) 105, 217, 226, 265, 365, 372
$\approx_L$	indistinguishability of behaviors 104, 105
$=_L$	equivalence relation of low states 104, 105
$\equiv$	equivalence 2, 237
$\triangleq$	defines 219
$\models$	semantic implication 44
$\vdash$	syntactic implication, proves 44, 105, 366–368, 370–373, 383
$[\cdot]$	program slot 373, 374, 383
$\cong$	isomorphism 369, 370, 382
$\llbracket _ \rrbracket$	function computed by program $_$ 2, 23, 104, 105
$\approx$	not up-to equivalent 276, 289
$\sim$	up-to equivalence 276, 289
0	absence of dependency . . . 43, 45, 152, 154, 159, 165, 166, 212, 219–221, 223, 228, 230–233, 307, 365–371, 373–375, 382
	<i>also</i> reinitialization 186–189
1	propagation 186–189
A_i	set of derivation choices 369, 374–376
α	coefficient in MWP 220, 228, 365, 366, 369, 374, 375, 382
$\vec{a}$	derivation assignment 369, 371–373, 376, 383
$\begin{pmatrix} \alpha \\ \beta \end{pmatrix}$	column vector of flow coefficients 367, 369, 370, 373–375, 383
$\{i^\alpha\}$	vector with α in i^{th} row and 0 otherwise 218, 366, 368
$\{i^\alpha, j^\beta\}$	component-wise vector addition 366, 367, 370
$\{i^\alpha \rightarrow j\}$	matrix M with $M_{ij} = \alpha$ and 0 elsewhere 229, 366, 368
$ $	BNF expression separator (alternatively $\parallel$) 105, 184, 217, 265, 365
β	coefficient in MWP 365, 366, 369, 374, 375, 382
$\vec{C}$	choice vector 225, 226, 234
$\mathcal{C}$	program 211, 212, 218, 235
	<i>also</i> complexity class 23
	<i>also</i> computational model 19
	<i>also</i> list of choice vectors 226
c	security class $(c, m, t, \dots)$ 261, 262, 274–278, 289, 290
$\text{Card}(_)$	cardinality of $_$ 369, 376
$\models C : M$	semantic relation of polynomial value growth 44
$\vdash C : M$	assignment of matrix M to command C . . . 44, 218, 229, 366, 368, 371, 373, 383
$\text{Corr}(_)$	correction of $_$ 187–189, 271–273
C	program 42–44, 104, 105, 214, 261, 265–279, 281, 289–291
C'	a sub-program of program C 266, 275, 289–291

$C[\vec{v}]_0$	in program C variable x_i was assigned value v_i	274
$C[\vec{v} \rightarrow \vec{v'}]_t$	in program C variable x_i has value $\vec{v'}$ after t updates	275, 276, 289
$\llbracket C \rrbracket(x_i \rightsquigarrow x'_i)$	data flow relation of initial and final variable value	42, 43, 165
Δ	sequence of derivation choices	220
δ	derivation choice	219–221, 227, 228, 369–377
DFG	data-flow graph	186, 187, 189, 190
$\mathcal{D}$	domain	219, 225–227
E^t	column vector for correction	188
$\vdash e : V$	assignment of vector V to expression e	218
$\tilde{F}$	program chunk	373, 374
f	function (also f or g)	24, 25, 278, 279, 281, 282, 372, 373
$\perp$	false	382
	also divergence	104
$f_{\vec{e}}$	list of freshly introduced variables $e_1, \dots, e_n$ unique to f	278–280, 296
$f_{\vec{e}}^{\text{in}}$	Variables $e_1, \dots, e_n$ in f signature	280–282
$f_{\vec{e}}^{\text{out}}$	Variables $e_1, \dots, e_n$ modified by f	281–283
Γ	typing context	105
γ	coefficient in MWP	365, 375
G	graph	191
G_W	condensation graph	190, 191
$\{P\}_{-}\{Q\}$	Hoare triple	115, 214, 215
i	matrix row	220, 228, 229, 365, 366, 368, 369, 382
	also value of derivation choice in A_i	369, 374, 375
	also variable index in X_i	218, 229, 366, 368
$\text{In}(_)$	variables used by $_$	185–187, 190, 264, 266, 269, 278, 281, 282, 290, 291
∞	derivation failure	43–45, 152, 153, 156–159, 165, 166, 171, 175, 212–214, 219, 221–223, 227–231, 233, 305, 368–370, 372, 375, 376, 379, 383–385
	also dependence	186–190
j	matrix column	220, 221, 228, 229, 365, 366, 368, 369, 382
	also variable index in X_j	229, 366, 368
	also index of a derivation choice	369, 374–376
k	degree of choice	219, 220, 222, 225–227, 234
	also derivation count	369, 372, 374–376
$\mathcal{L}$	programming language	23
L^∞	deterministic inference rule of loop	368, 370, 383
h	high security context	97, 104, 105
l	low security context	97, 104, 105
	also loop	235
ℓ	security class assignment	261, 267, 274, 276–279, 282, 289, 291
$\ell(_)$	security class assignment to variable $_$	261, 262, 267–269, 272, 273, 275, 276, 279, 282, 289, 291, 293–296
$\mathbb{M}(_)$	data-flow graph of $_$	186–190, 192
M	matrix	44, 218, 221, 229, 235, 365, 366, 368, 369, 371–373, 382, 383
m	maximally linear flow coefficient	43, 45, 152, 165, 166, 212, 219–223, 228, 229, 231, 233, 234, 299, 365–371, 373–375, 382, 383

$\mapsto$	derivation choice maps to a value 219, 230, 368–370 <i>also</i> variable maps to a security class 295 <i>also</i> input maps to an output 24
M^*	matrix closure/fixpoint 229, 365, 366, 368
$M \stackrel{j}{\leftarrow} V$	replace in matrix M the j^{th} column by V 366, 368, 370, 372, 374
$\mathbb{M}(_)$	security-flow matrix of $_$ 267–273, 276, 278, 279, 289, 290
$\mathbb{M}^e(_)$	security-flow matrix of $_$ with side effects 281, 282
$\mathbb{M}(_)(i, j)$	security flow coefficient at row i and column j 267–269, 289–291, 295
$\mathbb{M}(\text{MWP})$	the mwp semi-ring 365, 369, 374, 375, 383
MWP	set of flow coefficients 365, 368, 369, 371, 375
MWP^∞	MWP extended with ∞ -coefficient . . . 146, 219, 220, 305–308, 369, 371, 374–376, 383
$\mathbb{N}$	natural number 131, 219, 225, 226, 244, 274, 276, 300, 365
$\spadesuit$	non-interference violation . . . 264, 267–270, 272, 273, 277, 278, 281, 282, 289–296
$\mathcal{O}$	asymptotic upper bound 20, 145, 302
O	row vector for correction 188
o	asymptotic upper bound that is not tight 20
$\text{Occ}(_)$	variables occurring in $_$ 185, 186, 188, 189, 266–268, 272, 275–279, 281, 289–291
Ω	asymptotic lower bound 20
$\text{Out}(_)$	variables modified by $_$. . 185–190, 264, 266, 269, 272, 278, 280, 281, 290, 291, 296
P	precondition 115, 214, 215
p	polynomial flow coefficient . . . 43–45, 152, 165, 166, 212, 218, 219, 221, 223, 228, 229, 231–234, 299, 365–371, 373–375, 382 <i>also</i> program 23
ϕ	analyzed property 31
$\prod$	cartesian product of sets 369, 374–376
π	derivation 292, 367, 370, 371, 374
$\oplus$	component-wise addition 218, 229, 365, 366, 368, 370–374, 382, 383
$+\infty$	addition over MWP^∞ 369
poly	honest polynomial 45, 152, 166, 167, 222, 223, 366
$\bar{\Psi}$	injection over cartesian product 374
Q	postcondition 115, 214, 215
$\mathcal{R}$	restriction 23, 38, 39, 41
$\mathcal{S}$	set of derivation choice sequences 225–229, 234
$\mathbb{S}$	strong semi-ring 369, 382
S	set of states 104, 105
s	initial state 104, 105
s'	terminal state 104
SC	security lattice 261, 262, 274, 276, 278, 289
SC	partially ordered set of security classes 261
τ	type 105
t	time (counter) 274–276, 291
Θ	asymptotic tight bound 20
$\otimes$	matrix product 365, 366, 368, 370, 373, 374, 382

$\times^\infty$	multiplication over MWP^∞ 369, 379
$\top$	true 215
V	column vector 218, 366, 370, 373
$\text{var}(_)$	variables in $_$ 218, 366
$\vec{v}$	value list, i.e., $\vec{v} = v_1, \dots, v_n$, (alternatively $\vec{w}$) 274–276, 289, 291
W	mwp-bound ($W, V, U, \dots$) 45, 223, 233, 237
W^∞	deterministic inference rule of while 368, 370, 373, 383
$\tilde{W}$	parallel loop 191–193
w	sequential loop 189–193
w	weak polynomial flow coefficient . . 43, 45, 152, 165, 166, 212, 218, 219, 221, 223, 228, 229, 231, 233, 234, 299, 307, 365–371, 373–375, 382, 383
x	variable (alternatively $w, y, z, \dots$) . . 262, 264–267, 269, 270, 272–275, 277–279, 282, 289–291, 296
X_n	variable 42, 43, 165, 211, 218, 221, 223, 228, 229, 231, 233, 241
X'	variable referring to its final value . . 151–154, 158, 164, 165, 167, 169, 170, 223, 241, 243
x	variable's (initial) value 42, 43, 211
x'	variable's final value 42, 43, 45, 211
$\vec{x}$	variable list (also $\vec{y}, \vec{z}$) 45, 223, 366 <i>also</i> normal parameters 24, 25
$\vec{y}$	safe parameters 24, 25

12 Term Index

Symbols

λ -calculus 287, 325
 $\mathcal{O}$ -notation 20, 302
 π -calculus 125, 325
 T_{NI}^* 258–261, 263, 264, 266, 274, 276–278,
 280, 283–285, 287, 288
 NP-completeness 50, 120, 145

A

abstract interpretation 29, 239
 abstract syntax tree 49, 56, 154, 155
 access control 93
 ACL2 119
 Agda 119, 145
 Alloy 119
 Alt-Ergo 119
 amortization 34, 36, 37, 145
 amortized analysis 34
 antidependence 77, 78
 aperiodic programs 28, 109
 AProVE 36, 241, 379
 Ariane 5 112
 assertion 56, 115, 117, 127, 207, 208, 215,
 237, 238, 242, 250, 254
 Assertion Inference Paradox 115
 assumption 107, 208, 243, 248
 asymptotic analysis 19–21, 23, 29
 asymptotic bounds 20, 241
 Atelier B 119
 attack model 99
 attacker (adversary) 97, 104, 105, 262, 274
 attacker model 262
 Attestor 119
 authentication 92
 automata theory 6
 automated theorem proving 119, 120
 automatic amortized resource analysis
 34, 146
 automatic parallelization 182, 183,
 194–199

automatic resource analysis 8, 10, 312,
 320, 321, 328, 330, 331
 availability 92, 93, 96
 AWS authorization engine 129

B

Banerjee test 79
 barrier 84, 85, 87
 basic block 73
 Basic Feasible Functionals 109
 Bedrock 125
 bi-partite graph 186
 Bitwuzla 119
 blockchain 125
 Boogie 119, 127, 129
 Boolean circuit 28
 bounded loop 236, 238, 241, 242
 bytecode 36, 284

C

C 15, 36–38, 49, 51, 52, 60, 71, 82, 84,
 153–159, 164, 177, 194–197, 210, 217, 237,
 238, 241, 244, 254, 265, 313, 314
 C++ 82, 84, 194
 C4B 36, 146, 153, 154
 C# 124, 126
 CADE ATP System Competition 120
 Caesar 119
 Calf 145
 Cameleer 119
 canonical loop 72, 89, 90, 181–183, 194–196
 category theory 325
 Cecoa 38, 145
 Cerco 377
 Certicrypt 110
 certified-llvm 378
 Cetus 194, 195
 Chalice 129
 choice vector 157, 225–230, 232, 234
 circuit complexity 19
 CIVL 119

class invariant	115
Clava	194, 195, 198
Clp	119
Cobham-Edmonds Thesis	20, 25
code optimization	70, 73
Code2Inv	119
CoFloCo	36, 379
coinduction	124, 125
commutativity analysis	79, 80
Company-coq	123
CompCert	125, 378
completeness	30–33, 38–41, 109, 227
complexity analysis	39, 42, 111, 238, 240, 298, 299, 303
complexity bound	21–23, 25, 28, 36, 111, 287, 298
complexity classes	19–26, 29, 38–40, 108, 287, 299, 312, 324, 325
BPP	28
BQP	27
EQP	27
EXP	27
FP	25
Linspace	27
L	27, 28
NC	28
NP	28, 145, 316, 368
PP	28
PSPACE	26, 27
P	19, 20, 25–28, 38–40, 44, 47, 109, 110, 302, 365
ZQP	27
complexity measure	19, 33
complexity theory	6, 11, 19–23, 47, 51, 70, 144, 145, 210, 243, 248, 298, 299, 302, 324, 325
ComplexityParser	38, 377
compositionality	211, 237, 259, 286, 299, 300, 305, 313, 326
compound assignment operator	265
computable function	19
concurrent programs	6, 34, 82, 129, 242, 286
condensation graph	190–192
confidentiality	92–94, 100, 258, 259, 286
CONS-free programs	27
conservative analysis	32
constraint analysis	29
control dependence	78
control flow analysis	29
control flow graph	73
Cook-Levin Theorem	145, 303
Coq	<i>see also</i> Rocq, 298–303, 378
Coquelicot	125
cost equation	35
cost equation system	15, 34–36
COSTA	377
CP-SAT	119
CPAChecker	119
CPLEX	119
CProver	119
CPU	83, 197
CREST	119
CrossHair	119
CrowdStrike	112
cryptography	6, 100, 110, 125, 287, 312
cuPDL-C	119
cvc5	119
Cyclist	119
D	
dℓT	28, 110, 111
Dafny	15, 119, 121, 124, 126–129, 132, 134, 138, 139, 147, 249, 250
Daikon	119, 239, 240, 243, 244, 251, 252, 254
data dependence	77, 111, 266, 287, 300, 312
data flow analysis	22, 29, 111, 285, 325
data flow graph	185, 186, 211–213, 232
data flow independence	232, 307, 309, 310
data flow relation	42–44, 64, 164
data race freedom	116
deadlock freedom	41
deadlocking	94, 116
Decalf	145
declassification	96, 101, 109, 286
decryption	93
deductive verification	121, 126, 139
delta graph	376, 377
dependence	212
anti	77, 78

control 78
 output 78
 true 78
 dependence analysis . . . 79, 80, 181, 183, 185,
 193, 200, 267, 287, 309
 dependence graph 190, 193, 194
 Dependency Core Calculus . . 111, 287, 332
 dependent types 28, 121, 124
 derivability . . 44, 45, 49, 50, 52, 56, 58–60, 63,
 65, 166, 213, 214, 219, 229–231, 234, 235,
 238
 derivation choice 219–222, 224, 225, 227,
 228, 230
 derivation failure 45, 156, 212, 213,
 222–224, 229–231, 305–307
 derivation tree 34
 Desargues’s Theorem 125
 DIG 119, 239
 digital circuit 22
 distributed systems 129
 domain-specific language 160, 312
 downgrading 96, 286
 dReal 119
 Duet 209, 240, 242, 244, 251, 253, 254
 dynamic code loading 260
 dynamic program analysis 239, 243

E

E 119
 EasyCrypt 110, 119
 eco-imp 38
 effect system 29
 Eiffel 119, 126
 Eldarica 119
 Emacs 123
 encryption 93
 endorsement 96
 equivalence relation 98, 104
 EvoSpex 119, 239
 expressiveness 6–8, 29, 39, 45, 209, 214,
 218, 232, 246–248, 280, 286, 306, 313
 extensional completeness 39

F

F* 119

finite group theory 123
 firewall 93
 fixed point algorithm 29
 Flocq 125
 flow coefficient . . 43, 44, 47, 52–54, 59, 62, 64,
 65, 165, 166, 212, 218–220, 222, 224, 225,
 228–230, 232–234, 305
 Flux 119
 fork 82
 fork-join programming model 82
 formal methods 103, 113, 114, 133, 206,
 214, 247
 formal model 114
 formal security analysis 262
 formal verification 6, 113, 115–118, 122,
 210, 214, 247, 248, 298, 324, 325
 Fortran 82
 Four Color Theorem 125
 Four Color theorem 113, 123
 Frama-C 119
 FreqHorn 119
 function algebras 325
 functional correctness 120, 124
 functional equivalence 2, 3
 functional programs 34

G

Gallina 121
 GCC compiler 193, 197
 GCD test 79
 ghost construct 126
 ghost method 127
 Gillian 119
 Glop 119
 GLPK 119
 Go 126
 Gobra 119
 Goldreich-Levin Theorem 110
 GPU 83
 grammar 37, 104, 106, 107, 265, 283, 284,
 294, 313
 graph covering 190–193
 graph database 6
 GRASShopper 119
 Gröbner basis 375

guard variable 54, 71, 107, 108, 241, 255
 Gurobi 119
 Gödel-Rosser Incompleteness Theorem 125

H

Halting Problem 302
 Haskell 36, 122
 Hasse diagram 262, 268, 294
 HiGHS 119
 Hoare triple 115, 214
 HOL Light 119, 120
 hollow matrix 267, 268, 291
 Homotopy Type Theory 125
 honest polynomial 45, 152, 223, 300, 301
 HoSA 37
 hyperproperty 101

I

identity matrix 53, 56
 IFSPEC 96
 IMP language 51, 300
 imperative programs 10, 34, 42, 47, 50–52,
 54, 56, 60, 111, 164, 211, 213, 216, 217, 235,
 237, 262, 265, 287, 298, 299, 302, 305, 308,
 325, 328, 329
 incompleteness 239
 indistinguishability 98, 99, 104
 inference rule 34, 42, 44, 45, 47, 48, 52,
 54–56, 165, 211, 214–216, 218, 219, 221, 229,
 264, 292, 305
 information flow 93–96, 98–103, 107, 109,
 111, 116, 259–261, 285, 287, 295
 control 94–96, 262
 explicit 95, 101, 261, 264
 implicit 95, 97, 101, 261, 264, 271, 286
 policy 94–96, 98, 99, 101, 111, 258, 261,
 262, 264, 268, 284, 287, 294, 310
 trace-based 98
 transitive 271, 292
 initialization expression 71, 72
 integer program 37, 241
 integrity 92, 93, 96
 Intel’s C++ compiler 194, 195
 inter-procedural analysis 29

intermediate representation 73, 160, 182,
 217, 284, 368
 intermediate verification language 127
 Internet Archive 16
 Invariance Thesis 19
 invariant 115, 116, 129, 139, 177, 207, 208,
 215, 238, 239, 242–244, 250–252
 inductive 115, 207, 215, 238, 250
 linear 244, 254
 non-linear 242, 244, 255
 invariant inference 207, 239, 240, 243, 244,
 247, 250, 252
 invariant template 243
 iProver 119
 Iris 119
 Isabelle 119, 120
 iteration dependence 43, 236
 iteration independence 43, 236
 iteration space 71, 180–183, 185, 192, 193,
 197, 198, 236, 237, 241, 309
 iteration variable 71, 238

J

Java 15, 38, 52, 106, 124, 126, 217, 239, 263,
 265, 283, 284, 288
 Java bytecode 36, 37, 284
 JKind 119
 join 82, 83

K

K Framework 119
 Karp reduction 312
 KeY 119
 KLEE 119
 Kleene algebra 24
 KoAT 37, 239–242, 244, 251, 254
 Kontrol 119
 Kvasir 251

L

language-based security 98, 108, 111, 261,
 286, 287
 lattice 95, 98, 109, 261
 leak *see also* violation, 93, 96–98, 106,
 258–260, 264, 268, 280, 310
 Lean 119

- likely invariant 239, 243, 244, 254
- linear algebra 301
- linear logic 22, 325
- LiquidHaskell 119
- liveness property 117
- LLVM bitcode 37, 241
- LLVM compiler 73, 193
- loop body . . 71, 72, 91, 235, 242, 244, 254, 255
- loop boundedness 238, 316
- loop fission 386
- loop header 71, 253
- loop invariant . . . 70, 115, 116, 126, 139, 207, 215, 238, 243, 244, 250, 254
 - loop transformation
 - distribution 181, 308
 - fission 180–199
 - fusion 200, 317
 - tiling 183
 - unroll 182, 200
- loop transformations . . . 70, 73, 89–91, 317, 329
 - distribution 267, 309
 - fission 73, 91
 - fusion 73, 89
 - interchange 76, 89, 317
 - peel 76
 - reversal 74, 89
 - split 74, 76, 89
 - stripe 89
 - tile 74, 89, 90
 - tiling 317
 - unroll 76, 89
- loop-carried dependency 194
- loop-level parallelism 181
- LOOPUS 153, 154
- lower bound 20, 36, 37, 248
- LQICM 37
- Ltac 122

- M**
- machine model 19, 20, 22, 299, 325
- Mars Climate Orbiter 112
- Mathematical Components . . 122–124, 134, 137–139, 300
- MathSAT 119

- matrix-as-type 218, 301
- Maude 119
- MaxCore 38
- maximally linear flow 43, 165
- mechanized proof 118, 120, 131
- memory safety 116, 298
- Metamath 119
- MiBench 196, 199
- Mizar 119
- modal logic 22
- modeling language 114, 119, 129
- monad 302
- monomial . . . 220, 221, 227, 228, 234, 375, 376
- Move 129
- multigraph 212
- multilevel security 93, 100
- mwp_ℓ 210, 237, 240–247, 254, 255
- mwp_f 237, 244–247, 255
- mwp-bound 38, 42, 45–50, 52, 58, 64, 67, 152, 157, 166, 167, 173, 209, 210, 213, 216, 223, 224, 228–230, 233, 235–238, 244–246, 255, 300, 306, 316, 376, 377
 - iteration-dependent 209, 236, 238
 - iteration-independent 209, 236, 307
 - linear 209, 236
 - optimal 209, 233, 237, 247
 - mwp-bounds
 - optimal 237
- mwp-calculus . . . 10, 15, 27, 42–52, 54–56, 60, 146, 164, 165, 177, 207, 211, 213–216, 218–220, 222, 224, 228–230, 232, 233, 237–239, 242, 244–248, 255, 300, 301, 305, 307, 308, 315, 316, 319, 328, 387
- mwp-matrix 44, 45, 48, 49, 52–59, 62–65, 158, 216–226, 228–232, 234, 235, 300, 301, 305, 306, 365, 367–370, 372, 374–377, 382–384

- N**
- Nagini 119
- NAS Parallel Benchmarks 196, 199
- nested loop 11, 72, 190, 235, 245, 255, 310
- Nidhugg 119
- non-canonical loop 72, 91, 181, 200

- non-functional property . . . 3, 116, 311, 317,
321, 324, 327, 329, 333
- non-influence 103
- non-interference . . . 95, 99–104, 106, 107, 111,
258–260, 264, 267, 277, 284–287, 291, 293,
308, 310, 311, 329, 331
- anytime . . 111, 258–260, 262, 263, 271, 274,
276–278, 280, 282–285, 287, 289–291,
293–295, 311
- bipartite 102
- deadlock-sensitive 102
- intransitive 103
- nondeterministic 102
- progress-insensitive 102
- progress-sensitive 102, 260
- termination-insensitive . . . 102, 260, 285
- termination-sensitive 102
- non-leakage 103
- non-size-increasing program 27
- nondeterminism 35, 45, 47, 48, 54, 55, 98,
150, 155, 156, 158, 160, 166, 170, 212, 214,
218, 220, 224, 241, 245, 300, 305, 330,
366–369, 373, 376
- numerical loops 206–208, 210, 239, 244,
308, 313

O

- object invariant 115
- object oriented SAFE programs . . . 28, 109
- OCaml 37, 122, 143, 377
- Odd Order Theorem 125
- Odd Order theorem 123
- Omega test 79
- oneTBB 82
- OpenJML 119
- OpenMP 15, 81–84, 181–183, 193–199
- OpenMP directives 82, 83, 91
 - atomic 85
 - barrier 85
 - critical 85
 - for 86–88
 - full 89
 - loop 87
 - nowait 85, 87, 196
 - order 87

- parallel 84, 86–88, 184, 194–196
- parallel loop 87
- partial 89, 90
- pragma omp 84, 86, 88, 196
- private 88, 89, 196
- schedule 87
- shared 88
- single 87, 91, 196, 197
- unroll 89
- output dependence 78
- owi 119

P

- Par4All 194, 195
- parallel programming 70, 181, 182, 196,
295
- parallel programming models . . . 81, 84, 91
 - fork-join 82
 - oneTBB 82
 - OpenMP 72, 81, 82, 91
 - PPL 81
- parallel region 82, 84, 85, 87, 88
- parallel time 19
- parallelization directive 83, 84, 89,
182–184, 195, 197–199
- parallelization potential . . . 90, 182, 309, 329
- parallelizing compilation 182, 194–196,
198
- Parikh’s Theorem 6
- parse tree 283
- partial correctness 44, 117, 213
- Pastis 37, 146
- perfectly nested loop 72
- PicoSAT 119
- Pluto 194, 195
- pointer analysis 29
- pointer machines 28
- PolyBench/C 195–199, 295
- polyhedral optimization 183
- polynomial flow 43, 166
- polynomial structure 220–222, 224, 228,
305
- postcondition 115, 116, 126, 206–211,
214–216, 228, 230, 233–239, 241–243,
245–247, 249, 250, 252, 254, 255

PPL	81	randomness	246
precision	29, 31, 33, 40, 50, 218, 248, 284, 309	Rebeca	119
precondition	115, 126, 127, 208, 209, 214, 215, 238, 254	recurrence relation	35
primary thread	82, 85	recursive function	24
primitive recursion	22, 24, 25	refactoring	2
Princess	119	region	83
Prinsys	119	resource analysis	29, 33, 116, 207, 313
PRISM	119	resource bounds	34, 40, 116, 313
probabilistic programs	34, 318	resource usage	19, 20, 29, 298
program centric attacker model	262	Rice's Theorem	30, 39, 259
program synthesis	317	RMC	119
program trace	239, 243, 244, 251, 252	Rocq	14, 15, 119–125, 130–132, 134–136, 139, 141, 145
programming model	82	Rocq tactics	121, 122
Prolog	36	auto	131
proof assistant	118–121, 124, 131, 303	discriminate	131
Proof General	123	intros	122
proof script	121, 131	reflexivity	122
proof state	121, 122	rewrite	131, 139
properties	116	simpl	122
functional	116	split	131
liveness	117	ROSE	188, 193–197, 199
safety	117	Rust	41
Prover9	119	S	
Prusti	119	SAFE processes	27, 109
PUBS	36	SAFE programs	27, 108, 109
PVS	119	safe recursion	24–26, 110, 111, 125, 145, 312
pyparser	155	safety property	117
pymwp	38, 48–51, 56, 60, 150–160, 164–168, 171, 172, 175, 177, 178, 210, 227, 237, 245, 250, 256, 308, 317, 331, 377–379, 384–386	Sally	119
Python	154, 155, 158, 167, 168, 377, 384	SASyLF	119
Q		Sat4j	119
quantitative type theory	28	Satallax	119
quantum computing	318	saturated covering	191, 192
quantum languages	325	Scheme	122
quasi-interpretation	27, 40, 145	secure compilation	6, 103, 286
quasi-invariant	11, 236, 307, 377	security analysis	99
R		security class	94, 98, 100, 107, 258–262, 264, 273–278, 284–286, 294, 295, 310
ramification	26, 27, 109	security class inference	284, 288
ramified programs	27, 109	security monoid	267, 268, 271
RaML	37, 318, 377	security objective	99, 262
		security property	99
		security type	98, 262

security type system . . . 98, 103, 104, 106, 108,
 109, 258, 259, 262, 286, 287
 security-flow matrix 267, 268, 270, 279,
 282, 284, 285, 287, 288, 291, 293, 296
 semi-ring . . . 271, 300, 364, 365, 368–370, 372,
 374–376, 379, 382
 session types 34
 shape analysis 109
 shared memory 82
 side channel 94, 97
 side effect 260, 262, 263, 277, 280–283
 simulation 99
 size bound 25, 37, 239, 241, 242, 244, 251
 size-change termination principle . . . 27, 33
 small-steps semantics 301
 Software Foundations 300, 301
 Software Heritage 16
 Software System Award 121, 125
 software testing 30, 104, 113, 207
 soundness . . . 30–33, 38–40, 103, 209, 227, 232,
 274, 305, 308, 326
 soundness of mwp-calculus . . . 10, 44, 146,
 213, 215, 229, 301, 328, 331, 387
 soundy 32, 33
 source-to-source compilation 180, 182,
 194
 space complexity 19, 33, 303
 SPARK 119
 SPASS 119
 Spec# 129
 specifications . . . 112–117, 120, 122, 124, 126,
 127, 132, 133, 136, 139, 206, 207, 214, 215,
 238, 242, 247–249, 258, 308, 317, 319, 324,
 325, 332, 333
 SPEED 377
 SSProve 125
 SSReflect 15, 123, 124, 134, 137, 139, 143
 Stainless 119
 state explosion 182, 224, 367, 368
 static program analysis . . 29, 31, 40, 103, 116,
 239, 260, 333
 static single assignment 378
 stratified programs 28, 109
 stratified recurrence 26
 strong normalization 33

structured block 83, 84, 89, 194
 subtype 132
 SV-Comp 36
 symbolic execution 6
 Symbolic JPF 119
 system model 99, 262

T

tactics 131
 taint analysis 284, 288
 task 84
 TcT 37
 team 83, 87
 term rewrite system 36, 40, 325
 term rewriting 22
 termination . . . 30, 33, 40, 42, 44, 64, 94, 116,
 131, 136, 139, 145, 176, 208, 213–215, 218,
 227, 241, 244, 260, 274, 285, 299, 309
 Termination Competition . . 36, 37, 313, 314
 termination condition 71, 72
 Therac-25 112
 thread 83–89, 242
 threat model 99
 tiering 26, 109
 tight bound 20
 time complexity 19, 33, 110, 300, 303
 Tinker 119
 TLA+ 119
 TLC 119
 total correctness 117, 127, 139, 324
 trace 98, 103
 trace-based model 98, 100
 TRACO 194, 195
 transition ideal 242, 251
 true dependence 77, 78
 Turing Machine 19, 21, 25, 303
 Turing-completeness 6, 7, 30, 32, 39–41
 tyni 263, 283, 284, 288
 type checking 262
 type system . . . 22, 29, 110, 124, 287, 299, 301,
 325

U

unbounded loop 208, 236, 238, 248

undecidability . . . 30, 31, 39, 40, 117, 125, 298,
324
 Universal Turing Machine 145, 303
 unroll factor 90
 update expression 71, 72
 UPPAAL 119
 upper bound 20, 36, 40, 248

V

Vampire 119
 Verasco 377
 VerCors 119
 VeriFast 119
 veriT 119
 violation 96, 260, 262, 264, 266–273, 276,
279, 290–295, 310
 Viper 119
 Visual Studio Code 128

W

Wayback Machine 16
 weak polynomial flow 43, 165
 Whiley 119
 Why3 119
 worksharing loop 84, 87
 worksharing loop property 84
 worst-case execution time 116, 302

X

X-by-Construction 42, 317

Y

Yices 119

Z

Z specification language 14
 Z3 119, 129
 zero matrix 53